

# Rust Core

Language fundamentals for programmers coming from  
another language

Viacheslav Chub

2026

# Contents

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Preface</b>                                          | <b>1</b>  |
| 1.1      | Who these notes are for . . . . .                       | 1         |
| 1.2      | Why Rust is a different bet . . . . .                   | 1         |
| 1.3      | Production standards — what compile time does not teach | 2         |
| 1.4      | What you will not find here . . . . .                   | 3         |
| 1.5      | What these notes are (and are not) . . . . .            | 3         |
| 1.6      | Try it in the Rust Playground . . . . .                 | 4         |
| 1.7      | How to work through a chapter . . . . .                 | 4         |
| 1.8      | Afterparty: aim, importance, and how to use . . . . .   | 5         |
| 1.8.1    | Aim . . . . .                                           | 5         |
| 1.8.2    | Importance . . . . .                                    | 5         |
| 1.8.3    | How to use . . . . .                                    | 6         |
| 1.8.4    | Where to find prompts . . . . .                         | 7         |
| 1.9      | When something is unclear, prompt it . . . . .          | 7         |
| 1.10     | Toolchain: rustup and Cargo . . . . .                   | 8         |
| 1.11     | Conventions . . . . .                                   | 9         |
| 1.12     | Reading ahead . . . . .                                 | 9         |
| 1.13     | See also . . . . .                                      | 10        |
| <b>2</b> | <b>Chapter 1: Paradigm Shift</b>                        | <b>11</b> |
| 2.1      | Hook . . . . .                                          | 11        |
| 2.2      | Compiled vs interpreted . . . . .                       | 11        |
| 2.3      | Ownership vs garbage collection . . . . .               | 12        |
| 2.3.1    | Move edge cases (what the compiler catches) . . . .     | 12        |
| 2.4      | Zero-cost abstractions . . . . .                        | 13        |
| 2.5      | Fearless concurrency (preview) . . . . .                | 13        |
| 2.6      | Stack and heap . . . . .                                | 14        |
| 2.6.1    | Stack: fast and scoped . . . . .                        | 14        |
| 2.6.2    | Heap: growable data with an owner . . . . .             | 15        |
| 2.6.3    | Moves and the heap . . . . .                            | 16        |
| 2.6.4    | Copy on stack vs heap-backed types . . . . .            | 17        |
| 2.6.5    | References, borrowing, and dereferencing . . . . .      | 19        |
| 2.6.6    | Borrow checker edge cases . . . . .                     | 25        |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| 2.6.7    | When the compiler says no . . . . .                       | 27        |
| 2.6.8    | Why this matters for long-running programs . . . .        | 28        |
| 2.7      | Idiom spotlight . . . . .                                 | 28        |
| 2.8      | Go deeper . . . . .                                       | 28        |
| 2.9      | See also . . . . .                                        | 28        |
| 2.9.1    | Afterparty . . . . .                                      | 29        |
| <b>3</b> | <b>Chapter 2: Types and Expressions</b>                   | <b>33</b> |
| 3.1      | Hook . . . . .                                            | 33        |
| 3.2      | Integer types . . . . .                                   | 33        |
| 3.3      | Floating point, bool, and char . . . . .                  | 34        |
| 3.4      | Inference and annotations . . . . .                       | 35        |
| 3.5      | Tuples and arrays . . . . .                               | 35        |
| 3.6      | Slices . . . . .                                          | 36        |
| 3.7      | Strings and UTF-8 . . . . .                               | 37        |
| 3.7.1    | &str vs String in APIs . . . . .                          | 38        |
| 3.7.2    | Length, iteration, and slicing text . . . . .             | 39        |
| 3.7.3    | Common &str helpers (no allocation) . . . . .             | 40        |
| 3.8      | Variables and mutability . . . . .                        | 41        |
| 3.9      | Control flow as expressions . . . . .                     | 41        |
| 3.10     | match preview . . . . .                                   | 42        |
| 3.11     | Idiom spotlight . . . . .                                 | 43        |
| 3.12     | Go deeper . . . . .                                       | 43        |
| 3.13     | See also . . . . .                                        | 43        |
| 3.13.1   | Afterparty . . . . .                                      | 43        |
| <b>4</b> | <b>Chapter 3: Functions and Methods</b>                   | <b>46</b> |
| 4.1      | Hook . . . . .                                            | 46        |
| 4.2      | Scope — a brief tour . . . . .                            | 46        |
| 4.3      | Function signatures . . . . .                             | 47        |
| 4.4      | Parameters and ownership . . . . .                        | 47        |
| 4.5      | Expression bodies and early return . . . . .              | 48        |
| 4.6      | The unit type () and diverging functions . . . . .        | 49        |
| 4.7      | Methods and associated functions . . . . .                | 50        |
| 4.8      | Multiple impl blocks and privacy . . . . .                | 51        |
| 4.9      | Generic functions (preview) . . . . .                     | 52        |
| 4.10     | Functions that return Result (preview) . . . . .          | 53        |
| 4.11     | impl Trait in return position . . . . .                   | 53        |
| 4.12     | Draining with mem::take . . . . .                         | 54        |
| 4.13     | where clauses — readable bounds . . . . .                 | 55        |
| 4.14     | Error constructor factories . . . . .                     | 55        |
| 4.15     | Config-holder structs (lightweight builders) . . . . .    | 57        |
| 4.16     | Extension traits — add behaviour without newtypes . . . . | 58        |
| 4.16.1   | Function edge cases . . . . .                             | 59        |
| 4.17     | When the compiler says no . . . . .                       | 60        |
| 4.18     | Idiom spotlight . . . . .                                 | 60        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| 4.19     | Go deeper . . . . .                                   | 60        |
| 4.20     | See also . . . . .                                    | 60        |
| 4.20.1   | Afterparty . . . . .                                  | 60        |
| <b>5</b> | <b>Chapter 4: Iterators</b>                           | <b>63</b> |
| 5.1      | Hook . . . . .                                        | 63        |
| 5.2      | for is syntax sugar . . . . .                         | 64        |
| 5.3      | Three ways to walk a Vec . . . . .                    | 64        |
| 5.3.1    | iter vs into_iter — what changes . . . . .            | 65        |
| 5.3.2    | for forms — quick trap sheet . . . . .                | 67        |
| 5.4      | Lazy iterators . . . . .                              | 68        |
| 5.4.1    | Why closures see &&i32 (double reference) . . . . .   | 68        |
| 5.5      | Common adapters . . . . .                             | 69        |
| 5.5.1    | Adapter edge cases (empty, short, uneven) . . . . .   | 69        |
| 5.6      | Consumers — finishing the pipeline . . . . .          | 71        |
| 5.6.1    | collect and type hints . . . . .                      | 71        |
| 5.6.2    | Consumer edge cases . . . . .                         | 72        |
| 5.7      | Implementing Iterator . . . . .                       | 73        |
| 5.7.1    | Port range scanner . . . . .                          | 73        |
| 5.7.2    | Non-empty line iterator . . . . .                     | 74        |
| 5.7.3    | Implementing Iterator edge cases . . . . .            | 75        |
| 5.8      | When the compiler says no . . . . .                   | 77        |
| 5.9      | Java / Python contrast . . . . .                      | 77        |
| 5.10     | Idiom spotlight . . . . .                             | 77        |
| 5.11     | Go deeper . . . . .                                   | 78        |
| 5.12     | See also . . . . .                                    | 78        |
| 5.12.1   | Afterparty . . . . .                                  | 78        |
| <b>6</b> | <b>Chapter 5: Lifetimes</b>                           | <b>81</b> |
| 6.1      | Hook . . . . .                                        | 81        |
| 6.2      | Scope — a brief tour . . . . .                        | 81        |
| 6.3      | References always have a lifetime . . . . .           | 82        |
| 6.4      | What lifetimes are for . . . . .                      | 82        |
| 6.4.1    | How 'a helps (it's just a label) . . . . .            | 83        |
| 6.4.2    | What if you omit <'a>? . . . . .                      | 83        |
| 6.5      | Elision (why you rarely write lifetimes) . . . . .    | 85        |
| 6.6      | Named lifetimes on structs . . . . .                  | 86        |
| 6.6.1    | What if the struct omits <'a>? . . . . .              | 87        |
| 6.7      | 'static trap — formatted strings . . . . .            | 88        |
| 6.8      | Two lifetimes — when 'a and 'b diverge . . . . .      | 89        |
| 6.9      | Config<'a> — borrowed slices vs owned fix . . . . .   | 89        |
| 6.9.1    | Lifetime edge cases . . . . .                         | 90        |
| 6.10     | When the compiler says no . . . . .                   | 90        |
| 6.11     | Java / Python contrast . . . . .                      | 90        |
| 6.12     | Idiom spotlight . . . . .                             | 91        |
| 6.13     | Playground: two inputs, one shared lifetime . . . . . | 91        |

|          |                                                        |            |
|----------|--------------------------------------------------------|------------|
| 6.14     | Go deeper . . . . .                                    | 91         |
| 6.15     | See also . . . . .                                     | 92         |
| 6.15.1   | Afterparty . . . . .                                   | 92         |
| <b>7</b> | <b>Chapter 6: Enums and Pattern Matching</b>           | <b>93</b>  |
| 7.1      | Hook . . . . .                                         | 93         |
| 7.2      | Option<T> — no null . . . . .                          | 94         |
| 7.2.1    | if let, while let, and let ... else . . . . .          | 94         |
| 7.2.2    | Option combinators (before heavy match) . . . . .      | 95         |
| 7.2.3    | Option edge cases and compiler traps . . . . .         | 95         |
| 7.3      | Result<T, E> — errors as values . . . . .              | 96         |
| 7.3.1    | Result edge cases . . . . .                            | 97         |
| 7.4      | Custom enums — sum types . . . . .                     | 98         |
| 7.4.1    | Enum shapes (pattern matching preview) . . . . .       | 98         |
| 7.5      | Pattern matching mechanics . . . . .                   | 99         |
| 7.5.1    | Exhaustive match . . . . .                             | 99         |
| 7.5.2    | Wildcard _ — power and footgun . . . . .               | 102        |
| 7.5.3    | match on references — borrow vs move . . . . .         | 102        |
| 7.5.4    | Match guards . . . . .                                 | 103        |
| 7.5.5    | if let vs match . . . . .                              | 104        |
| 7.6      | Java / Python contrast (deeper) . . . . .              | 104        |
| 7.7      | Service-shaped example . . . . .                       | 105        |
| 7.8      | Advanced patterns . . . . .                            | 105        |
| 7.8.1    | Slice patterns — split a command line . . . . .        | 105        |
| 7.8.2    | matches! — enum check without full match . . . . .     | 106        |
| 7.8.3    | if let chains — bind host and port together . . . . .  | 107        |
| 7.8.4    | @bindings — match and bind in one arm . . . . .        | 108        |
| 7.8.5    | Advanced pattern edge cases . . . . .                  | 108        |
| 7.9      | When the compiler says no . . . . .                    | 109        |
| 7.10     | Idiom spotlight . . . . .                              | 109        |
| 7.11     | Go deeper . . . . .                                    | 110        |
| 7.12     | See also . . . . .                                     | 110        |
| 7.12.1   | Afterparty . . . . .                                   | 110        |
| <b>8</b> | <b>Chapter 7: Structs, Traits, and Generics</b>        | <b>113</b> |
| 8.1      | Hook . . . . .                                         | 113        |
| 8.2      | Structs and methods . . . . .                          | 113        |
| 8.3      | Traits — interfaces done right . . . . .               | 114        |
| 8.4      | Enums, structs, and traits together . . . . .          | 115        |
| 8.4.1    | Two impl blocks on one type . . . . .                  | 118        |
| 8.4.2    | Struct-in-enum vs enum-in-struct . . . . .             | 118        |
| 8.4.3    | Trait on enum: match is mandatory . . . . .            | 120        |
| 8.4.4    | #[derive(...)] on enums and structs . . . . .          | 124        |
| 8.4.5    | Enums + traits edge cases and compiler traps . . . . . | 125        |
| 8.4.6    | Idiom spotlight (enums + traits) . . . . .             | 129        |
| 8.5      | Generics . . . . .                                     | 129        |

|          |                                                                       |            |
|----------|-----------------------------------------------------------------------|------------|
| 8.6      | Trait bounds and where . . . . .                                      | 129        |
| 8.7      | Trait objects (dyn Trait) . . . . .                                   | 130        |
| 8.7.1    | What you actually store . . . . .                                     | 130        |
| 8.7.2    | When dyn Trait is idiomatic . . . . .                                 | 131        |
| 8.7.3    | impl Trait vs dyn Trait vs enum . . . . .                             | 134        |
| 8.7.4    | Object safety — not every trait can be dyn . . . . .                  | 135        |
| 8.7.5    | More edge cases and compiler traps . . . . .                          | 136        |
| 8.8      | Trait decomposition — small interfaces . . . . .                      | 138        |
| 8.9      | Strategy enum — closed dispatch without a top-level vtable . . . . .  | 140        |
| 8.10     | Transform traits — parse once, map to domain . . . . .                | 141        |
| 8.11     | Extension traits in your crate . . . . .                              | 143        |
| 8.12     | Associated types and supertraits . . . . .                            | 143        |
| 8.12.1   | Iterator::Item — one output type per iterator . . . . .               | 144        |
| 8.12.2   | Custom trait with type Output . . . . .                               | 145        |
| 8.12.3   | Associated type vs generic trait parameter . . . . .                  | 146        |
| 8.12.4   | Supertraits — stacking requirements . . . . .                         | 146        |
| 8.12.5   | Associated types and object safety . . . . .                          | 147        |
| 8.12.6   | UFCS when two traits share a method name . . . . .                    | 147        |
| 8.13     | Idiom spotlight . . . . .                                             | 148        |
| 8.14     | Go deeper . . . . .                                                   | 148        |
| 8.15     | See also . . . . .                                                    | 148        |
| 8.15.1   | Afterparty . . . . .                                                  | 148        |
| <b>9</b> | <b>Chapter 8: Errors and Testing</b>                                  | <b>152</b> |
| 9.1      | Hook . . . . .                                                        | 152        |
| 9.2      | Panic, unwind, and why it is not Result . . . . .                     | 153        |
| 9.2.1    | Problems unwind creates in production . . . . .                       | 153        |
| 9.3      | Error propagation . . . . .                                           | 154        |
| 9.3.1    | Three propagation tools . . . . .                                     | 156        |
| 9.3.2    | Internal vs boundary . . . . .                                        | 157        |
| 9.3.3    | Example: read first line from a file ( <b>Cargo only</b> ) . . . . .  | 157        |
| 9.3.4    | Propagation edge cases . . . . .                                      | 158        |
| 9.4      | Std library errors — fine for learning, weak for production . . . . . | 160        |
| 9.5      | Custom errors . . . . .                                               | 161        |
| 9.5.1    | Manual enum — learn the model . . . . .                               | 161        |
| 9.5.2    | thiserror in production — count it in . . . . .                       | 163        |
| 9.5.3    | anyhow for binaries (sidebar) . . . . .                               | 166        |
| 9.5.4    | Crate-local Result alias . . . . .                                    | 166        |
| 9.5.5    | Transparent #[from] variants . . . . .                                | 167        |
| 9.5.6    | Error aggregation for batch work . . . . .                            | 168        |
| 9.6      | Extension traits on Option . . . . .                                  | 169        |
| 9.6.1    | Custom error edge cases . . . . .                                     | 170        |
| 9.7      | Errors and enums . . . . .                                            | 172        |
| 9.7.1    | Variant shapes for errors . . . . .                                   | 172        |
| 9.7.2    | Error enum vs domain enum — keep them separate . . . . .              | 172        |
| 9.7.3    | Recovery by variant . . . . .                                         | 173        |

|        |                                                        |     |
|--------|--------------------------------------------------------|-----|
| 9.7.4  | Nested error enums . . . . .                           | 174 |
| 9.7.5  | Errors-and-enums edge cases . . . . .                  | 175 |
| 9.8    | panic! vs recover — decision table . . . . .           | 176 |
| 9.8.1  | Panic and unwind edge cases . . . . .                  | 177 |
| 9.9    | Testing . . . . .                                      | 177 |
| 9.9.1  | Testing edge cases . . . . .                           | 179 |
| 9.9.2  | Trait-based mocks — test orchestration, not I/O . .    | 180 |
| 9.9.3  | Golden-file and shared parser tests . . . . .          | 181 |
| 9.9.4  | Comparable test DTOs — normalize before equality       | 182 |
| 9.9.5  | Test-only derives with <code>cfg_attr</code> . . . . . | 184 |
| 9.9.6  | Async tests . . . . .                                  | 184 |
| 9.10   | Idiom spotlight . . . . .                              | 184 |
| 9.11   | Go deeper . . . . .                                    | 185 |
| 9.12   | See also . . . . .                                     | 185 |
| 9.12.1 | Afterparty . . . . .                                   | 185 |

## I Crates, Memory, and Data 188

|         |                                                                              |     |
|---------|------------------------------------------------------------------------------|-----|
| 10      | Chapter 9: Modules, Paths, and Crates                                        | 189 |
| 10.1    | Hook . . . . .                                                               | 189 |
| 10.2    | Scope — a brief tour . . . . .                                               | 189 |
| 10.3    | Crate, package, and workspace . . . . .                                      | 190 |
| 10.4    | Declaring modules . . . . .                                                  | 190 |
| 10.5    | Paths: <code>self</code> , <code>super</code> , <code>crate</code> . . . . . | 191 |
| 10.6    | Visibility: the pub ladder . . . . .                                         | 192 |
| 10.7    | <code>use</code> and re-exports . . . . .                                    | 193 |
| 10.8    | Binary vs library crate . . . . .                                            | 193 |
| 10.9    | Tests as modules . . . . .                                                   | 194 |
| 10.10   | Conditional compilation and Cargo features . . . . .                         | 195 |
| 10.10.1 | Common <code>cfg</code> attributes . . . . .                                 | 196 |
| 10.10.2 | Feature edge case — optional dependency . . . . .                            | 196 |
| 10.11   | Integration tests . . . . .                                                  | 197 |
| 10.11.1 | Re-export edge case — prelude module . . . . .                               | 197 |
| 10.12   | Workspaces (brief) . . . . .                                                 | 198 |
| 10.13   | Orphan rule (crate boundary) . . . . .                                       | 198 |
| 10.14   | Documentation comments . . . . .                                             | 198 |
| 10.15   | When the compiler says no . . . . .                                          | 199 |
| 10.16   | Idiom spotlight . . . . .                                                    | 199 |
| 10.17   | Go deeper . . . . .                                                          | 199 |
| 10.18   | See also . . . . .                                                           | 199 |
| 10.18.1 | Afterparty . . . . .                                                         | 200 |
| 11      | Chapter 10: Smart Pointers and Interior Mutability                           | 202 |
| 11.1    | Hook . . . . .                                                               | 202 |
| 11.2    | Scope — a brief tour . . . . .                                               | 202 |

|           |                                                           |            |
|-----------|-----------------------------------------------------------|------------|
| 11.3      | Box<T> — heap, single owner                               | 203        |
| 11.3.1    | Recursive enum (why Box is required)                      | 203        |
| 11.3.2    | Box edge cases                                            | 204        |
| 11.4      | Rc<T> and Arc<T> — shared ownership                       | 204        |
| 11.4.1    | clone on the smart pointer vs .clone() on the inner value | 205        |
| 11.4.2    | Cycles and Weak                                           | 205        |
| 11.4.3    | Rc / Arc edge cases                                       | 206        |
| 11.5      | Cell and RefCell — interior mutability                    | 206        |
| 11.5.1    | Cell for small Copy values                                | 207        |
| 11.5.2    | RefCell basics                                            | 207        |
| 11.5.3    | Rc<RefCell<T>> — shared mutable graph on one thread       | 207        |
| 11.5.4    | RefCell edge cases                                        | 208        |
| 11.6      | Deref and deref coercion                                  | 209        |
| 11.6.1    | Deref edge cases                                          | 210        |
| 11.7      | Drop — cleanup on scope end                               | 210        |
| 11.7.1    | Drop order edge cases                                     | 211        |
| 11.8      | Choosing a pointer (quick table)                          | 212        |
| 11.9      | When the compiler says no                                 | 212        |
| 11.10     | Idiom spotlight                                           | 212        |
| 11.11     | Go deeper                                                 | 213        |
| 11.12     | See also                                                  | 213        |
| 11.12.1   | Afterparty                                                | 213        |
| <b>12</b> | <b>Chapter 11: Collections</b>                            | <b>215</b> |
| 12.1      | Hook                                                      | 215        |
| 12.2      | Scope — a brief tour                                      | 215        |
| 12.3      | Choosing a collection                                     | 215        |
| 12.4      | Vec<T>                                                    | 216        |
| 12.4.1    | Safe indexing and mutation                                | 217        |
| 12.4.2    | Vec methods worth knowing                                 | 217        |
| 12.4.3    | Vec edge cases                                            | 217        |
| 12.5      | HashMap<K, V>                                             | 218        |
| 12.5.1    | The entry API                                             | 219        |
| 12.5.2    | HashMap edge cases                                        | 220        |
| 12.6      | HashSet<T>                                                | 221        |
| 12.7      | VecDeque<T>                                               | 222        |
| 12.8      | BTreeMap and BTreeSet                                     | 222        |
| 12.8.1    | Range queries                                             | 223        |
| 12.9      | Iterators and collection methods                          | 224        |
| 12.9.1    | .windows, .chunks, .enumerate                             | 224        |
| 12.9.2    | sort_by and custom order                                  | 225        |
| 12.10     | Collecting into collections                               | 225        |
| 12.10.1   | into_iter vs iter when building collections               | 226        |
| 12.11     | When the compiler says no                                 | 226        |



|           |                                                               |            |
|-----------|---------------------------------------------------------------|------------|
| 12.12     | Idiom spotlight . . . . .                                     | 226        |
| 12.13     | Go deeper . . . . .                                           | 227        |
| 12.14     | See also . . . . .                                            | 227        |
| 12.14.1   | Afterparty . . . . .                                          | 227        |
| <b>13</b> | <b>Chapter 12: Closures and the Fn Traits</b>                 | <b>229</b> |
| 13.1      | Hook . . . . .                                                | 229        |
| 13.2      | Scope — a brief tour . . . . .                                | 229        |
| 13.3      | Closure syntax . . . . .                                      | 230        |
| 13.4      | Capture modes . . . . .                                       | 230        |
| 13.5      | The move keyword . . . . .                                    | 231        |
| 13.6      | Fn, FnMut, FnOnce . . . . .                                   | 231        |
| 13.7      | Closures and iterator adapters . . . . .                      | 232        |
| 13.8      | Closures vs function pointers . . . . .                       | 232        |
| 13.9      | Returning closures . . . . .                                  | 233        |
| 13.10     | Callback registry — storing closures . . . . .                | 233        |
| 13.11     | Sort and filter with closures . . . . .                       | 234        |
| 13.12     | Generic helpers — impl Fn vs boxing . . . . .                 | 235        |
| 13.13     | Capture edge cases . . . . .                                  | 235        |
| 13.14     | Thread handoff — move on spawn . . . . .                      | 237        |
| 13.15     | Async closure note . . . . .                                  | 237        |
| 13.16     | Java / Python contrast . . . . .                              | 238        |
| 13.17     | When the compiler says no . . . . .                           | 238        |
| 13.18     | Idiom spotlight . . . . .                                     | 238        |
| 13.19     | Go deeper . . . . .                                           | 238        |
| 13.20     | See also . . . . .                                            | 239        |
| 13.20.1   | Afterparty . . . . .                                          | 239        |
| <b>14</b> | <b>Chapter 13: Standard Traits and Conversions</b>            | <b>241</b> |
| 14.1      | Hook . . . . .                                                | 241        |
| 14.2      | Scope — a brief tour . . . . .                                | 241        |
| 14.3      | Debug and Display . . . . .                                   | 242        |
| 14.3.1    | Debug vs Display — when which . . . . .                       | 243        |
| 14.3.2    | Formatting edge cases . . . . .                               | 244        |
| 14.3.3    | Formatting flags — quick reference . . . . .                  | 245        |
| 14.4      | Default . . . . .                                             | 246        |
| 14.5      | PartialEq, Eq, Hash, and Ord . . . . .                        | 246        |
| 14.6      | Clone and Copy (brief) . . . . .                              | 247        |
| 14.7      | From and Into . . . . .                                       | 248        |
| 14.7.1    | Chaining From without duplication . . . . .                   | 248        |
| 14.8      | TryFrom, TryInto, and FromStr . . . . .                       | 249        |
| 14.8.1    | FromStr for structured values — paths and endpoints . . . . . | 251        |
| 14.8.2    | Display + FromStr round-trip for CLI enums . . . . .          | 252        |
| 14.8.3    | Display on field-name enums — stable log labels . . . . .     | 253        |
| 14.8.4    | TryFrom edge cases . . . . .                                  | 254        |
| 14.9      | AsRef and Borrow . . . . .                                    | 255        |

|                                                                  |     |
|------------------------------------------------------------------|-----|
| 14.9.1 AsRef vs &T parameter . . . . .                           | 255 |
| 14.10 Cow — clone on write . . . . .                             | 256 |
| 14.10.1 into_owned() — finish with a plain owned value . . . . . | 257 |
| 14.10.2 ToOwned and .to_owned() on references . . . . .          | 258 |
| 14.11 Deref — cheap access through smart pointers . . . . .      | 260 |
| 14.12 Extension traits on &str . . . . .                         | 260 |
| 14.13 Common derive sets (cheat sheet) . . . . .                 | 261 |
| 14.14 Java / Python contrast . . . . .                           | 261 |
| 14.15 When the compiler says no . . . . .                        | 262 |
| 14.16 Idiom spotlight . . . . .                                  | 262 |
| 14.17 Go deeper . . . . .                                        | 263 |
| 14.18 See also . . . . .                                         | 263 |
| 14.18.1 Afterparty . . . . .                                     | 263 |

## II Concurrency 266

|                                                                              |            |
|------------------------------------------------------------------------------|------------|
| <b>15 Chapter 14: Multithreading</b> . . . . .                               | <b>267</b> |
| 15.1 Hook . . . . .                                                          | 267        |
| 15.2 Scope — a brief tour . . . . .                                          | 267        |
| 15.3 What multithreading is . . . . .                                        | 268        |
| 15.4 Send and Sync — the thread boundary rules . . . . .                     | 268        |
| 15.5 Examples: elementary -> hard . . . . .                                  | 269        |
| 15.5.1 Level 1 — Elementary: spawn and join . . . . .                        | 269        |
| 15.5.2 Level 2 — Elementary: move into the thread . . . . .                  | 270        |
| 15.5.3 Level 3 — Medium: mpsc channel . . . . .                              | 271        |
| 15.5.4 Level 4 — Medium: Arc<Mutex<T>> shared counter . . . . .              | 271        |
| 15.5.5 Level 5 — Hard: Send trap — Rc cannot enter a thread . . . . .        | 273        |
| 15.5.6 Level 6 — Hard: join without unwrap + sensor poll<br>worker . . . . . | 274        |
| 15.6 RwLock — many readers, rare writers . . . . .                           | 275        |
| 15.7 OnceLock — initialize once . . . . .                                    | 276        |
| 15.8 thread::scope — borrow stack data into threads . . . . .                | 277        |
| 15.8.1 Sync primitive edge cases . . . . .                                   | 278        |
| 15.9 Techniques at a glance . . . . .                                        | 278        |
| 15.10 Idiom spotlight . . . . .                                              | 279        |
| 15.11 Go deeper . . . . .                                                    | 279        |
| 15.12 See also . . . . .                                                     | 279        |
| 15.12.1 Afterparty . . . . .                                                 | 280        |
| <b>16 Chapter 15: Atomics and Lock-Free Basics</b> . . . . .                 | <b>282</b> |
| 16.1 Hook . . . . .                                                          | 282        |
| 16.2 Scope — a brief tour . . . . .                                          | 282        |
| 16.3 Where atomics fit — threads, async, and Chapter 14 . . . . .            | 283        |
| 16.4 Data races — why atomics exist . . . . .                                | 283        |
| 16.5 Examples: elementary -> hard . . . . .                                  | 284        |

|           |                                                                     |            |
|-----------|---------------------------------------------------------------------|------------|
| 16.5.1    | Level 1 — Elementary: AtomicBool shutdown flag .                    | 284        |
| 16.5.2    | Level 2 — Elementary: fetch_add counter . . . . .                   | 285        |
| 16.5.3    | Level 3 — Medium: compare_exchange — cap a<br>counter . . . . .     | 286        |
| 16.5.4    | Level 4 — Medium: Release/Acquire publish pattern                   | 288        |
| 16.5.5    | Level 5 — Hard: when Relaxed lies — ordering foot-<br>gun . . . . . | 289        |
| 16.5.6    | Level 6 — Hard: gateway sketch . . . . .                            | 290        |
| 16.6      | Memory ordering (practical subset) . . . . .                        | 292        |
| 16.7      | Atomics vs Mutex . . . . .                                          | 292        |
| 16.8      | Edge cases and compiler traps . . . . .                             | 293        |
| 16.9      | Idiom spotlight . . . . .                                           | 294        |
| 16.10     | Go deeper . . . . .                                                 | 294        |
| 16.11     | See also . . . . .                                                  | 294        |
| 16.11.1   | Afterparty . . . . .                                                | 294        |
| <b>17</b> | <b>Chapter 16: Async Rust and Tokio</b>                             | <b>297</b> |
| 17.1      | Hook . . . . .                                                      | 297        |
| 17.2      | Scope — a brief tour . . . . .                                      | 297        |
| 17.2.1    | What this chapter skips — and where to learn it . .                 | 298        |
| 17.3      | What async is . . . . .                                             | 299        |
| 17.3.1    | Level 0 — Mental model (sync stand-in, not async) .                 | 299        |
| 17.4      | Examples: elementary -> hard . . . . .                              | 300        |
| 17.4.1    | Level 1 — Elementary: minimal Tokio spawn . . . .                   | 300        |
| 17.4.2    | Level 2 — Elementary: sequential vs concurrent<br>.await . . . . .  | 301        |
| 17.4.3    | Level 3 — Medium: join! and timeout . . . . .                       | 302        |
| 17.4.4    | Level 4 — Medium: select! — first branch wins .                     | 303        |
| 17.4.5    | Level 5 — Hard: blocking footgun + spawn_blocking                   | 303        |
| 17.4.6    | Level 6 — Hard: async gateway supervisor . . . . .                  | 306        |
| 17.4.7    | Level 7 — Medium: bounded concurrency with<br>Semaphore . . . . .   | 309        |
| 17.4.8    | Level 8 — Medium: batch spawn and await?? . . .                     | 310        |
| 17.4.9    | Async trait boundaries for testing . . . . .                        | 311        |
| 17.4.10   | Pipeline sketch — reader task, channel, workers . .                 | 313        |
| 17.4.11   | Polling loop — long-running HTTP or job status . .                  | 314        |
| 17.5      | Techniques at a glance . . . . .                                    | 315        |
| 17.6      | Async I/O (brief) . . . . .                                         | 316        |
| 17.7      | Async vs OS threads . . . . .                                       | 317        |
| 17.8      | Edge cases and compiler traps . . . . .                             | 317        |
| 17.9      | Idiom spotlight . . . . .                                           | 319        |
| 17.10     | Go deeper . . . . .                                                 | 319        |
| 17.11     | See also . . . . .                                                  | 319        |
| 17.11.1   | Afterparty . . . . .                                                | 319        |

|                                                                                                          |            |
|----------------------------------------------------------------------------------------------------------|------------|
| <b>III Metaprogramming and Unsafe</b>                                                                    | <b>322</b> |
| <b>18 Chapter 17: Metaprogramming</b>                                                                    | <b>323</b> |
| 18.1 Hook . . . . .                                                                                      | 323        |
| 18.2 Scope — a brief tour . . . . .                                                                      | 323        |
| 18.3 Compile-time pipeline . . . . .                                                                     | 324        |
| 18.4 Under the hood — what the compiler actually does . . . .                                            | 324        |
| 18.5 <code>macro_rules!</code> . . . . .                                                                 | 325        |
| 18.5.1 Fragment specifiers . . . . .                                                                     | 325        |
| 18.5.2 Repetition . . . . .                                                                              | 326        |
| 18.5.3 Level 2 — automation: register map . . . . .                                                      | 326        |
| 18.5.4 Hygiene and <code>\$crate</code> . . . . .                                                        | 327        |
| 18.6 Syntax forbidden or restricted in macros — and why . . . .                                          | 328        |
| 18.6.1 Follow-set: keywords after <code>expr / stmt</code> . . . . .                                     | 328        |
| 18.6.2 Expression position vs statement expansion ( <code>let</code> ,<br>for, multiple lines) . . . . . | 330        |
| 18.7 Standard library macros you already use . . . . .                                                   | 332        |
| 18.8 Derive attributes . . . . .                                                                         | 332        |
| 18.8.1 Not annotations or decorators . . . . .                                                           | 333        |
| 18.8.2 Mechanics . . . . .                                                                               | 334        |
| 18.8.3 Std derives, ecosystem derives, and custom derives                                                | 335        |
| 18.9 Clone and Copy — derive in practice . . . . .                                                       | 336        |
| 18.10 Derive ecosystem — existing solutions . . . . .                                                    | 337        |
| 18.11 Proc macros — three kinds . . . . .                                                                | 341        |
| 18.12 <code>const</code> and compile-time evaluation . . . . .                                           | 342        |
| 18.13 When custom metaprogramming pays off . . . . .                                                     | 343        |
| 18.14 Restrictions — what macros cannot do . . . . .                                                     | 343        |
| 18.15 Why macro code is hard to trace . . . . .                                                          | 344        |
| 18.16 Edge cases and compiler traps . . . . .                                                            | 344        |
| 18.16.1 Follow-set: <code>for</code> , <code>let</code> , and expression position . . . .                | 344        |
| 18.16.2 <code>macro_rules!</code> (other traps) . . . . .                                                | 349        |
| 18.16.3 Derive / attributes . . . . .                                                                    | 349        |
| 18.16.4 Clone / build-time / automation . . . . .                                                        | 350        |
| 18.17 Cons, warnings, and team hygiene . . . . .                                                         | 351        |
| 18.18 Idiom spotlight . . . . .                                                                          | 351        |
| 18.19 Go deeper . . . . .                                                                                | 351        |
| 18.20 See also . . . . .                                                                                 | 352        |
| 18.20.1 Afterparty . . . . .                                                                             | 352        |
| <b>19 Chapter 18: Unsafe and When to Stop</b>                                                            | <b>356</b> |
| 19.1 Hook . . . . .                                                                                      | 356        |
| 19.2 Scope — a brief tour . . . . .                                                                      | 356        |
| 19.3 Why <code>unsafe</code> exists — the aim . . . . .                                                  | 357        |
| 19.4 What <code>unsafe</code> allows . . . . .                                                           | 357        |
| 19.4.1 Soundness vs safety . . . . .                                                                     | 358        |
| 19.5 Examples: elementary <code>-&gt;</code> hard . . . . .                                              | 359        |

|         |                                                                              |     |
|---------|------------------------------------------------------------------------------|-----|
| 19.5.1  | Level 1 — Elementary: deref a stack pointer . . . .                          | 359 |
| 19.5.2  | Level 2 — Elementary: viewing bytes + dangling trap                          | 359 |
| 19.5.3  | Level 3 — Intermediate: <code>unsafe fn</code> in a small module             | 361 |
| 19.5.4  | Level 4 — Hard: <code>unsafe impl Send</code> for an opaque handle . . . . . | 362 |
| 19.5.5  | Level 5 — Hard: FFI sketch (Cargo only) . . . . .                            | 363 |
| 19.6    | Practical cases in services and embedded . . . . .                           | 364 |
| 19.7    | <code>unsafe fn</code> and <code>unsafe trait</code> . . . . .               | 365 |
| 19.8    | FFI — checklist and pitfalls . . . . .                                       | 365 |
| 19.9    | Miri — when to run it . . . . .                                              | 366 |
| 19.10   | When <b>not</b> to use <code>unsafe</code> . . . . .                         | 366 |
| 19.11   | Edge cases and compiler traps . . . . .                                      | 366 |
| 19.12   | Idiom spotlight . . . . .                                                    | 367 |
| 19.13   | Go deeper . . . . .                                                          | 367 |
| 19.14   | See also . . . . .                                                           | 367 |
| 19.14.1 | Afterparty . . . . .                                                         | 368 |

## IV Standard Library I/O 371

|           |                                                            |            |
|-----------|------------------------------------------------------------|------------|
| <b>20</b> | <b>Chapter 19: I/O, Processes, and Bits</b>                | <b>372</b> |
| 20.1      | Hook . . . . .                                             | 372        |
| 20.2      | Scope — a brief tour . . . . .                             | 372        |
| 20.3      | Why Read and Write — the aim . . . . .                     | 373        |
| 20.4      | Read and Write essentials . . . . .                        | 373        |
| 20.5      | Examples: elementary -> hard . . . . .                     | 373        |
| 20.5.1    | Level 1 — Elementary: generic echo . . . . .               | 374        |
| 20.5.2    | Level 2 — Elementary: line-oriented config . . . . .       | 374        |
| 20.5.3    | Level 3 — Intermediate: frame struct + CRC . . . . .       | 375        |
| 20.5.4    | Level 4 — Intermediate: subprocess output . . . . .        | 377        |
| 20.5.5    | Level 5 — Hard: pipeline sketch (Cargo only) . . . . .     | 377        |
| 20.5.6    | Level 6 — Hard: sync file transform (Cargo only) . . . . . | 378        |
| 20.6      | File I/O patterns . . . . .                                | 379        |
| 20.7      | Bitwise and binary protocols . . . . .                     | 379        |
| 20.8      | Sync vs async I/O . . . . .                                | 379        |
| 20.9      | Practical I/O scenarios . . . . .                          | 380        |
| 20.10     | CLI utility — paths, env, and time . . . . .               | 381        |
| 20.10.1   | Path and env . . . . .                                     | 381        |
| 20.10.2   | File metadata before read . . . . .                        | 382        |
| 20.10.3   | Stdin prompt alongside args . . . . .                      | 382        |
| 20.10.4   | Timeout with <code>Duration</code> . . . . .               | 383        |
| 20.10.5   | CLI I/O edge cases . . . . .                               | 383        |
| 20.11     | PTY (note) . . . . .                                       | 383        |
| 20.12     | Edge cases and compiler traps . . . . .                    | 384        |
| 20.13     | Idiom spotlight . . . . .                                  | 384        |
| 20.14     | Go deeper . . . . .                                        | 384        |

|                          |     |
|--------------------------|-----|
| 20.15 See also . . . . . | 384 |
|--------------------------|-----|

## **V Production Standards 388**

### **21 Chapter 20: Production Rust Standards 389**

|                                                                                     |     |
|-------------------------------------------------------------------------------------|-----|
| 21.1 Hook . . . . .                                                                 | 389 |
| 21.2 Scope — a brief tour . . . . .                                                 | 389 |
| 21.3 How to use the checklist . . . . .                                             | 390 |
| 21.4 Types — newtypes and enums . . . . .                                           | 390 |
| 21.4.1 Weak — primitives hide two different mistakes . . . . .                      | 390 |
| 21.4.2 Strong — newtypes for identity, enums for state . . . . .                    | 391 |
| 21.5 Lifetimes — correct and minimal . . . . .                                      | 393 |
| 21.6 Option and Result — not null-like sentinels . . . . .                          | 394 |
| 21.7 Avoid unwrap and expect in production . . . . .                                | 395 |
| 21.8 Panic audit — indexing, panic!, and production paths . . . . .                 | 395 |
| 21.9 Propagate with ? consistently . . . . .                                        | 397 |
| 21.10 Custom errors — thiserror in libraries, anyhow in small<br>binaries . . . . . | 397 |
| 21.11 Errors store data, not messages . . . . .                                     | 399 |
| 21.12 Focused error enums per layer . . . . .                                       | 400 |
| 21.13 Cloning — appropriate, not a band-aid . . . . .                               | 401 |
| 21.14 Avoid unnecessary .clone() and .to_owned() . . . . .                          | 402 |
| 21.15 Rc::clone / Arc::clone — not .clone() on the smart<br>pointer . . . . .       | 403 |
| 21.16 Borrow instead of move when you only read . . . . .                           | 404 |
| 21.17 Heap allocations — only when needed . . . . .                                 | 404 |
| 21.18 Vec::with_capacity when size is bounded . . . . .                             | 405 |
| 21.19 Workspace dependencies — { workspace = true } . . . . .                       | 406 |
| 21.20 Tests — equality, not field-by-field drift . . . . .                          | 407 |
| 21.21 Tests — avoid time and global state pitfalls . . . . .                        | 408 |
| 21.22 Quick reference — full checklist . . . . .                                    | 410 |
| 21.23 Idiom spotlight . . . . .                                                     | 411 |
| 21.24 Go deeper . . . . .                                                           | 411 |
| 21.25 See also . . . . .                                                            | 411 |
| 21.25.1 Afterparty . . . . .                                                        | 411 |

### **A Appendices 413**

### **B Playground Guide 414**

|                                              |     |
|----------------------------------------------|-----|
| B.1 When to use the playground . . . . .     | 414 |
| B.2 Rules (matches STYLE_GUIDE.md) . . . . . | 414 |
| B.3 Sharing code . . . . .                   | 415 |
| B.4 Edition and channel . . . . .            | 415 |
| B.5 Playground stand-ins . . . . .           | 415 |
| B.6 Local run for Cargo labs . . . . .       | 415 |

|          |                                                               |            |
|----------|---------------------------------------------------------------|------------|
| B.7      | Troubleshooting . . . . .                                     | 415        |
| B.8      | See also . . . . .                                            | 416        |
| <b>C</b> | <b>Java / Python / Rust — Mental Model Map</b>                | <b>417</b> |
| C.1      | Execution and memory . . . . .                                | 417        |
| C.2      | Types and polymorphism . . . . .                              | 417        |
| C.3      | Collections and loops . . . . .                               | 418        |
| C.4      | Concurrency . . . . .                                         | 418        |
| C.5      | I/O and systems . . . . .                                     | 419        |
| C.6      | Package management . . . . .                                  | 419        |
| C.7      | Habits to unlearn . . . . .                                   | 419        |
| C.8      | AI Afterparty . . . . .                                       | 419        |
| <b>D</b> | <b>AI Prompt Index</b>                                        | <b>420</b> |
| D.1      | How to use Afterparty . . . . .                               | 420        |
| D.2      | Preface . . . . .                                             | 420        |
| D.3      | Chapter 1 — Paradigm shift . . . . .                          | 421        |
| D.4      | Chapter 2 — Types and expressions . . . . .                   | 423        |
| D.5      | Chapter 3 — Functions and methods . . . . .                   | 423        |
| D.6      | Chapter 4 — Iterators . . . . .                               | 425        |
| D.7      | Chapter 5 — Lifetimes . . . . .                               | 427        |
| D.8      | Chapter 6 — Enums and pattern matching . . . . .              | 428        |
| D.9      | Chapter 7 — Structs, traits, generics . . . . .               | 429        |
| D.10     | Chapter 8 — Errors and testing . . . . .                      | 430        |
| D.11     | Chapter 9 — Modules, paths, and crates . . . . .              | 430        |
| D.12     | Chapter 10 — Smart pointers and interior mutability . . . . . | 432        |
| D.13     | Chapter 11 — Collections . . . . .                            | 434        |
| D.14     | Chapter 12 — Closures and the Fn traits . . . . .             | 435        |
| D.15     | Chapter 13 — Standard traits and conversions . . . . .        | 437        |
| D.16     | Chapter 14 — Multithreading . . . . .                         | 438        |
| D.17     | Chapter 15 — Atomics . . . . .                                | 439        |
| D.18     | Chapter 16 — Async and Tokio . . . . .                        | 439        |
| D.19     | Chapter 17 — Metaprogramming . . . . .                        | 440        |
| D.20     | Chapter 18 — Unsafe . . . . .                                 | 443        |
| D.21     | Chapter 19 — I/O and processes . . . . .                      | 445        |
| D.22     | Chapter 20 — Production standards . . . . .                   | 448        |
| D.23     | By theme . . . . .                                            | 448        |
| D.24     | See also . . . . .                                            | 449        |

# Chapter 1

## Preface

### 1.1 Who these notes are for

You already program in at least one language. You do not need a tutorial that starts with for loops. You need a **paradigm map**: how Rust thinks about memory, types, concurrency, and errors — and how to write **idiomatic** Rust instead of transliterating syntax from whatever you know today.

Backend developers, CLI authors, and curious systems programmers are all welcome. **Rust Core** is the language-fundamentals track inside **Vibe Learning**. Part V applies ideas to std I/O; Parts I–IV stand on their own.

Where tables compare **Java** and **Python**, treat them as **optional anchors** — handy if you know those languages, not required to follow the notes.

### 1.2 Why Rust is a different bet

Most languages make you pick a poison you already know: stay close to the metal and spend years chasing memory bugs, or buy safety from a **GC** or a **VM** and pay latency, pauses, and opaque runtime behavior. Rust breaks that menu. **Ownership** and **borrow checking** reject huge classes of use-after-free, data race, and aliasing mistakes at **compile time** — with no garbage collector, no mandatory runtime, and no surrender of layout, syscalls, or `no_std` control. That is not a faster C or a stricter Java. It is a



different contract between you, the compiler, and the machine.

The contract scales across surfaces that used to demand different stacks. The same language can run on an **ARM64** cloud instance, serve HTTP with **Axum**, embed as a **WebAssembly** module in the browser or beside **Node** via **N-API** / **FFI**, ship a desktop shell with **Tauri**, and still compile firmware for **embedded** targets or a daemon that talks to hardware. You are not maintaining four mental models for four deployment shapes — you are reusing types, tests, and habits.

At runtime Rust stays **lean**: release builds are native **machine code**, not bytecode behind another interpreter. At compile time the language is **expressive**: **algebraic enums**, **traits** instead of inheritance tax, **generics** and **iterators** that optimize like hand-written loops, **macros** when the type system needs a lever. Small binary and rich syntax are not opposites here — zero-cost abstractions and monomorphization are the point.

The industry vote is already in the tree, not in slide decks. **Firefox** shipped memory-safe components from the Servo line; **Windows** and **Android** move security-critical paths to Rust; Chromium hardens parsers at the Rust boundary. In the **Linux kernel**, Rust entered in **6.1** (2022); at the **2025 Kernel Maintainers Summit** it graduated from *experimental* to a **core kernel language** alongside C and assembly — with production drivers and subsystems on the path maintainers expect to keep. You are not learning a hype cycle. You are learning the language those merges already assume.

That is why these notes exist: not to sell Rust in the abstract, but to hand you the paradigm map before the compiler enforces it.

## 1.3 Production standards — what compile time does not teach

Most Rust material stops when the program builds. Production does not. Reviewers ask whether your error types belong in a public API, whether that `.clone()` hides a borrow smell, whether two `u8` parameters can be swapped without the compiler noticing, and whether a workspace crate still shares dependencies correctly after the last refactor. Those questions rarely appear in language tutorials. They surface in merge requests, on-call threads, and code you maintain for years under enterprise SLAs.

**Chapter 20: Production Rust standards** closes that gap. It is a review

checklist — typed boundaries, panic-free paths, deliberate allocation, workspace and test conventions — distilled from my own work shipping Rust in **production and enterprise** teams. The patterns there are not generic best-practice quotes recycled from docs. They are habits I apply after every diff: anti-patterns that compiled cleanly but failed in staging, conventions that kept multi-crate repos reviewable, and the bar I use before calling code merge-ready.

Parts I–IV give you the paradigm map. Chapter 20 gives you the production bar. Read it when you are ready to write Rust that survives code review, not just cargo check.

## 1.4 What you will not find here

- A line-by-line clone of *The Rust Book* (excellent; use it as reference — current stable edition is **2024**, Rust 1.90+)
- Framework churn (Axum vs Actix debates)
- Long proofs about memory models
- Long prosaic-style text (density over narrative padding)

## 1.5 What these notes are (and are not)

These are **lecture notes**, not a textbook. Treat each chapter like dense slides from a course: a map of ideas, runnable examples, and pointers for self-study — not exhaustive reference pages you read cover to cover once and shelve.

### These notes are:

- paradigm maps for programmers coming from any mainstream language
- optional **Java** / **Python** comparison tables where they clarify a habit
- **Playground** snippets you can run, break, and fix
- edge-case drills through compiler errors — Rust is about restrictions; fluency is understanding the compiler’s logic, not just syntax
- hooks for drill and review (Afterparty prompts, crosslinks, optional deep dives)

### These notes are not:

- a hand-holding tutorial that explains every keyword in order

- a substitute for the official book or compiler documentation

AI drafts code; you still read, review, and steer. Success depends on three skills:

- you **understand** what the machine is doing
- you **feel** when a design is wrong
- your **technological thinking** is sharp enough to steer the result

These notes are written for that workflow.

## 1.6 Try it in the Rust Playground

Most examples are tagged **Playground** — paste them into the online editor and click **Run**:

<https://play.rust-lang.org/>

Tweak values, remove a semicolon, borrow after a move. The compiler errors are part of the lesson. You do not need a local install to start Part I.

| Use the playground              | Use Cargo locally                    |
|---------------------------------|--------------------------------------|
| Syntax, ownership, types        | Files, Command, serial ports         |
| Quick experiments (< 50 lines)  | External crates (tokio, serialport)  |
| Sharing a link in chat or notes | Integration tests, multi-file crates |

Rules for playground snippets: one file with `fn main()`, **std only**, no filesystem or subprocess APIs. Full details: Playground Guide.

## 1.7 How to work through a chapter

1. **Read once for the map** — skim headings and tables; do not memorize every line.
2. **Run the Playground example** — confirm output, then break it on purpose.
3. **Prompt what is still fuzzy** — open a **new chat for this chapter**, paste the [starter context](#), then a micro-prompt or Afterparty drill.
4. **Move on** — revisit later via AI Prompt Index IDs (**P001** onward).

## 1.8 Afterparty: aim, importance, and how to use

At the end of every chapter you will find **Afterparty** — numbered, copy-paste prompts for an AI tutor.

These notes give you a **map**, not an encyclopedia. No book — not even a full reference — can hold everything Rust can do or everything you might need in production. The space of possible knowledge is larger than any single track. Memorizing it all is neither required nor realistic.

**Afterparty** is how you choose your depth. Each prompt is a Lego block for **practice**: one angle on the chapter — a quiz, an error to decode, a comparison, a mini-design. Paste it, push back, verify in the playground, stop when it clicks — or keep going until it sticks. **You** set the pace; the book does not pretend to exhaust the topic.

Compare that to **Skills Lego blocks** — capability you assemble over time outside any one chapter: reading a crate, wrapping FFI safely, designing an error enum for a gateway, tuning Tokio under load. Those skills are **optional extensions**. They stack from prompts, side projects, and real code — not from reading cover to cover. Afterparty starts the stack; Skills blocks are what you add when a job or curiosity demands more.

This track ships **prompts**, not a fixed skill tree. That is intentional: the capacity available to you (AI tutor, docs, crates, production code) is greater than what fits in these pages. Prompts are the interface. Depth is your decision.

### 1.8.1 Aim

Afterparty drills the same concepts from different angles: quizzes, error decoding, comparisons, mini-design exercises. Each prompt is a complete sentence you paste into a chat — no setup required.

### 1.8.2 Importance

Afterparty is not a shortcut around learning Rust. It builds the skill that matters in an AI-assisted workflow:

- **interrogate** generated answers instead of accepting them
- **catch** ownership and type mistakes before they ship
- **stress-test** mental models that prompts alone cannot replace

Use the model as a sparring partner. You stay responsible for correctness and idiomatic style.

### 1.8.3 How to use

**One chapter, one chat.** Start a fresh conversation for each chapter. Keep Afterparty prompts in that thread; open a new chat when you move on — old ownership context will confuse async topics.

**Session flow** (after [reading and running examples](#)):

1. Paste the **starter context** below once.
2. Paste **one** Afterparty prompt; verify any code before the next.
3. Push back on vague answers; ask for exact compiler errors.
4. Repeat step 2 in the same chat until done.

**Starter context — paste once at the top of each chapter chat:**

```
You are my Rust tutor. I am working through
Rust Core.
I already program in: [your language(s) ---
    e.g. Python, C++, Go].
```

Answer rules:

- Rust edition 2024, stable toolchain unless I ask otherwise
- Prefer single-file Playground snippets (fn main, std only) unless I say Cargo
- Quote compiler errors literally; explain ownership/borrow moves step by step
- Do not invent crates or APIs; if unsure, say so
- I will verify your code --- keep snippets short and runnable

```
Current chapter: [number and title --- e.g.
    Chapter 1 Paradigm shift]
```

```
Topics I just read: [one line from the chapter
    hook, or paste a section heading]
```

**When to start a new chat instead of continuing:**

| Situation                                                             | Action                                            |
|-----------------------------------------------------------------------|---------------------------------------------------|
| Finished the chapter's Afterparty                                     | New chat for the next chapter                     |
| Model repeats wrong advice after you corrected it twice               | New chat + shorter starter context                |
| You jumped to a unrelated topic (e.g. Tokio while still on lifetimes) | New chat scoped to one chapter                    |
| Context is huge and answers feel generic                              | New chat; paste only the section you are stuck on |

**What good answers look like:** concrete Rust code, named compiler errors (E0382, E0502), and “this fails because ...” — not hand-wavy “Rust is strict about memory.” **Your job** is to run the snippet and say “that did not compile — here is the error” until the explanation matches reality.

### 1.8.4 Where to find prompts

Each chapter ends with its own block. All prompts are numbered globally in appendix/AI\_PROMPT\_INDEX.md (P001 onward) so you can reuse them later without rereading the chapter.

## 1.9 When something is unclear, prompt it

You do **not** need to wait for Afterparty. Any paragraph, table row, compiler error, or half-understood example is fair game — especially the parts that feel unclear after a first read.

### Micro-prompt recipe:

1. **Quote** the passage, snippet, or error text.
2. **State** what you expected to happen.
3. **Ask** for a concrete answer: the exact compiler message, one sentence with an analogy from a language you know (e.g. Java or Python), or a fixed 5-line snippet.

Example you can paste today:

“In Chapter 2, the notes say to prefer `&str` in parameters and `String` when storing. I am building a config parser. Explain why that split matters — give one function signature for `parse_line` and show what breaks if I use `String` everywhere.”

You remain the **reader and reviewer**. The model drafts explanations; you verify against the playground or `cargo check`. That habit turns lecture notes into understanding faster than rereading the same paragraph five times.

## 1.10 Toolchain: rustup and Cargo

Install Rust with **rustup** (toolchain manager) and build with **Cargo** (package manager, build system, test runner):

```
curl --proto '=https' --tlsv1.2 -sSf
  https://sh.rustup.rs | sh
cargo new my_app --bin
cd my_app && cargo run
```

| Concept         | In Rust                                                           |
|-----------------|-------------------------------------------------------------------|
| Package manager | <b>Cargo</b> — deps, build, test, run                             |
| Manifest        | <code>Cargo.toml</code> — name, version, edition, dependencies    |
| Lockfile        | <code>Cargo.lock</code> — pinned versions for reproducible builds |
| Entry point     | <code>fn main()</code> in <code>src/main.rs</code>                |
| Crate registry  | <a href="https://crates.io">crates.io</a>                         |

Every binary crate follows the same shape: one `Cargo.toml`, one `src/main.rs`, commands through `cargo run` and `cargo test`. Prefer that workflow over invoking `rustc` directly.

`env!("CARGO_PKG_NAME")` embeds a **compile-time** string from `Cargo.toml` into your binary — not the same as reading OS environment variables at runtime (`std::env::var`).

**Cargo only** — first project locally:

```
# Cargo.toml
[package]
name = "hello_rust"
version = "0.1.0"
edition = "2024"
```

```
// src/main.rs
fn main() {
```

```
println!("Hello from {}",
    env!("CARGO_PKG_NAME"));
}
```

**One binary, one manifest.** Commit `Cargo.lock` for applications (CLIs, services); library crates often omit it from version control.

## 1.11 Conventions

How the notes are formatted — not Rust syntax itself.

| Convention                               | Meaning                                                                                                                                                                                                                                                                                                                                  |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Playground / Cargo only</b>           | Every code block is tagged. <b>Playground</b> = one <code>fn main()</code> , <code>std</code> only, <a href="https://play.rust-lang.org">play.rust-lang.org</a> . <b>Cargo only</b> = needs <code>Cargo.toml</code> , filesystem, subprocess, or external crates. Details: <code>STYLE_GUIDE.md</code> , <code>Playground Guide</code> . |
| <b>Chapter shape</b>                     | Hook -> sections with examples -> <b>Idiom spotlight</b> -> <b>Go deeper</b> -> <b>See also</b> -> <b>Afterparty</b> .                                                                                                                                                                                                                   |
| <b>Levels</b>                            | Many chapters label examples <b>Level 1 ... N</b> . Run the snippet, then read <b>What happened</b> before the next level.                                                                                                                                                                                                               |
| <b>Crosslinks</b>                        | Relative paths ( <code>08_errors_and_testing.md</code> ). Chapter numbers match <code>CONTENTS.md</code> .                                                                                                                                                                                                                               |
| <b>Java / Python columns</b>             | Optional in tables — skip if you do not need the analogy.                                                                                                                                                                                                                                                                                |
| <b><code>unwrap()</code> in examples</b> | Avoided in teaching snippets unless the section is about <code>panic</code> or <code>Result</code> (Chapter 8).                                                                                                                                                                                                                          |
| <b>Prompt IDs</b>                        | Afterparty lines are catalogued as <b>P001+</b> in <code>AI Prompt Index</code> for reuse across chats.                                                                                                                                                                                                                                  |
| <b>Edition</b>                           | Rust <b>2021</b> , stable toolchain, unless a note says otherwise.                                                                                                                                                                                                                                                                       |

## 1.12 Reading ahead

Some syntax appears in examples before its dedicated chapter:

- **?** — “If this fails, return the error to the caller.” Explained fully in Chapter 8: Errors and testing. Until then, read it as a placeholder, not



something you must master on day one.

## 1.13 See also

- CONTENTS.md — full map
- Playground Guide — rules and stand-ins
- AI Prompt Index — all Afterparty prompts (**P001+**)
- Java/Python/Rust map — optional one-page cheat sheet
- Chapter 1: Paradigm shift — start here after this page

# Chapter 2

## Chapter 1: Paradigm Shift

### 2.1 Hook

Many mainstream languages let you **share** data freely: references, object graphs, mutable globals — **Java** and **Python** are familiar examples. Rust says: **one owner at a time**, checked at compile time.

That rule replaces a GC and prevents most data races. Learn to read compiler errors as hints, not obstacles.

### 2.2 Compiled vs interpreted

| Aspect      | Java                | Python                   | Rust                           |
|-------------|---------------------|--------------------------|--------------------------------|
| Execution   | Bytecode on JVM     | Interpreter / bytecode   | Native machine code            |
| Type checks | Mostly compile-time | Runtime                  | Compile-time                   |
| Memory      | GC                  | GC (refcount + cycle GC) | Ownership + deterministic drop |

Rust has **no GC**. When a value's owner goes out of scope, `drop` runs immediately. No stop-the-world pauses — valuable for long-running loops and low-latency tools.

## 2.3 Ownership vs garbage collection

**Python:** `b = a` makes two names point at one list; mutating through `b` affects `a`.

**Java:** references to the same object; GC reclaims when unreachable.

**Rust:** `let s2 = s1` for a `String` **moves** ownership. `s1` is invalid afterward unless you borrow or clone.

```
// Playground
fn main() {
    let s1 = String::from("hello");
    let s2 = s1; // move
    println!("{}", s2);
    // println!("{}", s1); // would not compile
}
```

Think of a move as **renaming** the value, not copying.

### 2.3.1 Move edge cases (what the compiler catches)

| Situation                  | Java / Python                    | Rust                                                                               |
|----------------------------|----------------------------------|------------------------------------------------------------------------------------|
| Reassign name to new value | old object GC'd when unreachable | old owner <b>dropped</b> when <code>let s1 = ...</code> overwrites <code>s1</code> |
| Pass to function by value  | reference still in caller        | <b>move</b> — caller binding dead unless returned                                  |
| Use after move             | often still works                | <b>compile error</b>                                                               |

```
// Playground --- uncomment one failing line at
a time
fn main() {
    let s1 = String::from("plc_a");
    let s2 = s1; // move: s1 -> s2
    // println!("{}", s1);
    // ERROR: use of moved value: `s1`

    let mut log = String::from("start");
    log = String::from("replaced");
    // drop old "start" buffer, bind new owner
}
```

```
println!("{}", log);  
}
```

**Wrong — use heap value after moving into a function:**

```
// Playground --- does not compile  
fn consume(s: String) {  
    println!("{}", s);  
}  
  
fn main() {  
    let label = String::from("plc1");  
    consume(label);  
    println!("{}", label);  
    // ERROR: borrow of moved value: `label`  
}
```

**Idiomatic fixes:** borrow for read (`consume_borrow(&label)`), take and return (`fn transform(s: String) -> String`), or `.clone()` when you truly need two owners (sparingly).

## 2.4 Zero-cost abstractions

Rust’s iterators, traits, and generics are designed to compile down to code as tight as hand-written C **when you use release builds** (`cargo build -release`). You do not pay for “elegant” APIs at runtime the way heavy OOP patterns can cost in Java.

## 2.5 Fearless concurrency (preview)

The same borrow rules that prevent use-after-free also prevent **data races** in safe Rust: you cannot mutate shared state from two threads without synchronization (Mutex, channels, atomics — Part III). The compiler enforces this; you do not rely on discipline alone.

## 2.6 Stack and heap

Ownership makes more sense once you see **where** values live. Rust uses two main memory regions:

- **Stack** — per-function frames, very fast allocate/deallocate (pointer bump). Size known at compile time.
- **Heap** — shared pool for data whose size is unknown or may grow. Slower; requires an explicit allocator and later **free** (in Rust: **drop** when the owner leaves scope).

In many GC-managed languages (Java, Python, C#, ...) you rarely think about stack vs heap: the runtime puts much of the data on the heap and cleans up later. Rust puts **small, fixed-size data on the stack by default** and uses the heap only when the type needs it (`String`, `Vec`, and similar).

| Aspect                          | Java                                 | Python                          | Rust                                           |
|---------------------------------|--------------------------------------|---------------------------------|------------------------------------------------|
| <code>int</code> / small number | often stack (local) or heap (object) | heap ( <code>PyObject</code> )  | stack ( <code>i32</code> , <code>Copy</code> ) |
| growable text                   | heap ( <code>String</code> object)   | heap ( <code>str</code> object) | stack struct <b>pointing</b> at heap buffer    |
| who frees heap?                 | GC                                   | GC                              | owner's <code>drop</code> at end of scope      |
| local variable cost             | reference push                       | name binding                    | often zero-cost stack slot                     |

### 2.6.1 Stack: fast and scoped

Each function call gets a **stack frame**. Locals like `i32`, `bool`, and tuples of `Copy` types sit there. When the function returns, the frame is popped — no separate “free” step.

```
// Playground
fn stack_demo() -> i32 {
    let a: i32 = 10;
    let b: i32 = 32;
    a + b
    // a and b die when this function returns
}

fn main() {
```

```
println!("{}", stack_demo());
}
```

`a` and `b` die when `stack_demo` returns — their stack slots vanish with the frame. You still see **42** because `a + b` is not asking the caller to keep `a` or `b`. It **computes a new value**, and that value becomes the **return value** before the frame is destroyed.

Order matters: Rust evaluates `a + b`, places **42** in the return slot (for `i32`, a bitwise **copy**), then pops the frame and drops `a` and `b`. `main` receives the copied **42** and passes it to `println!`. The locals are gone; the **result** escaped via `return`.

“Die” means the **bindings** and their stack slots are destroyed — not that every value computed inside the function disappears. Only what you **return** (or otherwise move out) survives past the closing brace. With heap-backed types like `String`, the same rule applies: the owner binding dies, but the data can live on if ownership **moves** into the caller’s binding.

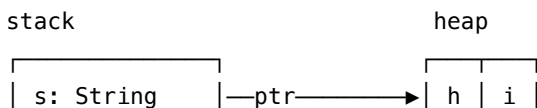
Nested blocks shrink live ranges — useful for **borrowing** below:

```
// Playground
fn main() {
    let outer = 1;
    {
        let inner = 2;
        println!("{}", outer, inner);
    } // inner dropped here
    println!("outer still live: {}", outer);
}
```

## 2.6.2 Heap: growable data with an owner

Types like `String` and `Vec` store **metadata on the stack** (pointer, length, capacity) and **payload on the heap**. The stack part is small and fixed; the heap part can grow.

Mental picture for `let s = String::from("hi");`





When `s` goes out of scope, Rust runs `drop` on `String`, which frees the heap buffer. No GC scan — cost is **predictable** and tied to scope.

```
// Playground
fn main() {
    let x: i32 = 42;
    // entirely on stack
    let s = String::from("hi");
    // stack handle -> heap bytes
    let r = &s;
    // borrow: no heap copy --- see
    // [References](#references-borrowing-and-de
    // referencing)
    println!("{}", x, r);
} // s dropped here -> heap "hi" freed
```

### 2.6.3 Moves and the heap

Assigning one `String` to another **moves** the stack handle (pointer/len/-cap) to the new owner; the heap buffer is **not** copied. That is why `s1` becomes invalid after `let s2 = s1` — there is only one owner responsible for freeing that heap memory.

```
// Playground
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;
    // move handle; heap buffer stays in place
    println!("{}", s2);
    // println!("{}", s1);
    // error: s1 no longer owns the heap data
}
```

Java would copy a **reference** (two refs, one object). Python would bind another name to the same object. Rust **transfers responsibility** — fewer copies, stricter rules.

2.6.4 Copy on stack vs heap-backed types

2.6.4.1 Why Rust splits the world this way

Rust's default is *move* (transfer ownership). *Copy* is a narrow, opt-in exception — not an arbitrary rule:

*Every value has exactly one owner who is responsible for cleaning up any resources it holds.*

If assignment silently duplicated a `String` or `Vec` handle, you would get **two owners for one heap allocation**. When both go out of scope, Rust would free the same memory twice — a classic double-free. C and C++ leave that to discipline; Rust makes the dangerous case **illegal at compile time**.

*Copy* is allowed only when a bitwise duplicate is **semantically identical** to the original: the type is self-contained, has no `Drop` that must run once, and duplicating it never shares external memory. Integers and floats are just stack bits — cheap and safe to copy. Heap-backed types **own** external memory, so the compiler **moves** unless you explicitly `.clone()`.

Rust treats ownership transfer as the normal case (handing someone a key) and silent duplication as a special case for values that cannot alias. Assignment feels “strict” compared to languages with implicit sharing because Rust prefers **provable correctness** over silent aliasing.

2.6.4.2 Which types are Copy vs move?

**Copy types** (assignment duplicates bits; both variables stay valid):

| Category          | Examples                                                                                                                                                                                                                              |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Integer scalars   | <code>i8</code> , <code>i16</code> , <code>i32</code> , <code>i64</code> , <code>i128</code> , <code>isize</code> , <code>u8</code> , <code>u16</code> , <code>u32</code> , <code>u64</code> , <code>u128</code> , <code>usize</code> |
| Floating point    | <code>f32</code> , <code>f64</code>                                                                                                                                                                                                   |
| Other scalars     | <code>bool</code> , <code>char</code>                                                                                                                                                                                                 |
| Unit type         | <code>()</code>                                                                                                                                                                                                                       |
| Fixed-size arrays | <code>[T; N]</code> when <code>T</code> : <code>Copy</code> (e.g. <code>[u8; 4]</code> )                                                                                                                                              |
| Tuples            | <code>(T1, T2, ...)</code> when every element is <code>Copy</code>                                                                                                                                                                    |
| References        | <code>&amp;T</code> , <code>&amp;mut T</code> (the reference itself is copied; it still <i>points</i> to the same data)                                                                                                               |
| Raw pointers      | <code>*const T</code> , <code>*mut T</code>                                                                                                                                                                                           |
| Function pointers | <code>fn(...) -&gt; ...</code>                                                                                                                                                                                                        |



| Category                                     | Examples                                       |
|----------------------------------------------|------------------------------------------------|
| <code>NonNull</code> , <code>NonZero*</code> | Many niche std types that wrap a plain integer |

**Move types** (assignment transfers ownership; the source is invalid afterward unless you borrow):

| Category                            | Examples                                                                                                                                                                     |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Growable / owned heap data          | <code>String</code> , <code>Vec&lt;T&gt;</code> , <code>Box&lt;T&gt;</code> , <code>HashMap&lt;K, V&gt;</code> , <code>HashSet&lt;T&gt;</code> , <code>BTreeMap</code> , ... |
| Shared / interior mutability        | <code>Rc&lt;T&gt;</code> , <code>Arc&lt;T&gt;</code> , <code>RefCell&lt;T&gt;</code> , <code>Mutex&lt;T&gt;</code> , <code>RwLock&lt;T&gt;</code>                            |
| Most user-defined structs/enums     | Unless you add <code>#[derive(Copy, Clone)]</code> <b>and</b> every field is <code>Copy</code>                                                                               |
| Types with custom <code>Drop</code> | File handles, network sockets, anything that must run cleanup once                                                                                                           |

### Rules of thumb:

1. **Primitives on the stack -> usually Copy.**
2. **Anything that owns heap memory or runs `Drop` -> move by default.**
3. **Copy and Drop are mutually exclusive** — a type cannot implement both.
4. **`.clone()` is the explicit, potentially expensive deep copy** when you truly need two independent heap values.

You can check any type in the playground or docs: `Copy` is a trait; if a type implements it, assignment copies; otherwise it moves.

```
// Playground
fn main() {
    let a = 5;
    let b = a;
    // Copy --- two independent i32 on stack
    println!("{}", a, b);

    let v1 = vec![1, 2, 3];
    let v2 = v1;
    // move --- v1 invalid; one owner for heap
```

```
// array
println!("{:?}", v2);

let s1 = String::from("hi");
let s2 = s1.clone();
// explicit deep copy --- both valid, two
// heap buffers
println!("{}", s1, s2);
}
```

## 2.6.5 References, borrowing, and dereferencing

Moves and `.clone()` are not the only way to use data. A **reference** is a borrow: you get access without becoming the owner. Rust has two safe reference forms:

| Type                    | Access       | Alias rule (preview)                                                           |
|-------------------------|--------------|--------------------------------------------------------------------------------|
| <code>&amp;T</code>     | read-only    | many <code>&amp;T</code> borrows at once                                       |
| <code>&amp;mut T</code> | read + write | <b>one</b> <code>&amp;mut</code> at a time, no overlapping <code>&amp;T</code> |

Use the `&` operator to **create** a reference from an owner:

```
// Playground
fn main() {
    let s = String::from("sensor_a");
    let r = &s;
    // r: &String --- immutable borrow; s still
    // owns the heap buffer

    let mut count = 0;
    let m = &mut count;
    // m: &mut i32 --- exclusive borrow for
    // mutation
    *m += 1; // writes through m into count (1)

    // Works: after *m += 1, the mutable borrow
    // of count is over (last use of m)
}
```

```
println!("{}", r, count); // sensor_a 1
}
```

Walk through the two borrows separately:

- **r and s:** `r` is a read-only handle to `s`. The heap buffer still has one owner (`s`); `r` only lets you read it. Many immutable borrows of the same value can coexist.
- **m and count:** `count` must be `mut` before you can take `&mut count`. The `*m += 1` line **dereferences** `m` and updates the owner's slot in place — `count` becomes 1 without moving it.

The closing `println!` uses `count` directly, not `m`. That compiles because the **mutable borrow ended** when `m` was last used (`*m += 1`). The binding `m` still exists, but until you use `m` again, `count` is free for a read.

Try adding `m` to the same `println!` — it fails:

```
// Playground --- does not compile
fn main() {
    let s = String::from("sensor_a");
    let r = &s;

    let mut count = 0;
    let m = &mut count;
    *m += 1;

    // ERROR E0502: cannot borrow `count` as
    // immutable because it is also borrowed
    // as mutable
    println!("{}", r, m, count);
}
```

`println!` expands to code that must **use every argument** for the duration of the call. Passing `m` (`&mut i32`) **re-opens** the exclusive borrow of `count`. Passing `count` in the same macro asks for an **immutable** read of the owner while that mutable borrow is active. Rust rejects the overlap — you cannot read and exclusively borrow `count` at once.

Fixes (smallest first):

1. **Read through one path** — `println!("{}", r, *m, count)` still

fails for the same reason (count plus active `m`). Use `println!("{}", r, *m)` or `println!("{}", r, count)` — not both `m` and `count`.

2. **End the borrow with a block** — `{ let m = &mut count; *m += 1; }` then `println!("{}", r, count)`.
3. **Drop `m` explicitly** — `drop(m)`; before printing `count` (less idiomatic than a block; Chapter 10 covers `drop` in depth).

**Java / Python:** two variables can reach the same object; neither owns it in the Rust sense. **Rust:** the **owner** (`s`, `count`) stays put; `r` and `m` are temporary handles the compiler checks (Chapter 5).

References are **Copy**: `let r2 = r1` copies the **pointer**, not the heap data. There is still **one owner** of the buffer.

### 2.6.5.1 Dereferencing with `*`

If `r: &i32`, then `*r` is the `i32` **at** that address. The unary `*` operator **dereferences** — it follows the reference to the underlying value.

| Operator           | Reads as                      | Example                         |
|--------------------|-------------------------------|---------------------------------|
| <code>&amp;</code> | “borrow this”                 | <code>&amp;s, &amp;mut n</code> |
| <code>*</code>     | “value behind this reference” | <code>*m, *m += 1</code>        |

Rust **auto-derefs** in many everyday spots: `println!("{}", r)` works with `&i32`, and method calls like `r.abs()` reach through the reference for you. You still write explicit `*` when you need the **inner value** in an expression — especially **assigning or mutating through `&mut T`**:

```
// Playground
fn bump(n: &mut i32) {
    *n += 1;
    // without *, you would try to reassign the
    // reference itself
}

fn main() {
    let x = 10;
    let r = &x;
    let sum = x + *r;
    // explicit: add the i32 behind r
```

```
println!("{}", sum);

let mut ticks = 0;
bump(&mut ticks);
println!("ticks = {}", ticks);
}
```

**Read-only borrow:** `*r` gives you a copy when `T: Copy` (like `i32`). You cannot write `*r = 5` if `r: &i32` — mutating requires `&mut`.

**Wrong — mutate through &:**

```
// Playground --- does not compile
fn main() {
    let x = 10;
    let r = &x;
    // *r = 20;
    // ERROR: cannot assign to `*r`, which is
    // behind a `&` reference
    println!("{}", r);
}
```

**Mutable borrow:** `*mut` = value updates the **owner's** data in place. That is how a function increments a caller's counter or fills a buffer without taking ownership.

```
// Playground
fn main() {
    let mut frame = [0u8; 4];
    let header = &mut frame[0..2];
    header[0] = 0xDE;
    // slice methods auto-deref; index writes
    // through &mut
    header[1] = 0xAD;
    println!("{:02X?}", frame);
}
```

### 2.6.5.2 References vs raw pointers (name only)

The type table above lists **raw pointers** `*const T` and `*mut T`. They also use `*` to dereference, but only inside **unsafe** code — common in embedded

and FFI, not day-one application Rust. Safe code uses `&T` / `&mut T`; the compiler enforces borrow rules instead of trusting you.

Rules of thumb:

- 1. **Need read-only access without moving?** -> `&T`
- 2. **Need in-place mutation of the owner?** -> `&mut T` (one at a time)
- 3. **Callee should take ownership** (store, send, transform-and-return)? -> pass by value — **move**
- 4. **Caller must reuse the same allocation** (loop buffer, counter, in-place edit)? -> `&mut T`

2.6.5.3 Move vs `&mut`: who keeps the value? (rules 3–4)

These look similar in other languages — “pass something to a function and let it change” — but Rust splits them on **ownership**.

| Aspect                | Move                                             | Mutable borrow                                         |
|-----------------------|--------------------------------------------------|--------------------------------------------------------|
|                       | <code>fn f(s: String)</code>                     | <code>fn f(s: &amp;mut String)</code>                  |
| Ownership             | Transfers to the callee                          | Stays with the caller                                  |
| Caller after the call | Original binding is <b>invalid</b>               | Original binding is <b>still valid</b> , often changed |
| Heap buffer           | Same buffer, new owner                           | Same buffer, same owner                                |
| Callee’s job          | Own, transform, drop, or <b>return</b> the value | Temporarily mutate, then <b>give control back</b>      |
| Typical use           | “Take this and finish with it”                   | “Tweak my value in place”                              |

**Move:** the function becomes the owner. If it takes `String` by value, the caller’s `label` is gone after the call — like handing over the key. The callee may mutate and drop it, or return it to transfer ownership.

**`&mut`:** the caller keeps ownership. The function gets a **loan** — exclusive access for the call — to edit the existing data. When the call returns, the borrow ends and the caller reads the updated value.

```
// Playground
fn consume(mut s: String) {
    s.push('!');
    // callee owns s; mutates its own local
    // owner
} // s dropped here unless returned
```

```

fn append_bang(s: &mut String) {
    s.push('!');
    // mutates caller's buffer through the
    // borrow
}

fn main() {
    let label = String::from("plc1");

    // Move path: ownership leaves main
    let moved = label;
    consume(moved);
    // println!("{}", moved);
    // error: moved value

    // Mut borrow path: caller keeps label
    let mut label = String::from("plc1");
    append_bang(&mut label);
    // lend exclusive access for one call
    println!("{}", label);
    // plc1! --- same owner, updated heap data
}

```

Use a move when the callee **should** take ownership — sending a message on a channel, storing in a struct field, or returning a transformed value. After `consume(moved)`, `main` no longer frees that buffer.

Use `&mut` when the caller must **reuse** the same allocation — a `String` label in a loop, a reusable `Vec<u8>` frame buffer, or a tick counter. `&mut` mutates in place with no handoff and no `.clone()`.

For stack Copy types like `i32`, “move” is a bitwise copy — both sides can still use their copy. The distinction matters most for **heap-backed** owners (`String`, `Vec`, ...) where move means the source name dies.

5. **Need to assign or arithmetic on the pointee?** -> often `explicit *`
6. **Need an independent duplicate?** -> `.clone()` or pass by value — not a longer-lived borrow

## 2.6.6 Borrow checker edge cases

Rust's alias rules are strict. These patterns look reasonable if you come from a language with implicit sharing (Java, Python, JavaScript, ...). The compiler rejects them **before** run time.

### 1. Immutable borrow blocks mutation of the owner

```
// Playground --- does not compile
fn main() {
    let mut s = String::from("plc");
    let r = &s;
    // shared read loan starts
    s.push('!');
    // ERROR: cannot borrow `s` as mutable
    // while `r` is live
    println!("{}", r);
}
```

**Fix:** end the read borrow before mutating — nested block, or `println!` first so `r` is not used after `push`:

```
// Playground
fn main() {
    let mut s = String::from("plc");
    {
        let r = &s;
        println!("{}", r);
    } // r ends here
    s.push('!');
    println!("{}", s);
}
```

### 2. Only one `&mut` at a time

```
// Playground --- does not compile
fn main() {
    let mut v = vec![1, 2, 3];
    let a = &mut v;
    let b = &mut v;
    // ERROR: cannot borrow `v` as mutable more
    // than once
}
```



```

    a.push(4);
    println!("{:?}", b);
}

```

### 3. & and &mut cannot overlap

```

// Playground --- does not compile
fn main() {
    let mut n = 0;
    let r = &n;
    let m = &mut n;
    // ERROR: cannot borrow as mutable while
    // immutable borrow exists
    println!("{}", r, m);
}

```

### 4. Cannot move while borrowed

```

// Playground --- does not compile
fn main() {
    let s = String::from("plc");
    let r = &s;
    let moved = s;
    // ERROR: cannot move `s` because it is
    // borrowed
    println!("{}", r);
}

```

References are Copy, but they **point at** an owner. Moving the owner while a borrow is active would leave a dangling & — same class of bug as use-after-free, ruled out at compile time.

### 5. Returning a reference to a local (preview)

```

// Playground --- does not compile
fn broken() -> &str {
    let s = String::from("tmp");
    &s
    // ERROR: `s` does not live long enough ---
    // returned ref would dangle
}

```

The owner dies at the end of `broken`; the caller cannot hold `&str` afterward. Return owned `String`, or borrow from the caller's data (Chapter 5).

## 6. `&mut` to one slice element vs whole `Vec`

```
// Playground --- does not compile
fn main() {
    let mut frame = vec![0u8; 4];
    let a = &mut frame[0];
    let b = &mut frame[1];
    // usually ok --- different elements
    *a = 1;
    *b = 2;

    let whole = &mut frame;
    // ERROR if `a` still live: borrow of
    // entire vec
    // whole.push(5);
    println!("{:?}", frame);
}
```

Borrowing **overlapping** parts of the same collection is the subtle case. Prefer one `&mut` to the whole `Vec`, or index without holding two element borrows across a mutating call.

### 2.6.7 When the compiler says no

Common errors in this chapter:

| Error (typical wording)                                          | You probably did                                                  | Smallest fix                                                |
|------------------------------------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------|
| use of moved value                                               | <code>let s2 = s1</code> , or passed <code>String</code> by value | borrow <code>&amp;s1</code> , or clone, or use return value |
| borrow of moved value                                            | used binding after <code>consume(s)</code>                        | <code>consume(&amp;s)</code> or receive owned return        |
| cannot borrow as <b>mutable</b> more than once                   | two <code>&amp;mut</code> to same value                           | one <code>mut</code> borrow; restructure loop               |
| cannot borrow as mutable while <b>immutable</b> borrow is active | <code>let r = &amp;s</code> then <code>s.push</code>              | shrink <code>r</code> 's scope with <code>{ }</code>        |
| cannot move out of ... because it is borrowed                    | move owner while <code>&amp;owner</code> exists                   | drop borrows first, then move                               |

| Error (typical wording)                             | You probably did                             | Smallest fix                                     |
|-----------------------------------------------------|----------------------------------------------|--------------------------------------------------|
| cannot assign to <code>*r</code> behind & reference | <code>*r = 5</code> on <code>&amp;i32</code> | use <code>&amp;mut</code>                        |
| does not live long enough                           | <code>return &amp; to local</code>           | <code>return String</code> or borrow from caller |

Read errors **top to bottom** — the first note is usually the root cause; later notes are often follow-on noise.

### 2.6.8 Why this matters for long-running programs

Control loops and serial parsers often run for hours. **Stack allocation is essentially free.** Heap allocation is fine when you need it, but freeing at a known scope avoids GC pauses and makes worst-case latency easier to reason about — the same theme as [ownership vs GC](#) above.

## 2.7 Idiom spotlight

**Let the type system carry intent.** Prefer `Result` and `Option` over sentinel values (`null`, `-1`, magic strings). Formalized in Chapter 6 and Chapter 8.

## 2.8 Go deeper

- [The Rust Book — Understanding Ownership](#)
- [Functional Rust — Option basics](#)

## 2.9 See also

- Chapter 4: Iterators — `iter` vs `into_iter` and borrow errors in chains
- Chapter 5: Lifetimes — when references must be named in types
- Chapter 14: Multithreading

## 2.9.1 Afterparty

### 2.9.1.1 Ownership, moves, and drop

1. **Move drill** — “Give 6 tiny Rust snippets mixing `String`, `Vec`, and `i32`. I predict ok or compile error; you reveal answers, quote the error message, and give the smallest fix.”
2. **Use-after-move decoder** — “Show one `println!` after a move that fails to compile. I explain the error in plain English; you refine my explanation and show the three fixes: `borrow`, `.clone()`, or restructure scope.”
3. **Return transfers ownership** — “Write a function `fn take(s: String) -> String` and a `main` that calls it. I trace who owns the heap buffer at each line, including after the return.”
4. **Move into a call** — “Snippet: `process(build_label())` where `build_label() -> String`. Explain when the heap allocation happens, when ownership moves into `process`, and when `drop` runs if `process` takes `String` by value.”
5. **Scope and drop** — “Give a nested-block snippet with two `Strings` and one `Vec`. I mark the exact line where each heap buffer is freed vs where stack slots disappear; you correct and explain `drop` order.”
6. **Single-owner principle** — “State Rust’s rule ‘every value has exactly one owner’ in one sentence, then give one `String` example where breaking that rule would double-free. Use the key-handoff metaphor.”

### 2.9.1.2 Stack, heap, and memory layout

7. **Stack vs heap quiz** — “For 10 declarations (`i32`, `bool`, `[u8; 4]`, `String`, `Vec<i32>`, `&str`, `&String`, `(i32, f64)`, `Box<i32>`, `()`), I say stack-only, heap involved, or both; you draw the pointer picture for any I miss.”
8. **String layout sketch** — “For `let s = String::from("hi")`, I describe stack fields (`ptr`, `len`, `cap`) and heap bytes; you correct and extend to `let v = vec![1, 2, 3]`.”
9. **Stack frame drill** — “I give a 3-function call chain with `i32` locals and one `String` passed by value. Trace stack frame push/pop, when each slot dies, and when the `String` heap buffer is dropped.”
10. **Nested block live ranges** — “Snippet with `{ let inner = ... }` inside `main`. I list which bindings are alive on each line; you explain why shorter live ranges matter for borrowing later.”
11. **Borrow without heap copy** — “Show `let s = String::from("x"); let r = &s;` — I explain what is on stack vs heap and why `r` does

not duplicate the buffer; you add one line that would fail because of move/borrow conflict.”

### 2.9.1.3 References and dereferencing (& and \*)

12. **& vs &mut pick** — “Five tasks (log a label, increment a tick counter, parse into a temp struct, share read-only config, swap two buffers in place). I choose pass-by-value, &T, or &mut T; you explain owner count and alias rules.”
13. **Create the borrow** — “Given `let mut n = 10;` and `let s = String::from(\"plc\");`, I write the types and expressions for one & and one &mut borrow; you verify the owner is still usable afterward.”
14. **When is \* required?** — “Four expressions mixing &i32, &mut i32, and `println!`. I mark where explicit \*r is needed vs where auto-deref handles it; you correct with compiled examples.”
15. **Mutate through \*** — “Fill in `fn reset(n: &mut u32) { ... }` and a main that calls it. I must use \* to zero the caller’s value; you show a broken version that tries to reassign n instead.”
16. **Reference copy vs move** — “After `let r1 = &s;` `let r2 = r1;` vs `let s2 = s1;` I compare heap owner count, which bindings stay valid, and whether heap data was copied.”
17. **Type of \*r** — “Bindings `r: &i32`, `m: &mut i32`, `b: Box<i32>`. I name the type of \*r, \*m, and whether \*m = 5 mutates the owner; you extend with one &T where \*r = 5 fails.”
18. **Borrow blocks mutation** — “Snippet: `immutable let r = &s;` then `s.push('!')`. I explain the compile error; you fix by shrinking r’s scope with a nested block.”
19. **Automation counter** — “Sketch a 1 kHz loop with `ticks: u64` and a helper `fn maybe_rollover(t: &mut u64)`. I write the call site and one \*t mutation inside the helper; you review without full thread code.”

### 2.9.1.4 Move vs mutable borrow

20. **Move or &mut quiz** — “Six tasks (append to a reusable log line, send a message on a channel, fill a frame `Vec<u8>` in a loop, store a device name in a struct, transform-and-return a label, increment a counter). I pick `fn take(T)` move vs `fn tweak(&mut T)`; you explain who owns the heap data after the call.”
21. **After the call** — “Two functions: `consume(String)` and `append_bang(&mut String)`. I trace which bindings in main are

- valid **after** each call and whether the heap buffer was dropped, reused, or returned.”
22. **Fix the signature** — “Show code that moves a `String` into a helper but then tries to `println!` it in `main`. I explain the error and choose the smallest fix: change to `&mut String`, restructure with a return value, or `.clone()` — you rank the idiomatic options.”
  23. **Loop reuse** — “A serial parser reuses `let mut buf = Vec::with_capacity(256)` every tick. I explain why `parse_frame(buf)` (move) is wrong and write `parse_frame(&mut buf)` instead; you add one line showing the caller reading updated length after `parse`.”
  24. **Transform-and-return** — “Task: uppercase a `String` and give it back. I write `fn upper(s: String) -> String` and call site; you contrast with an anti-pattern that takes `&mut String` when ownership should transfer.”
  25. **i32 vs String move** — “Same pattern with `fn add_one(n: i32)` and `fn add_bang(s: String)`: I explain why the caller can still use `n` after the call but not `s`; you map each to Copy vs heap move.”
  26. **Call it twice** — “Snippet that needs to pass the same `String` to two helpers in one scope. I show why two by-value calls fail, then fix with `&str/&mut` borrows or one move plus `.clone()`; you flag the smell.”

#### 2.9.1.5 Copy, move, and `.clone()`

27. **Copy eligibility quiz** — “Quiz me on 12 types (`i32`, `String`, `&str`, `Vec<i32>`, `[u8; 8]`, `(i32, String)`, `Box<i32>`, `fn()`, `bool`, `char`, `Rc<i32>`, struct with only `i32` fields). Copy, move, or ‘Copy only if derived’? Cite the rule each time.”
28. **Copy vs Drop paradox** — “Why can’t a type be both Copy and Drop? Use a `File` or `socket handle`: show what goes wrong if assignment bitwise-copies the handle.”
29. **Semantic copy test** — “For `i32`, `&T`, and `String`: if I duplicate the stack bits, is the result always semantically identical? When does it fail, and what does Rust do instead?”
30. **Double-free thought experiment** — “Hypothetical: `String` were Copy. Walk line-by-line through `let a = ...; let b = a; }` end of block — show both drops freeing the same address. Contrast with real move semantics.”
31. **When to derive Copy** — “I put `#[derive(Copy, Clone)]` on a struct with a `String` field — explain the error. Then give three struct shapes:

safe to derive `Copy`, must stay move-only, and where `.clone()` belongs in the public API.”

32. **Reference is Copy** — “Four snippets: `let r2 = r1` for `&String` vs `let s2 = s1` for `String`, plus one with `&mut`. I predict ok/error; you count owners of the heap buffer after each line.”
33. **`.clone()` judgment** — “Five scenarios (store config `String`, pass into `fn` twice, cache key in a map, loop append, return from `fn`). For each, say move, borrow, or `.clone()` — and flag when `.clone()` is a design smell.”

## Chapter 3

# Chapter 2: Types and Expressions

### 3.1 Hook

Rust's type system is not ceremony. The compiler uses it to check ownership, catch bugs, and keep code predictable **before** you run anything. This chapter covers built-in types, inference, and expression-style control flow.

### 3.2 Integer types

Rust has **fixed-width** signed and unsigned integers. Pick the width that matches your domain (protocol field, array index, counter).

| Type          | Width              | Typical use                          |
|---------------|--------------------|--------------------------------------|
| i8 ... i128   | 8–128 bit signed   | Binary protocols, embedded registers |
| u8 ... u128   | 8–128 bit unsigned | Bytes, sizes, hashes                 |
| isize / usize | pointer-sized      | indexing, len(), collection sizes    |



```
// Playground
fn main() {
    let port: u16 = 502;
    // Modbus-style port --- explicit width
    let count = 42;
    // infers i32 by default
    let byte = 0xFFu8;
    let big: i64 = 1_000_000;
    println!("{}", port, count, byte,
               big);
}
```

**Defaults:** integer literals infer `i32` unless you suffix (`42u64`) or annotate.

For wire formats and hardware, **always** choose an explicit width. Do not rely on “whatever the platform uses.”

**Overflow:** in debug builds, arithmetic that overflows **panics**. In release, integers wrap (two’s complement).

Use `checked_add`, `saturating_add`, or `wrapping_add` when the policy matters.

### 3.3 Floating point, `bool`, and `char`

| Type                                | Notes                                                                 |
|-------------------------------------|-----------------------------------------------------------------------|
| <code>f32</code> , <code>f64</code> | IEEE floats; <code>f64</code> is the default inference                |
| <code>bool</code>                   | exactly true or false — not an integer in safe Rust                   |
| <code>char</code>                   | one <b>Unicode scalar value</b> (4 bytes), e.g. <code>'(crab)'</code> |
| <code>()</code>                     | <b>unit type</b> — “no meaningful value”; used as empty return        |

```
// Playground
fn main() {
    let pi: f64 = 3.14159;
    let ok = true;
    let crab = '(crab)';
    let unit: () = (); // only value of type ()
    println!("{}", pi, ok, crab,
```

```
        unit);
    }
```

char is not a UTF-8 byte — it is a single code point.

Text strings use `str` / `String` (below); ownership of those buffers is Chapter 1.

### 3.4 Inference and annotations

The compiler infers types when unambiguous. Add annotations or suffixes when it is not:

```
// Playground
fn main() {
    let xs = vec![1, 2, 3];      // Vec<i32>
    let ys: Vec<u8> = Vec::new();
    // empty vec needs a hint
    let n = 10u32;
    let ratio = n as f64 / 3.0;
    // cast integer to float
    println!("{:?} {:?} {}", ys, xs, ratio);
}
```

**Casts** use `as` between numeric types (`u16 as u32`). **Conversions** that can fail (string parsing, narrowing) use methods like `.parse()` or `TryFrom` — covered in Chapter 8.

### 3.5 Tuples and arrays

**Tuples** group fixed types; access by index (`.0`, `.1`):

```
// Playground
fn main() {
    let pair: (u16, &str) = (502, "modbus");
    println!("port {} proto {}", pair.0,
            pair.1);
}
```

**Arrays** `[T; N]` have compile-time fixed length and stack storage when `T`

is Copy:

```
// Playground
fn main() {
    let frame: [u8; 4] = [0xDE, 0xAD, 0xBE,
        0xEF];
    println!("len {} first {:02X}",
        frame.len(), frame[0]);
}
```

Growable `Vec<T>` and slice ownership return in Chapter 1 and Chapter 11.

## 3.6 Slices

A **slice** is a **view** into contiguous memory: pointer plus length, no ownership.

The type is written `&[T]` — a borrow of `[T]`. `[T]` is **unsized**: the compiler does not know its length at compile time.

| Form                   | Meaning                                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>&amp;[T]</code>  | borrowed view of <code>T</code> elements                                                                        |
| <code>&amp;[u8]</code> | byte buffer view — wire payloads, file chunks, <code>Vec&lt;u8&gt;</code> data                                  |
| <code>&amp;str</code>  | UTF-8 <b>text</b> slice — same fat-pointer idea, different element type ( <code>u8</code> with UTF-8 invariant) |

Create slices from arrays, vectors, or other slices with **range syntax**:

```
// Playground
fn main() {
    let frame: [u8; 6] = [0x01, 0x02, 0x03,
        0x04, 0x05, 0x06];
    let payload: &[u8] = &frame[2..5];
    // bytes at indices 2, 3, 4
    let all: &[u8] = &frame[..];
    // whole array as slice

    let nums = vec![10, 20, 30, 40];
```

```

let mid = &nums[1..3];
// &[i32] borrowing the Vec's buffer

println!("payload {:?} all {:?} mid {:?}",
        payload, all, mid);
}

```

**Indexing:** `[i]` on a slice is a direct access. Out-of-range indices **panic** at runtime.

Prefer `.get(i) -> Option<T>` when the index comes from user input, parsed fields, or unproven loop math.

```

// Playground
fn main() {
    let frame: [u8; 4] = [0xDE, 0xAD, 0xBE,
                          0xEF];
    let i = 2;
    match frame.get(i) {
        Some(byte) => println!("byte {:02X}",
                               byte),
        None => println!("index {} out of
                        range", i),
    }
}

```

Half-open ranges match for loops: `&xs[0..3]` includes indices 0, 1, 2. An empty slice is valid: `&xs[0..0]`.

## 3.7 Strings and UTF-8

Rust strings are **UTF-8** byte sequences, not fixed-width characters. That keeps them compact and compatible with the web and most modern protocols.

It also means **byte index**  $\neq$  **character index**.

| Type             | Role                                                                       |
|------------------|----------------------------------------------------------------------------|
| <code>str</code> | unsized UTF-8 text — always used as <code>&amp;str</code> (borrowed slice) |

| Type                  | Role                                                                               |
|-----------------------|------------------------------------------------------------------------------------|
| <code>&amp;str</code> | borrowed view into valid UTF-8 (literal, substring, or <code>String</code> borrow) |
| <code>String</code>   | owned, growable UTF-8 buffer on the heap                                           |

```
// Playground
fn main() {
    let literal: &str = "fixed";
    // stored in read-only segment
    let owned = String::from("growable");
    // heap-backed owner
    let also: &str = &owned;
    // borrow the String as &str

    let mut label = String::new();
    label.push_str("port=");
    label.push_str("502");

    println!("{}", literal, owned, also);
    println!("{}", label);
}
```

**Literals** have type `&str`. `String::from`, `.to_string()`, and `format!` allocate when you need an owned buffer.

`push_str` / `push` extend a `String` in place. Use them when building output in a loop — log lines, CSV rows, protocol text fields.

3.7.1 `&str` vs `String` in APIs

| Callee needs                        | Parameter type                                                                             |
|-------------------------------------|--------------------------------------------------------------------------------------------|
| Read or compare text only           | <code>&amp;str</code>                                                                      |
| Store text beyond the call          | <code>String</code> (or return owned from caller)                                          |
| Accept literal or owned caller data | <code>&amp;str</code> — callers pass <code>"literal"</code> or <code>&amp;my_string</code> |

```
// Playground
fn log_label(label: &str) {
    println!("label={}", label);
}

fn store_tag(tag: String) -> String {
    tag // owner returns owned text
}

fn main() {
    let name = String::from("sensor_a");
    log_label("startup");      // &str literal
    log_label(&name);
    // &String coerces to &str
    let saved = store_tag(name);
    println!("{}", saved);
}
```

Rust **coerces** `&String` to `&str` at call sites. So `fn f(s: &str)` is the default for read-only string parameters.

Take `String` when the function must **own** the data — to store it, spawn a thread, or key a map.

### 3.7.2 Length, iteration, and slicing text

- `.len()` on `&str` / `String` is **byte** length, not grapheme or char count.
- `.chars()` iterates Unicode scalar values (`char`); use when you care about characters.
- `.bytes()` iterates raw `u8` values — useful next to binary parsers.
- `.lines()` splits on line endings — common for config and serial-line protocols.

```
// Playground
fn main() {
    let s = "naïve"; // 'i' is two UTF-8 bytes
    println!("bytes={} chars={}", s.len(),
        s.chars().count());

    for ch in s.chars() {
```

```

        print!("{}", ch);
    }
    println!();

    for line in "a\nb\n".lines() {
        println!("line: {}", line);
    }
}

```

**String slicing** uses the same range syntax as byte slices. Ranges must fall on **UTF-8 character boundaries**.

Slicing at a bad byte index **panics**:

```

// Playground
fn main() {
    let s = "hello";
    let hello = &s[0..5];
    // ok --- ASCII, one byte per char

    let emoji = "hi (crab)";
    let hi = &emoji[0..2]; // ok
    // let bad = &emoji[0..3];
    // panic --- splits the crab emoji
    println!("{}", hello, hi);
}

```

At protocol and config boundaries, treat **&[u8]** as opaque bytes on the wire. Treat **&str** as validated human-readable text — config keys, error messages, JSON string fields after parsing.

Do not cast arbitrary bytes to **&str** without validation. Use **std::str::from\_utf8** or **String::from\_utf8** when converting (Chapter 8).

### 3.7.3 Common **&str** helpers (no allocation)

| Method                                                  | Use                                |
|---------------------------------------------------------|------------------------------------|
| <code>.starts_with</code> / <code>.ends_with</code>     | protocol prefixes, file extensions |
| <code>.strip_prefix</code> / <code>.strip_suffix</code> | peel "CMD:" off a serial line      |
| <code>.split</code> / <code>.split_once</code>          | CSV-ish fields, key=value pairs    |

| Method                         | Use                                                  |
|--------------------------------|------------------------------------------------------|
| <code>.trim</code>             | whitespace around user input                         |
| <code>.parse::<t>()</t></code> | <code>"502".parse::<u16>() -&gt; Result</u16></code> |

```
// Playground
fn main() {
    let line = " PORT=502 ";
    let trimmed = line.trim();
    if let Some(rest) =
        trimmed.strip_prefix("PORT=") {
        match rest.parse::() {
            Ok(port) => println!("port {}",
                                port),
            Err(e) => println!("bad port: {}",
                               e),
        }
    }
}
```

### 3.8 Variables and mutability

```
// Playground
fn main() {
    let x = 10;           // immutable binding
    let mut y = 20;       // mutable binding
    y += x;
    println!("{}", y);
}
```

Default **let** is **immutable**. Use **mut** only where the value changes — it documents intent and helps the borrow checker later.

### 3.9 Control flow as expressions

```
// Playground
fn main() {
```



```

let n = 7;
let label = if n % 2 == 0 { "even" } else {
    "odd" };
println!("{}", n, label);

for i in 0..3 {
    println!("i = {}", i);
}

let mut sum = 0;
let mut k = 0;
while k < 3 {
    sum += k;
    k += 1;
}
println!("sum = {}", sum);
}

```

- **if** is an expression — both branches must return the same type.
- **0..3** is half-open (0, 1, 2); **0..=3** is inclusive (0 through 3).
- **loop**, **while**, and **for** cover the usual iteration shapes; **for** over a range is idiomatic for indexed passes. Iterator pipelines (**map**, **filter**, **collect**) are Chapter 4.

### 3.10 match preview

Full power in Chapter 6; here, matching integers:

```

// Playground
fn main() {
    let code = 404;
    let msg = match code {
        200 => "ok",
        404 => "not found",
        _ => "other",
    };
    println!("{}", msg);
}

```

`_` is the wildcard arm. `match` must be **exhaustive** — every possible value covered (easy for integers with `_`; enums need every variant).

## 3.11 Idiom spotlight

**Name your widths at boundaries.** Inside a function, inference is fine. At protocol edges, file headers, and public APIs, use explicit types (`u16`, `[u8; 8]`). That way refactors cannot silently change layout.

**Borrow text, own when storing.** Prefer `&str` and `&[u8]` in function parameters. Return or store `String` / `Vec<u8>` only when the data must outlive the caller. At binary/text boundaries, keep `&[u8]` for bytes and `&str` for validated UTF-8. Do not mix them silently.

## 3.12 Go deeper

- [The Rust Book — Data Types](#)
- [The Rust Book — String slices](#)

## 3.13 See also

- Preface — rustup and Cargo setup
- Chapter 1: Ownership and borrowing
- Chapter 4: Iterators
- Chapter 6: Enums and pattern matching

### 3.13.1 Afterparty

#### 3.13.1.1 Scalars and inference

1. **Integer pick** — “Quiz me: for 8 scenarios (Modbus port, byte buffer, collection index, money cents, timestamp millis, hash output, loop counter, enum discriminant), I pick `u8` / `u16` / `u32` / `u64` / `i32` / `usize`; you correct and explain overflow risk.”
2. **char vs byte** — “Explain why `'A'` is not the same type as `65u8`. Show one valid `char` literal and one invalid escape; mention UTF-8 only when discussing `str` / `String`.”

### 3.13.1.2 Compound types and text

3. **Tuple vs array** — “When do I use `(u16, u16)` vs `[u16; 2]` for a coordinate pair? Give one idiomatic example of each.”
4. **Array bounds** — “Snippet with `[u8; 4]` and an index from user input. I explain compile-time vs runtime safety; you show bounds checking with `.get()` vs `[i]`.”
5. **&str vs String API** — “Five function signatures for logging labels. I choose `&str` or `String` for each; you explain ownership and allocation cost.”
6. **Parse drill** — “Give `let s = \"502\";` — I write two ways to get `u16` (unwrap version and proper `Result` version); you grade idiomatic error handling.”

### 3.13.1.3 Slices and string idioms

7. **Slice from array** — “Given `[u8; 8]` with a 2-byte header and 6-byte payload, I write range expressions for header, payload, and full as `&[u8]`; you check half-open bounds.”
8. **.get vs [i]** — “Three indexing scenarios (fixed compile-time index, user-supplied index, loop variable). I pick `[i]` or `.get(i)` for each; you explain panic risk.”
9. **UTF-8 length trap** — “For `\"naïve\"` and `\"hi (crab)\"`, I predict `.len()` vs `.chars().count()`; you show one bad string slice that panics and the safe fix (`.chars()` or careful byte ranges).”
10. **Coercion quiz** — “Five call sites passing `&str`, `String`, `&String`, and a literal into `fn log(msg: &str)`. I say ok or what coercion happens; you quote the effective type.”
11. **Empty slice** — “Explain whether `&frame[0..0]` is valid, what `.len()` returns, and how `if slice.is_empty()` reads in a parser guard.”
12. **Lines and config** — “Multiline config string with comments and blank lines. I sketch a loop using `.lines()` and `.trim().starts_with('#')`; you extend with one `strip_prefix` for `KEY= rows`.”

### 3.13.1.4 Control flow and match

13. **Range quiz** — “For half-open vs inclusive ranges, I predict output of four for loops using `..` and `..=`; you correct with printed values.”
14. **if expression types** — “Show an `if` where branches return different types and fail to compile. I explain the error; you fix with a unified

type (same type in both arms or wrap in enum preview)."

15. **match warm-up** — "Extend a match on HTTP status codes with 3xx, 4xx, 5xx groupings using range patterns; I write arms; you check exhaustiveness."
16. **Loop choice** — "Four iteration tasks (infinite retry with break, consume iterator, index 0..len, early exit on condition). I pick loop/while/for; you confirm idiomatic choice."

# Chapter 4

## Chapter 3: Functions and Methods

### 4.1 Hook

You already know how to call functions. In Rust, every signature is a **contract** about ownership and types. A parameter is not “just a reference”; it tells you whether the callee **borrow**s, **mutably borrow**s, or **takes** the value.

This chapter covers standalone functions, methods, and return-type patterns you will reuse everywhere.

### 4.2 Scope — a brief tour

Function signatures, receivers, and return patterns — not full generics or `async`.

| This chapter covers                                            | Deferred                  |
|----------------------------------------------------------------|---------------------------|
| Signatures, ownership in params, <code>impl</code> methods     | Full trait system -> Ch 7 |
| <code>impl</code> Trait return preview, <code>mem::take</code> | Async fns -> Ch 16        |
| <code>Result</code> signature preview                          | Error design -> Ch 8      |

### 4.3 Function signatures

A Rust function names its inputs and output explicitly:

```
// Playground
fn add(a: i32, b: i32) -> i32 {
    a + b
}

fn main() {
    println!("{}", add(2, 3));
}
```

| Piece            | Meaning                                                             |
|------------------|---------------------------------------------------------------------|
| fn name          | declare a function                                                  |
| (a: i32, b: i32) | parameters with types                                               |
| -> i32           | return type                                                         |
| { ... }          | body; the <b>last expression</b> (no semicolon) is the return value |

#### Java / Python contrast

| Aspect         | Java                        | Python            | Rust                                                                |
|----------------|-----------------------------|-------------------|---------------------------------------------------------------------|
| Return         | return x; or last statement | return x optional | last <b>expression</b> returns; use return for early exit           |
| Missing return | void                        | implicit None     | -> () unit type                                                     |
| Overloading    | same name, different params | duck typing       | <b>no</b> overloading — use different names or generics (Chapter 7) |

### 4.4 Parameters and ownership

Parameters follow the same ownership rules as let bindings (Chapter 1):

```
// Playground
fn consume(s: String) {
    println!("{}", s);
}
```

```

} // s dropped here

fn borrow(s: &str) {
    println!("{}", s);
}

fn main() {
    let label = String::from("sensor");
    borrow(&label);
    consume(label);
    // borrow(&label);
    // ERROR: label was moved into consume
}

```

| Parameter type | Caller after call                 |
|----------------|-----------------------------------|
| T              | value <b>moved</b> unless T: Copy |
| &T             | still owns; shared borrow         |
| &mut T         | still owns; exclusive borrow      |

**Rule of thumb:** take &str / &[u8] when you only read; take owned String / Vec when you store or send the data elsewhere.

## 4.5 Expression bodies and early return

Omit braces for a single expression:

```

// Playground
fn double(x: i32) -> i32 {
    x * 2
}

fn abs_diff(a: i32, b: i32) -> i32 {
    if a > b { a - b } else { b - a }
}

fn main() {
    println!("{}", double(5), abs_diff(10,

```

```
    3));  
}
```

Use `return` when you exit before the end:

```
// Playground  
fn first_positive(v: &[i32]) -> Option<i32> {  
    for &n in v {  
        if n > 0 {  
            return Some(n);  
        }  
    }  
    None  
}  
  
fn main() {  
    println!("{:?}", first_positive(&[-1, 0, 4,  
    2]));  
}
```

Adding a semicolon after the last expression **suppresses** the return value and gives `()`:

```
// Playground --- does not compile if return  
    type is i32  
fn broken() -> i32 {  
    42;  
    // semicolon turns this into a statement,  
    // not the return value  
    // ERROR: expected i32, found ()  
}
```

## 4.6 The unit type `()` and diverging functions

Functions that do side effects only often return **unit**:

```
// Playground  
fn log_line(msg: &str) {  
    println!("[log] {}", msg);  
} // implicit -> ()
```



```
fn main() {
    log_line("ready");
}
```

Some functions **never return** — they diverge with `panic!`, infinite loops, or `std::process::exit`. Their return type is `!` (never type):

```
// Playground
fn fatal(msg: &str) -> ! {
    panic!("{}", msg);
}

fn main() {
    // fatal("stop");
    // would never reach code below
    println!("ok");
}
```

You will see `!` rarely in application code. Knowing it exists explains why match arms can have different “no return” branches.

## 4.7 Methods and associated functions

Rust splits **data** (struct / enum) from **behaviour** (impl blocks). Methods take `&self`, `&mut self`, or `self`; **associated functions** have no `self` (like static methods):

```
// Playground
struct Gauge {
    value: f64,
}

impl Gauge {
    // associated function --- call as
    // Gauge::new(0.0)
    fn new(value: f64) -> Self {
        Self { value }
    }
}
```

```

// method --- call as gauge.read()
fn read(&self) -> f64 {
    self.value
}

fn set(&mut self, v: f64) {
    self.value = v;
}

fn reset(self) -> Self {
    Self { value: 0.0 }
}
}

fn main() {
    let mut g = Gauge::new(12.5);
    println!("{}", g.read());
    g.set(99.0);
    let zero = g.reset();
    println!("{}", zero.read());
}

```

| Receiver                   | Meaning                        |
|----------------------------|--------------------------------|
| <code>&amp;self</code>     | borrow for read                |
| <code>&amp;mut self</code> | exclusive borrow for mutation  |
| <code>self</code>          | <b>consume</b> self (move out) |

**Java:** instance methods inside classes. **Python:** functions in a class with `self`. **Rust:** explicit `impl` blocks — no inheritance chain.

## 4.8 Multiple `impl` blocks and privacy

You can split `impl` for clarity; only items marked `pub` are visible outside the module (Chapter 9):

```

// Playground
pub struct Counter {
    n: u32,

```

```

}

impl Counter {
    pub fn new() -> Self {
        Self { n: 0 }
    }

    pub fn bump(&mut self) {
        self.n += 1;
    }

    pub fn get(&self) -> u32 {
        self.n
    }
}

fn main() {
    let mut c = Counter::new();
    c.bump();
    println!("{}", c.get());
}

```

Field `n` stays private; callers use methods — same encapsulation idea as Java private fields with public getters.

## 4.9 Generic functions (preview)

One function body can work for many types when they share a capability (**trait bound**). Full treatment is in Chapter 7; the shape is:

```

// Playground
use std::fmt::Display;

fn show_twice<T: Display>(x: T) {
    println!("{}", x, x);
}

fn main() {
    show_twice(42);
}

```

```
show_twice("hello");
}
```

The compiler **monomorphizes**—generates `show_twice_i32`, `show_twice_str`, etc. — so you pay no runtime dispatch cost unless you ask for `dyn Trait`.

## 4.10 Functions that return Result (preview)

Fallible operations return `Result` instead of throwing. Chapter 8 covers this in depth; the signature pattern is:

```
// Playground
fn parse_port(s: &str) -> Result<u16,
    std::num::ParseIntError> {
    s.parse::<u16>()
}

fn main() {
    match parse_port("8080") {
        Ok(p) => println!("port {}", p),
        Err(e) => println!("bad port: {}", e),
    }
}
```

Inside a `fn -> Result<...> body, ?` propagates errors upward. You will use it after Chapter 6 and Chapter 8.

## 4.11 impl Trait in return position

Return an iterator without naming the concrete adapter type:

```
// Playground
fn top_readings(readings: &[f64], n: usize) ->
    impl Iterator<Item = f64> + '_ {
    let mut sorted: Vec<f64> =
        readings.to_vec();
    sorted.sort_by(|a, b|
        b.partial_cmp(a).unwrap());
    sorted.into_iter().take(n)
```

```

}

fn main() {
    let vals = [1.2, 3.4, 2.1, 5.0];
    let top: Vec<_> = top_readings(&vals,
        2).collect();
    println!("{:?}", top);
}

```

Every return path must produce the **same** concrete type. Different iterator types in each arm fail to compile. Use `Box<dyn Iterator<Item = f64>>` or an enum instead.

## 4.12 Draining with `mem::take`

Move inner data out of a struct while leaving a valid empty value behind (Chapter 10):

```

// Playground
struct Buffer {
    inner: Vec<u8>,
}

impl Buffer {
    fn drain_into(&mut self, out: &mut Vec<u8>)
    {
        out.extend(std::mem::take(&mut
            self.inner));
    }
}

fn main() {
    let mut buf = Buffer { inner: vec![1, 2, 3] };
    let mut batch = Vec::new();
    buf.drain_into(&mut batch);
    println!("batch={:?} buf={:?}", batch,
        buf.inner);
}

```

take replaces inner with `Vec::default()` — safe because an empty `Vec` is valid.

## 4.13 where clauses — readable bounds

When generic bounds clutter the signature, move them to a `where` block:

```
// Playground
use std::fmt::Display;

fn log_pair<T, U>(a: T, b: U)
where
    T: Display,
    U: Display,
{
    println!("{}", a, b);
}

fn main() {
    log_pair(502, "ready");
}
```

Same contract as `fn log_pair<T: Display, U: Display>(...)` — pick whichever reads cleaner.

## 4.14 Error constructor factories

Production error enums often expose **named constructors** instead of spelling struct variants at every call site. Accept `impl Into<String>` so callers pass `&str`, `String`, or anything string-like:

```
// Playground
#[derive(Debug)]
// compile-time trait impl --- not a Java
// annotation or Python decorator; see Ch17
enum ParseError {
    MissingField { field: String, record:
        String },
    BadValue { field: String, raw: String },
```

```

}

impl ParseError {
    #[inline]
    pub fn missing_field(field: impl
        Into<String>, record: impl
        Into<String>) -> Self {
        Self::MissingField {
            field: field.into(),
            record: record.into(),
        }
    }

    #[inline]
    pub fn bad_value(field: impl Into<String>,
        raw: impl Into<String>) -> Self {
        Self::BadValue {
            field: field.into(),
            raw: raw.into(),
        }
    }
}

fn parse_port_field(raw: &str) -> Result<u16,
    ParseError> {
    raw.parse()
        .map_err(|_|
            ParseError::bad_value("port", raw))
}

fn main() {
    println!("{}", parse_port_field("502"));
    println!("{}", parse_port_field("oops"));
}

```

---

| Pattern                              | Why                                                |
|--------------------------------------|----------------------------------------------------|
| <code>impl Into&lt;String&gt;</code> | one constructor accepts literals and owned strings |

---

| Pattern                                 | Why                                                      |
|-----------------------------------------|----------------------------------------------------------|
| <code>#[inline]</code> on tiny wrappers | hints the compiler on hot parser paths — no runtime cost |
| named fn vs raw variant                 | call sites read like intent; field names stay consistent |

Use this in library crates with **thiserror** enums (Chapter 8) — the constructors stay even when `Display` is derived.

**`#[derive(...)]` preview:** the attribute auto-generates trait implementations at **compile time** (`impl Debug for ParseError { ... }`). It does not run at call time and is not runtime metadata. `Std` derives (`Debug`, `Clone`, ...), `ecosystem` derives (`Serialize`, `Error`, ...), and when custom derives are worth it — Chapter 17: Derive attributes.

## 4.15 Config-holder structs (lightweight builders)

You do not always need a builder crate. A small struct that **holds configuration** and exposes one method per use case is enough:

```
// Playground
struct RequestBuilder {
    timeout_ms: u64,
    retries: u32,
    endpoint: String,
}

impl RequestBuilder {
    fn new(endpoint: impl Into<String>) -> Self
    {
        Self {
            timeout_ms: 5000,
            retries: 3,
            endpoint: endpoint.into(),
        }
    }

    fn with_timeout(mut self, ms: u64) -> Self {
        self.timeout_ms = ms;
    }
}
```



```

        self
    }

    fn build_poll_body(&self, device_id: u32)
        -> String {
        format!(
            "GET
            /devices/{device_id}?timeout={}&
            retries={}",
            self.timeout_ms, self.retries
        )
    }
}

fn main() {
    let req =
        RequestBuilder::new("gateway.local")
        .with_timeout(1000)
        .build_poll_body(7);
    println!("{}", req);
}

```

Reach for **new + chainable setters + one build\_\* method** when parameters are fixed at construction time but the output shape varies. Full builder crates pay off when you have many optional fields and validation at build time.

## 4.16 Extension traits — add behaviour without newtypes

When you need a helper on a type you do not own, define a **trait in your crate** and implement it for the foreign type you control the orphan rule for — or implement for your wrapper. Common pattern: extend **Option** with domain-specific ? helpers (Chapter 8) and extend **&str** with string normalizers (Chapter 13).

```

// Playground
trait TrimOrEmpty {

```

```

    fn trim_or_empty(self) -> &str;
}

impl TrimOrEmpty for &str {
    fn trim_or_empty(self) -> &str {
        self.trim()
    }
}

fn label(raw: &str) -> &str {
    raw.trim_or_empty()
}

fn main() {
    println!("{}", label("  sensor-1  "));
}

```

Keep extension traits **small and focused** — one concern per trait, not a grab-bag of unrelated methods.

#### 4.16.1 Function edge cases

**Wrong — mismatched impl Trait return arms:**

```

// Playground --- does not compile
fn pick(_use_a: bool) -> impl Iterator<Item =
    i32> {
    if _use_a {
        vec![1, 2].into_iter()
    } else {
        [3, 4].into_iter()
        // different concrete iterator type
    }
}

```

**const fn preview:** Rust can evaluate some functions at compile time (const fn add(a: i32, b: i32) -> i32 { a + b }). Full const evaluation rules are in Chapter 17.

## 4.17 When the compiler says no

Common errors in this chapter:

| Error (typical)          | Cause                                                      | Fix                                        |
|--------------------------|------------------------------------------------------------|--------------------------------------------|
| expected T, found ()     | semicolon on last expression                               | remove ; or add return                     |
| use of moved value       | took <code>String</code> by value, caller uses again       | borrow with <code>&amp;str</code> or clone |
| cannot borrow as mutable | <code>&amp;mut</code> while <code>&amp;</code> still alive | shrink borrow scope                        |
| type annotations needed  | generic T unknown                                          | turbofish or annotate call site            |

## 4.18 Idiom spotlight

**Borrow in parameters, own at boundaries.** Library helpers take `&str` and `&[u8]`; store `String` / `Vec` only when the data outlives the call or crosses a thread/channel boundary.

## 4.19 Go deeper

- [The Rust Book — Functions](#)
- [Methods](#)

## 4.20 See also

- Chapter 2: Types and expressions — types in signatures
- Chapter 1: Ownership — move vs borrow in parameters
- Chapter 4: Iterators — functions as pipeline steps
- Chapter 7: Structs, traits, and generics — `impl`, traits, generics
- Chapter 8: Errors and testing — `Result` and `?`
- Chapter 17: Metaprogramming — `#[derive]` syntax, not annotations/decorators

### 4.20.1 Afterparty

#### 4.20.1.1 Signatures and ownership

1. **Parameter audit** — “Five function signatures for logging: `&str`, `String`, `&String`, `Cow<str>`, `impl AsRef<str>`. I pick one per use case; you

explain ownership cost.”

2. **Move vs borrow** — “Snippet calls `process(s)` then uses `s` again. I explain the error and show two fixes (`&s`, `clone`).”

#### 4.20.1.2 Methods and impl

3. **impl block** — “Struct `Timer` with `start`, `elapsed`, `reset`. I write `impl`; you check `&self` vs `&mut self`.”
4. **Associated fn** — “When is `Type::new()` idiomatic vs `Default::default()`? One example each.”
5. **Consume self** — “Method `fn into_inner(self) -> Vec<u8>` — why must it take `self` by value?”

#### 4.20.1.3 Returns and control flow

6. **Semicolon trap** — “Three tiny functions: one returns `i32` correctly, two fail due to `;`. I fix them.”
7. **Early return** — “Rewrite nested `if` in a parser as early return `None / ? style`.”
8. **Unit vs value** — “Which functions should return `()` vs `bool` vs `Option<T>`? Three CLI helper names, I choose.”

#### 4.20.1.4 Generics and errors preview

9. **Generic bounds** — “Fix `fn max(a: T, b: T)` without bounds; add `T: Ord` or `PartialOrd`.”
10. **Result signature** — “Design `fn read_config(path: &str) -> Result<Config, ...>`; list error variants, no body.”

#### 4.20.1.5 impl Trait and drain

11. **impl Iterator return** — “Write `top_n(vals: &[f64], n: usize) -> impl Iterator<Item = f64>` — explain why two different iterator types in `if arms` fail.”
12. **mem take drain** — “Buffer struct drains `Vec<u8>` to caller via `mem::take` — show before/after inner field.”
13. **where clause** — “Rewrite cluttered `fn f<T: A + B + C>(x: T)` with `where` block — same behaviour.”
14. **Self return** — “Method `fn into_inner(self) -> Vec<u8>` on wrapper — why `Self` not concrete type name?”

15. **const fn** — “One `const fn` port validator `fn is_valid(p: u16) -> bool` — what can and cannot run at compile time?”
16. **Error constructors** — “Add `missing_field(name, record)` on a parse error enum with `impl Into<String>`; compare to spelling the variant at each site.”
17. **Config holder** — “HTTP poll client: struct holds timeout/retries/endpoint; one `build_request(device_id)` — no builder crate.”
18. **Extension trait** — “Add `TrimOrEmpty` for `&str`; use in three parser call sites vs free functions.”

## Chapter 5

# Chapter 4: Iterators

### 5.1 Hook

**Iterators are a shared pattern, not a Rust quirk.** Walk a sequence once, transform or filter each step, then produce a result — without index loops.

| Stack      | How the pattern shows up                                                                     |
|------------|----------------------------------------------------------------------------------------------|
| Java       | <code>Stream&lt;T&gt;</code> — <code>map</code> , <code>filter</code> , <code>collect</code> |
| Python     | <code>for x in items</code> , list comprehensions, generators                                |
| C# / LINQ  | <code>IEnumerable</code> , <code>.Select()</code> , <code>.Where()</code>                    |
| JavaScript | <code>Array.prototype.map</code> / <code>filter</code> / <code>reduce</code>                 |
| SQL        | <code>SELECT</code> over rows (set-oriented, not index loops)                                |
| Unix       | pipes — one program's output is the next program's input stream                              |
| Rust       | <code>Iterator</code> trait — lazy adapters + consumers                                      |

**Rust's version:** the `Iterator` trait — lazy `.map()` / `.filter()` chains that run only when you `.collect()` or call another consumer. See Chapter 11 for the collection types these pipelines walk.

## 5.2 for is syntax sugar

Chapter 2 introduced `for i in 0..3` and `loop / while`. Behind `for`, Rust calls **IntoIterator** — “turn this value into something we can walk”:

```
// Playground
fn main() {
    for i in 0..3 {
        println!("{}", i); // 0, 1, 2
    }

    let words = vec!["modbus", "tcp"];
    for w in &words {
        println!("{}", w); // borrows each &str
    }
}
```

| What you write                 | What happens                                                       |
|--------------------------------|--------------------------------------------------------------------|
| <code>for x in 0..10</code>    | <code>range</code> implements <code>IntoIterator</code>            |
| <code>for x in &amp;vec</code> | <code>vec.iter()</code> — borrow each element                      |
| <code>for x in vec</code>      | <code>vec.into_iter()</code> — <b>consume</b> the <code>Vec</code> |

Ranges (`..`, `..=`) are iterators. So are `.lines()` and `.chars()` on `str` from Chapter 2.

## 5.3 Three ways to walk a Vec

| Method                    | You get                                        | Vec after loop                                     |
|---------------------------|------------------------------------------------|----------------------------------------------------|
| <code>.iter()</code>      | <code>Iterator&lt;Item = &amp;T&gt;</code>     | still usable                                       |
| <code>.iter_mut()</code>  | <code>Iterator&lt;Item = &amp;mut T&gt;</code> | still usable, elements may change                  |
| <code>.into_iter()</code> | <code>Iterator&lt;Item = T&gt;</code>          | <b>moved</b> — do not use <code>v</code> afterward |

```
// Playground
fn main() {
    let mut v = vec![1, 2, 3];
```

```

let sum_borrow: i32 = v.iter().sum();
println!("sum = {} vec = {:?}", sum_borrow,
        v);

for n in v.iter_mut() {
    *n *= 10;
}
println!("after mut: {:?}", v);

let v2 = vec![4, 5];
for n in v2.into_iter() {
    println!("consumed item {}", n);
}
// println!("{:?}", v2);
// error: v2 was moved by into_iter
}

```

**Rule of thumb:** default to `.iter()` when you only need to read. Use `.iter_mut()` to update in place. Use `into_iter()` / `for x in vec` when the collection itself should be consumed.

### 5.3.1 `iter` vs `into_iter` — what changes

The iterator method picks the **element type** the whole pipeline sees:

| Call                       | Item type (for <code>Vec&lt;T&gt;</code> ) | Who owns heap data after the chain?                              |
|----------------------------|--------------------------------------------|------------------------------------------------------------------|
| <code>v.iter()</code>      | <code>&amp;T</code>                        | still <code>v</code>                                             |
| <code>v.iter_mut()</code>  | <code>&amp;mut T</code>                    | still <code>v</code>                                             |
| <code>v.into_iter()</code> | <code>T</code>                             | moved out of <code>v</code> ; <code>v</code> is empty / unusable |

**Common habit in Java/Python-style loops:** iterating a list does not destroy the list. In Rust, for `x in v` is **`into_iter`** — equivalent to consuming `v`. That surprises newcomers and is a common compile-error source.

```

// Playground --- uncomment ONE wrong line at a
// time to see the error
fn main() {

```



```

let v = vec![String::from("a"),
             String::from("b")];

for s in &v {
    println!("{}", s);
    // borrows each &String
}
println!("{}", v);
// ok --- v still owned here

let v2 = vec![1, 2, 3];
for n in v2 {
    println!("{}", n);
    // moves each i32 out of v2
}
// println!("{}", v2);
// ERROR: value used after move
}

```

### Wrong pipeline — owned vs borrowed mismatch:

```

// Playground --- does not compile; read the
// explanation below
fn main() {
    let nums = vec![1, 2, 3];
    // Goal: Vec<i32> of doubled values --- but
    // iter yields &i32
    let doubled: Vec<i32> =
        nums.iter().collect();
    // ERROR: expected i32, found &i32 (or
    // similar mismatch on collect)
}

```

**Fix:** either borrow through the chain and copy at the end, or consume:

```

// Playground
fn main() {
    let nums = vec![1, 2, 3];
    let doubled: Vec<i32> =
        nums.iter().map(|&x| x * 2).collect();
}

```

```

    // peel & with |&x|
    let also: Vec<i32> =
        nums.into_iter().map(|x| x *
            2).collect(); // nums moved
    println!("{:?} {:?}", doubled, also);
}

```

Second pass on the same data:

```

// Playground
fn main() {
    let nums = vec![1, 2, 3];
    let sum: i32 = nums.iter().sum();
    println!("{}", sum);
    let max = nums.iter().fold(i32::MIN, |acc,
        &x| acc.max(x)); // ok --- borrow again

    let v = vec![10, 20];
    let total: i32 = v.into_iter().sum();
    // let again: i32 = v.into_iter().sum();
    // ERROR: v already moved
    println!("{}", total);
}

```

`into_iter()` **consumes** the collection once. `.iter()` lets you run many consumers (sum, then max, then print) as long as you do not also move `v`.

### 5.3.2 for forms — quick trap sheet

| You write                     | Typical mistake                                                              |
|-------------------------------|------------------------------------------------------------------------------|
| for x in v                    | Using v after the loop — v was moved into the loop                           |
| for x in &v                   | Expecting x to be T — x is &T (often fine for <code>println!</code> )        |
| for x in &mut v               | Forgetting <code>*x</code> to mutate the element                             |
| for x in 0..v.len() with v[i] | Out-of-bounds panic if v shrinks while looping — prefer <code>.iter()</code> |

## 5.4 Lazy iterators

Adapters return **another iterator** — they do not allocate a full intermediate `Vec` unless you `.collect()`.

```
// Playground
fn main() {
    let nums = vec![1, 2, 3, 4, 5];
    let doubled_evens: Vec<i32> = nums
        .iter()
        .filter(|&&x| x % 2 == 0)
        .map(|&x| x * 2)
        .collect();
    println!("{:?}", doubled_evens); // [4, 8]
    println!("nums still {:?}", nums);
}
```

`.filter` and `.map` take **closures** (`|x| ...`). How closures capture variables (`Fn`, `FnMut`, `FnOnce`) is covered in Chapter 12 — Closures.

### 5.4.1 Why closures see `&&i32` (double reference)

On `v.iter()`, each `Item` is `&i32`. Adapter methods like `.filter()` pass **`&Item`** to your closure — so the parameter is `&&i32`.

| Closure param                | Meaning                                                                                                   |
|------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code> x </code>             | <code>x</code> is <code>&amp;&amp;i32</code> ; <code>*x</code> coerces to <code>i32</code> in expressions |
| <code> \&amp;x </code>       | destructures one layer <code>-&gt; x: &amp;i32</code>                                                     |
| <code> \&amp;\&amp;x </code> | destructures to <code>x: i32</code>                                                                       |

Pick one style and stay consistent in a chain. Mixing `.iter()` with a closure that expects owned `i32` without peeling references is a frequent compile error.

The `Iterator` trait lives in `std` and defines `.next() -> Option<Item>`. Calling `.iter()` on standard types covers most code — when you need a custom walk, see [Implementing Iterator](#) below and Chapter 7 — associated types.

## 5.5 Common adapters

| Adapter                          | Effect                             |
|----------------------------------|------------------------------------|
| <code>.map(f)</code>             | transform each item                |
| <code>.filter(p)</code>          | keep items where p is true         |
| <code>.enumerate()</code>        | (index, item) pairs                |
| <code>.zip(other)</code>         | pair items until one iterator ends |
| <code>.take(n) / .skip(n)</code> | first n / skip first n             |
| <code>.chain(other)</code>       | append another iterator            |

```
// Playground
fn main() {
    let a = ['x', 'y', 'z'];
    let b = [1, 2, 3];

    for (i, ch) in a.iter().enumerate() {
        println!("{}", i, ch);
    }

    for (ch, n) in a.iter().zip(b.iter()) {
        println!("{}", ch, n);
    }

    let first_two: Vec<_> =
        a.iter().take(2).collect();
    println!("{}", first_two);
}
```

On text, remember Chapter 2 helpers: `"a\nb\n".lines()`, `"hi".chars()` — each returns an iterator over parts of the string.

### 5.5.1 Adapter edge cases (empty, short, uneven)

| Situation                            | What happens                                               |
|--------------------------------------|------------------------------------------------------------|
| <code>.zip</code> on unequal lengths | stops when <b>either</b> iterator ends — silent truncation |
| <code>.take(100)</code> on 3 items   | yields 3, no error                                         |

| Situation                         | What happens                                 |
|-----------------------------------|----------------------------------------------|
| <code>.skip(10)</code> on 3 items | yields nothing                               |
| <code>.chain</code>               | fused walk; still lazy until a consumer runs |

```
// Playground
fn main() {
    let ids = vec![1u16, 2];
    let vals = vec![10u16, 20, 30];
    // longer than ids
    let pairs: Vec<_> =
        ids.iter().zip(vals.iter()).collect();
    println!("{:?}", pairs);
    // [(1, 10), (2, 20)] --- third value
    // dropped

    let empty: Vec<i32> = vec![];
    let n: i32 = empty.iter().sum();
    // 0 --- not an error
    let hit = empty.iter().find(|&&x| x > 0);
    // None
    println!("sum={} find={:?}", n, hit);
}
```

**Wrong — two mutable walks at once:**

```
// Playground --- does not compile
fn main() {
    let mut v = vec![1, 2, 3];
    for a in v.iter_mut() {
        for b in v.iter_mut() {
            // ERROR: cannot borrow `v` as
            // mutable more than once
            *a += *b;
        }
    }
}
```

Rust forbids overlapping `&mut` borrows, including through nested `iter_mut()` loops. Fix: index by position, use `.enumerate()`, or `collect`

indices first — patterns from Chapter 1.

## 5.6 Consumers — finishing the pipeline

A **consumer** drains the iterator and produces a value (or side effect).

| Consumer                                      | Result                                                                    |
|-----------------------------------------------|---------------------------------------------------------------------------|
| <code>.collect()</code>                       | build <code>Vec</code> , <code>String</code> , <code>HashMap</code> , ... |
| <code>.sum()</code> / <code>.product()</code> | numeric fold                                                              |
| <code>.count()</code>                         | number of items                                                           |
| <code>.find(p)</code>                         | <code>Option&lt;Item&gt;</code> — first match                             |
| <code>.any(p)</code> / <code>.all(p)</code>   | <code>bool</code>                                                         |
| <code>.fold(init, f)</code>                   | single accumulated value                                                  |

```
// Playground
fn main() {
    let nums = vec![3, 1, 4, 1, 5];
    let sum: i32 = nums.iter().sum();
    let max = nums.iter().fold(i32::MIN, |acc,
        &x| acc.max(x));
    let has_even = nums.iter().any(|&x| x % 2
        == 0);
    println!("sum={} max={} has_even={}", sum,
        max, has_even);
}
```

### 5.6.1 collect and type hints

`collect()` can build many collection types. Sometimes Rust needs help:

```
// Playground
fn main() {
    let nums = vec![1, 2, 3];
    let doubled: Vec<i32> =
        nums.iter().map(|&x| x * 2).collect();
    let also: Vec<i32> = nums.iter().map(|&x| x
        * 2).collect::<Vec<_>>();
    println!("{:??} {:?}", doubled, also);
}
```

`Vec<_>` asks the compiler to infer the element type. Prefer an explicit binding type (`let v: Vec<i32> = ...`) when `collect` is the only clue.

#### Wrong — `collect` without enough type context:

```
// Playground --- does not compile
fn main() {
    let nums = vec![1, 2, 3];
    let doubled = nums.iter().map(|&x| x *
        2).collect();
    // ERROR: type annotations needed --- many
        types implement FromIterator
}
```

#### Wrong — collecting references, then dropping the owner:

```
// Playground --- does not compile
fn main() {
    let lines: Vec<String> =
        vec![String::from("PORT=502")];
    let tags: Vec<&str> = lines.iter().map(|s|
        s.as_str()).collect();
    drop(lines);
    // println!("{:?}", tags);
    // would dangle --- often fails earlier:
    // tags borrows `lines`, so drop(lines) is
        ERROR while tags is alive
}
```

Owned collection after parsing is the safe default: `.map(|s| s.clone())` or `collect into Vec<String>` before dropping the source buffer. Lifetimes formalize this in Chapter 5.

### 5.6.2 Consumer edge cases

| Call                    | Empty input          | Notes                                |
|-------------------------|----------------------|--------------------------------------|
| <code>.sum()</code>     | 0 for numbers        | type must implement <code>Sum</code> |
| <code>.product()</code> | 1 for multiplication |                                      |
| <code>.count()</code>   | 0                    |                                      |

| Call                                      | Empty input  | Notes                                                      |
|-------------------------------------------|--------------|------------------------------------------------------------|
| <code>.find(...)</code>                   | None         | not a panic — use <code>if let</code> / <code>match</code> |
| <code>.max()</code> / <code>.min()</code> | None         |                                                            |
| <code>.any</code> / <code>.all</code>     | false / true | empty all is vacuously true                                |

```
// Playground
fn main() {
    let ports: Vec<u16> = vec![];
    let ok = ports.iter().all(|&p| p >= 1 && p
        <= 65535);
    println!("all in range? {}", ok);
    // true on empty --- know your domain rule

    let logs = vec!["ok", "ERROR: timeout"];
    if let Some(line) = logs.iter().find(|&s|
        s.contains("ERROR")) {
        println!("first error line: {}", line);
    }
}
```

## 5.7 Implementing Iterator

Most code uses `.iter()` on collections. When you own the walk logic — port ranges, non-empty lines, frame parsing — implement `Iterator` yourself.

### 5.7.1 Port range scanner

Scan Modbus-style ports 502–505 without building a `Vec` first:

```
// Playground
struct PortScan {
    next_port: u16,
    end: u16,
}

impl Iterator for PortScan {
    type Item = u16;
```



```

fn next(&mut self) -> Option<Self::Item> {
    if self.next_port > self.end {
        return None;
    }
    let p = self.next_port;
    self.next_port += 1;
    Some(p)
}

fn main() {
    let ports: Vec<u16> = PortScan { next_port:
        502, end: 505 }.collect();
    let sum: u16 = PortScan { next_port: 502,
        end: 504 }.sum();
    println!("ports={:?} sum={}", ports, sum);
}

```

type `Item = u16` is an **associated type** — see Chapter 7. Adapters like `.map` and `.sum` use it to know what flows through the pipeline.

## 5.7.2 Non-empty line iterator

Skip blank lines while walking config text:

```

// Playground
struct NonEmptyLines<'a> {
    inner: std::str::Lines<'a>,
}

impl<'a> Iterator for NonEmptyLines<'a> {
    type Item = &'a str;

    fn next(&mut self) -> Option<Self::Item> {
        loop {
            let line = self.inner.next()?;
            let trimmed = line.trim();
            if !trimmed.is_empty() {
                return Some(trimmed);
            }
        }
    }
}

```

```

    }
  }
}

fn main() {
    let cfg = "PORT=502\n\nHOST=127.0.0.1\n";
    let keys: Vec<_> = NonEmptyLines { inner:
        cfg.lines() }
        .filter_map(|line|
            line.split('=').next())
        .collect();
    println!("{:?}", keys);
}

```

State lives in the struct fields. `next` returns `None` when the inner iterator is exhausted — that is how every iterator signals “done.”

### 5.7.3 Implementing Iterator edge cases

**Empty range — immediate `None`:**

```

// Playground
struct PortScan {
    next_port: u16,
    end: u16,
}

impl Iterator for PortScan {
    type Item = u16;

    fn next(&mut self) -> Option<Self::Item> {
        if self.next_port > self.end {
            return None;
        }
        let p = self.next_port;
        self.next_port += 1;
        Some(p)
    }
}

```

```

}

fn main() {
    let scan = PortScan { next_port: 502, end:
        501 };
    let n = scan.count();
    println!("{}", n); // 0
}

```

**Infinite range** — always pair with **.take(n)**:

```

// Playground
struct Counter {
    n: u64,
}

impl Iterator for Counter {
    type Item = u64;
    fn next(&mut self) -> Option<u64> {
        let v = self.n;
        self.n += 1;
        Some(v)
    }
}

fn main() {
    let first_five: Vec<_> = Counter { n: 0
    }.take(5).collect();
    println!("{}", first_five);
    // [0, 1, 2, 3, 4]
}

```

**IntoIterator vs Iterator:** collections implement `IntoIterator` so `for x in vec` works. Your type can implement both — `Iterator` for `.next()`, and `IntoIterator` with `type Item = Self`; `type IntoIter = Self` when consuming `self` in a `for` loop is natural.

## 5.8 When the compiler says no

Common errors in this chapter:

| Error message (typical)                    | Likely cause                              | Smallest fix                             |
|--------------------------------------------|-------------------------------------------|------------------------------------------|
| value used after move                      | for x in v or into_iter()<br>then v again | for x in &v or .iter()                   |
| cannot borrow as mutable<br>more than once | nested iter_mut                           | one mut borrow at a time;<br>restructure |
| type annotations needed for<br>collect     | ambiguous target collection               | let x: Vec<T> = ... or<br>turbofish      |
| expected T, found &T in<br>collect         | .iter() but collecting<br>owned values    | \&x  in map or use<br>into_iter()        |
| borrow of moved value                      | .into_iter().sum() then<br>reuse v        | clone input or use .iter()               |
| lifetimes / may not live long<br>enough    | Vec<&str> pointing into<br>dropped String | own the strings                          |

Read the **first** error top-down; iterator chains confuse the borrow checker, but the fix is usually “wrong walk mode (iter vs into\_iter)” or “references outlive owner.”

## 5.9 Java / Python contrast

| Aspect             | Java                              | Python                 | Rust                          |
|--------------------|-----------------------------------|------------------------|-------------------------------|
| Lazy pipeline      | Stream (often from<br>collection) | generator / itertools  | Iterator adapters             |
| Eager list comp    | —                                 | [f(x) for x in xs]     | .iter().map(f).collect()      |
| Consume collection | stream from list                  | for x in list          | into_iter() moves             |
| Index loop         | for (int i=0; ...)                | for i in<br>range(len) | prefer<br>.iter().enumerate() |

## 5.10 Idiom spotlight

**Prefer iterator chains over index loops.** Use `.iter()`, `.enumerate()`, or `.windows(2)` instead of `for i in 0..v.len()` with `v[i]`. Sliding windows: Chapter 11.

## 5.11 Go deeper

- [The Rust Book — Iterators](#)
- [Functional Rust — iterator topics](#)

## 5.12 See also

- Chapter 1: Ownership and borrowing — `iter` vs `into_iter`
- Chapter 2: Types and expressions — ranges, `for`, string iterators
- Chapter 5: Lifetimes — borrows inside iterator chains
- Chapter 7: Structs, traits, and generics — `type Item`, associated types
- Chapter 11: Collections — `Vec`, `HashMap`, `.windows`
- Chapter 12: Closures — `Fn` traits for adapters

### 5.12.1 Afterparty

#### 5.12.1.1 `for`, ranges, and three walk modes

1. **for desugar quiz** — “Four `for` loops (`0..n`, `&vec`, `vec`, `&mut vec`). I say which calls `iter`, `into_iter`, or `range`; you correct and show owner state after the loop.”
2. **iter vs into\_iter** — “Give 4 snippets using `Vec`; I predict whether `v` is usable after the loop; you explain `move` vs `borrow`.”
3. **iter\_mut drill** — “Task: double every element in `Vec<f64>` in place without allocating a new `Vec`. I write the loop; you review `*n` and `borrow` rules.”
4. **Range vs collect** — “When is `for i in 0..n` better than `(0..n).collect::<Vec<_>>()`? Give two automation examples (`retry count` vs `materializing indices`).”

#### 5.12.1.2 Adapters and lazy pipelines

5. **Adapter chain** — “Task: parse lines, trim, keep non-empty, parse as `u16`. I sketch `.lines().map(...).filter(...).collect()`; you refine.”
6. **zip pairs** — “Two `Vec<u16>`: register IDs and values. Build `Vec<(u16, u16)>` with `.zip()`; I write it; you handle length mismatch policy.”
7. **take and skip** — “Paginate a log: skip first 100 lines, take next 20. I use `.skip().take()` on `.lines()`; you show one pitfall if the source is not recomputed.”

8. **chain iterators** — “Concatenate header rows and body rows (two `&[u8]` slices) without copying bytes into one array first — sketch with `.iter().chain()`.”
9. **enumerate vs index** — “Same sum-over-evens task twice: index for vs `.enumerate()`. Compare readability and bounds-check risk.”
10. **Lazy vs eager** — “Explain when `filter().map()` allocates vs when `.collect()` forces work. One example with `println!` in `map` showing evaluation order.”

### 5.12.1.3 Consumers and types

11. **collect turbofish** — “Three `collect()` calls that fail without hints — I add type annotation or turbofish; you verify.”
12. **find and Option** — “Wire scan: `Vec<&str>` of lines, find first containing `ERROR`. I return `Option<&str>` with `.find()`; you contrast with `.filter().next()`.”
13. **fold vs sum** — “Compute max and count in one pass with `.fold()` vs calling `.max()` and `.len()` separately — when is fold worth it?”
14. **any and all** — “Validate a batch: all ports in `1..=65535`, any line starts with `#`. I write `.all()` / `.any()` predicates; you fix one double-reference mistake.”

### 5.12.1.4 Errors, ownership, and automation

15. **Moved Vec mistake** — “Show code that does `for x in v` then uses `v` again. I explain the error and fix with `.iter()` or `clone`; you rank fixes.”
16. **Borrow in chain** — “Snippet: build `Vec<&str>` from `String` lines then drop the `String`. I explain why it fails; you fix with owned `String` or different lifetime design.”
17. **Modbus-style scan** — “List of raw register values `Vec<u16>`: filter evens, map to `f64` scale 0.1, sum. I write the iterator chain; you check overflow and types.”
18. **Zero-cost check** — “Does `nums.iter().filter(...).map(...).sum()` allocate intermediate `Vecs`? Answer for `--release` and what to measure conceptually.”

### 5.12.1.5 Compile errors and edge cases

19. **Trap sheet drill** — “Give 6 snippets mixing `for x in v`, `for x in &v`, `into_iter`, and collect type errors. I predict compile ok or fail and why; you show the fixed line.”
20. **&&i32 decoder** — “One `.iter().filter().map()` chain: I label the closure parameter type at each step; you correct and show `|x|`, `|&x|`, and `|&&x|` versions that compile.”
21. **Empty iterator policy** — “Three tasks (`sum`, `find`, `all`) on possibly empty `Vec`. I state the result and whether it is a domain bug; you correct (e.g. `empty all` is true).”
22. **zip truncation** — “IDs `len 5`, values `len 100` — I write `zip collect`; you explain silent loss and sketch `zip + length check` or `enumerate` on the longer `vec`.”

### 5.12.1.6 Custom iterators

23. **PortScan impl** — “Implement `Iterator` for ports `502..=505`; collect to `Vec` and sum with `.sum()` — show type `Item` and `fn next` only.”
24. **Skip blanks** — “Config string with empty lines — write `NonEmptyLines` iterator that trims and skips `' '`; collect keys before `=`.”
25. **Infinite take** — “Counter from 0 without end — why must you `.take(n)` before `.collect()`? Show hang vs bounded collect.”
26. **IntoIterator pair** — “Same struct: implement `Iterator` and `IntoIterator` so both `scan.next()` and `for p in scan` work — minimal impl blocks.”
27. **Stateful parser** — “Byte buffer iterator yielding complete 4-byte frames; partial frame stays in struct — sketch `next()` state machine.”
28. **Capstone iterator** — “CSV line iterator: split fields, parse col 2 as `u16`, filter `> 0` — custom struct + one consumer chain.”

# Chapter 6

## Chapter 5: Lifetimes

### 6.1 Hook

In GC-managed languages (**Java**, **Python**, **C#**, ...), a collector keeps heap objects alive while any reference can reach them. You rarely ask whether a pointer is still valid.

Rust has no GC. Every `&T` must not outlive the value it borrows. **Lifetimes** are the compiler's way of proving that — usually without you writing any syntax.

### 6.2 Scope — a brief tour

Lifetimes are compile-time labels on references — not runtime values. This chapter covers elision, struct borrows, and `'static` traps. `HRTB`, `GATs`, and `Pin` are deferred.

| This chapter covers                               | Deferred                                      |
|---------------------------------------------------|-----------------------------------------------|
| <code>'a</code> on functions and structs, elision | Higher-ranked lifetimes ( <code>HRTB</code> ) |
| <code>'static</code> , owned vs borrowed fields   | <code>Pin</code> and self-referential structs |
| <code>T</code> : <code>'a</code> bounds preview   | Full struct chapter -> Ch 7                   |



## 6.3 References always have a lifetime

Every `&T` is valid for some span of code. If you return a reference to a local variable, that reference dies when the stack frame ends.

The compiler rejects that pattern.

```
// Playground --- this pattern is OK: reference
does not outlive owner
fn first_word(s: &str) -> &str {
    let bytes = s.as_bytes();
    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return &s[0..i];
        }
    }
    s
}

fn main() {
    let sentence = String::from("hello world");
    let word = first_word(&sentence);
    println!("{}", word);
}
```

## 6.4 What lifetimes are for

**Aim:** guarantee you never use a borrow after its owner is gone.

Chapter 1 said each `&T` must not outlive what it points to. That rule is easy inside one function.

It gets hard when you **return** a reference or **store** one in a struct. The caller cannot see your local variables, so the compiler needs a contract on the signature.

**Bad (won't compile):** return a pointer into memory that dies when the function returns.

```
// Playground --- uncomment to see the error
fn broken() -> &str {
    let s = String::from("tmp");
}
```

```

    &s
    // error: `s` dropped here; return would
    // dangle
}

```

**Good:** return a slice that still lives in the caller’s String:

```

fn first_word(s: &str) -> &str { /* ... */ }
// return borrows from `s`, not from locals

```

A **lifetime** is the span of code where a borrow is valid. You do not set it at runtime — the compiler checks it at compile time.

Lifetime syntax on functions and structs answers one question:

**“Which input (or owner) must still be alive while this reference exists?”**

### 6.4.1 How 'a helps (it’s just a label)

When several references in one signature must live **together**, give that span a name — 'a, 'b, 'input, whatever reads clearly.

Same idea as naming a generic type T: the name is for humans and the compiler. **'a is not magic.**

```

fn longest<'a>(x: &'a str, y: &'a str) -> &'a
    str {
    if x.len() >= y.len() { x } else { y }
}

```

Read this as: “The returned &str is valid while **both** x and y are valid.” Those borrows share one lifetime 'a.

If s1 is dropped while something still holds the return from longest (&s1, &s2), the program must not compile.

### 6.4.2 What if you omit <'a>?

The body does not change — you still return either x or y. The lifetime is **not** something you execute at runtime; it is a **signature** contract for the compiler. Try writing longest without any lifetime parameters:

```
fn longest(x: &str, y: &str) -> &str {
    // does not compile
    if x.len() >= y.len() { x } else { y }
}
```

Rust rejects this with **missing lifetime specifier** (or similar).

Reason: the return might point into **x** or **y**. Those borrows can come from **different owners** with **different lifetimes**. The compiler will not guess.

| With lifetimes                                                                 | Without lifetimes                                          |
|--------------------------------------------------------------------------------|------------------------------------------------------------|
| -> &'a str tied to x and y                                                     | Compiler does not know which input the return borrows from |
| Caller must keep <b>both</b> s1 and s2 alive while using the result            | No check that you drop the wrong String too early          |
| Elision cannot help: <b>two</b> reference inputs + <b>one</b> reference output | Signature is incomplete                                    |

Compare `first_word(s: &str) -> &str`: one borrowed input, one returned reference. Elision can assume “output lives as long as s.”

`longest` has **two** borrowed inputs, so elision stops. You must write 'a (or 'long, etc.) yourself.

```
// Playground
fn longest<'a>(x: &'a str, y: &'a str) -> &'a
    str {
    if x.len() >= y.len() { x } else { y }
}

fn main() {
    let s1 = String::from("long");
    let s2 = String::from("x");
    let r = longest(&s1, &s2);
    // drop s1;
    // error: r might still point into s1
    println!("{}", r);
}
```

Without `<'a>` on the function, this `main` could compile even when `r` still points at freed memory.

Lifetimes exist so that mistake is caught on the **function definition**, before any caller runs.

| Situation                                                                            | What you are telling the compiler                                                    |
|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <code>fn first&lt;'a&gt;(s: &amp;'a str) -&gt; &amp;'a str</code>                    | Return borrows from <code>s</code> only                                              |
| <code>fn longest&lt;'a&gt;(x: &amp;'a str, y: &amp;'a str) -&gt; &amp;'a str</code>  | Return may point into <code>x</code> <b>or</b> <code>y</code> ; both must stay alive |
| <code>fn pick&lt;'a, 'b&gt;(x: &amp;'a str, y: &amp;'b str) -&gt; &amp;'a str</code> | Return borrows from <code>x</code> only; <code>y</code> can die earlier              |

The only lifetime name with a **fixed** meaning is **'static**: valid for the whole program (e.g. string literals).

Do not slap **'static** on normal borrows to silence errors.

**Mental model:** owner = suitcase, &T = claim ticket. Lifetimes prove you never read the ticket after the suitcase was thrown away.

## 6.5 Elision (why you rarely write lifetimes)

Often the compiler infers the contract above — **lifetime elision**. You write the short form; Rust fills in **'a** for you:

```
// Playground --- elision expansion (signatures
    only; not runnable alone)
// fn len(s: &str) -> usize
//
// means: fn len<'a>(s: &'a str) -> usize ---
// no return reference, so no tie needed
//
// fn first_word(s: &str) -> &str
//
// means: return borrows from `s` (elision ties
// input and output)
```

Three **elision rules** cover most everyday signatures (the compiler applies them automatically):

1. Each reference parameter gets its own lifetime parameter (`&str -> &'a str, ...`).

2. If there is exactly **one** input lifetime, it is assigned to **all** output lifetimes (fn foo<'a>(x: &'a str) -> &'a str).
3. If there are multiple input lifetimes but one is **&self** or **&mut self**, the lifetime of **self** is assigned to all output lifetimes (methods only).

When elision fails — several references in and out, or an ambiguous return — write 'a / 'b explicitly. Any name works.

```
fn longest<'long>(x: &'long str, y: &'long str)
    -> &'long str {
    if x.len() >= y.len() { x } else { y }
}
```

## 6.6 Named lifetimes on structs

You have not met **structs** in depth yet (Chapter 7 covers `struct`, `impl`, and `traits`). For lifetimes, you only need this much:

A **struct** groups named fields into one type — like a labeled tuple or a small Java/Python data class:

```
struct Point {
    x: f64,
    y: f64,
}
```

When a field is a **borrow** (`&str`, `&[u8]`, ...) instead of an owned value (`String`, `Vec`), the struct does not own that memory. It only holds a pointer into someone else's buffer.

The same question applies as for `longest`:

**“Which owner must still be alive while this struct exists?”**

Put 'a on the **struct** (not just on a function) when a field stores a reference:

```
// Playground
struct Excerpt<'a> {
    text: &'a str,
    // borrows from outside; struct does not
    // own the bytes
}
```

```
fn main() {
    let novel = String::from("Call me Ishmael.
        Once upon a time...");
    let first =
        novel.split('.').next().unwrap_or("");
    let e = Excerpt { text: first };
    println!("{}", e.text);
}
// `e` and `first` dropped here; then `novel`
// can be dropped
```

Read `Excerpt<'a>` like `longest<'a>`: **'a is the lifetime of the borrow in text.** The struct may not outlive the data `text` points into — here, `novel`.

| Piece                             | Role                                                                               |
|-----------------------------------|------------------------------------------------------------------------------------|
| <code>novel: String</code>        | <b>Owner</b> of the heap text                                                      |
| <code>first: &amp;str</code>      | Borrow into <code>novel</code>                                                     |
| <code>e: Excerpt&lt;'a&gt;</code> | Bundles that borrow; 'a ties <code>e.text</code> to <code>novel</code> 's lifetime |

### 6.6.1 What if the struct omits `<'a>`?

```
struct Excerpt {    // does not compile
    text: &str,
}
```

Same idea as `fn longest(x: &str, y: &str) -> &str` without lifetimes. The compiler sees a reference inside the type but **no contract** for how long it is valid.

Error: **missing lifetime specifier** on `text`.

**Practical rule:** struct fields that are references almost always need a lifetime parameter on the struct. Fields that are owned (`String`, `u32`, `Vec<...>`) do not.

```
// Playground --- uncomment `drop` to see the
    error
fn main() {
    let novel = String::from("Call me
```

```

    Ishmael.");
    let first =
        novel.split('.').next().unwrap_or("");
    let e = Excerpt { text: first };
    // drop(novel);
    // error: `e.text` still borrows `novel`
    println!("{}", e.text);
}

```

Prefer **owned** fields in public APIs (`String` instead of `&str`) until you need zero-copy views.

Methods on structs (`impl Excerpt { ... }`) come in Chapter 7.

## 6.7 'static trap — formatted strings

**Wrong — reference to temporary:**

```

// Playground --- does not compile
fn label() -> &'static str {
    let s = format!("sensor-{}", 1);
    &s // ERROR: `s` dropped at end of function
}

```

**Right — return owned:**

```

// Playground
fn label() -> String {
    format!("sensor-{}", 1)
}

fn main() {
    println!("{}", label());
}

```

Only string literals and leaked allocations are `'static`. `format!` always produces an owned `String`.

## 6.8 Two lifetimes — when 'a and 'b diverge

Return borrows from **one** input only:

```
// Playground
fn first<'a, 'b>(x: &'a str, _y: &'b str) ->
    &'a str {
    x
}

fn main() {
    let a = String::from("left");
    let b = String::from("right");
    let r = first(&a, &b);
    drop(b);
    println!("{}", r);
    // OK --- r only borrows `a`
}
```

If the return could point into either argument, both inputs share one 'a like longest.

## 6.9 Config<'a> — borrowed slices vs owned fix

```
// Playground
struct Config<'a> {
    host: &'a str,
    port: u16,
}

fn parse_line<'a>(line: &'a str) ->
    Option<Config<'a>> {
    let mut parts = line.split(':');
    let host = parts.next()?;
    let port: u16 = parts.next()?.parse().ok()?;
    Some(Config { host, port })
}

fn main() {
```



```

let line = String::from("127.0.0.1:502");
if let Some(cfg) = parse_line(&line) {
    println!("{}", cfg.host, cfg.port);
}
}

```

Public APIs often use owned `String` fields instead — callers need not keep the original buffer alive.

### 6.9.1 Lifetime edge cases

**T: 'a** — generic container holding a borrow:

```

// Playground
struct Holder<'a, T: 'a> {
    value: &'a T,
}

fn main() {
    let n = 502u16;
    let h = Holder { value: &n };
    println!("{}", h.value);
}

```

**T: 'a** means “T must live at least as long as 'a.”

## 6.10 When the compiler says no

Common errors in this chapter:

Typical fixes:

- Return an **owned** `String` instead of `&str`.
- Take owned data as parameter and return owned data.
- Use `Rc`/`Arc` when shared ownership is real (Chapter 10).

## 6.11 Java / Python contrast

| Aspect                  | Java              | Python                | Rust                   |
|-------------------------|-------------------|-----------------------|------------------------|
| dangling pointer        | rare (GC + JIT)   | possible C extensions | compile error          |
| return inner ref        | object on heap OK | OK if object lives    | must tie lifetimes     |
| self-referential struct | awkward           | awkward               | needs<br>pins/advanced |

## 6.12 Idiom spotlight

**Return owned types from public APIs** unless you are building a zero-copy internal API and can document lifetime ties.

String and Vec are easy. &str in returns often fights the borrow checker for newcomers.

## 6.13 Playground: two inputs, one shared lifetime

When the return must be valid for **either** argument, both references share one lifetime parameter. Any name works — here 'a.

```
// Playground
fn longest<'a>(x: &'a str, y: &'a str) -> &'a
    str {
    if x.len() >= y.len() { x } else { y }
}

fn main() {
    let s1 = String::from("longer");
    let s2 = String::from("x");
    println!("{}", longest(&s1, &s2));
}
```

## 6.14 Go deeper

- [The Rust Book — Validating References with Lifetimes](#)
- [Functional Rust — pattern matching & types](#)

## 6.15 See also

- Chapter 1: Ownership and borrowing
- Chapter 4: Iterators — `.iter()` borrows reinforce lifetime thinking
- Chapter 7: Structs, traits, and generics — full `struct / impl` coverage (this chapter only needs borrowed fields + `'a`)

### 6.15.1 Afterparty

1. **Error archaeology** — “I paste a ‘lifetime may not live long enough’ error; walk me through owner vs reference diagram.”
2. **Return type choice** — “For API `fn title(book: &Book) -> ???` compare `&str` vs `String` trade-offs for a library.”
3. **Struct lifetime** — “Design `ConfigParser` holding `&str` slices into input buffer — when is it sound vs use owned `String`?”
4. **Elision quiz** — “Add explicit lifetimes to 4 function signatures where elision fails.”
5. **Fix mine** — “I return `&String` built inside function; show three idiomatic fixes ranked by simplicity.”

#### 6.15.1.1 Lifetimes in practice

6. **static trap** — “Function returns `&str` from `format!` — show error and owned fix.”
7. **two lifetimes** — “Write `fn first<'a, 'b>(x: &'a str, y: &'b str) -> &'a str` — drop `y` while result lives.”
8. **Config struct** — “Parse `host:port` into `Config<'a>` — when must caller keep line alive?”
9. **Owned refactor** — “Same parser returning owned `Config { host: String, port: u16 }` — tradeoffs in 3 bullets.”
10. **T: 'a bound** — “Explain `struct Holder<'a, T: 'a> { value: &'a T }` — what fails if `T` is shorter-lived?”
11. **Elision fail** — “Four signatures where elision works vs fails — I label each.”
12. **Iterator borrow** — “Collect `Vec<&str>` from `String` lines — why drop order matters (link Ch 4).”

# Chapter 7

## Chapter 6: Enums and Pattern Matching

### 7.1 Hook

**Java** uses `null` for “missing” and often throws exceptions for failures. **Python** uses `None` and exceptions, or informal unions (`int | str`) without the type checker enforcing branches. If you know other languages, map their “missing value” and error habits to the table below.

**Rust** encodes “one of several possibilities” in the **type system**:

| Idea                  | Java              | Python              | Rust                                                     |
|-----------------------|-------------------|---------------------|----------------------------------------------------------|
| Missing value         | <code>null</code> | <code>None</code>   | <code>Option&lt;T&gt;</code>                             |
| Failure               | exception         | exception           | <code>Result&lt;T, E&gt;</code>                          |
| Several shapes        | class hierarchy   | duck typing / Union | <code>enum + match</code>                                |
| Must handle all cases | runtime surprise  | runtime surprise    | <b>compile error</b> if <code>match</code> is incomplete |

`match` is not a fancy `switch`. It **destructures** data, and the compiler insists you cover every variant. That is how Rust replaces `null` checks and many runtime type errors with reviewable compile-time rules (Chapter 1).

Chapter 2 previewed `match` on integers. This chapter applies the same machinery to `Option`, `Result`, and your own `enums`.

## 7.2 Option<T> — no null

Option is an enum with two variants: Some(T) or None.

```
// Playground
fn divide(a: f64, b: f64) -> Option<f64> {
    if b == 0.0 { None } else { Some(a / b) }
}

fn main() {
    match divide(10.0, 2.0) {
        Some(v) => println!("{}", v),
        None => println!("undefined"),
    }
}
```

**Callers must decide** what None means — log, default, propagate, or abort. Rust will not let you treat Option like a plain T without handling None (unless you explicitly force it — see edge cases below).

### 7.2.1 if let, while let, and let ... else

When only **one** variant matters:

```
// Playground
fn main() {
    let maybe_port = Some(502u16);
    if let Some(p) = maybe_port {
        println!("port {}", p);
    }

    let raw = "8080";
    let Ok(n) = raw.parse:::<u16>() else {
        println!("bad port");
        return;
    };
    println!("parsed {}", n);
}
```

let ... else (Rust 1.65+) is idiomatic for “parse or bail early” without nested match.

7.2.2 Option combinators (before heavy match)

| Method                           | Use when                                                                    |
|----------------------------------|-----------------------------------------------------------------------------|
| <code>.map(f)</code>             | transform inner value if <code>Some</code>                                  |
| <code>.and_then(f)</code>        | chain fallible steps ( <code>f</code> returns <code>Option</code> )         |
| <code>.unwrap_or(default)</code> | supply fallback                                                             |
| <code>.ok_or(err)</code>         | turn <code>None</code> into <code>Err</code> for <code>Result</code> chains |

```
// Playground
fn main() {
    let s = "502";
    let port = s.trim().parse:::<u16>().ok();
    // Option<u16>
    let doubled = port.map(|p| p * 2);
    println!("{:?}", doubled);
}
```

Full error typing for `Result` chains: Chapter 8.

7.2.3 Option edge cases and compiler traps

Wrong — treat `Option` like nullable Java without branching:

```
// Playground --- does not compile
fn main() {
    let maybe: Option<i32> = Some(5);
    // let x: i32 = maybe;
    // ERROR: expected i32, found Option<i32>
    let x = maybe.unwrap();
    // compiles --- panics on None at runtime
}
```

| Approach                                                | None behavior               | Idiom                                         |
|---------------------------------------------------------|-----------------------------|-----------------------------------------------|
| <code>match</code> / <code>if let</code>                | you handle it               | <b>preferred</b>                              |
| <code>unwrap()</code> / <code>expect("...")</code>      | <b>panic</b>                | prototypes, tests,<br>“impossible” invariants |
| <code>unwrap_or</code> /<br><code>unwrap_or_else</code> | default                     | config with safe fallback                     |
| <code>? in fn -&gt; Option</code>                       | propagate <code>None</code> | chaining parsers                              |

**Wrong — unwrap in production paths:**

```
// Playground --- runs, then may panic in
production
fn read_port(s: &str) -> u16 {
    s.parse().ok().unwrap()
    // panic if config line is wrong
}
```

Prefer `match`, `if let`, or `Result` with `?` so a bad config file returns an error instead of crashing a long-running service.

**Exhaustive match on Option:**

```
// Playground --- does not compile if you omit
an arm
fn main() {
    let x = Some(1);
    match x {
        Some(n) => println!("{}", n),
        // None => {}
        // ERROR: non-exhaustive patterns:
        // `None` not covered
    }
}
```

## 7.3 Result<T, E> — errors as values

`Result` is also a two-variant enum: `Ok(T)` or `Err(E)`.

```
// Playground
fn parse_positive(s: &str) -> Result<i32,
    &'static str> {
    let n: i32 = s.parse().map_err(|_| "not a
        number")?;
    if n > 0 { Ok(n) } else { Err("not
        positive") }
}

fn main() {
    println!("{}", parse_positive("42"));
```

```
println!("{:?}", parse_positive("-1"));
println!("{:?}", parse_positive("oops"));
}
```

The `?` operator propagates `Err` early — like `throw`, but **typed** and visible in the signature. Custom error types, **thiserror**, and errors as enums: Chapter 8.

### 7.3.1 Result edge cases

**Wrong — ignore the error arm:**

```
// Playground --- does not compile
fn main() {
    let r: Result<i32, &str> = Ok(1);
    match r {
        Ok(n) => println!("{}", n),
        // Err(e) => ...
        // ERROR: non-exhaustive patterns
    }
}
```

**Wrong — unwrap on Result in library code:**

```
// Playground
fn load_timeout(s: &str) -> u32 {
    s.parse().unwrap()
    // panic on "abc" --- use ? or match instead
}
```

**Idiomatic boundary pattern:**

```
// Playground
fn parse_port(s: &str) -> Result<u16, String> {
    s.trim().parse::<u16>().map_err(|e|
        format!("port: {}", e))
}

fn main() {
    match parse_port("502") {
        Ok(p) => println!("ok {}", p),
    }
}
```



```
    Err(msg) => eprintln!("config error:
        {}", msg),
    }
}
```

At **boundaries** (main, HTTP handler, CLI), match on `Result` and map to user-facing messages. **Inside** helpers, prefer `?` to bubble `Err` up.

## 7.4 Custom enums — sum types

Your own enum lists allowed states or message kinds. Each variant can be a **unit**, **tuple**, or **struct** shape.

```
// Playground
enum Status {
    Idle,
    Running { rpm: u32 },
    Fault { code: u8 },
}

fn describe(s: &Status) -> String {
    match s {
        Status::Idle => "idle".into(),
        Status::Running { rpm } =>
            format!("running at {}", rpm),
        Status::Fault { code } =>
            format!("fault {}", code),
    }
}

fn main() {
    let s = Status::Running { rpm: 1500 };
    println!("{}", describe(&s));
}
```

### 7.4.1 Enum shapes (pattern matching preview)

| Variant syntax | Example        | Pattern in match       |
|----------------|----------------|------------------------|
| Unit           | Idle           | Status::Idle           |
| Tuple          | Ping(u32)      | Message::Ping(ms)      |
| Struct         | Fault { code } | Status::Fault { code } |

```
// Playground
enum Frame {
    Heartbeat,
    Data { len: u16, payload: [u8; 4] },
}

fn opcode(f: &Frame) -> u8 {
    match f {
        Frame::Heartbeat => 0x00,
        Frame::Data { len, .. } => {
            if *len > 0 { 0x01 } else { 0xFF }
        }
    }
}

fn main() {
    let f = Frame::Data { len: 2, payload: [1,
        2, 0, 0] };
    println!("op={:02X}", opcode(&f));
}
```

.. in a pattern ignores fields you do not need (Chapter 2 ranges use the same token for a different feature — context disambiguates).

## 7.5 Pattern matching mechanics

### 7.5.1 Exhaustive match

For an enum, **every variant** needs an arm (or a catch-all that is intentionally broad).

```
// Playground
enum Mode { Auto, Manual }
```

```
fn label(m: Mode) -> &'static str {
    match m {
        Mode::Auto => "auto",
        Mode::Manual => "manual",
    }
}

fn main() {
    println!("{}", label(Mode::Auto));
}
```

### 7.5.1.1 Why -> &'static str?

The return type is a **borrowed string slice** (&str), not an owned String. Each match arm returns a **string literal** — "auto", "manual" — which is stored in the program binary and lives for the **entire run**. Rust types that as &'static str: a reference valid for the **'static** lifetime (see Chapter 5 — Lifetimes).

| Return expression               | Lifetime meaning          | Who must stay alive?             |
|---------------------------------|---------------------------|----------------------------------|
| "auto" (literal)                | 'static                   | nobody — bytes are in the binary |
| s.as_str() from caller's String | usually 'a tied to s      | the owner String                 |
| format!(...)                    | cannot return &str at all | need owned String                |

So label does **not** borrow from m or from any local variable. It only returns pointers to immortal literals. That is why the signature can be &'static str without a generic 'a parameter.

### Wrong — return &str pointing at a local String:

```
// Playground --- does not compile
fn bad_label(m: Mode) -> &str {
    let s = format!("{:?}", m);
    // owned String on stack
    s.as_str()
    // ERROR: `s` does not live long enough ---
```

```
// returned ref would dangle
}
```

**Idiomatic alternatives** when text is built at run time:

```
// Playground
enum Mode { Auto, Manual }

fn label_owned(m: Mode) -> String {
    match m {
        Mode::Auto => "auto".to_string(),
        Mode::Manual => "manual".to_string(),
    }
}

fn main() {
    println!("{}", label_owned(Mode::Auto));
}
```

Use `&'static str` for fixed catalog strings; use `String` (or `&str` with an explicit lifetime tied to an input) when the text depends on borrowed or formatted data — Chapter 5 covers the contract, Chapter 2 covers `String` vs `&str`.

**Add a variant — compiler forces updates:**

```
// Playground --- does not compile until you
add `Mode::Safe => ...`
enum Mode { Auto, Manual, Safe }

fn label(m: Mode) -> &'static str {
    match m {
        Mode::Auto => "auto",
        Mode::Manual => "manual",
    }
    // ERROR: non-exhaustive patterns: `Safe`
    // not covered
}
```

After you add `Mode::Safe => "safe"`, the match compiles again — that is the safety net when protocols or state machines grow.

That is a **feature**: refactors cannot silently forget a new PLC mode or protocol opcode.

## 7.5.2 Wildcard `_` — power and footgun

On integers, `_` means “everything else.” On enums, `_` means “any variant I did not list”:

```
// Playground
fn severity(code: u16) -> &'static str {
    match code {
        0..=99 => "info",
        _ => "unknown",
        // catches 100+ --- fine for integers
    }
}
```

**Footgun on enums:** if you write `_` instead of listing variants, **adding a new variant will not force you to update this match**. The compiler assumes you meant to lump new cases into `_`. Prefer explicit arms for domain enums you control. Reserve `_` for “truly don’t care” cases or integer ranges.

## 7.5.3 `match` on references — borrow vs move

```
// Playground
enum Status {
    Idle,
    Running { rpm: u32 },
    Fault { code: u8 },
}

fn describe(s: &Status) -> String {
    match s {
        Status::Idle => "idle".into(),
        Status::Running { rpm } =>
            format!("running at {}", rpm),
        Status::Fault { code } =>
            format!("fault {}", code),
    }
}
```

```

}

fn main() {
    let s = Status::Running { rpm: 100 };
    match &s {
        Status::Running { rpm } =>
            println!("rpm {}", rpm),
        _ => println!("other"),
    }
    println!("still own s: {}", describe(&s));
}

```

match &s borrows; match s **moves** s into the match (often wrong unless you intentionally consume).

**Wrong — move field out of non-Copy enum:**

```

// Playground --- does not compile
enum Packet { Text(String) }

fn main() {
    let p = Packet::Text(String::from("hi"));
    match p {
        Packet::Text(s) => println!("{}", s),
        // moves s out of p
    }
    // use p again
    // ERROR: use of partially moved value: `p`
}

```

Use ref in the pattern to borrow inside fields: `Packet::Text(ref s)` or `match &p`.

## 7.5.4 Match guards

Add a boolean condition on an arm:

```

// Playground
fn classify(v: i32) -> &'static str {
    match v {
        n if n < 0 => "negative",
    }
}

```

```
    0 => "zero",
    n if n > 100 => "high",
    _ => "normal",
}

fn main() {
    println!("{}", classify(150));
}
```

Guards do not replace exhaustiveness — you still need arms that cover all cases (often with `_`).

7.5.5 if let vs match

| Use if let / while let                          | Use match                              |
|-------------------------------------------------|----------------------------------------|
| one variant matters                             | several variants or mixed handling     |
| quick peel <code>Some / Ok</code>               | equal weight per branch                |
| <code>while let Some(line) = iter.next()</code> | full <code>Result + Err</code> mapping |

Long chains of `if let` on the same value are a smell — switch to `match` for clarity.

7.6 Java / Python contrast (deeper)

| Pitfall               | Java / Python habit                          | Rust idiom                                           |
|-----------------------|----------------------------------------------|------------------------------------------------------|
| Forgot null check     | NPE / <code>AttributeError</code> at runtime | <code>Option + match / ?</code>                      |
| Swallowed exception   | empty catch                                  | <code>match on Err</code> or explicit <code>?</code> |
| Stringly-typed states | "IDLE", "RUN"                                | <code>enum Mode { Idle, Run }</code>                 |
| Unchecked union       | <code>isinstance</code> ladders              | one <code>match</code> on <code>enum</code>          |

**Python match (3.10+)** looks like Rust but matches object structure; Rust `match` ties to **algebraic types** and ownership.

## 7.7 Service-shaped example

```
// Playground
enum ReadOutcome {
    Value(u16),
    Timeout,
    CrcFault,
}

fn handle(r: ReadOutcome) {
    match r {
        ReadOutcome::Value(v) =>
            println!("register = {}", v),
        ReadOutcome::Timeout =>
            eprintln!("retry"),
        ReadOutcome::CrcFault => eprintln!("bus
            fault"),
    }
}

fn main() {
    handle(ReadOutcome::Value(42));
    handle(ReadOutcome::CrcFault);
}
```

Adding `ReadOutcome::Disconnected` later without updating `handle` -> **compile error**, not a silent log gap.

## 7.8 Advanced patterns

Patterns for parsers and config loaders — same domain as the service example above.

### 7.8.1 Slice patterns — split a command line

Parse "GET /path HTTP/1.1" into verb and path without index math:

```
// Playground
fn parse_request(line: &str) -> Option<(&str,
```



```

    &str, &str)> {
    match
      line.split_whitespace().collect::

```

For paths with spaces, collect into an owned String or match [method, rest @ .., version] and join rest into a local String before returning.

## 7.8.2 matches! — enum check without full match

Before — verbose match:

```

// Playground
#[derive(Debug)]
enum Mode { Auto, Manual, Off }

fn is_active(mode: &Mode) -> bool {
    match mode {
        Mode::Auto | Mode::Manual => true,
        Mode::Off => false,
    }
}

```

After — matches! macro:

```

// Playground
#[derive(Debug)]

```

```
enum Mode { Auto, Manual, Off }

fn is_active(mode: &Mode) -> bool {
    matches!(mode, Mode::Auto | Mode::Manual)
}

fn main() {
    println!("{}", is_active(&Mode::Auto));
}
```

Use `matches!` for boolean guards. Keep full match when arms return different values.

### 7.8.3 if let chains — bind host and port together

Parse `host:port` from a config line:

```
// Playground
fn parse_endpoint(line: &str) -> Option<(&str,
    u16)> {
    let mut parts = line.split(':');
    let host = parts.next()?.trim();
    if host.is_empty() {
        return None;
    }
    let port_str = parts.next()?;
    if parts.next().is_some() {
        return None; // reject host:port:extra
    }
    let port: u16 = port_str.parse().ok()?;
    if port == 0 {
        return None;
    }
    Some((host, port))
}

fn main() {
    println!("{}",
        parse_endpoint("127.0.0.1:502"));
}
```

```
}
```

Each condition must pass for the binding to succeed — good for multi-step parsing without nested match.

### 7.8.4 @ bindings — match and bind in one arm

Accept only valid port numbers while keeping the value:

```
// Playground
fn classify_port(p: u16) -> &'static str {
    match p {
        n @ 1..=1023 => "system",
        n @ 1024..=49151 => "registered",
        _ => "other",
    }
}

fn main() {
    println!("{}", classify_port(502));
}
```

### 7.8.5 Advanced pattern edge cases

Wrong — fixed-length slice on variable input:

```
// Playground --- logic bug, not compile error
fn main() {
    let words: Vec<&str> = vec!["GET", "/only"];
    match words.as_slice() {
        [method, path] => println!("{}", method, path),
        _ => println!("bad request"),
    }
}
```

Three or one token hits `_` — use `[method, path @ ..]` or check length explicitly.

Empty slice:

```
// Playground
fn main() {
    let empty: &[&str] = &[];
    match empty {
        [] => println!("empty"),
        [first, ..] => println!("first={}",
                                first),
    }
}
```

## 7.9 When the compiler says no

Common errors in this chapter:

| Error (typical)                           | Cause                                  | Fix                                |
|-------------------------------------------|----------------------------------------|------------------------------------|
| non-exhaustive patterns                   | missing enum variant or None/Err       | add arm or intentional _           |
| expected T, found Option<T>               | treated Option as value                | match / if let / ?                 |
| use of partially moved value              | field moved out in match               | match &e, ref in pattern, or clone |
| cannot move out of ... behind a reference | match on & but pattern moves           | ref / ref mut in pattern           |
| unreachable pattern                       | arm duplicated or shadowed by _ above  | reorder arms; remove redundant _   |
| ? in fn without Result return             | ? only propagates in Result/Option fns | change return type or use match    |

## 7.10 Idiom spotlight

**match on Result at boundaries, ? inside.** At main or API edge, convert to user-facing messages; inside libraries, propagate with ?.

**Prefer explicit enum variants over strings** for device and protocol states. Let the compiler remind you when the protocol adds a code.

**Avoid unwrap in long-running services** — use Option/Result so bad input is data, not a panic.

## 7.11 Go deeper

- [The Rust Book — Enums](#)
- [The Rust Book — Patterns and Matching](#)
- [Option basics](#) — examples 041–044
- [Result basics](#) — examples 045–048

## 7.12 See also

- Chapter 2: Types — integer match, if expressions, String vs &str
- Chapter 5: Lifetimes — 'static, returning &str, why bad\_label fails
- Chapter 4: Iterators — find returns Option
- Chapter 8: Errors — ?, custom Error, testing Result
- Chapter 7: Traits — traits on enums, impl blocks
- Chapter 17: Metaprogramming — matches! macro

### 7.12.1 Afterparty

#### 7.12.1.1 Option and null habits

1. **Null replacement** — “Translate 5 Java methods returning null into Option Rust; explain callsite changes.”
2. **unwrap audit** — “Paste 20 lines with 4 unwrap() calls; I mark panic risk; you rewrite with match or ?.”
3. **Combinator chain** — “Parse port Option<u16>, double if Some, default 502 — I write map/unwrap\_or; you add and\_then version.”
4. **let-else port** — “Rewrite nested match on parse() into let Ok(x) = ... else { ... }; preserve behavior.”

#### 7.12.1.2 Result and propagation

5. **Result railway** — “Chain parse -> validate -> compute with ?; I fill blanks, you verify.”
6. **Err arm missing** — “Show match on Result without Err arm; I quote error and fix; add boundary eprintln! pattern.”
7. **unwrap vs ?** — “Same parser twice: unwrap vs fn -> Result with ?; compare panic risk and signature honesty.”

**7.12.1.3 Enums and exhaustive match**

8. **Exhaustive match** — “Enum with 4 variants; I write match; you add variant Safe and show compile error until I fix.”
9. **Wildcard footgun** — “Explain why `_` on your own enum hides new variants; show explicit arms vs `_` refactor story.”
10. **Partial move** — “enum with String field: match moves field; I fix with `ref` or `match &`; you show error text.”
11. **State machine** — “TCP/serial states as enum; `connect/send/close` return `Result<(), IllegalTransition>`.”
12. **Python Union** — “Python `int | str` parameter -> Rust enum + match; no dyn.”

**7.12.1.4 Patterns and style**

13. **if let vs match** — “Three tasks: one variant, two variants, five variants — I pick `if let` or `match` each time.”
14. **Match on ref** — “Given owned Status, I choose `match s` vs `match &s`; predict move errors; you diagram ownership.”
15. **Guard drill** — “Classify i32 with guards (`<0`, `==0`, `>100`); I write arms; you check exhaustiveness.”
16. **Opcode table** — “Design enum for 3 frame types + match returning u8 opcode; add fourth type as compile-break exercise.”

**7.12.1.5 Errors and automation**

17. **ReadOutcome extend** — “Add Disconnected to automation `ReadOutcome`; show all match sites compiler lists.”
18. **Config sentinel** — “Rewrite `fn port() -> i32` returning -1 on failure to `Option<u16>` + match in main.”
19. **Checklist drill** — “Match 6 Chapter 6 compiler errors to snippets; I name fix (match arm, ref, ?, return type).”
20. **Java enum map** — “Java enum `State { IDLE, RUN }` with method — port to Rust enum + match + impl; contrast nullability.”

**7.12.1.6 Advanced patterns**

17. **Slice split** — “Parse `POST /api/v1/run HTTP/1.1` with [method, path @ .., \_ver] — I write; you handle single-token input.”
18. **matches refactor** — “Replace 6-arm match that returns bool with `matches!` — show before/after on Mode enum.”

19. **if let chain** — “Parse `host:port:extra` — chain should reject extra segments; fix my broken parser.”
20. **@ binding quiz** — “Three arms with @ ranges for port classes — I label which values hit which arm.”
21. **Exhaustive slice** — “[a, b] on Vec of len 1 or 3 — predict \_ arm vs bug; suggest .. rest pattern.”
22. **matches vs match** — “When must you keep full match instead of matches!? Give one example returning String.”
23. **\*\*Guard + @\*\*** — “Match port n @ 1024..`65535` with guard `n % 2 == 0` — sketch arm.”
24. **Capstone parse** — “Frame header [sync, len @ 1..`255`, payload @ ..] from byte slice — sketch match arms only.”

## Chapter 8

# Chapter 7: Structs, Traits, and Generics

### 8.1 Hook

Rust uses **structs** for data, `impl` for methods, **traits** for shared behaviour, and **enums** for closed alternatives. **Java** bundles data with inheritance; **Python** relies on duck typing (“if it quacks...”) — optional comparison points. Rust favors composition over inheritance, all checked at compile time.

### 8.2 Structs and methods

```
// Playground
struct Sensor {
    id: u32,
    value: f64,
}

impl Sensor {
    fn new(id: u32, value: f64) -> Self {
        Self { id, value }
    }
}
```



```

fn scaled(&self, factor: f64) -> f64 {
    self.value * factor
}

fn main() {
    let s = Sensor::new(1, 25.0);
    println!("{}", s.scaled(0.5));
}

```

## 8.3 Traits — interfaces done right

If you know **Java** or **Python**, a **trait** names a **capability** any type can opt into — shared behaviour **without inheritance**, checked at compile time. The table below maps the habit; then the examples.

### Why traits are a win

| Benefit               | Java / Python pain                                             | Rust trait answer                                                                           |
|-----------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Checked contracts     | interface optional; duck typing fails at runtime               | missing method = <b>compile error</b>                                                       |
| No hierarchy tax      | deep trees, fragile super chains                               | flat types + <code>impl</code> blocks                                                       |
| Cross-crate reuse     | can't add methods to <code>String</code> / third-party classes | implement <b>your</b> trait for <b>your</b> wrapper (orphan rule applies)                   |
| Performance           | interface dispatch / dynamic checks                            | <b>static dispatch</b> by default ( <code>impl Trait</code> -> monomorphization, no vtable) |
| Optional polymorphism | always reference types / ABCs                                  | <code>dyn Trait</code> only when you need mixed-type collections                            |

You will use traits everywhere: `Display`, `Clone`, `Iterator`, custom `Measurable` / `Summary` in automation code. Start with `impl Trait` for `MyType` and `fn f(x: &impl Trait)`; reach for `dyn Trait` later when types truly differ at runtime (see below).

```
// Playground
trait Summary {
    fn summarize(&self) -> String;
}

struct Reading { v: f64 }

impl Summary for Reading {
    fn summarize(&self) -> String {
        format!("reading: {}", self.v)
    }
}

fn print_summary(item: &impl Summary) {
    println!("{}", item.summarize());
}

fn main() {
    let r = Reading { v: 3.14 };
    print_summary(&r);
}
```

| Java                      | Python            | Rust                 |
|---------------------------|-------------------|----------------------|
| interface                 | informal protocol | trait                |
| implements                | “has method”      | impl Trait for Type  |
| default interface methods | mixin / ABC       | trait default bodies |

## 8.4 Enums, structs, and traits together

Chapter 6 gave you `enum` + exhaustive match. This section covers **how structs and traits attach to enums** — Rust’s substitute for a class hierarchy or a Python Union of unrelated types.

The idiomatic shape for automation and protocol code:

1. `enum` — closed set of states or message kinds (compiler tracks every variant).
2. `struct` — per-variant payload when a variant carries real data.

3. `trait` — shared behaviour across *different* types (`Reading`, `Alarm`, ...), or a uniform API over one enum.
4. `impl` — inherent methods on the enum *and/or* trait implementations that match inside.

```
// Playground
trait Measurable {
    fn value(&self) -> f64;
    fn unit(&self) -> &'static str;
}

struct Temperature {
    celsius: f64,
}

struct Pressure {
    bar: f64,
}

enum SensorReading {
    Temp(Temperature),
    Press(Pressure),
    Skipped { reason: String },
}

impl Measurable for SensorReading {
    fn value(&self) -> f64 {
        match self {
            SensorReading::Temp(t) => t.celsius,
            SensorReading::Press(p) => p.bar,
            SensorReading::Skipped { .. } =>
                f64::NAN,
        }
    }

    fn unit(&self) -> &'static str {
        match self {
            SensorReading::Temp(_) => "°C",
            SensorReading::Press(_) => "bar",
```

```

        SensorReading::Skipped { .. } =>
            "n/a",
    }
}

impl SensorReading {
    fn is_valid(&self) -> bool {
        !matches!(self, SensorReading::Skipped
            { .. })
    }
}

fn log(item: &impl Measurable) {
    println!("{}", item.value(),
        item.unit());
}

fn main() {
    let r = SensorReading::Temp(Temperature {
        celsius: 21.5 });
    log(&r);
    println!("valid={}", r.is_valid());
}

```

What `!matches!(self, SensorReading::Skipped { .. })` does

- `matches!(value, pattern)` — standard-library **macro** (Chapter 17: Metaprogramming) that expands to a `match` returning `true` or `false`. It is syntax sugar for “does this value fit this pattern?” without binding variables you do not need.
- `SensorReading::Skipped { .. }` — matches the `Skipped` struct variant and **ignores** the reason field (`..` = “other fields don’t matter for this test”). Same pattern token as in Chapter 6.
- `!` — negates the result: `Temp and Press -> true; Skipped { .. } -> false`.

Equivalent without the macro:

```
// Playground
enum SensorReading {
    Temp,
    Press,
    Skipped { reason: String },
}

fn is_valid_longhand(r: &SensorReading) -> bool
{
    match r {
        SensorReading::Skipped { .. } => false,
        _ => true,
    }
}

fn main() {
    println!("{}",
        is_valid_longhand(&SensorReading::Temp))
    ;
}
```

Use matches! for readable guards and filters; use full match when each arm returns different data. More on declarative macros: Chapter 17 — macro\_rules!.

### 8.4.1 Two impl blocks on one type

| Block                                     | Role                                                        | Java / Python analogy    |
|-------------------------------------------|-------------------------------------------------------------|--------------------------|
| impl SensorReading { ... }                | <b>Inherent</b> methods — always available on SensorReading | methods on the class     |
| impl Measurable for SensorReading { ... } | <b>Trait impl</b> — call only where Measurable is required  | interface implementation |

Both on the same enum is normal. Put variant-specific logic in inherent methods. Put cross-type contracts on traits.

### 8.4.2 Struct-in-enum vs enum-in-struct

| Layout                            | Sketch                                    | Reach for it when                                                |
|-----------------------------------|-------------------------------------------|------------------------------------------------------------------|
| <b>Struct inside enum variant</b> | enum Msg { Data(Frame),<br>Ping }         | variants own <i>different</i> shapes;<br>match is the dispatcher |
| <b>Enum field inside struct</b>   | struct Device { kind:<br>Kind, addr: u8 } | all rows share the same<br>fields; tag only selects<br>behaviour |
| <b>Unit variants only</b>         | enum Mode { Auto, Manual<br>}             | no payload —<br>traits/ methods ignore inner<br>data             |

```
// Playground --- shared metadata + tagged kind
enum DeviceKind {
    Modbus,
    OpcUa,
}

struct Device {
    name: String,
    kind: DeviceKind,
}

impl Device {
    fn default_port(&self) -> u16 {
        match self.kind {
            DeviceKind::Modbus => 502,
            DeviceKind::OpcUa => 4840,
        }
    }
}

fn main() {
    let d = Device {
        name: "plc-1".into(),
        kind: DeviceKind::Modbus,
    };
    println!("{}", d.name, d.default_port());
}
```

Prefer **enum + struct variants** when each variant would have been a sub-

class with different fields. Prefer **struct + enum field** when every instance shares the same columns and only behaviour differs.

### 8.4.3 Trait on enum: match is mandatory

Unlike a single struct, an enum trait body almost always uses `match self` (or `match &self`) — one arm per variant. That mirrors Chapter 6 exhaustiveness. Add a variant, and the compiler lists every `impl` and `match` you must update.

**Default trait methods** still work — override only where a variant differs:

```
// Playground
trait HasCode {
    fn code(&self) -> u8;
    /// Shared default formatting --- call this
    // from overrides to avoid recursion.
    fn default_label(&self) -> String {
        format!("code {}", self.code())
    }
    fn label(&self) -> String {
        self.default_label()
    }
}

enum Fault {
    OverTemp(u8),
    CommLost,
}

impl HasCode for Fault {
    fn code(&self) -> u8 {
        match self {
            Fault::OverTemp(c) => *c,
            Fault::CommLost => 0xFF,
        }
    }

    fn label(&self) -> String {
        match self {
```

```

        Fault::CommLost => "communication
            lost".into(),
        other => other.default_label(),
        // trait default body, not this
        // override
    }
}

fn main() {
    println!("{}", Fault::CommLost.label());
    println!("{}",
        Fault::OverTemp(0x0A).label()); //
    default: "code 10"
}

```

#### 8.4.3.1 Calling the trait’s default method from your override

Yes — `other.default_label()` deliberately calls the **default body written on the trait**, not your override. That is how you reuse the trait’s fallback for some cases (here, `OverTemp`) while customising others (`CommLost`).

**Footgun:** `HasCode::label(other)` or `other.label()` inside `impl HasCode` for `Fault` still dispatches to **this override** and can recurse forever. Extract the default into a separate trait method (here `default_label`) or inline `format!("{}", other.code())`.

| Call                               | What runs                                                                                                                  |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <code>fault.label()</code>         | <b>Dynamic dispatch on the impl</b> — always the overridden label in <code>impl HasCode</code> for <code>Fault</code>      |
| <code>fault.default_label()</code> | <b>Default trait body</b> — <code>format!("{}", self.code())</code> , which still uses your overridden <code>code()</code> |

**Not Java overloading.** Rust has **no** method overloading (same name, different parameter lists). There is only one `label(&self) -> String` here. What looks like “pick another version” is **explicit qualified call syntax**: `Trait::method(receiver)`. This is sometimes called UFCS (universal function call syntax).



Closer Java analogy: a default interface method, not overloads:

```
// Java --- conceptual parallel
interface HasCode {
    int code();
    default String label() { return "code " +
        code(); }
}
class Fault implements HasCode {
    public String label() {
        if (isCommLost()) return "communication
            lost";
        return HasCode.super.label();
        // call default, not recurse into
        // self.label()
    }
}
```

Wrong — “call the override again” (infinite recursion on OverTemp):

```
// Playground --- stack overflow at runtime for
    OverTemp
impl HasCode for Fault {
    fn label(&self) -> String {
        match self {
            Fault::CommLost => "communication
                lost".into(),
            other => other.label(),
            // ERROR path: calls THIS override
            // again, forever
        }
    }
}
```

Use `other.default_label()` (or duplicate the default logic) when you want the **trait default**; use `other.label()` only when you intend the **full override path** (including for `CommLost`).

#### 8.4.3.2 One `impl` block per trait (not `impl A, B for T`)

No — Rust does not allow multiple traits in one `impl` header:

```
// Playground --- does not compile
// impl HasCode, Display for Fault { ... }
// ERROR: expected `for`, found ``,``
```

A type may implement **many** traits, but each trait gets its **own** `impl` block:

```
// Playground
use std::fmt;

trait HasCode {
    fn code(&self) -> u8;
}

enum Fault {
    OverTemp(u8),
    CommLost,
}

impl HasCode for Fault {
    fn code(&self) -> u8 {
        match self {
            Fault::OverTemp(c) => *c,
            Fault::CommLost => 0xFF,
        }
    }
}

impl fmt::Display for Fault {
    fn fmt(&self, f: &mut fmt::Formatter<'_>)
        -> fmt::Result {
        write!(f, "fault 0x{:02X}", <Self as
            HasCode>::code(self))
    }
}

fn main() {
    println!("{}", Fault::CommLost);
}
```

Where **multiple traits appear together** is on **bounds**, not on `impl`:

| Location          | Example                                                | Meaning                                     |
|-------------------|--------------------------------------------------------|---------------------------------------------|
| Generic parameter | <code>fn show&lt;T: HasCode + Display&gt;(x: T)</code> | T must implement <b>both</b> traits         |
| where clause      | <code>where T: HasCode + Display</code>                | same, spelled out                           |
| Trait inheritance | <code>trait Loud: Display { ... }</code>               | every Loud type must also implement Display |

So: `impl HasCode` for `Fault` and `impl Display` for `Fault` are two separate blocks; `T: HasCode + Display` means “T implements both.”

#### 8.4.4 `#[derive(...)]` on enums and structs

Enums and structs in the same model often share derives:

```
// Playground
#[derive(Debug, Clone, PartialEq, Eq)]
enum Command {
    Stop,
    SetSpeed(u32),
}

#[derive(Debug, Clone, PartialEq, Eq)]
struct SetSpeedLog {
    at: u64,
    rpm: u32,
}

impl Command {
    /// Turn a command into a timestamped log
    row when it carries speed data.
    fn to_log(&self, at: u64) ->
        Option<SetSpeedLog> {
        match self {
            Command::SetSpeed(rpm) =>
                Some(SetSpeedLog { at, rpm:
                    *rpm }),
            Command::Stop => None,
        }
    }
}
```

```

    }
}

fn main() {
    let c = Command::SetSpeed(1500);
    assert_eq!(c, Command::SetSpeed(1500));

    let log =
        c.to_log(1_700_000_000).expect("SetSpeed
            maps to a log row");
    assert_eq!(log, SetSpeedLog { at:
        1_700_000_000, rpm: 1500 });
    println!("{:?}", log);
}

```

SetSpeedLog is the **struct side** of the same automation model. The enum is the wire/command shape (Stop, SetSpeed(1500)); the struct is the persisted audit row (at + rpm). Command::to\_log bridges them — only SetSpeed produces a SetSpeedLog; Stop returns None.

Both types share `#[derive(...)]` because you typically want the same tooling (Debug, Clone, equality) on commands and the records they generate. What `#[derive]` is (compile-time codegen, not annotations/decorators), ecosystem vs custom derives — Chapter 17: Derive attributes.

PartialEq/Eq on enums requires every payload type to be comparable. A variant holding f64 forces you to drop Eq or model floats differently — see edge cases below.

## 8.4.5 Enums + traits edge cases and compiler traps

**Wrong — non-exhaustive match in trait impl (add a variant, forget an arm):**

```

// Playground --- does not compile
enum Status { Idle, Running, Fault }

trait Label {
    fn label(&self) -> &'static str;
}

```

```
impl Label for Status {
    fn label(&self) -> &'static str {
        match self {
            Status::Idle => "idle",
            Status::Running => "running",
            // Status::Fault => ...
            // ERROR: non-exhaustive patterns
        }
    }
}
```

**Wrong — partial move: extract payload, then reuse self:**

```
// Playground --- does not compile
enum Packet {
    Raw(String),
    Empty,
}

impl Packet {
    fn dump(self) {
        if let Packet::Raw(s) = self {
            println!("{}", s.len());
        }
        // ERROR: use of partially moved value:
        `self`
        // Packet::Empty arm never ran, but
        `Raw` variant moved `self`
        let empty = matches!(self,
            Packet::Empty);
        println!("empty={}", empty);
    }
}
```

Fix: take `&self`, clone the `String` when you need ownership, or handle all variants in one match.

**Wrong — match self by value in one method, then call another method on self:**

```
// Playground --- does not compile
impl Packet {
    fn describe(self) -> String {
        match self {
            Packet::Raw(s) => format!("raw {}
                                bytes", s.len()),
            Packet::Empty => "empty".into(),
        }
    }

    fn tag(self) -> &'static str {
        match self {
            Packet::Raw(_) => "RAW",
            Packet::Empty => "EMPTY",
        }
    }

    fn full(self) -> String {
        format!("{}", self.tag(),
                self.describe()) // ERROR: `self`
                                moved
    }
}
```

Idiomatic fix: `fn full(&self) -> String` and `match` on references, or merge into a single `match`.

### Wrong — orphan rule (external trait + external type):

```
// Playground --- does not compile
// impl std::fmt::Display for Vec<u8> { ... }
// ERROR: impl doesn't apply to type defined
// outside of crate
```

You can `impl Display` for `YourEnum`, or wrap `Vec<u8>` in a newtype struct `Frame(pub Vec<u8>)`; and implement there.

### Wrong — assume `dyn Trait` works with every trait:

```
// Playground --- does not compile
trait Measurable {
```

```

fn value(&self) -> f64;
fn read() -> f64;
// no `self` --- not object-safe
}

// let items: Vec<Box<dyn Measurable>> = ...;
// ERROR: trait `Measurable` is not dyn
compatible

```

Only **object-safe** traits can become `dyn Trait` — `&self` methods, no generic methods on the trait itself. Enums with trait impls usually use **static dispatch** (`impl Trait / generics`) instead.

#### Wrong — Eq on enum with f64 payload:

```

// Playground --- does not compile
#[derive(Eq, PartialEq)]
enum Sample {
    Analog(f64),
    Digital(bool),
}
// ERROR: the trait bound `f64: Eq` is not
satisfied

```

Use integer fixed-point, ordered floats (`ordered-float`), or derive only `PartialEq`.

| Trap                              | Symptom                                    | Idiom                                                                                                    |
|-----------------------------------|--------------------------------------------|----------------------------------------------------------------------------------------------------------|
| New enum variant                  | errors in every match / trait impl         | treat as feature — update all arms                                                                       |
| Partial move in match             | “use of partially moved value”             | <code>match &amp;self</code> , clone, or one combined match                                              |
| Trait impl without match          | wrong for multi-variant enums              | one arm per variant                                                                                      |
| <code>dyn Trait</code> collection | “not dyn compatible”                       | object-safe trait, or enum of variants instead of heterogeneous <code>Vec&lt;Box&lt;dyn _&gt;&gt;</code> |
| Orphan rule                       | “impl doesn’t apply to type outside crate” | newtype wrapper or own trait                                                                             |

### 8.4.6 Idiom spotlight (enums + traits)

**Model closed sets as `enum`, not `strings`.** Put data in struct variants; implement traits with exhaustive `match`.

**New variant -> compiler updates your checklist** — no forgotten else when the PLC adds a fault code.

**Prefer `&self` on enums** unless you intentionally consume the value.

## 8.5 Generics

```
// Playground
fn largest<T: PartialOrd>(list: &[T]) -> &T {
    let mut max = &list[0];
    for item in &list[1..] {
        if item > max { max = item; }
    }
    max
}

fn main() {
    let nums = vec![3, 1, 4, 1, 5];
    println!("{}", largest(&nums));
}
```

## 8.6 Trait bounds and where

```
// Playground
use std::fmt::Display;

fn show<T: Display>(x: T) {
    println!("{}", x);
}

fn main() {
    show(42);
}
```



```
show("text");
}
```

## 8.7 Trait objects (dyn Trait)

**Generics** and **impl Trait** pick the concrete type at **compile time** — monomorphized, no vtable.

**dyn Trait** is runtime polymorphism through a vtable — like Java interface references or Python duck typing at runtime. Prefer **impl Trait** when types are known at compile time; use **dyn Trait** for mixed-type collections.

### 8.7.1 What you actually store

A trait object is a **wide pointer** (fat pointer):

| Part           | Points to                                          |
|----------------|----------------------------------------------------|
| Data pointer   | the concrete value (En, Fr, ...)                   |
| Vtable pointer | that type's <code>impl Greeter</code> method table |

Because size varies by concrete type, `dyn Greeter` is **dynamically sized** (!Sized). It almost always lives **behind a pointer**:

| Form                              | Owns value?         | Typical use                                                              |
|-----------------------------------|---------------------|--------------------------------------------------------------------------|
| <code>&amp;dyn Trait</code>       | no — borrow         | pass one of several types into <code>fn notify(x: &amp;dyn Trait)</code> |
| <code>&amp;mut dyn Trait</code>   | no — mutable borrow | plug-in you mutate in place                                              |
| <code>Box&lt;dyn Trait&gt;</code> | yes — heap          | <code>Vec&lt;Box&lt;dyn Trait&gt;&gt;</code> of mixed types              |
| <code>Arc&lt;dyn Trait&gt;</code> | yes — shared        | callbacks/handlers shared across threads ( <code>Send + Sync</code> )    |

```
// Playground
trait Greeter {
    fn greet(&self) -> String;
}
```

```

struct En;
struct Fr;

impl Greeter for En {
    fn greet(&self) -> String { "Hello".into() }
}
impl Greeter for Fr {
    fn greet(&self) -> String {
        "Bonjour".into() }
}

fn announce(g: &dyn Greeter) {
    println!("{}", g.greet());
}

fn main() {
    announce(&En);
    // no Box --- stack value, borrowed as
    // trait object
    announce(&Fr);

    let voices: Vec<Box<dyn Greeter>> =
        vec![Box::new(En), Box::new(Fr)];
    for v in &voices {
        println!("{}", v.greet());
    }
}

```

Box in the Vec **homogenizes** size: every element is one pointer pair. Without Box, Rust cannot lay out a vector of different struct sizes in one array.

## 8.7.2 When dyn Trait is idiomatic

| Situation                                                                | Prefer                                  | Why                          |
|--------------------------------------------------------------------------|-----------------------------------------|------------------------------|
| Function called with <b>one of a few types</b> , known when writing code | <code>fn f(x: &amp;impl Greeter)</code> | static dispatch, zero vtable |

| Situation                                              | Prefer                           | Why                                                            |
|--------------------------------------------------------|----------------------------------|----------------------------------------------------------------|
| <b>Vec / registry of plug-ins</b><br>loaded at runtime | Vec<Box<dyn Handler>>            | one collection, mixed concrete types                           |
| <b>Return type depends on runtime input</b> (factory)  | Box<dyn Greeter>                 | caller only knows the trait                                    |
| <b>Closed protocol</b> (fixed set of variants)         | enum + match                     | no vtable; exhaustiveness — see <a href="#">enums + traits</a> |
| Shared handler across threads                          | Arc<dyn AlarmSink + Send + Sync> | cheap clone, thread-safe ref count                             |

**Automation sketch — alarm sinks (open set of outputs):**

```
// Playground
trait AlarmSink {
    fn emit(&self, code: u8, msg: &str);
}

struct LogSink;
struct MetricsSink;

impl AlarmSink for LogSink {
    fn emit(&self, code: u8, msg: &str) {
        println!("{}", code, msg);
    }
}

impl AlarmSink for MetricsSink {
    fn emit(&self, code: u8, msg: &str) {
        println!("metric alarm_code={} len={}",
            code, msg.len());
    }
}

fn raise(sinks: &[&dyn AlarmSink], code: u8,
    msg: &str) {
    for s in sinks {
        s.emit(code, msg);
    }
}
```

```
fn main() {
    let log = LogSink;
    let metrics = MetricsSink;
    raise(&[&log, &metrics], 0x07, "over-temp");
}
```

Borrowed `&dyn AlarmSink` avoids heap allocation when callers already own `LogSink` / `MetricsSink` on the stack. Use a `Vec<Box<dyn AlarmSink>>` when sinks are constructed dynamically and stored for the process lifetime.

### Factory returning erased type:

```
// Playground
trait Greeter {
    fn greet(&self) -> String;
}

struct En;
struct Fr;

impl Greeter for En {
    fn greet(&self) -> String {
        "Hello".into()
    }
}

impl Greeter for Fr {
    fn greet(&self) -> String {
        "Bonjour".into()
    }
}

fn greeter_for(lang: &str) -> Box<dyn Greeter> {
    match lang {
        "fr" => Box::new(Fr),
        _ => Box::new(En),
    }
}

fn main() {
    let g = greeter_for("fr");
    println!("{}", g.greet());
}
```

```
}

```

→ `impl Greeter` cannot replace this if the two arms return **different** concrete types. `impl Trait` in return position hides a **single** concrete type per function body. For “either `En` or `Fr`”, use `Box<dyn Greeter>` or an `enum`.

### 8.7.3 `impl Trait` vs `dyn Trait` vs `enum`

```
// Playground --- same behaviour, three Rust
    styles

trait Driver {
    fn poll(&self) -> u32;
}

struct Modbus;
struct OpcUa;
impl Driver for Modbus { fn poll(&self) -> u32
    { 502 } }
impl Driver for OpcUa { fn poll(&self) -> u32 {
    4840 } }

// 1) Static dispatch --- best default
fn port_static(d: &impl Driver) -> u32 {
    d.poll() }

// 2) Runtime erasure --- plug-in list
fn ports_dynamic(drivers: &[&dyn Driver]) ->
    Vec<u32> {
    drivers.iter().map(|d| d.poll()).collect()
}

// 3) Closed sum --- best when variants are
    fixed in your crate
enum Device {
    Modbus,
    OpcUa,
}
```

```
impl Device {
    fn poll(&self) -> u32 {
        match self {
            Device::Modbus => Modbus.poll(),
            Device::OpcUa => OpcUa.poll(),
        }
    }
}

fn main() {
    assert_eq!(port_static(&Modbus), 502);
    assert_eq!(ports_dynamic(&[&Modbus,
        &OpcUa]), vec![502, 4840]);
    assert_eq!(Device::OpcUa.poll(), 4840);
}
```

### 8.7.4 Object safety — not every trait can be dyn

A trait is **dyn compatible** (object-safe) only if the vtable can list a fixed set of methods. Common **object-unsafe** patterns:

| Trait shape                                 | Why it breaks dyn                                  |
|---------------------------------------------|----------------------------------------------------|
| fn read() -> f64 — no self                  | no receiver to dispatch on                         |
| fn convert<T>(&self, x: T) — generic method | vtable cannot hold infinite monomorphizations      |
| fn clone(&self) -> Self                     | return type is concrete Self, unknown at call site |
| trait Foo: Sized                            | trait object itself is !Sized                      |

#### Wrong — not object-safe:

```
// Playground --- does not compile
trait Reader {
    fn read() -> f64; // no `self`
}

// fn load(r: &dyn Reader) { ... }
// ERROR: the trait `Reader` is not dyn
```

```
compatible ... consider marking as
`#[allow(...)]`
```

### Wrong — generic method on trait:

```
// Playground --- does not compile
trait Parse {
    fn parse<T: std::str::FromStr>(&self, s:
        &str) -> Option<T>;
}
// let p: &dyn Parse = &MyParser;
// ERROR: trait `Parse` is not dyn compatible
```

**Fix for → Self:** return a boxed trait object or associated type with a bound, or use static dispatch (`impl Clone` on generic `T` instead of `dyn Clone`-style patterns). Standard library `Clone` is not `dyn`-compatible for this reason. You clone concrete types or use `Box<dyn Display>` etc. for other traits.

Cross-reference: the same object-safety trap appears in enums + traits edge cases above.

## 8.7.5 More edge cases and compiler traps

### Wrong — dyn Trait by value on the stack:

```
// Playground --- does not compile
trait Greeter { fn greet(&self) -> String; }
struct En;
impl Greeter for En { fn greet(&self) -> String
    { "Hi".into() } }

// let g: dyn Greeter = En;
// ERROR: the size for values of type `dyn
// Greeter` cannot be known
```

Use `Box<dyn Greeter>`, `&dyn Greeter`, etc.

### Wrong — heterogeneous Vec without homogenizing pointer:

```
// Playground --- does not compile
// let v: Vec<dyn Greeter> = vec![En, Fr];
// ERROR: the size for values of type `dyn
// Greeter` cannot be known
```

**Lifetime on borrowed trait objects** — the reference must not outlive the concrete value:

```
// Playground --- OK inside one function:
`en`/`fr` live long enough
trait Greeter {
    fn greet(&self) -> String;
}
struct En;
struct Fr;
impl Greeter for En {
    fn greet(&self) -> String {
        "Hello".into()
    }
}
impl Greeter for Fr {
    fn greet(&self) -> String {
        "Bonjour".into()
    }
}

fn pick_in_place(use_fr: bool) {
    let en = En;
    let fr = Fr;
    let g: &dyn Greeter = if use_fr { &fr }
        else { &en };
    println!("{}", g.greet());
}

fn main() {
    pick_in_place(true);
}
```

Returning `&dyn Greeter` from a function that creates locals **does not compile** (dangling):

```
// Playground --- does not compile
fn broken(use_fr: bool) -> &dyn Greeter {
    let en = En;
    let fr = Fr;
```



```
if use_fr { &fr } else { &en }
// ERROR: returns a reference to data owned
// by the current function
}
```

Return `Box<dyn Greeter>` when the callee allocates/chooses the value.

**Auto traits on trait objects:** `Box<dyn AlarmSink + Send + Sync>` when the handler crosses thread boundaries; omit when single-threaded.

| Trap                                  | Symptom                                | Idiom                                                                                        |
|---------------------------------------|----------------------------------------|----------------------------------------------------------------------------------------------|
| Unsize dyn                            | “size cannot be known at compile time” | <code>&amp;</code> , <code>Box</code> , <code>Arc</code> , never bare <code>dyn</code> value |
| Non-object-safe trait                 | “trait X is not dyn compatible”        | fix trait API, or use <code>enum</code> / generics                                           |
| <code>-&gt; impl Trait</code> factory | arms return different types            | <code>Box&lt;dyn Trait&gt;</code> or <code>enum</code>                                       |
| Closed protocol                       | stringly plug-in names                 | <code>enum</code> beats <code>dyn</code> for speed + exhaustiveness                          |
| Dangling <code>&amp;dyn</code>        | borrow checker on return               | <code>Box&lt;dyn Trait&gt;</code> or borrow from caller’s value                              |

## 8.8 Trait decomposition — small interfaces

Large “god traits” are hard to mock and test. Split I/O boundaries into **focused traits** that each do one job. The same orchestration code runs in production, CLI, and tests by swapping implementations:

```
// Playground
use std::io::{self, Read};

trait ConfigLoader {
    fn load_text(&self, path: &str) ->
        io::Result<String>;
}

trait PortValidator {
    fn validate(&self, port: u16) -> Result<(),
        &'static str>;
}
```

```

struct FileConfigLoader;

impl ConfigLoader for FileConfigLoader {
    fn load_text(&self, path: &str) ->
        io::Result<String> {
        std::fs::read_to_string(path)
    }
}

struct PrivilegedPortGuard;

impl PortValidator for PrivilegedPortGuard {
    fn validate(&self, port: u16) -> Result<(),
        &'static str> {
        if port < 1024 {
            Err("port below 1024")
        } else {
            Ok(())
        }
    }
}

fn startup_port<L: ConfigLoader, V:
    PortValidator>(
    loader: &L,
    validator: &V,
    path: &str,
) -> io::Result<u16> {
    let text = loader.load_text(path)?;
    let port: u16 =
        text.trim().parse().map_err(|e|
            io::Error::new(io::ErrorKind::InvalidData, e))?;
    validator.validate(port).map_err(|msg|
        io::Error::new(io::ErrorKind::InvalidInput, msg))?;
    Ok(port)
}

```

```
fn main() {
    let loader = FileConfigLoader;
    let guard = PrivilegedPortGuard;
    // startup_port(&loader, &guard,
    //             "port.txt") in a real project
    let _ = (&loader as &dyn ConfigLoader,
            &guard as &dyn PortValidator);
    println!("traits split for test doubles");
}
```

| Split                | Role                                    |
|----------------------|-----------------------------------------|
| ConfigLoader         | where bytes come from (file, env, mock) |
| PortValidator        | business rule isolated from I/O         |
| generic orchestrator | one function, many backend pairs        |

In async services, the same split applies with `async fn` traits — see Chapter 16 — async trait boundaries.

## 8.9 Strategy enum — closed dispatch without a top-level vtable

When you have a **fixed set** of backends, an internal **enum** often beats `Box<dyn Trait>` at the runner layer. Each variant holds its parser/state; one match dispatches:

```
// Playground
enum SourceKind {
    Modbus { unit_id: u8 },
    OpcUa { endpoint: String },
}

impl SourceKind {
    fn default_port(&self) -> u16 {
        match self {
            SourceKind::Modbus { .. } => 502,
            SourceKind::OpcUa { .. } => 4840,
        }
    }
}
```

```

    }

    fn label(&self) -> &str {
        match self {
            SourceKind::Modbus { .. } =>
                "modbus",
            SourceKind::OpcUa { .. } => "opcua",
        }
    }
}

fn run_source(src: &SourceKind) {
    println!("{}", src.label(),
        src.default_port());
}

fn main() {
    run_source(&SourceKind::Modbus { unit_id: 1
    });
    run_source(&SourceKind::OpcUa {
        endpoint: "opc://plc".into(),
    });
}

```

Use **enum + match** when variants are known at compile time in your crate. Use **dyn Trait** when plug-ins arrive at runtime from outside. Many production runners combine both: enum at the top, impl Stream or trait objects **inside** a variant when needed.

## 8.10 Transform traits — parse once, map to domain

Keep **wire format** separate from **domain records**. A parser yields raw rows; a transformer maps them to what you persist:

```

// Playground
struct RawReading {
    tag: String,

```

```

    value: f64,
}

struct StoredReading {
    tag: String,
    value_milli: i64,
}

trait ToStored {
    fn to_stored(&self) -> StoredReading;
}

impl ToStored for RawReading {
    fn to_stored(&self) -> StoredReading {
        StoredReading {
            tag: self.tag.clone(),
            value_milli: (self.value *
                1000.0).round() as i64,
        }
    }
}

fn ingest(rows: &[RawReading]) ->
    Vec<StoredReading> {
    rows.iter().map(|r| r.to_stored()).collect()
}

fn main() {
    let raw = RawReading {
        tag: "temp".into(),
        value: 22.5,
    };
    println!("{:?}", ingest(&[raw]).len());
}

```

Each source (Modbus, OPC-UA, CSV) can implement the same transform trait on its row type. The runner stays generic over “anything that becomes `StoredReading`”.

## 8.11 Extension traits in your crate

Extension traits add methods without wrapping every value in a newtype. Typical targets: **Option**<T> for required-field checks, **&str** for normalizers (Chapter 13), **Stream** adapters in async pipelines (Chapter 16).

```
// Playground
trait RequiredField<T> {
    fn required(self, name: &str) -> Result<T,
        String>;
}

impl<T> RequiredField<T> for Option<T> {
    fn required(self, name: &str) -> Result<T,
        String> {
        self.ok_or_else(|| format!("missing
            field '{name}'"))
    }
}

fn parse_id(raw: Option<&str>) -> Result<u32,
    String> {
    let s = raw.required("device_id");
    s.parse().map_err(|e| e.to_string())
}

fn main() {
    println!("{:?}", parse_id(Some("42")));
    println!("{:?}", parse_id(None));
}
```

Implement extension traits in the **same module** as the error type they produce — keeps ? chains readable in hand-written parsers.

## 8.12 Associated types and supertraits

You already implement traits with methods. Many std traits also declare an **associated type** — one output type fixed per impl, not a separate generic parameter on the trait itself.

### 8.12.1 `Iterator::Item` — one output type per iterator

Chapter 4 — Implementing Iterator defines a port scanner. The associated type pins what `.next()` yields:

```
// Playground
struct PortScan {
    next_port: u16,
    end: u16,
}

impl Iterator for PortScan {
    type Item = u16;
    // associated type --- fixed for this impl

    fn next(&mut self) -> Option<Self::Item> {
        if self.next_port > self.end {
            return None;
        }
        let p = self.next_port;
        self.next_port += 1;
        Some(p)
    }
}

fn main() {
    let ports: Vec<u16> = PortScan { next_port:
        502, end: 505 }.collect();
    let sum: u16 = PortScan { next_port: 502,
        end: 504 }.sum();
    println!("ports={:?} sum={}", ports, sum);
}
```

`type Item = u16` tells every adapter (`.map`, `.sum`, `.collect`) what flows through the pipeline. Change it to `String` and the same `impl` body stops compiling — callers and adapters depend on that one associated type.

### 8.12.2 Custom trait with type Output

When a trait's return type varies **per implementor** but stays **one type per implementor**, use an associated type instead of a generic parameter on the trait:

```
// Playground
trait Summarizable {
    type Output;
    fn summarize(&self) -> Self::Output;
}

struct TempReading(f64);
struct PressReading(f64);

impl Summarizable for TempReading {
    type Output = String;
    fn summarize(&self) -> String {
        format!("temp={:.1}C", self.0)
    }
}

impl Summarizable for PressReading {
    type Output = String;
    fn summarize(&self) -> String {
        format!("press={:.0}kPa", self.0)
    }
}

fn log_reading<T: Summarizable>(r: &T)
where
    T::Output: std::fmt::Display,
{
    println!("{}", r.summarize());
}

fn main() {
    log_reading(&TempReading(22.5));
    log_reading(&PressReading(101.3));
}
```



### 8.12.3 Associated type vs generic trait parameter

| Style           | Example                                                            | When                                                               |
|-----------------|--------------------------------------------------------------------|--------------------------------------------------------------------|
| Associated type | <code>trait Iterator { type Item; ... }</code>                     | <b>One</b> output type per implementor (u16 ports, &str lines)     |
| Generic param   | <code>trait Storage&lt;T&gt; { fn get(&amp;self) -&gt; T; }</code> | Caller picks T at each call site — many operations per implementor |
| Both in std     | FromStr with type Err                                              | Error type is fixed per parsed type, like Item for iterators       |

Iterator could have been `trait Iterator<T> { fn next(&mut self) -> Option<T>; }` — but then every function taking “some iterator” would need an extra type parameter. Associated types keep call sites clean.

### 8.12.4 Supertraits — stacking requirements

A **supertrait** requires another trait as a prerequisite. Your type must implement both:

```
// Playground
use std::fmt::Display;

trait Loggable: Display {
    fn log(&self) {
        println!("[LOG] {}", self);
    }
}

struct Port(u16);
impl Display for Port {
    fn fmt(&self, f: &mut
        std::fmt::Formatter<'_>) ->
        std::fmt::Result {
        write!(f, "Port({})", self.0)
    }
}
impl Loggable for Port {}
```

```
fn emit(p: &impl Loggable) {
    p.log();
}

fn main() {
    emit(&Port(502));
}
```

Loggable: Display means “anything loggable must also be displayable.” The default method can call Display formatting inside the trait.

### 8.12.5 Associated types and object safety

Traits with type `Item` or type `Err` can still be object-safe when methods don’t return bare `Self`. `Clone` returns `Self` — that is why `dyn Clone` is awkward. `Iterator` is object-safe, but boxing iterators is rare; prefer `impl Iterator` or generics.

### 8.12.6 UFCS when two traits share a method name

Wrong — ambiguous method call:

```
// Playground --- does not compile
trait A { fn id(&self) -> u32; }
trait B { fn id(&self) -> u32; }
struct Tag;
impl A for Tag { fn id(&self) -> u32 { 1 } }
impl B for Tag { fn id(&self) -> u32 { 2 } }

fn main() {
    let t = Tag;
    // println!("{}", t.id());
    // ERROR: multiple applicable items in scope
    println!("A={} B={}", A::id(&t), B::id(&t));
}
```

Use **UFCS** — `TraitName::method(&self)` — when two traits define the same method name.

## 8.13 Idiom spotlight

**Default:** `impl Trait` / `generics (static dispatch)`. Use `dyn Trait` for runtime plug-in lists or return-type erasure — and `enum` when the variant set is closed.

`&dyn Trait` when callers own values; `Box<dyn Trait>` for owning heterogeneous collections.

**Split large traits** into small getters/savers; **enum-dispatch** fixed backends; **transform traits** bridge wire format -> domain.

## 8.14 Go deeper

- [The Rust Book — Structs and Methods](#)
- [The Rust Book — Generics, Traits, and Lifetimes](#)
- [The Rust Book — Trait Objects](#)
- [Records / structs — example 062](#)

## 8.15 See also

- Chapter 6: Enums and pattern matching — `match`, exhaustiveness, `Option/Result`
- Chapter 4: Iterators — custom `Iterator` and type `Item`
- Chapter 11: Collections
- Chapter 17: Derive attributes — `#[derive]` syntax, `std/ecosystem/custom`
- Chapter 16: Async traits (advanced)

### 8.15.1 Afterparty

#### 8.15.1.1 Structs and inherent `impl`

1. **new vs default** — “When is `Sensor::new` idiomatic vs `Default` + field update? One automation example each.”
2. **Method receiver** — “Same logic three ways: `fn f(self)`, `fn f(&self)`, `fn f(&mut self)` on a struct; I predict what calls compile.”

### 8.15.1.2 Enums, structs, and traits together

3. **Enum trait impl** — “Add variant `Fault` to `Status`; show every compile error until trait `impl` and inherent methods match.”
4. **Struct vs enum layout** — “Modbus/OpcUa device registry: justify struct `Device { kind: Enum }` vs enum `Device { Modbus(...), OpcUa(...) }`; sketch types.”
5. **SensorReading port** — “Python `Union[TempReading, PressReading, Skipped]` -> Rust enum + struct payloads + `Measurable` trait; show match in `impl`.”
6. **Two impl blocks** — “Same type: add inherent `is_valid()` and trait `Summary`; explain which call sites see which methods.”
7. **Partial move fix** — “Given enum `Packet { Raw(String), Empty }`, reproduce ‘partially moved value’ and rewrite with `&self` or one match.”
8. **matches! drill** — “Rewrite `!matches!(self, Skipped { .. })` as longhand match; then back to `matches!`; link to `macro_rules` conceptually.”

### 8.15.1.3 Default trait methods and UFCS

9. **Default override** — “Trait `HasCode` with default `label()`; override for one enum variant only; use `HasCode::label(self)` for the rest.”
10. **Recursion trap** — “Show infinite recursion when override calls `self.label()` instead of `HasCode::label(self)`; fix it.”
11. **One trait per impl** — “Show `impl HasCode, Display for T` compile error; split into two blocks; add `fn show<T: HasCode + Display>(x: T).`”

### 8.15.1.4 #[derive] and shared models

12. **Command + log row** — “enum `Command` + struct `SetSpeedLog + to_log()`; list which derives each type needs and why.”
13. **Eq on floats** — “Add `Analog(f64)` variant; show `#[derive(Eq)]` failure; fix with integer fixed-point or `PartialEq` only.”

### 8.15.1.5 Generics and trait bounds

14. **Generic bounds** — “Fix compiler error: `T` needs `Display + Clone`; minimal bound set on `fn duplicate_and_print<T>(x: T).`”
15. **largest pitfalls** — “Why does `largest` need non-empty slice? Add

Option return or document panic; compare to Java generics erasure story.”

16. **where clause** — “Rewrite `fn f<T: A + B + C>(x: T)` with a `where` block; same signature, longer trait list.”

#### 8.15.1.6 `impl Trait` vs `dyn Trait` vs `enum`

17. **dyn vs impl quiz** — “Four scenarios (plug-in `Vec`, single helper `fn`, closed protocol, factory return) — pick `impl`, `dyn`, or `enum` each time.”
18. **AlarmSink registry** — “Implement `&dyn AlarmSink` for log + metrics; then refactor to `enum Sink` if set is closed — compare trade-offs.”
19. **Factory return** — “`greeter_for(lang) -> Box<dyn Greeter>` vs `-> impl Greeter` — show why `impl` fails when arms return `En` and `Fr`.”
20. **Driver three ways** — “Same `poll()` behaviour with `&impl Driver`, `&dyn Driver`, and `enum Device`; benchmark story without running code.”
21. **Box vs borrow** — “When is `&dyn Trait` enough vs `Box<dyn Trait>` required? `Vec` of mixed types + dangling return examples.”

#### 8.15.1.7 Object safety and `dyn` traps

22. **Object safety audit** — “Mark each trait `dyn`-safe or not: no-self method, generic method, `-> Self`, trait `Foo: Sized`.”
23. **Reader fix** — “Trait with `fn read() -> f64` fails as `dyn`; redesign for `&dyn Reader` or use `enum`.”
24. **Clone not dyn** — “Why no `Box<dyn Clone>` pattern for heterogenous clone list; suggest `enum` or generic `T: Clone` instead.”
25. **Send + Sync** — “Alarm handler shared across threads: write type as `Arc<dyn AlarmSink + Send + Sync>`; what breaks if handler holds `Rc`?”
26. **Unsize trap** — “Show three snippets that fail: `let g: dyn Greeter = En, Vec<dyn Greeter>`, returning `&dyn` from locals; fix each.”

#### 8.15.1.8 Orphan rule and cross-crate patterns

27. **Orphan rule** — “Why `impl Display for Vec<u8>` fails; fix with new-type struct `Frame(pub Vec<u8>) + impl Display for Frame`.”
28. **External trait** — “Wrap third-party struct; implement your trait on the wrapper; call from automation `main`.”

**8.15.1.9 Capstone drills**

29. **Checklist drill** — “Match 8 Chapter 7 compiler errors to snippets (non-exhaustive match, partial move, orphan, not dyn compatible, unsized, moved self, Eq+f64, multi-trait impl).”
30. **PLC message model** — “Design full model: enum frames, struct payloads, two traits, one dyn registry for sinks, derive list — no code over 80 lines.”
31. **Refactor story** — “Start with `Vec<Box<dyn Driver>>`; protocol closes to two devices; refactor to enum; list what the compiler now catches.”

**8.15.1.10 Associated types and supertraits**

32. **Item type quiz** — “Change `PortScan`’s type `Item` from `u16` to `(u16, bool)` — list every call site in a `.map().collect()` chain that breaks and why.”
33. **Summarizable design** — “Add `Summarizable` with type `Output = String` for three sensor structs; one `fn report(r: &impl Summarizable)` — no duplicate return types in the trait.”
34. **Associated vs generic** — “Same cache API twice: `trait Get<T>` vs `trait Get { type Value; }` — when is each painful at call sites?”
35. **Supertrait bounds** — “Trait `Exportable: Display + Debug` with default `fn export` — show `impl` for `Port(u16)`; what fails if `Display` is missing?”
36. **UFCS fix** — “Type implements A and B, both define `name()` — show ambiguous call error and UFCS fix with `A::name(&x).`”

# Chapter 9

## Chapter 8: Errors and Testing

### 9.1 Hook

Many languages use exceptions for control flow (**Java** checked / unchecked throws, **Python** raise, ...). Rust splits **recoverable** (`Result`) from **unrecoverable** (`panic!`). Long-running code should treat errors as data, not surprises.

**Why `panic!` / `unwrap()` in production is a bad idea**

A **panic** stops the thread — often the whole process. Use it for bugs, not for expected failures like bad config or offline devices.

| Expected failure (data)                                                | Panic (crash)                                                       |
|------------------------------------------------------------------------|---------------------------------------------------------------------|
| Log, alert operator, retry next poll                                   | Whole gateway or PLC bridge <b>down</b> until someone restarts it   |
| Return <code>Err</code> to caller — choose fallback port or safe state | No chance to enter <b>safe mode</b> or drain the queue              |
| Error is <b>typed and testable</b> in the signature                    | Failure is a <b>surprise</b> at a random <code>unwrap()</code> line |

At service boundaries, use **? on `Result`** — not `unwrap` on ports, config, or field I/O. Full panic mechanics below; see also Chapter 6 — avoid `unwrap` in automation.

## 9.2 Panic, unwind, and why it is not Result

When Rust **panics** (`panic!`, failed `assert!`, `unwrap` on `Err/None`), the runtime treats the situation as **non-recoverable for this thread**. What happens next depends on the panic strategy:

| Strategy                                      | Behaviour                                                                                              | Typical target                |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------|
| <b>Unwind</b> (default on many hosts)         | Walk the stack <b>backwards</b> ; run <b>Drop on each frame</b> ; then thread dies or error propagates | desktop, server               |
| <b>Abort</b> ( <code>panic = "abort"</code> ) | <b>Immediate process exit</b> — no Drop, no cleanup                                                    | embedded, some release builds |

**Unwind** is Rust's structured stack retreat — closer to C++ exceptions or Java unwinding than to returning `Err`. It is **not** "return an error value".

```
normal Result path:  open()? ->
                    Err(io::Error) -> caller matches, logs,
                    retries
panic path:          unwrap() -> panic!
                    -> unwind stack -> thread/process gone
```

### 9.2.1 Problems unwind creates in production

1. **No typed error in the signature** — callers cannot match on what went wrong; the thread just stops.
2. **Drop runs under panic** — destructors flush buffers, close handles, release locks. If **Drop also panics**, Rust **aborts the whole process** (double panic).
3. **Partial cleanup story** — during unwind, some resources are dropped and some code never runs. Invariants may be left mid-update unless you designed for it.
4. **Unattended automation** — a Modbus poll loop that panics on one bad frame **stops all future polls**. A `Result` lets you log, skip, and continue.
5. **catch\_unwind is a niche tool** — it can catch unwinding panics in isolated blocks (e.g. foreign callbacks). It **must not** replace `Result` for business logic. It misses **abort panics**, has **UnwindSafe bounds**, and adds complexity.



```
// Playground --- illustration only; not
    idiomatic error handling
use std::panic;

fn main() {
    let outcome = panic::catch_unwind(|| {
        panic!("simulated bug");
    });
    println!("caught panic: {:?}",
        outcome.is_err());
}
```

For PLC gateways and serial bridges: **Result at boundaries, panic for programmer mistakes.**

## 9.3 Error propagation

Chapter 6 introduced `Result<T, E>`. This section is the **railway**: each step returns `Result`; `?` exits early on `Err` and passes the success value forward. No unwind, no panic.

*[Diagram omitted in print edition — see the web repository.]*

**Layered gateway load** (playground — uses manual `AppError` defined below):

```
// Playground
#[derive(Debug)]
enum AppError {
    Parse(String),
    OutOfRange(u16),
}

fn read_file(path: &str) -> Result<String,
    AppError> {
    // stand-in for std::fs::read_to_string in
    demos
    if path.is_empty() {
        Err(AppError::Parse("empty
            path".into()))
    }
}
```

```
    } else {
        Ok("8080".into())
    }
}

fn parse_port(text: &str) -> Result<u16,
    AppError> {
    text.trim()
        .parse()
        .map_err(|_|
            AppError::Parse(text.into()))
}

fn validate_port(port: u16) -> Result<u16,
    AppError> {
    if port < 1024 {
        Err(AppError::OutOfRange(port))
    } else {
        Ok(port)
    }
}

fn load_gateway_config(path: &str) ->
    Result<u16, AppError> {
    let text = read_file(path)?;
    let port = parse_port(&text)?;
    validate_port(port)?;
    Ok(port)
}

fn run() -> Result<u16, AppError> {
    load_gateway_config("config.toml")
}

fn main() {
    match run() {
        Ok(port) => println!("gateway on {}",
            port),
    }
}
```

```

        Err(e) => eprintln!("startup failed:
            {:?}" , e),
    }
    println!("{:?}",
        parse_port("8080").and_then(validate_port));
}

```

Each `?` is an **early return** of `Err` — same idea as Java throw, but the error type is visible in every signature. The match in `main` is the **boundary**; run and helpers bubble errors with `?`.

### 9.3.1 Three propagation tools

| Tool                         | Use when                                        | Effect on <code>Err</code>                                                  |
|------------------------------|-------------------------------------------------|-----------------------------------------------------------------------------|
| <code>?</code>               | Inside <code>fn -&gt; Result&lt;_, E&gt;</code> | return <code>Err</code> immediately (via <code>From</code> if types differ) |
| <code>.map / .map_err</code> | Transform value or error in place               | stay in <code>Result</code> ; no early exit from current function           |
| <code>.and_then</code>       | Next fallible step on success                   | chain; equivalent to <code>?</code> inside a closure                        |

```

// Playground
fn parse_doubled(s: &str) -> Result<u32,
    String> {
    s.trim()
        .parse::<u32>()
        .map_err(|e| e.to_string())
        .map(|p| p * 2)
}

fn main() {
    println!("{:?}", parse_doubled("21"));
}

```

`.and_then(validate_port)` in the gateway example chains two fallible steps without a separate function body.

9.3.2 Internal vs boundary

| Layer    | Who                                | Pattern                                                    |
|----------|------------------------------------|------------------------------------------------------------|
| Internal | library helpers, poll loop body    | ? bubble Err up — no printing here                         |
| Boundary | main, HTTP handler, PLC supervisor | one match or if let Err(e) = run() — log, exit code, retry |

In the gateway example above: `read_file / parse_port / validate_port / load_gateway_config` are **internal**; `main's match run()` is the **boundary**.

9.3.3 Example: read first line from a file (Cargo only)

```
// Cargo project --- needs std::fs
use std::fs::File;
use std::io::{self, Read};

fn read_first_line(path: &str) ->
    Result<String, io::Error> {
    let mut f = File::open(path)?;
    let mut buf = String::new();
    f.read_to_string(&mut buf)?;
    Ok(buf.lines().next().unwrap_or("").to_string())
}

fn main() {
    match read_first_line("Cargo.toml") {
        Ok(s) => println!("{}", s),
        Err(e) => println!("error: {}", e),
    }
}
```

| Line                         | What it does                                              |
|------------------------------|-----------------------------------------------------------|
| -> Result<String, io::Error> | Success = first line; failure = std I/O error (no panic). |

| Line                                         | What it does                                              |
|----------------------------------------------|-----------------------------------------------------------|
| <code>File::open(path)?</code>               | Propagate open failure — <b>return path</b> , not unwind. |
| <code>f.read_to_string(&amp;mut buf)?</code> | Same for read errors.                                     |
| <code>unwrap_or("")</code>                   | Empty file -> ""; <b>not</b> a panic path.                |
| <code>match in main</code>                   | <b>Boundary</b> — process keeps running after logging.    |

### Playground equivalent:

```
// Playground
fn parse_config(s: &str) -> Result<u32, String>
{
    s.trim().parse::<u32>().map_err(|e|
        e.to_string())
}

fn main() {
    println!("{:?}", parse_config("8080"));
    println!("{:?}", parse_config("oops"));
}
```

### What ? desugars to:

```
let mut f = match File::open(path) {
    Ok(v) => v,
    Err(e) => return Err(e),
};
```

## 9.3.4 Propagation edge cases

### Wrong — ? in a function that does not return Result (or Option):

```
// Playground --- does not compile
fn broken(s: &str) -> u32 {
    s.parse::<u32>()?
    // ERROR: the `?` operator can only be used
    // in a function that returns `Result` or
    // `Option`
}
```

**? on Option inside fn -> Option** (Chapter 6):

```
// Playground
fn first_word(s: &str) -> Option<&str> {
    let word = s.split_whitespace().next()?;
    // None -> return None
    Some(word)
}
```

**Wrong — error type mismatch with ?:**

```
// Playground --- does not compile
enum AppError { Io(std::io::Error) }

fn read(path: &str) -> Result<String, AppError>
{
    let mut buf = String::new();
    std::fs::File::open(path)?.read_to_string(&mut buf)?; // ERROR: `?' can't convert
    `io::Error` to `AppError`
    Ok(buf)
}
```

Fix: `map_err`, `From impl`, or `thiserror #[from]` — see below.

**Wrong — unwrap where ? belongs:**

```
// Playground --- runs, panics on bad input
(unwind path)
fn load_port(s: &str) -> u16 {
    s.parse().unwrap()
}
```

**Idiomatic poll loop — log and continue:**

```
// Playground
fn poll_port(s: &str) {
    match s.parse::<u16>() {
        Ok(p) => println!("poll ok {}", p),
        Err(e) => eprintln!("bad config, skip
            tick: {}", e),
    }
}
```

```
fn main() {
    poll_port("502");
    poll_port("oops");
}
```

| Helper                     | On failure            | Production use                      |
|----------------------------|-----------------------|-------------------------------------|
| ?                          | return Err            | library / internal propagation      |
| match / if let             | branch                | boundaries, loops, recovery         |
| unwrap() / expect("...")   | <b>panic</b> , unwind | tests, prototypes, truly impossible |
| unwrap_or / unwrap_or_else | default value         | safe fallback (not for I/O errors)  |

## 9.4 Std library errors — fine for learning, weak for production

Std error types are **correct** and **zero-dependency** — perfect for tutorials and tiny scripts. Real automation pain comes from **ergonomics and maintainability**, not from `io::Error` being wrong.

| Std type                                    | Problem in production automation                                                                                     |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>io::Error</code>                      | Opaque; <code>ErrorKind</code> matching is brittle; hard to attach <i>which config file</i> or <i>which device</i>   |
| <code>String</code> / <code>&amp;str</code> | Not <b>matchable</b> by variant; context lost when re-wrapped; easy to typo the same message twice                   |
| <code>ParseIntError</code> etc.             | Tiny; no path/key context; every callsite needs manual <code>map_err</code>                                          |
| Manual impl <code>Error</code>              | Boilerplate: <code>Display</code> , <code>source()</code> , <code>From</code> for each wrapped type — easy to forget |

| Stage                        | Error type                                                | Tooling                                                                         |
|------------------------------|-----------------------------------------------------------|---------------------------------------------------------------------------------|
| Playground / first week      | <code>String</code> , <code>&amp;str</code> , manual enum | none                                                                            |
| Library crate / gateway core | <b>enum</b> + <b>thiserror</b>                            | <code>derive Error</code> , <code>#[from]</code> , <code>#[error("...")]</code> |

| Stage                       | Error type                  | Tooling                                              |
|-----------------------------|-----------------------------|------------------------------------------------------|
| Binary / CLI / service main | <code>anyhow::Result</code> | <code>.context("while loading config")</code> chains |

**Honest take:** add `thiserror` to any library or gateway core you will maintain. Use std errors while learning; switch before you ship.

## 9.5 Custom errors

### 9.5.1 Manual enum — learn the model

Real automation crates combine variants (parse, I/O, device timeout) instead of a bare `String`:

```
// Playground
#[derive(Debug)]
enum AppError {
    Parse(String),
    OutOfRange(u32),
}

fn set_port(s: &str) -> Result<u16, AppError> {
    let p: u16 = s.parse().map_err(|_|
        AppError::Parse(s.into()))?;
    if p < 1024 {
        Err(AppError::OutOfRange(p as u32))
    } else {
        Ok(p)
    }
}

fn main() {
    println!("{:?}", set_port("8080"));
    println!("{:?}", set_port("80"));
    println!("{:?}", set_port("oops"));
}
```



| Line                     | What it does                                              |
|--------------------------|-----------------------------------------------------------|
| enum AppError            | Closed set of failure modes — <b>match</b> at boundaries. |
| map_err(...)             | Turn parse failure into <b>your</b> variant with context. |
| ?                        | Propagate <code>Err(AppError::Parse(...))</code> early.   |
| if p < 1024 { Err(...) } | Business rule as <b>data</b> — no panic.                  |

**What manual production code still needs** (what `thiserror` generates for you):

```
// Playground --- conceptual boilerplate; use
    thiserror instead
use std::fmt;

impl fmt::Display for AppError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>)
        -> fmt::Result {
        match self {
            AppError::Parse(s) => write!(f,
                "invalid port '{s}'"),
            AppError::OutOfRange(p) =>
                write!(f, "port {p} below
                    privileged range"),
        }
    }
}

impl std::error::Error for AppError {}
```

Add `source()` when wrapping `io::Error` — another ~10 lines per variant. **thiserror** generates this for you.

**Chaining with ? across error types** — implement From:

```
// Playground
use std::num::ParseIntError;

#[derive(Debug)]
```

```

enum AppError {
    Parse(String),
    OutOfRange(u32),
}

impl From<ParseIntError> for AppError {
    fn from(_: ParseIntError) -> Self {
        AppError::Parse("invalid
            integer".into())
    }
}

fn set_port_from(s: &str) -> Result<u16,
    AppError> {
    let p: u16 = s.parse()?;
    // `?` converts via From
    if p < 1024 {
        Err(AppError::OutOfRange(p as u32))
    } else {
        Ok(p)
    }
}

fn main() {
    println!("{}", set_port_from("8080"));
}

```

## 9.5.2 thiserror in production — count it in

For libraries and gateway cores, **thiserror** replaces manual `Display + Error + From` with one derive. `#[derive(Error)]` is an **ecosystem derive attribute** — same `#[derive]` syntax as `Debug`, implemented by a proc-macro crate, not a runtime annotation (Chapter 17).

Cargo.toml:

```

[dependencies]
thiserror = "2"

```

Cargo project example:

```
// Cargo project
use thiserror::Error;

#[derive(Debug, Error)]
pub enum AppError {
    #[error("failed to read config at {path}")]
    ConfigRead {
        path: String,
        #[source]
        source: std::io::Error,
    },

    #[error("invalid port '{raw}'")]
    ParsePort {
        raw: String,
        #[source]
        source: std::num::ParseIntError,
    },

    #[error("port {0} below privileged range")]
    OutOfRange(u16),

    #[error("device timeout after {ms} ms")]
    Timeout { ms: u64 },

    #[error("serial error")]
    Serial(#[from] SerialError),
}

#[derive(Debug, Error)]
pub enum SerialError {
    #[error("framing error")]
    Framing,
    #[error("crc mismatch")]
    Crc,
}
```

| Attribute / line                         | What it does                                                                     |
|------------------------------------------|----------------------------------------------------------------------------------|
| <code>#[derive(Error)]</code>            | Generates <code>Display + std::error::Error + source()</code> chain              |
| <code>#[error("...")]</code>             | Operator-facing message template — no manual <code>fmt</code>                    |
| <code>#[source]</code>                   | Underlying error preserved for logs / <code>error.chain()</code>                 |
| <code>Serial(#[from] SerialError)</code> | Auto <code>From&lt;SerialError&gt;</code> — use <code>?</code> on serial helpers |

### Same railway, less noise:

```
// Cargo project --- with thiserror AppError
above
use std::fs;

pub fn load_gateway_config(path: &str) ->
    Result<u16, AppError> {
    let text =
        fs::read_to_string(path).map_err(|source|
            | AppError::ConfigRead {
                path: path.into(),
                source,
            })?;
    let port = text
        .trim()
        .parse()
        .map_err(|source| AppError::ParsePort {
            raw: text.clone(),
            source,
        })?;
    if port < 1024 {
        return Err(AppError::OutOfRange(port));
    }
    Ok(port)
}
```

With `#[from]` on `io::Error` variant you could shorten further. Trade explicit context vs brevity.

### 9.5.3 anyhow for binaries (sidebar)

Use **anyhow** in **main** and binary crates — not in library public APIs:

```
// Cargo project --- binary crate
// anyhow = "1"

use anyhow::Context;

fn load_gateway_config(_path: &str) ->
    Result<u16, std::io::Error> {
    // stand-in for lib helper from the
    // thiserror example above
    Ok(8080)
}

fn run() -> anyhow::Result<()> {
    let port =
        load_gateway_config("config.toml")
        .context("loading gateway config")?;
    println!("port {}", port);
    Ok(())
}

fn main() -> anyhow::Result<()> {
    run()
}
```

**Pairing:** use a **thiserror** enum in **lib** for typed errors for callers. Use **anyhow** in **bin** for context chains at the top. See [errors and enums](#) for matching variants at the boundary.

### 9.5.4 Crate-local Result alias

Large modules repeat the same error type on every signature. A **type alias** fixes the error side once:

```
// Playground
#[derive(Debug)]
pub enum GatewayError {
    Timeout,
```

```

    Parse(String),
}

pub type Result<T> = std::result::Result<T,
    GatewayError>;

pub fn poll() -> Result<f64> {
    Ok(42.0)
}

fn main() {
    println!("{:?}", poll());
}

```

Use `pub type Result<T> = std::result::Result<T, MyError>` in library roots — not in public APIs that re-export `std::result::Result` ambiguously. Document the alias in crate docs.

### 9.5.5 Transparent `#[from]` variants

Wrap subsystem errors without a custom message — `#[error(transparent)]` forwards `Display` and `source()`:

```

// Cargo project --- conceptual with thiserror
use thiserror::Error;

#[derive(Debug, Error)]
pub enum AppError {
    #[error("config read failed")]
    Config { source: std::io::Error },

    #[error(transparent)]
    Serial(#[from] SerialError),
}

#[derive(Debug, Error)]
pub enum SerialError {
    #[error("framing error")]
    Framing,
}

```

```

}

fn read_frame() -> Result<(), AppError> {
    Err(SerialError::Framing.into())
    // `?' works on SerialError helpers
}

fn main() {
    let _ = read_frame();
}

```

### 9.5.6 Error aggregation for batch work

Stream and batch pipelines often collect **per-item failures** and report them together instead of failing silently on the first error. A small trait keeps the merge logic reusable:

```

// Playground
#[derive(Debug)]
struct ItemError {
    line: usize,
    msg: String,
}

#[derive(Debug)]
enum BatchError {
    Multiple { errors: Vec<ItemError> },
}

trait FromMultipleErrors<E> {
    fn from_multiple(errors: Vec<E>) -> Self;
}

impl FromMultipleErrors<ItemError> for
    BatchError {
    fn from_multiple(errors: Vec<ItemError>) ->
        Self {
        Self::Multiple { errors }
    }
}

```

```

}

fn flush_errors(errors: Vec<ItemError>) ->
    Result<(), BatchError> {
    if errors.is_empty() {
        Ok(())
    } else {
        Err(BatchError::from_multiple(errors))
    }
}

fn main() {
    let errs = vec![
        ItemError { line: 3, msg: "bad
port".into() },
        ItemError { line: 9, msg: "missing
tag".into() },
    ];
    println!("{:?}", flush_errors(errs));
}

```

Format the **Display** message for Multiple variants to list each sub-error — operators need the full list, not “something failed”.

## 9.6 Extension traits on Option

Hand-written parsers repeat `ok_or_else` for required fields. An **extension trait** on `Option<T>` centralizes the error shape (Chapter 7):

```

// Playground
#[derive(Debug)]
enum ParseError {
    MissingField { name: String },
}

trait RequiredField<T> {
    fn required(self, name: &str) -> Result<T,
        ParseError>;
}

```



```

impl<T> RequiredField<T> for Option<T> {
    fn required(self, name: &str) -> Result<T,
        ParseError> {
        self.ok_or_else(||
            ParseError::MissingField {
                name: name.to_string(),
            })
    }
}

fn parse_unit_id(fields:
    &std::collections::HashMap<&str, &str>) ->
    Result<u8, ParseError> {
    let raw =
        fields.get("unit_id").copied().required(
            "unit_id"?;
    raw.parse::<u8>()
        .map_err(|_| ParseError::MissingField {
            name: "unit_id".into() })
    }

fn main() {
    let mut m =
        std::collections::HashMap::new();
    m.insert("unit_id", "7");
    println!("{:?}", parse_unit_id(&m));
}

```

Name constructors on the error enum (`ParseError::missing_field(...)`) keep call sites even shorter (Chapter 3).

### 9.6.1 Custom error edge cases

**Wrong — `Result<T, String>` in a library public API:**

```

// Playground --- works, but avoid in libraries
pub fn connect(host: &str) -> Result<(),
    String> {
    Err("timeout".into())
}

```

```

    // callers cannot match variants --- only
    // string compare
}

```

### Wrong — panic inside error path:

```

// Playground --- anti-pattern
fn bad(s: &str) -> Result<u16, AppError> {
    s.parse().unwrap_or_else(|_| panic!("bad
        port {}", s))
}

```

### Match at boundary for operator-facing messages:

```

// Playground
#[derive(Debug)]
enum AppError {
    Parse(String),
    OutOfRange(u32),
}

fn set_port(s: &str) -> Result<u16, AppError> {
    let p: u16 = s
        .parse()
        .map_err(|_|
            AppError::Parse(s.into()))?;
    if p < 1024 {
        Err(AppError::OutOfRange(p as u32))
    } else {
        Ok(p)
    }
}

fn main() {
    match set_port("80") {
        Ok(p) => println!("listening on {}", p),
        Err(AppError::OutOfRange(p)) =>
            eprintln!("port {} needs root", p),
        Err(AppError::Parse(s)) =>
            eprintln!("not a port: {:?}{}", s),
    }
}

```

```
}  
}
```

## 9.7 Errors and enums

Your error type **is an enum** — same machinery as `Option`, `Result`, and domain enums in Chapter 6. Use `match` in helpers and `#[derive(Error)]` instead of hand-written trait impls (Chapter 7).

### 9.7.1 Variant shapes for errors

| Shape                        | Example                                  | Use                              |
|------------------------------|------------------------------------------|----------------------------------|
| Unit                         | <code>Timeout</code>                     | simple tag                       |
| Tuple                        | <code>OutOfRange(u16)</code>             | single payload                   |
| Struct                       | <code>ConfigRead { path, source }</code> | context + <code>#[source]</code> |
| <code>#[from]</code> wrapper | <code>Serial(#[from] SerialError)</code> | transparent propagation          |

```
// Playground --- error enum mirrors Ch6 enum  
    shapes  
#[derive(Debug)]  
enum GatewayError {  
    Timeout,  
    // unit  
    OutOfRange(u16),  
    // tuple  
    ConfigRead { path: String },  
    // struct  
}
```

### 9.7.2 Error enum vs domain enum — keep them separate

| Type                          | Models             | match purpose      |
|-------------------------------|--------------------|--------------------|
| <code>Command</code> (domain) | what the PLC sends | dispatch behaviour |

| Type                    | Models          | match purpose                   |
|-------------------------|-----------------|---------------------------------|
| <b>AppError</b> (error) | what went wrong | recovery / logging / exit codes |

Do not merge domain commands and error variants into one enum — they have different lifecycles.

### 9.7.3 Recovery by variant

Automation often **branches on error kind** without panicking:

```
// Playground --- conceptual poll with typed
    recovery
#[derive(Debug)]
enum AppError {
    Timeout { ms: u64 },
    ParsePort { raw: String },
    DeviceOffline,
}

fn poll_device() -> Result<f64, AppError> {
    Err(AppError::Timeout { ms: 500 })
}

fn handle_poll() {
    match poll_device() {
        Ok(v) => println!("stored {}", v),
        Err(AppError::Timeout { .. }) =>
            eprintln!("retry next tick"),
        Err(AppError::ParsePort { raw }) =>
            eprintln!("fix config: {}", raw),
        Err(AppError::DeviceOffline) =>
            eprintln!("alert operator"),
    }
}

fn main() {
    handle_poll();
}
```

| Variant       | Typical recovery                |
|---------------|---------------------------------|
| Timeout       | retry, backoff                  |
| ParsePort     | alert config team, skip startup |
| DeviceOffline | alarm, safe state               |

### 9.7.4 Nested error enums

Subsystem errors as their own enum, embedded in the top-level error:

```
// Playground
#[derive(Debug)]
enum SerialError {
    Framing,
    Crc,
}

#[derive(Debug)]
enum AppError {
    Serial(SerialError),
    Timeout { ms: u64 },
}

fn read_frame() -> Result<(), AppError> {
    Err(AppError::Serial(SerialError::Crc))
}

fn main() {
    if let
        Err(AppError::Serial(SerialError::Crc))
        = read_frame() {
        eprintln!("crc fail --- request
            retransmit");
    }
}
```

Same pattern as **struct-in-enum** in Chapter 7.

### 9.7.5 Errors-and-enums edge cases

**Exhaustiveness — add a variant, update every match:**

```
// Playground --- does not compile until you
    handle DeviceOffline everywhere
#[derive(Debug)]
enum AppError {
    Timeout { ms: u64 },
    DeviceOffline, // new
}

fn describe(e: &AppError) -> &'static str {
    match e {
        AppError::Timeout { .. } => "timeout",
        // AppError::DeviceOffline =>
            "offline", // ERROR: non-exhaustive
    }
}
```

**source() vs matching the enum** — match **AppError** variants for recovery policy. Inspect **source()** only when you need the underlying `io::Error` kind for logging.

**#[non\_exhaustive]** on public library error enums allows adding variants without breaking downstream match exhaustiveness. Downstream must keep a wildcard arm.

**Wrong — Result inside an enum variant** (usually an anti-pattern):

```
// Playground --- awkward; prefer Result at the
    fn level
enum Messy {
    Pending(Result<u16, AppError>),
}
```

Model outcomes with `fn -> Result<T, E>` or a dedicated domain enum. Do not nest `Result` inside unrelated enums.

| Trap                          | Idiom                         |
|-------------------------------|-------------------------------|
| String errors in lib API      | <code>enum + thiserror</code> |
| Mixed domain + error variants | two enums                     |

| Trap                     | Idiom                                                   |
|--------------------------|---------------------------------------------------------|
| New error variant        | compiler lists every match — treat as feature           |
| Need underlying I/O kind | <code>#[source]</code> + logging, not string formatting |

9.8 panic! vs recover — decision table

| Mechanism       | Control flow    | Cleanup                           | Caller can react?                                     |
|-----------------|-----------------|-----------------------------------|-------------------------------------------------------|
| Result / Option | normal return   | you decide in match               | <b>yes</b>                                            |
| panic! / unwrap | unwind or abort | Drop on unwind;<br>abort skips it | <b>no</b> (except<br><code>catch_unwind</code> niche) |

| Use              | When                                                             |
|------------------|------------------------------------------------------------------|
| Result           | Missing file, bad input, timeout, CRC fail,<br>device NAK        |
| Option           | Optional value, “not found” without error<br>detail              |
| panic! / assert! | Logic bug, violated invariant, test failure                      |
| expect("...")    | Same as unwrap but message names the<br>invariant (still panics) |

**Libraries should not panic on bad user input.** Binaries may use `main()` → `Result<(), E>` and print once at the top:

```
// Playground --- CLI pattern (conceptual)
fn run() -> Result<(), AppError> {
    let port = set_port("8080")?;
    println!("port {}", port);
    Ok(())
}

fn main() {
    if let Err(e) = run() {
        eprintln!("fatal: {:?}", e);
        std::process::exit(1);
    }
}
```

Exit code + message at **one** boundary — not scattered unwraps.

### 9.8.1 Panic and unwind edge cases

**Panic in Drop -> abort:**

```
// Playground --- do not do this; conceptual
struct Bad;
impl Drop for Bad {
    fn drop(&mut self) {
        panic!("drop panicked");
    }
}
// let _b = Bad;
// drop triggers panic; if already panicking ->
    abort
```

**Thread panic without handler** — spawned thread panics can **join as Err**. If ignored, the runtime may **abort the whole program** on drop of an un-joined panicking thread.

**assert! in production paths** — failed assert is a panic (unwind/abort), not a graceful Result. Prefer explicit checks returning Err.

| Trap                       | Symptom                          | Idiom                                           |
|----------------------------|----------------------------------|-------------------------------------------------|
| unwrap on config/I/O       | process exit on bad deploy       | ? + log at boundary                             |
| ? wrong return type        | compile error                    | change signature or map_err / From              |
| panic in Drop              | double panic -> <b>abort</b>     | never panic in Drop                             |
| catch_unwind as policy     | fragile, not for business errors | use Result                                      |
| empty file vs missing file | unwrap_or("") vs open()?         | separate “not found” from “empty” if it matters |

## 9.9 Testing

Tests use **assert!** / **assert\_eq!** — a failed test **panics** (that is intentional; CI catches it). Production error paths should not rely on panics for expected cases.

**Cargo only:**



```
// src/lib.rs or same file with #[cfg(test)]
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

pub fn parse_port(s: &str) -> Result<u16,
String> {
    let p: u16 = s.parse().map_err(|e|
        e.to_string())?;
    Ok(p)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn adds() {
        assert_eq!(add(2, 2), 4);
    }

    #[test]
    fn parse_valid_port() {
        assert_eq!(parse_port("502").unwrap(),
            502);
    }

    #[test]
    fn parse_rejects_non_numeric() {
        assert!(parse_port("oops").is_err());
        // prefer is_err over unwrap in
        // negative tests
    }
}
```

What each piece does:

| Piece                                   | Role                                                                              |
|-----------------------------------------|-----------------------------------------------------------------------------------|
| <code>pub fn</code>                     | tests in <code>mod tests</code> import via <code>use super::*</code>              |
| <code>#[cfg(test)]</code>               | module compiled <b>only</b> when running cargo test — zero cost in release binary |
| <code>assert_eq!</code>                 | equality check; fails test with diff on mismatch                                  |
| <code>parse_port("502").unwrap()</code> | <b>OK in tests</b> — failure means test broken, not field conditions              |
| <code>is_err()</code>                   | checks error path <b>without</b> panicking the test runner                        |

cargo test

Also: **doc tests** in `///` examples and **integration tests** in `tests/*.rs` (external crate view of your API).

### 9.9.1 Testing edge cases

**Table-driven tests** — one function, many inputs:

```
// Playground
fn parse_port(s: &str) -> Result<u16, String> {
    let p: u16 = s.parse().map_err(|e:
        std::num::ParseIntError|
            e.to_string())?;
    Ok(p)
}

#[cfg(test)]
mod table {
    use super::parse_port;

    #[test]
    fn ports() {
        for (input, ok) in [("502", true),
            ("0", true), ("oops", false)] {
            assert_eq!(parse_port(input).is_ok(),
                ok, "input={input}");
        }
    }
}
```

```
}
```

```
fn main() {}
```

**Testing Result errors** — use `match`, `.unwrap_err()`, or crates like `assert_matches` instead of panicking on unexpected `Ok`.

## 9.9.2 Trait-based mocks — test orchestration, not I/O

Production code depends on **traits** (Chapter 7). Tests supply a **mock struct** that implements the same trait and records calls:

```
// Playground
use std::cell::RefCell;

trait DeviceWriter {
    fn write_reading(&self, tag: &str, value: f64);
}

struct LiveWriter;

impl DeviceWriter for LiveWriter {
    fn write_reading(&self, tag: &str, value: f64) {
        println!("live {}={}", tag, value);
    }
}

struct MockWriter {
    pub calls: RefCell<Vec<(String, f64)>>,
}

impl DeviceWriter for MockWriter {
    fn write_reading(&self, tag: &str, value: f64) {
        self.calls.borrow_mut().push((tag.to_string(), value));
    }
}
```

```

}

fn ingest<W: DeviceWriter>(w: &W, tag: &str,
    value: f64) {
    w.write_reading(tag, value);
}

#[cfg(test)]
mod mock_tests {
    use super::*;

    #[test]
    fn records_one_write() {
        let mock = MockWriter {
            calls: RefCell::new(Vec::new()),
        };
        ingest(&mock, "temp", 22.5);
        assert_eq!(mock.calls.borrow().len(),
            1);
    }
}

fn main() {}

```

Use **RefCell** or **Mutex** in mocks when the trait takes `&self` but tests need interior mutability. Async traits use the same pattern with `#[async_trait]` and `#[tokio::test]` (Chapter 16).

### 9.9.3 Golden-file and shared parser tests

Parsers fit a **shared test helper**: read input fixture, run the parser, compare to expected output. One helper covers every source format:

```

// Playground --- pattern sketch
fn parse_line(line: &str) -> Result<(String,
    f64), String> {
    let (tag, val) = line
        .split_once('=')
        .ok_or_else(|| "missing

```

```

        '='.to_string()?);
    Ok((tag.into(), val.parse().map_err(|e|
        e.to_string()?))
}

#[cfg(test)]
mod golden {
    use super::parse_line;

    #[test]
    fn sample_lines() {
        let inputs = ["temp=22.5",
            "press=101.3"];
        let expected: Vec<_> = inputs
            .iter()
            .map(|s| parse_line(s).unwrap())
            .collect();
        assert_eq!(expected.len(), 2);
    }
}

```

For large nested structs, `pretty_assertions::assert_eq` (dependency) prints colored diffs on failure — worth it in data-heavy tests.

### 9.9.4 Comparable test DTOs — normalize before equality

Golden comparisons often fail on **benign differences** (whitespace, sort order). Build a **test-only comparable type** that normalizes:

```

// Playground
#[derive(Debug, PartialEq)]
struct Reading {
    tag: String,
    value: f64,
}

#[derive(Debug, PartialEq)]
struct ComparableReading {

```

```

    tag: String,
    value_milli: i64,
}

impl ComparableReading {
    fn from_reading(r: Reading) -> Self {
        Self {
            tag: r.tag.trim().to_lowercase(),
            value_milli: (r.value *
                1000.0).round() as i64,
        }
    }
}

#[cfg(test)]
mod compare {
    use super::*;

    #[test]
    fn normalizes_whitespace_and_float() {
        let a =
            ComparableReading::from_reading(Reading {
                tag: " Temp ".into(),
                value: 22.499,
            });
        let b =
            ComparableReading::from_reading(Reading {
                tag: "temp".into(),
                value: 22.5,
            });
        assert_eq!(a, b);
    }
}

fn main() {}

```

### 9.9.5 Test-only derives with `cfg_attr`

Production types may skip **Deserialize** or **PartialEq** to keep dependencies lean. Tests opt in with `cfg_attr`:

```
// Playground
#[derive(Debug, Clone, PartialEq)]
#[cfg_attr(test, derive(serde::Deserialize))]
struct RegionOffsets {
    start: u64,
    end: u64,
}

fn main() {
    let _ = RegionOffsets { start: 0, end: 10 };
}
```

Requires `serde` as a **dev-dependency** when you deserialize fixtures in tests only.

### 9.9.6 Async tests

Stream and parser tests that use `.await` need a runtime:

```
// Cargo project --- needs tokio with "macros"
+ "rt"
#[tokio::test]
async fn delayed_ok() {
    tokio::time::sleep(std::time::Duration::from_millis(1)).await;
    assert_eq!(2 + 2, 4);
}
```

See Chapter 16 for `#[tokio::test]` and spawn patterns.

## 9.10 Idiom spotlight

**Libraries:** `enum + thiserror` — typed variants, `#[from]` for `?`, `#[error("...")]` for messages. **Never** `Result<T, String>` in a public API.

**Binaries:** **anyhow** at `main` for context chains; `thiserror` errors at the boundary.

**Propagation:** `?` inside helpers; **one** `match` or `main()`  $\rightarrow$  Result at the edge. Expected failures are **data**; panics **unwind** (or **abort**) — wrong tool for retry loops on a factory floor.

**Batch pipelines:** aggregate per-item errors; **extension traits** on **Option** for required fields. **Mock traits** in tests — same interface as production I/O.

## 9.11 Go deeper

- [The Rust Book — Error Handling](#)
- [The Rust Book — Writing Automated Tests](#)
- [Result railway](#)
- [Unit test patterns](#) — 744+
- [thiserror docs](#)

## 9.12 See also

- Chapter 6: Result enum
- Chapter 7: Enums + traits — same enum machinery for errors
- Chapter 19: I/O errors
- Chapter 17: Derive attributes — `#[derive(Error)]`, annotations vs decorators

### 9.12.1 Afterparty

#### 9.12.1.1 Error propagation

1. `?` **chain** — “Refactor nested `match` on `Results` to `?` railway; explain each desugared `return Err.`”
2. **map** / **and\_then** — “Same parser with `.map_err` + `.map` vs `.and_then(validate)`; when to use each.”
3. **Wrong return type** — “Show `?` compile error in `fn`  $\rightarrow$  `u32`; fix signature and boundary `match.`”
4. **Poll loop** — “Rewrite `unwrap` `Modbus config parse` to `log-and-continue`; process must survive bad line.”



5. **Internal vs boundary** — “Mark 8 functions in a gateway crate: bubble with ? or handle in main?”

#### 9.12.1.2 Panic, unwind, and production

6. **Unwind vs Result** — “Diagram stack for `open()`? vs `unwrap()` on missing file; who runs `Drop`?”
7. **panic audit** — “Mark 10 `unwrap`/expect sites: keep (test/invariant) vs `Result` (I/O/config).”
8. **Drop double panic** — “Explain why panicking in `Drop` aborts; sketch safe cleanup pattern.”
9. **catch\_unwind scope** — “When is `catch_unwind` appropriate vs abuse? One automation anti-example.”
10. **abort strategy** — “Embedded firmware: `panic = abort` — what cleanup is skipped vs `unwind`?”

#### 9.12.1.3 Std errors, thiserror, anyhow

11. **Std limits** — “List 4 pain points of `io::Error` + `String` errors in a Modbus gateway; no FUD.”
12. **Manual vs thiserror** — “Same `AppError` twice: hand-written `Display/Error/From` vs `#[derive(Error)]`; count lines saved.”
13. **thiserror design** — “Design `GatewayError` with `ConfigRead`, `ParsePort`, `Timeout`, `Serial` sub-enum; full derive attrs.”
14. **anyhow vs thiserror** — “Pick for automation binary vs library crate; show `main -> anyhow::Result + .context()`.”
15. **Why not String** — “Explain why `pub fn connect() -> Result<(), String>` is weak; propose enum API.”

#### 9.12.1.4 Errors and enums

16. **Error enum design** — “Design `AppError` for `config` + `serial` + `timeout`; variant shapes + recovery match.”
17. **Recovery match** — “Poll loop: `Timeout -> retry`, `ParsePort -> alert`, `DeviceOffline -> safe state` — sketch match.”
18. **Nested SerialError** — “Add `AppError::Serial(SerialError)`; show propagation with `#[from]`.”
19. **Exhaustive trap** — “Add `DeviceOffline` variant; quote non-exhaustive match errors until fixed.”

### 9.12.1.5 Custom errors and boundaries

- 20. **Boundary pattern** — “`run() -> Result + main` maps to exit code; no `unwrap` in between.”
- 21. **map\_err drill** — “Convert `ParseIntError -> AppError::Parse` with and without `From / #[from]`.”

### 9.12.1.6 Testing

- 22. **Table-driven ports** — “Tests for `parse_port`: valid, zero, non-numeric, too large for `u16`.”
- 23. **Integration test** — “Sketch tests/`config_load.rs` that expects `Err` on missing file without panicking.”

## **Part I**

# **Crates, Memory, and Data**

# Chapter 10

## Chapter 9: Modules, Paths, and Crates

### 10.1 Hook

Packages and modules group code by namespace in **Java**, **Python**, and many other stacks. Rust uses **modules** inside a **crate** — one compiled unit. The module tree splits projects across files, controls visibility, and exposes a public API.

Most of this chapter is **Cargo only** — you need a filesystem and cargo build. Playground snippets show the syntax in a single file; multi-file layout is the production pattern.

### 10.2 Scope — a brief tour

Module tree, pub, and Cargo layout — not publishing or semver policy.

| This chapter covers                                   | Deferred                                   |
|-------------------------------------------------------|--------------------------------------------|
| Module tree, pub, use, workspaces                     | <code>cargo publish</code> , semver policy |
| <code>#[cfg]</code> and Cargo <code>[features]</code> | Full cross-compilation matrix              |
| Unit and integration tests                            | Fuzzing, property tests                    |
| <code>///</code> docs and <code>cargo doc</code>      | Custom rustdoc themes                      |

## 10.3 Crate, package, and workspace

| Term             | Meaning                                                                 |
|------------------|-------------------------------------------------------------------------|
| <b>Crate</b>     | One library (rlib) or binary (bin) the compiler builds from a root file |
| <b>Package</b>   | One Cargo.toml + source that produces one or more crates                |
| <b>Workspace</b> | Several packages in one repo sharing one Cargo.lock                     |

A default binary package looks like:

```
my_app/
  Cargo.toml
  src/
    main.rs      # crate root for the binary
```

A library + binary in one package:

```
my_lib/
  Cargo.toml
  src/
    lib.rs      # library crate root --- `pub`
                API lives here
    main.rs     # binary uses the library via
                `use my_lib:....`
    config.rs   # submodule file
```

Run `cargo build` from the package directory. The **crate name** in `Cargo.toml` is the root for use `crate_name:....` in dependent packages.

## 10.4 Declaring modules

Two ways to add a module:

**Inline** — code in the same file:

```
// Playground
mod math {
    pub fn double(x: i32) -> i32 {
        x * 2
    }
}
```

```

    }
}

fn main() {
    println!("{}", math::double(21));
}

```

**File-backed** — in `lib.rs` or `main.rs`:

```

// src/lib.rs or main.rs
mod config;
// loads src/config.rs or src/config/mod.rs

// src/config.rs
pub fn app_name() -> &'static str {
    "gateway"
}

```

| Form                           | Use when                             |
|--------------------------------|--------------------------------------|
| <code>mod foo { ... }</code>   | tiny helpers, tests, one-off nesting |
| <code>mod foo; + foo.rs</code> | normal production split              |

## 10.5 Paths: `self`, `super`, `crate`

Paths mirror the module tree:

| Prefix                | Points to                                             |
|-----------------------|-------------------------------------------------------|
| <code>crate::</code>  | root of <b>this</b> crate                             |
| <code>super::</code>  | parent module                                         |
| <code>self::</code>   | current module (often omitted)                        |
| <code>foo::bar</code> | child module <code>foo</code> , item <code>bar</code> |

```

// Playground
mod outer {
    pub fn tag() -> &'static str {
        "outer"
    }
}

```

```

pub mod inner {
    pub fn full() -> String {
        format!("{}", inner, super::tag())
    }
}

fn main() {
    println!("{}", outer::inner::full());
}

```

**Java:** `com.example.app.Config`. **Python:** `package.submodule`. **Rust:** explicit tree from crate root; no classpath scanning.

## 10.6 Visibility: the pub ladder

By default, items are **private** to their module. Mark what callers may use:

| Visibility               | Visible from                     |
|--------------------------|----------------------------------|
| (none)                   | same module + child modules      |
| pub                      | anywhere that can reach the path |
| pub(crate)               | anywhere in this crate           |
| pub(super)               | parent module                    |
| pub(in path::to::module) | specific ancestor branch         |

```

// Playground
mod api {
    pub fn public_entry() -> i32 {
        helper()
    }

    fn helper() -> i32 {
        42
    }
}

fn main() {
    println!("{}", api::public_entry());
}

```

```
// api::helper(); // ERROR: private
}
```

**Library design:** keep fields private; expose `pub fn` constructors and accessors (Chapter 3). Hide internals so you can refactor without breaking callers.

## 10.7 use and re-exports

`use` brings names into scope so you do not repeat long paths:

```
// Playground
mod shapes {
    pub mod circle {
        pub fn area(r: f64) -> f64 {
            std::f64::consts::PI * r * r
        }
    }
}

use shapes::circle;

fn main() {
    println!("{}", circle::area(2.0));
}
```

Re-export a dependency's type under your crate's API:

```
// lib.rs pattern (Cargo only --- conceptual)
// pub use serde::Deserialize;
```

Callers write `use my_crate::Deserialize` instead of depending on path details you might change later.

## 10.8 Binary vs library crate

| Crate   | Root file                | Typical role                                           |
|---------|--------------------------|--------------------------------------------------------|
| Binary  | <code>src/main.rs</code> | CLI entry, <code>fn main</code> , thin wiring          |
| Library | <code>src/lib.rs</code>  | reusable logic, most <code>pub API</code> , unit tests |



**Cargo only** — binary calling library in the same package:

```
// src/lib.rs
pub fn greet(name: &str) -> String {
    format!("Hello, {name}")
}

// src/main.rs
fn main() {
    println!("{}", my_lib::greet("Rust"));
}
```

Replace `my_lib` with the `[package]` name from `Cargo.toml`. Keep `main` small: parse args, call library functions, map errors to exit codes (Chapter 8).

## 10.9 Tests as modules

Unit tests live in a nested module guarded by `#[cfg(test)]`:

```
// Playground
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn adds() {
        assert_eq!(add(2, 2), 4);
    }
}

fn main() {
    println!("{}", add(1, 1));
}
```

`cargo test` compiles and runs tests; normal `cargo build` skips them. Integration tests go in `tests/*.rs` at the package root (**Cargo only**).

## 10.10 Conditional compilation and Cargo features

Gate optional serial support behind a feature flag.

**Cargo.toml:**

```
# Cargo only
[package]
name = "gateway"
version = "0.1.0"
edition = "2024"

[features]
default = []
serial = []

[dependencies]
serialport = { version = "4", optional = true }
```

**src/lib.rs:**

```
// Cargo only --- conceptual
#[cfg(feature = "serial")]
pub mod serial_io {
    pub fn list_ports() -> Vec<String> {
        serialport::available_ports()
            .unwrap_or_default()
            .into_iter()
            .map(|p| p.port_name)
            .collect()
    }
}

#[cfg(not(feature = "serial"))]
pub mod serial_io {
    pub fn list_ports() -> Vec<String> {
        vec![]
    }
}
```

Build with cargo `build --features serial` to compile the real backend. Without the flag, the stub stays in the binary. The API path stays identical.

### 10.10.1 Common `cfg` attributes

```
// Playground
#[cfg(test)]
fn only_in_tests() -> i32 {
    42
}

fn main() {
    #[cfg(debug_assertions)]
    println!("debug build");

    println!("running");
}
```

| Attribute                               | Effect                                    |
|-----------------------------------------|-------------------------------------------|
| <code>#[cfg(test)]</code>               | compiled only for cargo <code>test</code> |
| <code>#[cfg(debug_assertions)]</code>   | debug builds only                         |
| <code>#[cfg(feature = "serial")]</code> | enabled when feature is on                |
| <code>cfg!(feature = "serial")</code>   | <b>runtime</b> bool — code still compiled |

`#[cfg(...)]` removes code at compile time. `if cfg!(...) { ... }` keeps both branches in the binary.

### 10.10.2 Feature edge case — optional dependency

```
# Cargo only
tokio = { version = "1", optional = true,
    features = ["rt", "macros"] }

[features]
async = ["dep:tokio"]
```

Enabling `async` pulls in `tokio` with the listed features — one switch for callers.

## 10.11 Integration tests

Integration tests live in `tests/` at the package root. Each file is a separate crate that links your library:

```
gateway/
  Cargo.toml
  src/lib.rs
  tests/
    parse_config.rs
```

```
// Cargo only --- tests/parse_config.rs
// use gateway::parse_port;

// #[test]
// fn reads_port_from_line() {
//     assert_eq!(parse_port("502").unwrap(),
//                502);
// }
```

Run with `cargo test`. Unlike `#[cfg(test)] mod tests`, integration tests only see the **public** API — they simulate external callers.

### 10.11.1 Re-export edge case — prelude module

```
// Playground
mod api {
    pub mod inner {
        pub fn connect() -> i32 {
            502
        }
    }
    pub use inner::connect;
}

fn main() {
    println!("{}", api::connect());
}
```

`pub use` re-exports `connect` at the `api` level so callers skip `inner`.

## 10.12 Workspaces (brief)

When one repo holds multiple packages:

```
workspace/
Cargo.toml      # [workspace] members =
    ["core", "cli"]
core/
  Cargo.toml
  src/lib.rs
cli/
  Cargo.toml
  src/main.rs   # depends on core via path
                dependency
```

Each member has its own crate graph. Shared versions live in the workspace `Cargo.lock`. Start with one package. Split when a library clearly deserves reuse.

## 10.13 Orphan rule (crate boundary)

You may implement **your trait** for **your type**, or either side from **your crate**. You cannot add `impl Display for Vec<u8>` in your app — both trait and type are defined elsewhere. Fix with a **newtype** wrapper (Chapter 7).

## 10.14 Documentation comments

Public API items use `///` doc comments. Run `cargo doc --open` to browse generated HTML locally:

```
// Playground
/// Parses a port number from decimal text.
///
/// # Errors
/// Returns `ParseIntError` when the string is
    not a valid `u16`.
pub fn parse_port(s: &str) -> Result<u16,
    std::num::ParseIntError> {
    s.parse()
}
```

```
fn main() {
    println!("{}", parse_port("502").unwrap());
}
```

Use `# Examples`, `# Errors`, and `# Panics` sections for items callers depend on. Hide internal helpers with `#[doc(hidden)]` when re-exporting.

## 10.15 When the compiler says no

Common errors in this chapter:

| Error (typical)                            | Cause                                                  | Fix                                                 |
|--------------------------------------------|--------------------------------------------------------|-----------------------------------------------------|
| file not found for module <code>foo</code> | missing <code>foo.rs</code> or <code>foo/mod.rs</code> | create file or use <code>inline mod</code>          |
| private item                               | forgot <code>pub</code> on <code>fn/struct</code>      | add <code>pub</code> or use <code>pub(crate)</code> |
| unresolved import                          | wrong path or typo                                     | use <code>crate::...</code> from crate root         |
| found duplicate module                     | <code>mod foo; twice</code>                            | one declaration per module                          |

## 10.16 Idiom spotlight

**Thin main, fat library.** Put logic in `lib.rs` modules; test there with `mod tests`. The binary is just the process entry point.

## 10.17 Go deeper

- [The Rust Book — Modules](#)
- [Cargo workspaces](#)

## 10.18 See also

- Preface — `rustup` and Cargo setup
- Chapter 3: Functions and methods — `pub` on methods
- Chapter 8: Errors and testing — `mod tests`, `cargo test`
- Chapter 10: Smart pointers — `Arc` across modules
- Chapter 7: Traits — orphan rule

## 10.18.1 Afterparty

### 10.18.1.1 Layout and paths

1. **File tree** — “Design module tree for a CLI that reads config and runs commands. Directories + mod lines only, no bodies.”
2. **Path quiz** — “From `crate::service::worker::run`, how do I reach `crate::config::load`? Show use and fully qualified call.”
3. **lib vs bin** — “What belongs in `main.rs` vs `lib.rs` for a tool with 500 lines of logic?”

### 10.18.1.2 Visibility

4. **pub audit** — “List items that should be pub vs private in a library crate exposing `Client::connect`.”
5. **pub(crate)** — “When is `pub(crate)` better than `pub` for test helpers?”
6. **Re-export** — “Sketch `pub use` so users see `my_crate::Error` but you wrap `thiserror` internally.”

### 10.18.1.3 Crates and workspaces

7. **Workspace split** — “Two crates: core library + cli binary. Write `Cargo.toml` dependency path only.”
8. **Integration test** — “Where does `tests/smoke.rs` live and how does it use the library?”
9. **Orphan fix** — “I want `Display` on `Vec8` — show newtype wrapper module layout.”

### 10.18.1.4 Practice

10. **Split monolith** — “Given one `main.rs` with config + parser + runner, name three modules and what each owns.”
11. **cfg test** — “Explain why `mod tests` uses `#[cfg(test)]` and use `super::*`.”
12. **Capstone** — “Generate `src/` tree for `sensor_core` library + `sensor_cli` binary in one workspace; I implement.”

### 10.18.1.5 Features and cfg

13. **Feature flag** — “Add serial feature gating `mod serial_io` — write `Cargo.toml` [features] and one `#[cfg]` line.”

14. **cfg vs cfg!** — “Same debug log twice: `#[cfg(debug_assertions)]` block vs `if cfg!(debug_assertions)` — what stays in release binary?”
15. **Optional dep** — “Wire optional tokio behind feature `async` — show `[features]` and `dep:tokio` line.”
16. **Platform gate** — “Sketch `#[cfg(target_os = \"linux\")]` module for `/dev/ttyUSB0` path only on Linux.”

#### 10.18.1.6 Integration and docs

17. **Integration layout** — “Draw tree for `tests/load_config.rs` calling `public load()` — what is invisible to the test?”
18. **pub use prelude** — “Users should call `my_crate::connect` not `my_crate::internal::connect` — show re-export.”
19. **doc hidden** — “When to mark helper `#[doc(hidden)]` on a public re-export surface?”
20. **Capstone crate** — “Design gateway crate: `serial` feature, integration test, `///` on public parse fn — tree + TOML only.”



# Chapter 11

## Chapter 10: Smart Pointers and Interior Mutability

### 11.1 Hook

Some languages treat almost everything as heap references (**Python** names, typical **Java** objects). Rust defaults to **stack + unique ownership**. Reach for **smart pointers** only when you need heap indirection, shared ownership, or mutation through a shared borrow.

Module layout for splitting code across files is Chapter 9. This chapter is about **memory patterns**.

### 11.2 Scope — a brief tour

Std smart-pointer patterns — not custom allocators or every `NonNull` API.

| This chapter covers           | Deferred                                |
|-------------------------------|-----------------------------------------|
| Box, Rc, Arc                  | Custom allocators, <code>NonNull</code> |
| Cell, RefCell, Rc<RefCell<T>> | Mutex/RwLock depth -> Chapter 14        |
| Deref coercion, Drop          | Pin/Unpin -> Chapter 16, Chapter 18     |
| Weak (break cycles)           | Full graph GC designs                   |

## 11.3 Box<T> — heap, single owner

Box allocates on the heap but keeps **one owner** — a unique pointer with known size for the type system:

```
// Playground
fn main() {
    let b = Box::new(42);
    println!("{}", b);
}
```

Use Box for:

- **Trait objects** — Box<dyn Trait> when types differ at runtime (Chapter 7)
- **Recursive types** — enum variants that contain themselves (e.g. AST nodes)
- **Large blobs** — move heavy data without copying the stack frame

### 11.3.1 Recursive enum (why Box is required)

A type cannot contain itself with infinite size. Indirection fixes that:

```
// Playground
#[derive(Debug)]
enum List {
    Cons(i32, Box<List>),
    Nil,
}

use List::{Cons, Nil};

fn main() {
    let list = Cons(1, Box::new(Cons(2,
        Box::new(Nil))));
    println!("{:?}", list);
}
```

**What happened:** Cons stores i32 plus a **Box<List>** (pointer-sized). The chain terminates at Nil. Without Box, the compiler rejects infinite size.

### 11.3.2 Box edge cases

| Situation           | What happens                                              |
|---------------------|-----------------------------------------------------------|
| Move Box            | ownership moves; old binding unusable                     |
| *box_ref            | dereference moves inner value <b>out</b> if T is not Copy |
| Drop last Box owner | heap freed immediately                                    |

**Wrong — use after move:**

```
// Playground --- does not compile
fn main() {
    let a = Box::new(1);
    let b = a;
    println!("{}", a); // ERROR: value moved
}
```

**Move inner value out of Box:**

```
// Playground
fn main() {
    let b = Box::new(String::from("hi"));
    let s: String = *b;
    // moves String out; b is empty/unusable
    println!("{}", s);
}
```

## 11.4 Rc<T> and Arc<T> — shared ownership

When many parts of the program need to **own** the same data (read-mostly), use reference counting:

| Type   | Thread-safe | Use                                 |
|--------|-------------|-------------------------------------|
| Rc<T>  | no          | single-thread graphs, shared config |
| Arc<T> | yes         | Chapter 14 shared state             |

```
// Playground
use std::rc::Rc;
```

```
fn main() {
    let a = Rc::new(String::from("shared"));
    let b = Rc::clone(&a);
    println!("{}", refs, Rc::strong_count(&a));
    println!("{}", a, b);
}
```

**What happened:** `Rc::clone(&a)` bumps the strong count — it does **not** deep-clone the `String`. Both `a` and `b` point at the same heap buffer.

11.4.1 **clone on the smart pointer vs .clone() on the inner value**

| Call                           | Effect                                          |
|--------------------------------|-------------------------------------------------|
| <code>Rc::clone(&amp;a)</code> | new handle, same allocation, count +1           |
| <code>(*a).clone()</code>      | deep copy of T, new allocation, count unchanged |

**Wrong habit from Java/Python:** calling `.clone()` on the `String` inside when you only needed another handle:

```
// Playground
use std::rc::Rc;

fn main() {
    let a = Rc::new(String::from("config"));
    let b = Rc::clone(&a);
    // cheap --- shared handle
    let c = (*a).clone();
    // expensive --- duplicate string on heap
    println!("{}", a, b, c);
}
```

11.4.2 **Cycles and Weak**

Cycles (A holds `Rc<B>`, B holds `Rc<A>`) leak memory — counts never hit zero. Break cycles with **Weak**: a non-owning handle that does not keep the allocation alive.

```
// Playground --- conceptual pattern (no cycle
    formed)
use std::rc::{Rc, Weak};

fn main() {
    let strong = Rc::new(42);
    let weak: Weak<i32> =
        Rc::downgrade(&strong);
    println!("upgrade = {:?}", weak.upgrade());
    // Some(42) while strong lives
    drop(strong);
    println!("after drop = {:?}",
        weak.upgrade()); // None
}
```

Parent/child trees often use Rc for the parent and Weak for the back-pointer to child or parent.

11.4.3 Rc / Arc edge cases

| Error / symptom        | Cause               | Fix                  |
|------------------------|---------------------|----------------------|
| Rc in thread::spawn    | not Send            | Arc                  |
| Memory never freed     | Rc cycle            | Weak, redesign graph |
| strong_count stays > 0 | leaked clone handle | audit who holds Rc   |

Arc uses atomic ref count — more CPU on clone/drop than Rc, but required for thread-safe sharing.

11.5 Cell and RefCell — interior mutability

Sometimes you need mutation **through a shared reference** — counters in single-thread code, or fields behind Rc.

**Cell** and **RefCell** enforce borrow rules **at runtime** instead of compile time:

| Type       | Access                         | Panics when                                      |
|------------|--------------------------------|--------------------------------------------------|
| Cell<T>    | .get() / .set() for Copy types | —                                                |
| RefCell<T> | .borrow() / .borrow_mut()      | double borrow_mut, borrow while mut borrow alive |

### 11.5.1 Cell for small Copy values

```
// Playground
use std::cell::Cell;

fn main() {
    let n = Cell::new(0);
    n.set(n.get() + 1);
    println!("{}", n.get());
}
```

Cell has no .borrow() — values are copied in and out. Non-Copy types belong in RefCell (or redesign ownership).

### 11.5.2 RefCell basics

```
// Playground
use std::cell::RefCell;

fn main() {
    let total = RefCell::new(0);
    *total.borrow_mut() += 1;
    *total.borrow_mut() += 2;
    println!("{}", total.borrow());
}
```

### 11.5.3 Rc<RefCell<T>> — shared mutable graph on one thread

```
// Playground
use std::cell::RefCell;
use std::rc::Rc;
```

```
fn main() {
    let counter = Rc::new(RefCell::new(0));
    let c2 = Rc::clone(&counter);
    *counter.borrow_mut() += 1;
    *c2.borrow_mut() += 10;
    println!("{}", counter.borrow());
}
```

Do **not** share `RefCell` across threads — it is not `Sync`. For threads, use `Arc<Mutex<T>>` or channels (Chapter 14).

### 11.5.4 `RefCell` edge cases

**Wrong** — hold an immutable borrow, then borrow mutably:

```
// Playground --- panics at runtime (double
    borrow)
use std::cell::RefCell;

fn main() {
    let cell = RefCell::new(vec![1, 2, 3]);
    let r = cell.borrow();
    // immutable borrow active
    cell.borrow_mut().push(4);
    // panic: already borrowed
    drop(r);
}
```

**Fix:** shrink the immutable borrow scope with a nested block, or restructure so borrows do not overlap.

**Wrong** — re-enter `borrow_mut` while guard lives:

```
// Playground --- panics
use std::cell::RefCell;

fn bump_twice(cell: &RefCell<i32>) {
    let mut g1 = cell.borrow_mut();
    let mut g2 = cell.borrow_mut();
    // panic --- g1 still alive
}
```

```
*g1 += 1;
*g2 += 1;
}
```

**Fix:** finish one mutation, drop the guard, then borrow again — or use a single `borrow_mut` block.

**Compile-time vs runtime:** the borrow checker catches overlapping `&mut` at compile time. `RefCell` moves that check to **runtime** and **panics** on violation. Same logic, different moment you find the bug.

| Pattern                           | Compile-time borrow                 | RefCell                     |
|-----------------------------------|-------------------------------------|-----------------------------|
| Overlapping <code>&amp;mut</code> | compile error                       | runtime panic               |
| Cost                              | zero                                | small bookkeeping           |
| Thread-safe                       | N/A for shared <code>RefCell</code> | no — use <code>Mutex</code> |

11.6 Deref and deref coercion

Smart pointers implement **Deref** so you can call methods on the inner value:

```
// Playground
fn main() {
    let s = Box::new(String::from("hi"));
    println!("len = {}", s.len());
    // Deref: Box<String> -> &String -> &str
    // for .len()
}
```

**Deref coercion** converts references at call sites:

| From                           | Often coerces to      |
|--------------------------------|-----------------------|
| <code>&amp;String</code>       | <code>&amp;str</code> |
| <code>&amp;Box&lt;T&gt;</code> | <code>&amp;T</code>   |
| <code>&amp;Rc&lt;T&gt;</code>  | <code>&amp;T</code>   |

That is why `fn f(s: &str)` accepts `&String` and `Box<String>` after appropriate refs.



### 11.6.1 Deref edge cases

- Coercion applies to **references**, not owned values moving into `fn take(s: String)`.
- Chains stop at the target type the function expects — no infinite deref ladder in safe code.
- **Deref to `str` for custom newtypes** is a common pattern for string-like wrappers (Chapter 13).

## 11.7 Drop — cleanup on scope end

When an owner goes out of scope, Rust runs **drop** automatically (Chapter 1).

| Type                               | When heap / cleanup runs           |
|------------------------------------|------------------------------------|
| <code>Box&lt;T&gt;</code>          | when <code>Box</code> dropped      |
| <code>Rc</code> / <code>Arc</code> | when <b>strong count</b> hits 0    |
| Custom <code>Drop</code>           | before memory freed, on last owner |

Custom cleanup:

```
// Playground
struct Logger;

impl Drop for Logger {
    fn drop(&mut self) {
        println!("Logger shut down");
    }
}

fn main() {
    let _log = Logger;
    println!("working..");
} // prints "Logger shut down" here
```

**Never panic in Drop** — a panicking destructor during unwind aborts the process (Chapter 8).

### 11.7.1 Drop order edge cases

Locals drop in **reverse declaration order** (last declared, first dropped):

```
// Playground
struct Tag(&'static str);

impl Drop for Tag {
    fn drop(&mut self) {
        println!("drop {}", self.0);
    }
}

fn main() {
    let _a = Tag("a");
    let _b = Tag("b");
    println!("running");
}
// prints: running, then drop b, then drop a
```

**Rc drop:** inner T drops only when the **last** Rc/Arc handle goes away — not when one clone drops.

```
// Playground
use std::rc::Rc;

struct Loud(i32);
impl Drop for Loud {
    fn drop(&mut self) {
        println!("drop Loud({})", self.0);
    }
}

fn main() {
    let a = Rc::new(Loud(1));
    let b = Rc::clone(&a);
    drop(a);
    println!("b still alive");
} // drop Loud(1) here when b goes out of scope
```

## 11.8 Choosing a pointer (quick table)

| Need                                  | Type                        |
|---------------------------------------|-----------------------------|
| heap, one owner                       | Box<T>                      |
| shared read, one thread               | Rc<T>                       |
| shared read, many threads             | Arc<T>                      |
| mutate through shared ref, one thread | RefCell<T> / Rc<RefCell<T>> |
| mutate across threads                 | Arc<Mutex<T>> or channels   |
| break Rc cycles                       | Weak<T>                     |

## 11.9 When the compiler says no

Common errors in this chapter:

| Error (typical)                          | Cause                       | Fix                        |
|------------------------------------------|-----------------------------|----------------------------|
| Rc cannot be sent between threads        | not Send                    | Arc                        |
| RefCell cannot be shared between threads | not Sync                    | Arc<Mutex<T>>              |
| already borrowed (runtime)               | overlapping RefCell borrows | shrink scope, restructure  |
| recursive type has infinite size         | direct self in enum         | Box, Rc, or Arc            |
| value moved out of Box                   | *b moved T                  | clone inner or keep in Box |
| T: Copy not satisfied for Cell           | non-Copy in Cell            | RefCell or owned mutation  |

## 11.10 Idiom spotlight

**Reach for Box/Arc when ownership is genuinely shared or recursive — not by default.** Most data should stay owned or borrowed.

**Rc::clone is not .clone() on the data.** Count handles; deep-copy only when semantics require a duplicate.

**RefCell trades compile-time errors for runtime panics —** use for single-thread patterns; prefer compile-time borrows when you can.

## 11.11 Go deeper

- [Smart pointers — Rust Book](#)
- [Interior mutability](#)
- [Reference cycles with Weak](#)

## 11.12 See also

- Chapter 9: Modules — crate layout
- Chapter 12: Closures — Fn traits with captured handles
- Chapter 14: Multithreading — Arc, Mutex, Send/Sync
- Chapter 7: Trait objects — Box<dyn Trait>
- Chapter 1: Ownership — drop and move

### 11.12.1 Afterparty

#### 11.12.1.1 Box and heap ownership

1. **Box why** — “When is Box<T> better than Vec<T> on the stack? Two cases.”
2. **Recursive list** — “Draw memory for Cons(1, Box::new(Cons(2, Nil))) — stack vs heap boxes.”
3. **Move out of Box** — “What happens after let s = \*box\_string? When is that idiomatic vs keeping the Box?”
4. **Trait object box** — “Three plugin types implement Plugin — sketch Vec<Box<dyn Plugin>> factory; why not Vec<Plugin>?”

#### 11.12.1.2 Rc, Arc, and Weak

5. **Rc cycle** — “Explain why Rc cycles leak; contrast with Python reference cycles and Weak fix.”
6. **Handle vs deep clone** — “Audit snippet with Rc<String> and both Rc::clone and (\*rc).clone() — label cost of each.”
7. **Arc vs Rc** — “Thread spawn with Rc — show error; fix with Arc; explain atomic count overhead in one sentence.”
8. **Weak upgrade** — “Parent/child with Rc parent and Weak child back-ref — I sketch types; you explain cycle break.”
9. **strong\_count debug** — “strong\_count stays at 2 after I thought I dropped all refs — list 5 places handles hide.”

### 11.12.1.3 RefCell and interior mutability

10. **RefCell trap** — “Show double borrow\_mut panic; fix with scoped borrows.”
11. **Immutable then mut** — “Hold let r = cell.borrow() and call borrow\_mut — explain panic; fix with nested block.”
12. **Cell vs RefCell** — “Counter u32 vs Vec cache — I pick Cell or RefCell each; you correct.”
13. **Rc RefCell graph** — “Two nodes share Rc<RefCell<Node>> — one updates field, one reads — sketch borrow rules on one thread.”
14. **Compile vs runtime** — “Same overlapping-mut pattern: show compile error with &mut and runtime panic with RefCell side by side.”

### 11.12.1.4 Deref, Drop, and threads

15. **Deref coercion** — “Why does fn takes\_str(s: &str) accept &String, &Box<String>, and &Rc<String>? Trace steps.”
16. **Drop order** — “Three Drop structs in one function — I predict print order; you confirm reverse declaration rule.”
17. **Rc last handle** — “Two Rc clones dropped at different times — when does inner Drop run? Step through with println in Drop.”
18. **Drop panic** — “Explain double-panic abort if Drop panics during unwind — link to Ch8.”
19. **Arc Mutex sketch** — “Diagram thread-safe cache with Arc<Mutex<HashMap>> — no full code.”
20. **Pick pointer** — “Five scenarios (AST node, thread cache, GUI callback graph, config string, plugin list) — I pick Box/Rc/Arc/RefCell/Weak; you grade.”

### 11.12.1.5 Capstone

21. **Java heap map** — “Map Java ‘everything is reference’ to Rust ownership — when Arc, when plain &, when neither.”
22. **Refactor to smart ptr** — “I paste struct with Box, Rc, or raw Vec tree — you suggest minimal smart pointer fix and justify.”
23. **Leak hunt** — “Sketch Rc cycle in observer pattern; refactor one edge to Weak and explain count after each drop.”

# Chapter 12

## Chapter 11: Collections

### 12.1 Hook

Lists and maps are workhorses in most languages (**Python** lists/dicts, **Java** `ArrayList/HashMap`, ...). Rust's standard collections are **generic**, **ownership-aware**, and pair with iterator pipelines from Chapter 4. Closures in `.sort_by` and `.retain` appear in Chapter 12.

### 12.2 Scope — a brief tour

Daily std collections — not third-party crates or custom allocators.

| This chapter covers                                | Deferred                         |
|----------------------------------------------------|----------------------------------|
| Vec, HashMap, HashSet, VecDeque, BTreeMap/BTreeSet | Persistent/immutable collections |
| entry, iterators, .windows                         | Full algorithm analysis          |
| Ownership with collect / insert                    | Database/query engines           |

### 12.3 Choosing a collection

| Type           | Keys / order           | Typical use                         |
|----------------|------------------------|-------------------------------------|
| Vec<T>         | indexed, contiguous    | default sequence, stack-like growth |
| VecDeque<T>    | indexed, double-ended  | queue, BFS, push/pop both ends      |
| HashMap<K, V>  | unordered keys         | fast lookup, counts, caches         |
| HashSet<T>     | unique keys, unordered | membership, dedup                   |
| BTreeMap<K, V> | sorted keys            | ordered map, range queries          |
| BTreeSet<T>    | sorted unique          | ordered set                         |

All live in `std::collections` except `Vec` (always available).

Java / Python contrast

| Task          | Java       | Python                             | Rust     |
|---------------|------------|------------------------------------|----------|
| Growable list | ArrayList  | list                               | Vec      |
| Map           | HashMap    | dict                               | HashMap  |
| Set           | HashSet    | set                                | HashSet  |
| Queue         | ArrayDeque | deque                              | VecDeque |
| Sorted map    | TreeMap    | sortedcontainers<br>(stdlib: none) | BTreeMap |

12.4 Vec<T>

```
// Playground
fn main() {
    let mut v = vec![10, 20, 30];
    v.push(40);
    println!("{:?}", v);
    println!("second = {}", v[1]);
}
```

Indexing with `[i]` **panics** out of bounds. Use `.get(i)` for `Option`.

Walk with `.iter()` / `.iter_mut()` / `.into_iter()` — Chapter 4.

**Capacity:** `Vec` over-allocates for amortized push. Use `Vec::with_capacity(n)` when you know `size` upfront.

### 12.4.1 Safe indexing and mutation

| API                       | On missing index | On success   |
|---------------------------|------------------|--------------|
| <code>v[i]</code>         | <b>panic</b>     | T or &T      |
| <code>v.get(i)</code>     | None             | Some(&T)     |
| <code>v.get_mut(i)</code> | None             | Some(&mut T) |

```
// Playground
fn main() {
    let v = vec![10, 20];
    println!("{:?}", v.get(5));
    // None --- no panic
    // println!("{}", v[5]);
    // panic if uncommented
}
```

### 12.4.2 Vec methods worth knowing

```
// Playground
fn main() {
    let mut v = vec![3, 1, 4, 1, 5];
    v.sort();
    // in-place sort
    v.dedup();
    // remove consecutive duplicates (sort
    // first!)
    v.retain(|&x| x % 2 == 0);
    // keep evens only
    v.extend([9, 8]);
    // append from iterator
    println!("{:?}", v);
}
```

**What happened:** `dedup` only removes **adjacent** duplicates — sort first if you need global dedup. `retain` takes a closure (Chapter 12).

### 12.4.3 Vec edge cases

**Wrong** — use `Vec` as a queue with `remove(0)`:



```
// Playground --- works but O(n) per pop front
fn main() {
    let mut v = vec![1, 2, 3];
    while !v.is_empty() {
        let front = v.remove(0);
        // shifts entire buffer left each time
        println!("{}", front);
    }
}
```

**Fix:** use VecDeque for FIFO (see below).

**Wrong — hold reference into Vec, then mutate:**

```
// Playground --- does not compile
fn main() {
    let mut v = vec![1, 2, 3];
    let r = &v[0];
    v.push(4);
    // ERROR: cannot borrow `v` as mutable
    // because it is also borrowed as immutable
    println!("{}", r);
}
```

**Why:** push may reallocate. The old reference would dangle. Finish using `r` before mutating.

| Symptom                   | Cause                            | Fix                                              |
|---------------------------|----------------------------------|--------------------------------------------------|
| panic on <code>[i]</code> | out of bounds                    | <code>.get(i)</code> or check <code>len()</code> |
| slow queue on Vec         | <code>remove(0)</code> is $O(n)$ | VecDeque                                         |
| borrow error on push      | ref into element still alive     | shrink ref scope                                 |
| dedup leaves dupes        | not sorted                       | sort then dedup                                  |

## 12.5 HashMap<K, V>

```
// Playground
use std::collections::HashMap;

fn main() {
```

```

let mut counts = HashMap::new();
for word in ["a", "b", "a"] {
    *counts.entry(word).or_insert(0) += 1;
}
println!("{:?}", counts);
}

```

Keys need **Eq + Hash**. Owned String keys are common. Lookup accepts &str via **Borrow** — `map.get("alice")` works when keys are String.

## 12.5.1 The entry API

`.entry(key)` returns an **Entry**. Use it to insert, update, or skip in one lookup:

```

// Playground
use std::collections::HashMap;

fn main() {
    let mut scores = HashMap::new();
    scores.insert("alice", 10);

    let e = scores.entry("bob").or_insert(0);
    *e += 5;

    scores.entry("alice").and_modify(|v| *v += 1).or_insert(0);

    println!("{:?}", scores);
}

```

| Method                         | Effect                                            |
|--------------------------------|---------------------------------------------------|
| <code>or_insert(v)</code>      | insert if missing, return &mut V                  |
| <code>and_modify(f)</code>     | run f on existing value only                      |
| <code>or_default()</code>      | insert <code>Default::default()</code> if missing |
| <code>or_insert_with(f)</code> | lazy insert — call f only if missing              |

**Lazy insert** avoids work when the key already exists:

```
// Playground
use std::collections::HashMap;

fn main() {
    let mut cache = HashMap::new();
    cache.entry("key").or_insert_with(|| {
        println!("building expensive value");
        vec![1, 2, 3]
    });
    cache.entry("key").or_insert_with(|| {
        println!("should not run");
        vec![99]
    });
}
```

**What happened:** the closure runs **once** — on first insert only.

## 12.5.2 HashMap edge cases

**insert overwrites** — returns the old value if any:

```
// Playground
use std::collections::HashMap;

fn main() {
    let mut m = HashMap::new();
    m.insert("port", 502);
    let old = m.insert("port", 503);
    println!("old = {:?}, now = {:?}", old,
        m.get("port"));
}
```

**Wrong — double lookup (get then insert):**

```
// Playground --- compiles but wasteful /
    awkward for mutation
use std::collections::HashMap;

fn bump(m: &mut HashMap<String, i32>, key:
    &str) {
```

```

    if m.contains_key(key) {
        *m.get_mut(key).unwrap() += 1;
    } else {
        m.insert(key.to_string(), 1);
    }
}

```

**Idiomatic:** `*m.entry(key.to_string()).or_insert(0) += 1;`

**Borrow while iterating:** you cannot mutably insert while iterating the same map. Collect keys first, or use `entry` outside the loop.

| Error / symptom           | Cause                           | Fix                                         |
|---------------------------|---------------------------------|---------------------------------------------|
| key type needs Hash       | custom struct key               | <code>#[derive(Hash, Eq, PartialEq)]</code> |
| cannot borrow map mutably | iterator or get ref still alive | drop borrows first                          |
| silent overwrite          | insert on existing key          | check return or use <code>entry</code>      |
| f32 as key                | floats not Hash in std          | integer keys or <code>newtype</code>        |

## 12.6 HashSet<T>

Unique values with set algebra:

```

// Playground
use std::collections::HashSet;

fn main() {
    let a: HashSet<_> = [1, 2,
        3].into_iter().collect();
    let b: HashSet<_> = [3, 4,
        5].into_iter().collect();
    let union: HashSet<_> =
        a.union(&b).copied().collect();
    let inter: HashSet<_> =
        a.intersection(&b).copied().collect();
    println!("union {:?} inter {:?}", union,
        inter);
}

```

| Operation  | Method                                           | Notes          |
|------------|--------------------------------------------------|----------------|
| membership | <code>set.contains(&amp;x)</code>                | $O(1)$ average |
| dedup      | <code>collect::&lt;HashSet&lt;_&gt;&gt;()</code> | order lost     |
| difference | <code>a.difference(&amp;b)</code>                | in a not in b  |

**Dedup preserving order** — `HashSet` loses order. For stable unique order, use a `Vec + HashSet` seen-set, or `sort + dedup`.

## 12.7 VecDeque<T>

Efficient push/pop at **front and back**:

```
// Playground
use std::collections::VecDeque;

fn main() {
    let mut q = VecDeque::new();
    q.push_back(1);
    q.push_back(2);
    q.push_front(0);
    println!("pop front = {:?}", q.pop_front());
    println!("rest = {:?}", q);
}
```

| Operation    | Vec                                | VecDeque |
|--------------|------------------------------------|----------|
| push_back    | $O(1)$ amortized                   | $O(1)$   |
| pop_front    | $O(n)$ with <code>remove(0)</code> | $O(1)$   |
| random index | $O(1)$                             | $O(1)$   |

Use as a **FIFO queue** or **BFS frontier**. Use `Vec` when you only grow at the end.

## 12.8 BTreeMap and BTreeSet

**Sorted keys** — useful for ordered iteration and range scans:

```
// Playground
use std::collections::BTreeMap;

fn main() {
    let mut map = BTreeMap::new();
    map.insert(30, "c");
    map.insert(10, "a");
    map.insert(20, "b");
    for (k, v) in &map {
        println!("{}", k, v);
    }
}
```

Keys need **Ord**. Iteration is always **sorted by key**.

### 12.8.1 Range queries

```
// Playground
use std::collections::BTreeMap;

fn main() {
    let mut map = BTreeMap::new();
    for k in [100, 150, 200, 250] {
        map.insert(k, format!("reg_{}", k));
    }
    for (k, v) in map.range(120..=220) {
        println!("{}", k, v);
    }
}
```

**What happened:** prints keys 150 and 200 — inclusive range 120..=220.

| Need                      | Pick                                          |
|---------------------------|-----------------------------------------------|
| fastest single lookup     | HashMap                                       |
| sorted walk or range      | BTreeMap                                      |
| min/ max key always handy | BTreeMap (first_key_value,<br>last_key_value) |

BTreeMap is  $O(\log n)$  per op; HashMap is  $O(1)$  average. Extra cost buys

order.

## 12.9 Iterators and collection methods

Collection methods delegate to iterators — see Chapter 4:

```
// Playground
fn main() {
    let nums = vec![1, 2, 3, 4];
    let sum: i32 = nums.iter().sum();
    let evens: Vec<_> =
        nums.iter().filter(|&&x| x % 2 ==
            0).collect();
    println!("{}", sum, evens);
}
```

### 12.9.1 .windows, .chunks, .enumerate

```
// Playground
fn main() {
    let samples = vec![1.0, 1.1, 2.5, 2.4];
    let rising: Vec<_> = samples
        .windows(2)
        .filter(|w| w[1] > w[0])
        .collect();
    println!("rising pairs {:?}", rising);

    let bytes = vec![0xDE, 0xAD, 0xBE, 0xEF];
    for chunk in bytes.chunks(2) {
        println!("{:02X?}", chunk);
    }
}
```

| Adapter                   | Yields                                      |
|---------------------------|---------------------------------------------|
| <code>.windows(n)</code>  | overlapping slices of length <code>n</code> |
| <code>.chunks(n)</code>   | non-overlapping chunks                      |
| <code>.enumerate()</code> | (index, item)                               |

## 12.9.2 sort\_by and custom order

```
// Playground
fn main() {
    let mut items = vec![("b", 3), ("a", 1),
        ("c", 2)];
    items.sort_by(|a, b| a.1.cmp(&b.1));
    // sort by numeric field
    println!("{:?}", items);
}
```

Requires **Ord** on compared fields or a custom comparator closure (Chapter 12).

## 12.10 Collecting into collections

`collect()` builds many types when the element type is known:

```
// Playground
use std::collections::{HashMap, HashSet};

fn main() {
    let pairs = vec![("a", 1), ("b", 2)];
    let map: HashMap<_, _> =
        pairs.into_iter().collect();
    let uniq: HashSet<_> = vec![1, 1,
        2].into_iter().collect();
    println!("{:?}", map, uniq);
}
```

**Duplicate keys in collect to HashMap:** later pairs **overwrite** earlier ones — know your source data.

**Wrong — ambiguous collect:**

```
// Playground --- does not compile
fn main() {
    let nums = vec![1, 2, 3];
    let v = nums.iter().map(|&x| x *
        2).collect();
    // ERROR: type annotations needed
```



```
}

```

**Fix:** `let v: Vec<i32> = ... or .collect::<Vec<_>>().`

12.10.1 into\_iter vs iter when building collections

| Source walk              | You get in pipeline | Source after    |
|--------------------------|---------------------|-----------------|
| .iter()                  | &T                  | Vec still owned |
| .into_iter() / owned vec | T                   | Vec consumed    |

Building `HashMap<String, i32>` from owned strings requires **into\_iter** or cloning keys.

12.11 When the compiler says no

Common errors in this chapter:

| Error (typical)          | Cause                       | Fix                       |
|--------------------------|-----------------------------|---------------------------|
| T cannot be hashed       | key missing Hash            | derive or newtype         |
| T: Ord not satisfied     | BTreeMap key not ordered    | derive Ord or use HashMap |
| cannot borrow as mutable | ref into Vec/HashMap alive  | end borrow scope          |
| type annotations needed  | ambiguous collect()         | turbofish or annotate     |
| iterator invalidation    | mutate map during for (k,v) | collect keys first        |
| dedup ineffective        | unsorted Vec                | sort_unstable then dedup  |

12.12 Idiom spotlight

**Iterator chains over index loops.** `for i in 0..v.len()` plus `v[i]` is a smell when `.iter().enumerate()` or `.windows(2)` states intent.

**.entry for HashMap updates** — one lookup, not `contains_key + get_mut + insert`.

**Pick the collection for the access pattern** — `queue -> VecDeque`; `sorted range -> BTreeMap`; `default list -> Vec`.

## 12.13 Go deeper

- [Rust Book — Collections](#)
- [HashMap entry API](#)
- [Functional Rust — iterator topics](#)

## 12.14 See also

- Chapter 4: Iterators
- Chapter 12: Closures — `.sort_by`, `.retain`
- Chapter 13: Standard traits — `Borrow`, `Eq`, `Hash`
- Chapter 1: Borrowing
- Chapter 7: Generics

### 12.14.1 Afterparty

#### 12.14.1.1 Choosing and comparing collections

1. **Pick collection** — “Five tasks (dedup, sorted range scan, FIFO queue, index by id, min-key lookup) — I pick `Vec`/`HashMap`/`BTree`/`VecDeque`/`HashSet` each.”
2. **Hash vs BTree** — “Same 10k insert + range scan workload — when `HashMap` wins vs `BTreeMap`; one sentence each.”
3. **Queue anti-pattern** — “Review `while !v.is_empty() { v.remove(0) }` — cost and fix with `VecDeque`.”

#### 12.14.1.2 Vec drills

4. **Loop port** — “Rewrite C-style indexed loop as iterator chain; preserve behavior.”
5. **get vs index** — “Four access patterns — I pick `[i]` vs `.get(i)` vs `.get_mut` vs `if let Some`.”
6. **sort dedup** — “Dedup `[3,1,4,1,5]` wrong vs right — show sort + dedup pipeline.”
7. **retain vs filter** — “Remove evens in-place vs new `Vec` — compare `retain` and `filter().collect()`.”
8. **Borrow push trap** — “Explain `let r = &v[0]; v.push(1)` error; fix with `scope`.”

### 12.14.1.3 HashMap and HashSet

9. **entry drill** — “Word frequency from `Vec<str>` using only `.entry` — no double lookup.”
10. **or\_insert\_with** — “Lazy cache: expensive `Vec` built once per key — sketch with `or_insert_with`.”
11. **HashMap merge** — “Two maps of scores — merge by max per key; iterator + entry style.”
12. **insert overwrite** — “Track old value on port remap 502 -> 503 using `insert` return.”
13. **Set ops** — “Tags on two records — union, intersection, difference with `HashSet`.”
14. **Stable dedup** — “Unique `String` lines preserving first-seen order — no `HashSet`-only collect.”

### 12.14.1.4 BTree, windows, collect

15. **BTree range** — “List keys in `BTreeMap<u32, _>` between 100 and 200 inclusive.”
16. **Windows** — “Detect rising edges in `Vec<f64>` with `.windows(2)`; extend to `.windows(3)` for slope.”
17. **chunks vs windows** — “Parse byte stream into 4-byte frames — `chunks(4)` vs `windows(4)` when?”
18. **collect types** — “Three `collect()` calls that need type hints — fix with `turbofish`.”
19. **Duplicate keys** — “collect to `HashMap` from duplicate-key pairs — predict final map; explain overwrite rule.”

### 12.14.1.5 Performance and capstone

20. **Performance myth** — “Do Rust iterators optimize to loops? When might they not?”
21. **Capacity hint** — “Read 1M lines into `Vec` — when with `_capacity` matters; rough sizing rule.”
22. **Capstone** — “Design in-memory store: register id `u16` -> last reading `f64`, need range scan by id — pick map type, list three API methods, no impl.”

# Chapter 13

## Chapter 12: Closures and the Fn Traits

### 13.1 Hook

You already used closures in Chapter 4 — `|x| x * 2` inside `.map()`. A **closure** is an anonymous function that can **capture** variables from its scope. Rust classifies captures as **Fn**, **FnMut**, or **FnOnce**. That controls where you can store and call them.

### 13.2 Scope — a brief tour

Capture modes, `Fn*` traits, storage, and traps — not async closures or thread pools.

| This chapter covers                                                                    | Deferred                                             |
|----------------------------------------------------------------------------------------|------------------------------------------------------|
| Syntax, <code>move</code> , <code>Fn</code> / <code>FnMut</code> / <code>FnOnce</code> | Thread pools, <code>rayon</code> -> Ch 14 Afterparty |
| <code>impl Fn</code> helpers, <code>Box&lt;dyn Fn()&gt;</code> storage                 | HRTB, higher-ranked bounds                           |
| <code>.sort_by</code> , <code>.retain</code> , callback registries                     | Full async <code>Future</code> internals -> Ch 16    |
| Capture edge cases, <code>RefCell</code> + closure                                     | <code>Pin</code> / async trait objects -> Ch 16 / 18 |

### 13.3 Closure syntax

```
// Playground
fn main() {
    let factor = 3;
    let scale = |x| x * factor;
    println!("{}", scale(10));
}
```

| Form                                        | Notes                          |
|---------------------------------------------|--------------------------------|
| <code>\ x\  x + 1</code>                    | parameter types often inferred |
| <code>\ x: i32\  -&gt; i32 { x + 1 }</code> | explicit types when needed     |
| <code>\  \  println!("tick")</code>         | no parameters                  |

Closures are **expressions** — you can pass them to functions, store them (with trait bounds), or return them (carefully).

### 13.4 Capture modes

The compiler decides how a closure uses outer variables:

| Capture             | Example                         | Closure trait (typical) |
|---------------------|---------------------------------|-------------------------|
| By shared borrow    | <code>read factor</code>        | <code>Fn</code>         |
| By mutable borrow   | <code>*count += 1</code>        | <code>FnMut</code>      |
| By move (ownership) | <code>move \  \  drop(s)</code> | <code>FnOnce</code>     |

```
// Playground
fn main() {
    let mut total = 0;
    let mut add = |n| total += n;
    // FnMut --- mutably borrows total
    add(5);
    add(2);
    println!("{}", total);
}
```

## 13.5 The move keyword

move forces the closure to **take ownership** of captured values. Use when the closure may **outlive** the current scope — threads, spawn, or storing in a struct:

```
// Playground
fn main() {
    let label = String::from("worker");
    let print = move || println!("{}", label);
    print();
    // println!("{}", label);
    // ERROR: label moved into closure
}
```

Chapter 14 uses move on closures passed to `thread::spawn`.

## 13.6 Fn, FnMut, FnOnce

These are **traits** implemented automatically for closures:

| Trait    | Call style                       | When                      |
|----------|----------------------------------|---------------------------|
| Fn()     | call many times, shared<br>&self | read-only capture         |
| FnMut()  | call many times, &mut self       | mutates captured state    |
| FnOnce() | consumes self on first call      | moves captured values out |

**Subtrait chain:** Fn implies FnMut implies FnOnce. A function expecting FnOnce accepts any of them; expecting Fn is the strictest.

```
// Playground
fn call_twice(f: impl Fn(i32) -> i32) {
    println!("{}", f(1));
    println!("{}", f(2));
}

fn main() {
    let double = |x| x * 2;
    call_twice(double);
}
```

## 13.7 Closures and iterator adapters

`.map`, `.filter`, and `.for_each` take closures. The iterator holds the closure and calls it per item. The closure must match the adapter's trait bound:

```
// Playground
fn main() {
    let nums = vec![1, 2, 3];
    let doubled: Vec<_> = nums.iter().map(|&x|
        x * 2).collect();
    println!("{:?}", doubled);
}
```

On `.iter()`, items are references — see [Chapter 4 — double reference](#).

**Wrong — move closure with borrow after:**

```
// Playground --- does not compile
fn main() {
    let name = String::from("log");
    let v = vec![1, 2];
    let _ = v.into_iter().map(move |_|
        name.len()); // name moved into closure
    // println!("{}", name); // ERROR
}
```

## 13.8 Closures vs function pointers

Named functions can coerce to `fn()` (function pointer) when they capture nothing:

```
// Playground
fn add_one(x: i32) -> i32 {
    x + 1
}

fn apply(f: fn(i32) -> i32, x: i32) -> i32 {
    f(x)
}

fn main() {
```

```
println!("{}", apply(add_one, 5));
}
```

Closures that capture variables are **not** plain `fn()` — they are unique anonymous types. Use `impl Fn` or generics when you need to accept closures.

## 13.9 Returning closures

Returning `impl Fn()` works when the closure owns its data or only captures 'static references:

```
// Playground
fn make_adder(n: i32) -> impl Fn(i32) -> i32 {
    move |x| x + n
}

fn main() {
    let add5 = make_adder(5);
    println!("{}", add5(10));
}
```

Returning a closure that borrows local stack variables fails — lifetimes (Chapter 5) forbid dangling borrows.

## 13.10 Callback registry — storing closures

Filter log lines through pluggable rules. Each rule is a `Box<dyn Fn(&str) -> bool>`:

```
// Playground
fn main() {
    let mut rules: Vec<Box<dyn Fn(&str) ->
        bool>> = Vec::new();
    rules.push(Box::new(|line|
        line.contains("ERROR")));
    rules.push(Box::new(|line|
        line.starts_with("WARN"))));
}
```



```

let lines = ["INFO ok", "ERROR timeout",
             "WARN retry"];
for line in lines {
    if rules.iter().all(|rule| rule(line)) {
        println!("blocked: {}", line);
    } else if rules.iter().any(|rule|
                               rule(line)) {
        println!("matched: {}", line);
    }
}

```

`Box<dyn Fn(...)>` homogenizes closures into one type for a `Vec`. Each closure may capture different data, but the **signature** must match. Prefer `impl Fn` in helpers that do not need storage. Boxing adds heap allocation and dynamic dispatch.

## 13.11 Sort and filter with closures

Chapter 11 methods take closures when the logic is local:

```

// Playground
#[derive(Debug)]
struct SensorReading {
    id: u32,
    value: f64,
    valid: bool,
}

fn main() {
    let mut readings = vec![
        SensorReading { id: 1, value: 22.1,
                        valid: true },
        SensorReading { id: 2, value: -1.0,
                        valid: false },
        SensorReading { id: 3, value: 21.8,
                        valid: true },
    ];
}

```

```

readings.retain(|r| r.valid);
readings.sort_by(|a, b| {
    b.value
    .partial_cmp(&a.value)
    .unwrap_or(std::cmp::Ordering::Equal
    )
});

println!("{:?}", readings);
}

```

.retain needs FnMut — it may mutate the vector while iterating. .sort\_by compares pairs; use sort\_by\_key when you only need one field (|r| r.id).

## 13.12 Generic helpers — impl Fn vs boxing

Call a closure twice without boxing:

```

// Playground
fn apply_twice(f: impl Fn(i32) -> i32, x: i32)
    -> i32 {
    f(f(x))
}

fn main() {
    let scale = 3;
    let triple = |n| n * scale;
    println!("{}", apply_twice(triple, 2));
    // 18
}

```

impl Fn monomorphizes — the compiler generates a specialized copy per closure type. Use Box<dyn Fn()> only when you need a homogeneous collection or trait object return type.

## 13.13 Capture edge cases

Wrong — closure moves String in a loop:

```
// Playground --- does not compile
fn main() {
    let labels = vec![String::from("a"),
        String::from("b")];
    let mut fns: Vec<Box<dyn Fn()>> =
        Vec::new();
    for label in labels {
        fns.push(Box::new(move ||
            println!("{}", label)));
        // ERROR: label moved into closure,
        // used again next iteration
    }
}
```

**Right — clone before move:**

```
// Playground
fn main() {
    let labels = vec![String::from("a"),
        String::from("b")];
    let mut fns: Vec<Box<dyn Fn()>> =
        Vec::new();
    for label in labels {
        fns.push(Box::new(move ||
            println!("{}", label)));
    }
    for f in &fns {
        f();
    }
}
```

**RefCell + FnMut — shared counter:**

```
// Playground
use std::cell::RefCell;

fn main() {
    let count = RefCell::new(0);
    let mut bump = |n| *count.borrow_mut() += n;
    bump(5);
}
```

```

    bump(2);
    println!("{}", count.borrow());
}

```

Interior mutability (Chapter 10) lets a closure mutate state through a shared borrow.

## 13.14 Thread handoff — move on spawn

Closures passed to Chapter 14 `thread::spawn` must own their captures:

```

// Playground
use std::thread;

fn main() {
    let label = String::from("worker-1");
    let handle = thread::spawn(move || {
        println!("running {}", label);
    });
    handle.join().expect("thread panicked");
}

```

Without `move`, the closure would borrow `label`. The thread may outlive the stack frame. `move` transfers ownership into the closure.

## 13.15 Async closure note

Tokio and async code often use `async move { ... }` blocks. Same capture rules apply: use `move` when the task outlives the current scope. Full async trait and `Pin` details are in Chapter 16.

```

// Cargo only --- needs tokio in Cargo.toml:
    tokio = { version = "1", features = ["rt",
        "macros"] }
// async fn read_lines() {
//     let path = String::from("config.toml");
//     tokio::spawn(async move {
//         println!("read {}", path);
//     }).await.unwrap();

```

// }

## 13.16 Java / Python contrast

| Aspect         | Java                      | Python             | Rust                             |
|----------------|---------------------------|--------------------|----------------------------------|
| Lambda         | SAM interfaces            | lambda, nested def | closure + Fn* traits             |
| Capture        | effectively final         | full closure       | borrow / move checked            |
| Store in field | functional interface type | any callable       | Box<dyn Fn()> or generic F: Fn() |

## 13.17 When the compiler says no

Common errors in this chapter:

| Error (typical)                     | Cause                                      | Fix                                       |
|-------------------------------------|--------------------------------------------|-------------------------------------------|
| expected Fn, found FnOnce           | closure moves captured var                 | move + redesign, or clone before          |
| cannot move out of ...              | FnOnce consumed                            | call once or clone                        |
| borrowed after move                 | move closure took ownership                | restructure scopes                        |
| FnMut in concurrent context         | shared mutation                            | Mutex, channels (Chapter 14)              |
| expected Fn, found FnOnce           | closure consumes captured value on call    | clone before capture, or use FnOnce bound |
| FnMut required                      | .retain / .sort_by mutates through closure | change bound to FnMut or use mut closure  |
| type mismatch in Vec<Box<dyn Fn()>> | closures with different signatures         | same param/return types for every box     |

## 13.18 Idiom spotlight

Prefer `impl Fn in helpers` that only need to call a closure a few times. Use `move` when the closure crosses threads or outlives the stack frame.

## 13.19 Go deeper

- [The Rust Book — Closures](#)

- [Functional Rust — closure topics](#)

## 13.20 See also

- Chapter 4: Iterators — adapters that consume closures
- Chapter 11: Collections — `.retain`, `.sort_by`
- Chapter 7: Traits — trait bounds on generics
- Chapter 14: Multithreading — `move` + `spawn`

### 13.20.1 Afterparty

#### 13.20.1.1 Capture and traits

1. **Fn quiz** — “Four closures: I label each `Fn` / `FnMut` / `FnOnce`; you correct and explain capture.”
2. **move drill** — “Thread spawn snippet missing `move` — show compile error and fix.”
3. **Iterator chain** — “`.filter` closure that uses `&config` — why `Fn` not `FnMut`?”
4. **RefCell bump** — “Closure mutates `RefCell` counter — which `Fn` trait and why?”

#### 13.20.1.2 Types and storage

5. **fn vs closure** — “When can you pass `fn()` vs `impl Fn()` to the same helper?”
6. **Box dyn Fn** — “Store heterogeneous callbacks in a `Vec` — sketch trait object version.”
7. **Return closure** — “Write `make_multiplier(f: f64) -> impl Fn(f64) -> f64` and explain `move`.”
8. **Callback registry** — “Three log filters in `Vec<Box<dyn Fn(&str) -> bool>>` — all must match signature; I add a wrong one; you fix.”
9. **Loop move trap** — “Building `Vec<Box<dyn Fn()>>` in a `for` loop over `String` — show `move` error and `clone` fix.”

#### 13.20.1.3 Collections and pipelines

10. **Double reference** — “Fix `.iter().filter(|x| ...) type error on Vec<String> — show |s| vs &s| patterns.”`

11. **sort\_by** — “Sort `Vec<(String, u32)>` by count descending with `sort_by` closure.”
12. **sort\_by\_key** — “Same sort with `sort_by_key` — when is key extraction cleaner?”
13. **retain valid** — “Drop invalid `SensorReading` rows in-place with `.retain` — `FnMut` bound.”
14. **for\_each vs for** — “Same side-effect loop twice: `for` vs `.for_each(|...|)` — style tradeoffs.”

#### 13.20.1.4 Errors and concurrency

15. **Fn bound too strict** — “Helper takes `impl Fn()` but caller passes closure that mutates — fix signature.”
16. **Thread move** — “spawn closure borrows `String` — show error without move and fix.”
17. **Send on Box Fn** — “When does `Box<dyn Fn() + Send>` matter for thread pool callbacks?”

#### 13.20.1.5 Capstone

18. **Capstone** — “Pipeline: read lines, filter non-empty, parse `u16`, sum — all with closures; I write; you review `Fn` bounds.”

## Chapter 14

# Chapter 13: Standard Traits and Conversions

### 14.1 Hook

Rust’s standard library is built on **traits** — shared behaviour you opt into with `impl` or `#[derive]`. This chapter covers formatting, comparison, conversion, and flexible-argument traits you see in almost every crate. Deep mechanics (`dyn`, object safety) stay in Chapter 7. Here the focus is **idioms**.

### 14.2 Scope — a brief tour

Formatting, comparison, and conversion traits — not custom `Formatter` depth or `Serde` derives.

| This chapter covers                                                                                        | Deferred                                                                                            |
|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>Debug</code> , <code>Display</code> , <code>Default</code>                                           | Custom <code>Formatter</code> flags in depth                                                        |
| <code>PartialEq</code> , <code>Eq</code> , <code>Hash</code> , <code>Ord</code>                            | Float ordering edge cases (nominal)                                                                 |
| <code>From</code> / <code>Into</code> , <code>TryFrom</code> / <code>TryInto</code> , <code>FromStr</code> | Full error-type design -> Chapter 8                                                                 |
| <code>AsRef</code> , <code>Borrow</code> , <code>Cow</code>                                                | <code>#[derive]</code> mechanics, <code>serde</code> / <code>custom</code> derives<br>-> Chapter 17 |



## 14.3 Debug and Display

**Debug** — developer-oriented formatting via `{:?}`. Derive for most structs with `#[derive(Debug)]` — compile-time codegen, not a decorator (Chapter 17):

```
// Playground
#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 1, y: 2 };
    println!("{:?}", p);
    println!("{:#?}", p);
    // pretty-print with newlines
}
```

**Display** — user-facing formatting via `{}`. No derive — implement manually:

```
// Playground
use std::fmt;

struct Point {
    x: i32,
    y: i32,
}

impl fmt::Display for Point {
    fn fmt(&self, f: &mut fmt::Formatter<'_>)
        -> fmt::Result {
        write!(f, "({}, {})", self.x, self.y)
    }
}

fn main() {
    println!("{}", Point { x: 3, y: 4 });
}
```

```
}

```

| Trait   | Format       | Typical use               |
|---------|--------------|---------------------------|
| Debug   | {:?}", {:#?} | logs, tests, derive       |
| Display | {}           | CLI output, user messages |

**ToString:** any type with `Display` gets `.to_string()` via a blanket impl — you rarely implement `ToString` yourself.

14.3.1 Debug vs Display — when which

| Situation                        | Use                            |
|----------------------------------|--------------------------------|
| dbg!, tests, internal logs       | Debug + derive                 |
| CLI status line, user error text | Display                        |
| Secret field in struct           | manual Debug with redaction    |
| Enum in production logs          | derive Debug or compact custom |

**Redacted Debug** — do not derive if logs would leak tokens:

```
// Playground
use std::fmt;

struct ApiConfig {
    pub name: String,
    api_key: String,
}

impl fmt::Debug for ApiConfig {
    fn fmt(&self, f: &mut fmt::Formatter<'_>)
        -> fmt::Result {
        f.debug_struct("ApiConfig")
            .field("name", &self.name)
            .field("api_key", &"[REDACTED]")
            .finish()
    }
}
```

```
fn main() {
    let c = ApiConfig {
        name: String::from("prod"),
        api_key: String::from("secret-123"),
    };
    println!("{:?}", c);
}
```

### 14.3.2 Formatting edge cases

**Wrong — {} on type with only Debug:**

```
// Playground --- does not compile
#[derive(Debug)]
struct OnlyDebug(i32);

fn main() {
    // println!("{}", OnlyDebug(1));
    // ERROR: `OnlyDebug` doesn't implement
    // Display
    println!("{:?}", OnlyDebug(1)); // ok
}
```

**Newtype for orphan rule** — implement Display on your wrapper, not on Vec<u8>:

```
// Playground
use std::fmt;

struct HexBytes(pub Vec<u8>);

impl fmt::Display for HexBytes {
    fn fmt(&self, f: &mut fmt::Formatter<'_>)
        -> fmt::Result {
        for b in &self.0 {
            write!(f, "{:02X}", b)?;
        }
        Ok(())
    }
}
```

```
fn main() {
    println!("{}", HexBytes(vec![0xDE, 0xAD]));
}
```

14.3.3 Formatting flags — quick reference

Use these in println!, format!, and write! (Chapter 17 covers macro syntax):

```
// Playground
#[derive(Debug)]
struct Reading {
    id: u32,
    value: f64,
}

fn main() {
    let r = Reading { id: 1, value: 22.456 };
    println!("{}", r);
    // Reading { id: 1, value: 22.456 }
    println!("{:#?}", r);
    // pretty-printed multiline Debug
    println!("{:.2}", r.value);
    // 22.46 --- precision on floats
    println!("{:04}", r.id);
    // 0001 --- zero-padded width
}
```

| Flag          | Effect            | Typical use                 |
|---------------|-------------------|-----------------------------|
| {:?}          | Debug             | logs, tests                 |
| {:#?}         | pretty Debug      | nested struct inspection    |
| {:.2}         | 2 decimal places  | sensor readings             |
| {:04}         | width 4, zero-pad | register ids                |
| {:X} / {:02X} | hex               | byte dumps (HexBytes above) |

For CLI path helpers, accept impl AsRef<Path>. User-facing path formatting is in Chapter 19.

## 14.4 Default

Types with a sensible “empty” or “zero” value implement **Default**:

```
// Playground
#[derive(Default, Debug)]
struct Config {
    timeout_ms: u64,
    retries: u32,
}

fn main() {
    let c = Config {
        retries: 3,
        ..Default::default()
    };
    println!("{:?}", c);
}
```

Enums need `#[default]` on one variant (Rust 1.62+) to derive **Default**:

```
// Playground
#[derive(Default, Debug)]
enum Mode {
    #[default]
    Auto,
    Manual,
}

fn main() {
    println!("{:?}", Mode::default());
}
```

Use **Default** with struct update syntax (`..Default::default()`) and `HashMap::entry(...).or_default()`.

## 14.5 PartialEq, Eq, Hash, and Ord

Collection APIs from Chapter 11 require specific traits:

| Trait     | Enables                               |
|-----------|---------------------------------------|
| PartialEq | ==, !=                                |
| Eq        | full equivalence (no NaN-style holes) |
| Hash      | HashMap / HashSet keys                |
| Ord       | BTreeMap / BTreeSet, sorting          |

```
// Playground
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
struct UserId(u64);

fn main() {
    let a = UserId(1);
    let b = UserId(1);
    println!("{}", a == b, a);
}
```

**Wrong — Eq with f64 field:**

```
// Playground --- does not compile
// #[derive(Eq, PartialEq)]
// struct Sample { value: f64 }
// ERROR: binary operation `==` cannot be
//       applied to type `f64` in Eq context
```

**Fix:** use integer fixed-point, ordered-float crates, or derive only `PartialEq` (no `Eq`, no `Hash` on floats).

| Goal             | Minimum derives           |
|------------------|---------------------------|
| test equality    | PartialEq (+ Eq if valid) |
| hash map key     | Eq + Hash                 |
| btree key / sort | Ord (implies Eq)          |

## 14.6 Clone and Copy (brief)

| Trait | Meaning                                           |
|-------|---------------------------------------------------|
| Copy  | bitwise duplicate on assign — no move (Chapter 1) |
| Clone | explicit <code>.clone()</code> — may allocate     |

Derive `Clone` on small config types; skip on unique handles (file descriptors, connection pools). `#[derive(Copy, Clone)]` together when all fields are `Copy`.

## 14.7 From and Into

**From<T>** defines conversion **into** your type. **Into<T>** is auto-implemented when you implement `From`:

```
// Playground
struct Port(u16);

impl From<u16> for Port {
    fn from(n: u16) -> Self {
        Port(n)
    }
}

fn main() {
    let p: Port = 8080.into();
    let q = Port::from(443);
    println!("{}", p.0, q.0);
}
```

**Rule:** implement **From** on your type — not **Into** on the source.

### 14.7.1 Chaining From without duplication

```
// Playground
struct Label(String);

impl From<String> for Label {
    fn from(s: String) -> Self {
        Label(s)
    }
}

impl From<&str> for Label {
    fn from(s: &str) -> Self {
```

```

        Label(s.to_string())
    }
}

fn main() {
    let a = Label::from("hello");
    let b = Label::from(String::from("world"));
    println!("{}", a.0, b.0);
}

```

Std library is full of From: `String::from("text")`, `Vec::from([1, 2])`, etc.

? **and From:** ? converts errors via From (Chapter 8). Implement `From<ParseIntError>` for `MyError` once. Then `parse()? works in fn -> Result<_, MyError>`.

## 14.8 TryFrom, TryInto, and FromStr

Fallible conversions use **TryFrom** / **TryInto** when validation can fail:

```

// Playground
use std::convert::TryFrom;

#[derive(Debug)]
struct Port(u16);

impl TryFrom<i32> for Port {
    type Error = &'static str;

    fn try_from(n: i32) -> Result<Self, Self::Error> {
        if (1..=65535).contains(&n) {
            Ok(Port(n as u16))
        } else {
            Err("port out of range")
        }
    }
}
}

```



```
fn main() {
    println!("{}", Port::try_from(8080));
    println!("{}", Port::try_from(0));
}
```

**FromStr** — parse from string slice (what `.parse()` calls):

```
// Playground
use std::str::FromStr;

#[derive(Debug)]
struct Port(u16);

impl FromStr for Port {
    type Err = std::num::ParseIntError;

    fn from_str(s: &str) -> Result<Self,
        Self::Err> {
        let n: u16 = s.parse()?;
        Ok(Port(n))
    }
}

fn main() {
    let p: Port = "8080".parse().unwrap();
    println!("{}", p);
}
```

| Mechanism              | Input               | Error type       |
|------------------------|---------------------|------------------|
| From / into()          | known-valid value   | none             |
| TryFrom / try_into()   | value needing check | associated Error |
| str::parse() / FromStr | text                | FromStr::Err     |

**When to use which:** use `s.parse::<u16>()` for plain numbers. Use custom `FromStr` for **domain types** (`Port`, `HostPort`) with extra rules.

### 14.8.1 FromStr for structured values — paths and endpoints

Parse **one string into a sum type** at the boundary. Validation happens once; the rest of the code pattern-matches on variants:

```
// Playground
use std::str::FromStr;

#[derive(Debug, PartialEq)]
enum DevicePath {
    Tcp { host: String, port: u16 },
    Serial { port_name: String },
}

impl FromStr for DevicePath {
    type Err = String;

    fn from_str(s: &str) -> Result<Self,
        Self::Err> {
        if let Some(rest) =
            s.strip_prefix("tcp://") {
            let (host, port) = rest
                .split_once(':')
                .ok_or_else(|| format!("invalid
                    tcp path: {s}"))?;
            Ok(DevicePath::Tcp {
                host: host.into(),
                port: port.parse().map_err(|_|
                    format!("bad port in
                    {s}"))?,
            })
        } else if let Some(name) =
            s.strip_prefix("serial://") {
            Ok(DevicePath::Serial {
                port_name: name.into(),
            })
        } else {
            Err(format!("expected tcp:// or
```

```

        serial://, got {s}"))
    }
}

fn main() {
    let p: DevicePath =
        "tcp://plc.local:502".parse().unwrap();
    assert_eq!(
        p,
        DevicePath::Tcp {
            host: "plc.local".into(),
            port: 502,
        }
    );
    println!("{:?}", p);
}

```

Same idea for `s3://`, `file://`, or `host:port` — **FromStr** at the CLI/config edge, typed enum inside the core.

## 14.8.2 Display + FromStr round-trip for CLI enums

CLI tools often need **human-readable flags** that round-trip through strings. Implement both traits on the same enum:

```

// Playground
use std::fmt;
use std::str::FromStr;

#[derive(Debug, Clone, PartialEq)]
enum LogFormat {
    Json,
    Plain,
}

impl fmt::Display for LogFormat {
    fn fmt(&self, f: &mut fmt::Formatter<'_>)
        -> fmt::Result {

```

```

        match self {
            LogFormat::Json => write!(f,
                "json"),
            LogFormat::Plain => write!(f,
                "plain"),
        }
    }
}

impl FromStr for LogFormat {
    type Err = String;

    fn from_str(s: &str) -> Result<Self,
        Self::Err> {
        match s {
            "json" => Ok(LogFormat::Json),
            "plain" => Ok(LogFormat::Plain),
            other => Err(format!("unknown log
                format: {other}")),
        }
    }
}

fn main() {
    let fmt = LogFormat::Json;
    let s = fmt.to_string();
    assert_eq!(LogFormat::from_str(&s).unwrap(),
        fmt);
    println!("{}", s);
}

```

Display strings should match what FromStr accepts — tests can assert `parse(display(x)) == x`.

### 14.8.3 Display on field-name enums — stable log labels

Small enums that name **schema fields** or **metric keys** implement Display so error and log messages stay consistent:

```
// Playground
use std::fmt;

enum MetricField {
    PollLatencyMs,
    ErrorCount,
}

impl fmt::Display for MetricField {
    fn fmt(&self, f: &mut fmt::Formatter<'_>)
        -> fmt::Result {
        match self {
            MetricField::PollLatencyMs =>
                write!(f, "poll_latency_ms"),
            MetricField::ErrorCount =>
                write!(f, "error_count"),
        }
    }
}

fn main() {
    let field = MetricField::PollLatencyMs;
    println!("metric {} exceeded threshold",
        field);
}
```

Prefer enums over string literals for field names — renames stay compile-time checked.

#### 14.8.4 TryFrom edge cases

Wrong — as cast that silently truncates:

```
// Playground
fn main() {
    let big: i32 = 70000;
    let p = big as u16; // wraps --- no error
    println!("{}", p);
    // 14464 --- not what you want for ports
}
```

```
}
```

Fix: `u16::try_from(big)` or custom `TryFrom` with validation.

## 14.9 AsRef and Borrow

Flexible APIs accept many input forms **without allocating**:

| Trait                        | Role                                                                                                        |
|------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>AsRef&lt;T&gt;</code>  | cheap ref conversion ( <code>String -&gt; &amp;str</code> ,<br><code>Vec&lt;u8&gt; -&gt; &amp;[u8]</code> ) |
| <code>Borrow&lt;T&gt;</code> | like <code>AsRef</code> but tied to <b>hashing/equality</b> for<br>map keys                                 |

```
// Playground
fn print_label<S: AsRef<str>>(s: S) {
    println!("{}", s.as_ref());
}

fn main() {
    print_label("literal");
    print_label(String::from("owned"));
}
```

Why `HashMap<String, V>` accepts `get(&str)`: keys implement `Borrow<str>`. You can look up with `&str` without allocating a `String`.

```
// Playground
use std::collections::HashMap;

fn main() {
    let mut m = HashMap::new();
    m.insert(String::from("alice"), 10);
    println!("{}", m.get("alice"));
    // &str borrows as key
}
```

### 14.9.1 AsRef vs &T parameter

| Signature                                   | Accepts                                                                         | Notes                                                     |
|---------------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------|
| <code>fn f(s: &amp;str)</code>              | <code>&amp;str</code> only                                                      | callers with <code>String</code> pass <code>&amp;s</code> |
| <code>fn f(s: impl AsRef&lt;str&gt;)</code> | <code>&amp;str</code> , <code>String</code> , <code>Cow&lt;str&gt;</code> , ... | ergonomic public API                                      |
| <code>fn f(s: &amp;String)</code>           | <code>&amp;String</code> only                                                   | usually a smell                                           |

Same pattern: `impl AsRef<Path>`, `impl AsRef<[u8]>`.

## 14.10 Cow — clone on write

`Cow<'a, T>` (clone-on-write) is either **borrowed** or **owned**:

```
// Playground
use std::borrow::Cow;

fn normalize<'a>(s: Cow<'a, str>) -> Cow<'a,
    str> {
    if s.contains(' ') {
        Cow::Owned(s.replace(' ', "_"))
    } else {
        s
    }
}

fn main() {
    let borrowed =
        normalize(Cow::Borrowed("hello"));
    let owned = normalize(Cow::Borrowed("hello
        world"));
    println!("{}", borrowed, owned);
}
```

| Variant                            | When                              |
|------------------------------------|-----------------------------------|
| <code>Cow::Borrowed(&amp;T)</code> | no allocation; return as-is       |
| <code>Cow::Owned(T)</code>         | had to mutate or build owned data |

### 14.10.1 `into_owned()` — finish with a plain owned value

`Cow` is for **maybe** allocating. Once you know the caller needs an owned `String` or `Vec` long-term, call `.into_owned()` — it returns `T` without cloning when the data is already `Cow::Owned`:

```
// Playground
use std::borrow::Cow;

fn slugify<'a>(s: Cow<'a, str>) -> String {
    let cow = if s.contains(' ') {
        Cow::Owned(s.replace(' ', "-"))
    } else {
        s
    };
    cow.into_owned()
    // String --- cheap move if already Owned,
    // one clone if Borrowed
}

fn store_label(label: String) {
    println!("stored: {}", label);
}

fn main() {
    store_label(slugify(Cow::Borrowed("sensor
1"))); // allocates once inside slugify
    store_label(slugify(Cow::Borrowed("sensor-2"
)))); // into_owned clones the &str
}
```

| Call                       | If Cow was...                 | Cost                                            |
|----------------------------|-------------------------------|-------------------------------------------------|
| <code>.into_owned()</code> | <code>Borrowed(&amp;T)</code> | clone <code>T</code> into owned                 |
| <code>.into_owned()</code> | <code>Owned(T)</code>         | move <code>T</code> out — <b>no extra clone</b> |

Use `into_owned()` at the boundary when the next API takes `String` / `Vec<T>`, not `Cow`.



### 14.10.2 ToOwned and .to\_owned() on references

The **ToOwned** trait generalizes “borrowed view -> owned copy”. For **&str**, **.to\_owned()** is the same idea as **.to\_string()** — both produce a **String**:

```
// Playground
fn cache_key(prefix: &str, id: u32) -> String {
    format!("{}", prefix.to_owned(), id)
}

fn main() {
    let literal = "modbus";
    let owned = String::from("opcua");
    println!("{}", cache_key(literal, 1));
    println!("{}", cache_key(&owned, 2));
}
```

Typical implementations:

| Borrowed                              | .to_owned() gives                    |
|---------------------------------------|--------------------------------------|
| <b>&amp;str</b>                       | <b>String</b>                        |
| <b>&amp;[T]</b> where <b>T: Clone</b> | <b>Vec&lt;T&gt;</b>                  |
| <b>&amp;Path</b>                      | <b>PathBuf</b>                       |
| <b>Cow&lt;'a, T&gt;</b>               | <b>T</b> (via <b>.into_owned()</b> ) |

#### When to reach for .to\_owned()

- You hold **&str** / **&[u8]** and the next step needs **String** / **Vec<u8>** for storage or Send across threads.
- You are **normalizing** borrowed input once at the boundary (trim, replace) before pushing into a collection.

#### When to skip it

- The value is **already** **String** — use it as-is; **.to\_owned()** on **&String** allocates a **duplicate** (Chapter 20).
- You only **read** the data — take **&str** / **impl AsRef<str>** and borrow (**AsRef** above).

```
// Playground --- weak: double allocation
fn weak(label: &String) -> String {
    label.to_owned()
}
```

```

    // clones entire String when &str would
    // borrow
}

// Playground --- strong: borrow until you store
fn strong(label: &str) -> String {
    label.to_string()
    // or to_owned() --- one allocation when
    // storage is required
}

fn main() {
    let s = String::from("plc-1");
    println!("{}", weak(&s), strong(&s));
}

```

**.clone()** vs **.to\_owned()**: on **&T**, prefer **.to\_owned()** when you mean “give me an owned copy of this borrowed data”. On **String** / **Vec**, **.clone()** duplicates the owned value. The names signal intent to readers.

**API pattern:** accept **Cow<'a, str>** when you might normalize or pass through unchanged; return **String** (or call **.into\_owned()** before storing). Common in parsers and HTTP header helpers.

| Method               | On what                 | Result                                                         | Cost                                                                                           |
|----------------------|-------------------------|----------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>.to_owned()</b>   | borrowed <b>&amp;T</b>  | owned copy of <b>T</b><br>( <b>&amp;str</b> -> <b>String</b> ) | <b>heap alloc + copy</b> — same price as <b>.clone()</b> on owned data; skip if you can borrow |
| <b>.clone()</b>      | already-owned <b>T</b>  | duplicate of <b>T</b><br>( <b>String</b> -> <b>String</b> )    | <b>heap alloc + copy</b> for <b>String</b> / <b>Vec</b> ; cheap for <b>Copy</b> types          |
| <b>.into_owned()</b> | <b>Cow&lt;'a, T&gt;</b> | owned <b>T</b>                                                 | <b>move</b> if already Owned; <b>alloc + copy</b> if Borrowed                                  |

**One line:** borrow -> **.to\_owned()**; own and keep two -> **.clone()**; Cow -> plain owned -> **.into\_owned()**.

## 14.11 Deref — cheap access through smart pointers

Deref lets you treat **Arc<T>**, **Box<T>**, and newtypes like **T** for field access. In async code, tasks often hold **Arc<Config>** — destructure without cloning the whole struct:

```
// Playground
use std::ops::Deref;
use std::sync::Arc;

struct GatewayConfig {
    host: String,
    port: u16,
}

fn connect(cfg: Arc<GatewayConfig>) {
    let GatewayConfig { host, port, .. } =
        cfg.deref();
    println!("connecting to {}:{}", host, port);
}

fn main() {
    connect(Arc::new(GatewayConfig {
        host: "plc.local".into(),
        port: 502,
    }));
}
```

Rust also **deref-coerces** **&String** -> **&str** via **Deref**. Newtype wrappers often implement **Deref** to reach inner data (Chapter 10).

## 14.12 Extension traits on **&str**

Add string helpers as traits on **&str** (or your newtype) instead of a pile of free functions. Keeps call chains readable and each helper unit-testable:

```
// Playground
trait NormalizeWhitespace {
```

```
fn normalize_whitespace(self) -> String;
}

impl NormalizeWhitespace for &str {
    fn normalize_whitespace(self) -> String {
        self.split_whitespace().collect::<Vec<_>
            >().join(" ")
    }
}

fn clean_label(raw: &str) -> String {
    raw.normalize_whitespace()
}

fn main() {
    println!("{}", clean_label(" over temp
    "));
}
```

Use a **state machine or split loop** for hot paths; reach for regex only when the pattern truly needs it. Pair with Cow<str> when most inputs need no allocation (Cow above).

14.13 Common derive sets (cheat sheet)

| Type role       | Typical derives                      |
|-----------------|--------------------------------------|
| log / test DTO  | Debug, Clone, PartialEq              |
| config file     | Debug, Clone, Deserialize, Default   |
| map key         | Eq, Hash, Clone, Debug               |
| sortable record | Eq, Ord, PartialOrd, Debug           |
| error enum      | Debug, Error (thiserror) — Chapter 8 |

Do not derive Clone on unique handles. Do not derive Debug on secrets without redaction.

14.14 Java / Python contrast

| Idea              | Java                  | Python           | Rust                       |
|-------------------|-----------------------|------------------|----------------------------|
| toString          | toString()            | __str__          | Display / Debug            |
| equals / hashCode | equals, hashCode      | __eq__, __hash__ | PartialEq, Eq, Hash        |
| parsing           | constructors, valueOf | int(s)           | FromStr, TryFrom, .parse() |
| flexible param    | overloads             | duck typing      | AsRef, impl Trait          |

## 14.15 When the compiler says no

Common errors in this chapter:

| Error (typical)                     | Cause                    | Fix                                      |
|-------------------------------------|--------------------------|------------------------------------------|
| X doesn't implement Display         | used {}                  | {:?}", or impl Display                   |
| the trait bound Eq is not satisfied | f64 or partial type      | fixed-point or PartialEq only            |
| T: Hash not satisfied               | key not hashable         | derive Hash + Eq                         |
| conflicting From impls              | two From to same type    | one canonical path                       |
| orphan rule                         | impl Display for Vec<u8> | newtype wrapper                          |
| ? can't convert error               | missing From impl        | From or map_err                          |
| Borrow / key mismatch               | wrong lookup type        | use Borrow target (&str for String keys) |

## 14.16 Idiom spotlight

**Implement From, accept impl AsRef<str>** in public helpers. Callers pass literals or owned strings; you stay allocation-free when borrowing suffices.

**Display for users, Debug for developers** — derive Debug freely; invest in Display where humans read the output.

**Validate at the boundary** — TryFrom / FromStr at parse time; trust types inside the core.

**Pair Display + FromStr** on CLI enums; use **FromStr on path strings** to build sum types in one step.

## 14.17 Go deeper

- [The Rust Book — Useful Traits](#)
- [The Rust Book — Derivable Traits \(Appendix C\)](#)
- [From/Into](#)
- [Borrow trait](#)

## 14.18 See also

- Chapter 7: Structs, traits, and generics — orphan rule, `impl Trait`
- Chapter 8: Errors and testing — `From` in `?`, custom errors
- Chapter 11: Collections — trait bounds on keys
- Chapter 3: Functions — flexible parameters, first `#[derive(Debug)]` on errors
- Chapter 17: Derive attributes — `std` vs ecosystem vs custom derives
- Chapter 19: I/O — `AsRef<Path>`, CLI path/env patterns

### 14.18.1 Afterparty

#### 14.18.1.1 Debug, Display, Default

1. **Debug vs Display** — “When derive `Debug` only vs implement `Display` for a CLI status line?”
2. **Redacted Debug** — “Struct with `api_key: String` — sketch manual `Debug` with `[REDACTED]`.”
3. **Pretty debug** — “Same struct with `{:#?}` vs `{:?}` — when does pretty-print help in tests?”
4. **Display impl** — “Implement `Display` for `Port(u16)` showing `Port(8080)` — I write `fmt`, you review.”
5. **Default enum** — “Three-variant mode enum — derive `Default` with `#[default]` on `Auto`; show update syntax.”

#### 14.18.1.2 Eq, Hash, Ord

6. **Derive set quiz** — “Map key, log line, sortable row, error enum — I list derives each needs.”
7. **Eq on floats** — “Struct with `f64` field — show `Eq` derive failure; three fixes.”
8. **HashMap key** — “Why does `UserId(String)` need `Eq + Hash` for `HashMap` keys?”

9. **Ord sort** — “Sort `Vec<MyRecord>` by timestamp field — bounds needed on `MyRecord`?”

#### 14.18.1.3 From, TryFrom, FromStr

10. **From chain** — “`String -> MyLabel` via `From`; add `From<&str>` without duplicating logic.”
11. **TryFrom port** — “Port validation with custom enum error `OutOfRange` instead of `&str`.”
12. **parse vs TryFrom** — “When `s.parse:<u16>()` vs `u16::try_from(x)` vs custom `FromStr`?”
13. **FromStr type** — “Parse `host:port` into struct — sketch `FromStr` with split and two parse steps.”
14. **Silent cast trap** — “Show `70000i32 as u16` vs `TryFrom` — predict values; argue for validation.”
15. **From in errors** — “Wire `ParseIntError` into `AppError` with `From` so ? works — list impl only.”

#### 14.18.1.4 AsRef, Borrow, Cow

16. **AsRef drill** — “Rewrite three functions taking `&String` to impl `AsRef<str>`.”
17. **AsRef bytes** — “Function logging wire payload — signature with impl `AsRef<[u8]>`; accept `Vec, &[u8], array`.”
18. **Borrow lookup** — “Explain `HashMap<String, V>.get(&str)` — role of `Borrow<str>`.”
19. **Cow API** — “Normalize slug: accept `Cow<str>`, return borrowed if already valid else owned.”
20. **into\_owned** — “When caller needs `String` after your `Cow` helper — where call `into_owned()`?”
21. **to\_owned vs clone** — “Three snippets: `&str->store`, `&String->store`, `already-String` — pick to `owned`, `borrow`, or `move`; explain each.”

#### 14.18.1.5 API design and capstone

21. **Newtype Display** — “Wrap `Vec<u8>` as `HexBytes` — implement `Display` without orphan violation.”
22. **Derive audit** — “Config with secrets + TOML — list safe vs unsafe derives.”

23. **Mini crate API** — “Public `HostPort { host, port }` with `Display`, `TryFrom<&str>` for `host:port` — list impl blocks only.”
24. **Capstone** — “Design public API for `RateLimit { max: u32, window_secs: u64 }`: parsing, display, equality — traits only, no bodies.”



# **Part II**

# **Concurrency**

# Chapter 15

# Chapter 14: Multithreading

## 15.1 Hook

Concurrency models differ by language. **Python** threads fight the GIL for CPU work. **Java** threads share the heap with locks. Rust threads are **OS threads**. **Safe Rust** blocks many data races at compile time via Send and Sync. Use channels and mutexes when sharing is real.

## 15.2 Scope — a brief tour

Intro to OS threads, channels, and Arc<Mutex<T>> — not pools, IPC, or lock-free depth.

| This chapter covers                              | Deferred to See also / Afterparty             |
|--------------------------------------------------|-----------------------------------------------|
| thread::spawn, join, move, thread::scope (brief) | detached threads, custom pools                |
| mpsc channels                                    | bounded channels, backpressure, worker pools  |
| Arc<Mutex<T>>, RwLock, OnceLock (brief)          | Condvar, Barrier, LazyLock depth              |
| Send / Sync basics                               | RefCell across threads, pinning               |
| One worker sketch (poll + channel)               | rayon, custom thread pools, cross-process IPC |
| —                                                | Lock-free atomics -> Chapter 15               |
| —                                                | Async concurrency -> Chapter 16               |

### 15.3 What multithreading is

A **thread** is an independent call stack scheduled by the OS. Your program can run multiple threads at once — or interleaved on one CPU.

| Idea              | Plain language                                                                                                                      |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Thread            | Its own stack; runs code in parallel with other threads                                                                             |
| Why bother        | Overlap <b>I/O wait</b> (serial port + network) or parallelize <b>isolated</b> work units                                           |
| Rust twist        | Ownership and borrowing apply <b>across threads</b> — the compiler rejects many sharing mistakes that become data races in C++/Java |
| Two safe patterns | <b>Message passing</b> (mpsc channels) or <b>shared state</b> (Arc<Mutex<T>>)                                                       |

main thread: spawn worker → worker runs →  
join() waits → use result

Arc exists largely because threads need **shared ownership** — Rc is single-threaded only.

### 15.4 Send and Sync — the thread boundary rules

When data crosses a thread boundary, the compiler checks two marker traits:

| Trait | Meaning (simplified)                                          |
|-------|---------------------------------------------------------------|
| Send  | Owning value may be <b>moved</b> to another thread            |
| Sync  | Shared <b>&amp;T</b> may be used from multiple threads safely |

Most types you write are both Send and Sync. Common exceptions:

| Type  | Send? | Sync? | Why                       |
|-------|-------|-------|---------------------------|
| Rc<T> | no    | no    | ref count not thread-safe |

| Type                          | Send?                       | Sync?                       | Why                                    |
|-------------------------------|-----------------------------|-----------------------------|----------------------------------------|
| <code>RefCell&lt;T&gt;</code> | yes*                        | no                          | interior mutability<br>not thread-safe |
| <code>Mutex&lt;T&gt;</code>   | yes if <code>T: Send</code> | yes if <code>T: Send</code> | lock enforces<br>exclusive access      |

\*Moving `RefCell` to another thread is allowed; sharing `&RefCell` across threads is not.

`thread::spawn` requires a **Send** closure with **'static** captures. Captured data must outlive the spawn and be safe to transfer. That is why `Rc` inside a spawned thread fails but `Arc` works (Chapter 10).

## 15.5 Examples: elementary -> hard

Work through the levels in order. After each snippet: **run it**, then read **what happened**.

### 15.5.1 Level 1 — Elementary: spawn and join

```
// Playground
use std::thread;
use std::time::Duration;

fn main() {
    let handle = thread::spawn(|| {
        thread::sleep(Duration::from_millis(10))
        ;
        42
    });
    println!("joined = {:?}", handle.join());
}
```

**What happened:**

- Main spawns a child thread; child sleeps 10 ms, returns 42.
- **joined = 0k(42)** — `join()` waits for the child and wraps its return value in `0k`.

- If the child **panics**, `join()` returns `Err(...)` (not a normal return) — see Chapter 8 — panic and unwind and Level 6 below.

## 15.5.2 Level 2 — Elementary: move into the thread

Data used inside the closure must be **owned** or **'static**. Usually you **move** values into the new thread:

```
// Playground
use std::thread;

fn main() {
    let msg = String::from("poll");
    let handle = thread::spawn(move || {
        println!("{msg}");
    });
    handle.join().unwrap();
}
```

What happened:

- Prints **poll** — the child owns `msg` inside its closure.
- **move** transfers ownership from `main` into the thread; `msg` is **not** usable in `main` after `spawn`.

Wrong — use `msg` after `move`:

```
// Playground --- does not compile
use std::thread;

fn main() {
    let msg = String::from("poll");
    let handle = thread::spawn(move ||
        println!("{msg}");
    println!("{msg}");
    // ERROR: borrow of moved value: `msg`
    handle.join().unwrap();
}
```

Fix: `msg.clone()` before `spawn` if both sides need a copy. That **duplicates the String on the heap**. Large buffers can get costly. Cheaper patterns: return data via `join()` or a **channel**, or share with **Arc** (Level 4) when

many threads need one allocation.

### 15.5.3 Level 3 — Medium: mpsc channel

**Multi-producer, single-consumer** — send messages instead of sharing mutable state:

```
// Playground
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();
    thread::spawn(move || {
        tx.send(1).unwrap();
        tx.send(2).unwrap();
    });
    for val in rx {
        println!("got {}", val);
    }
}
```

**What happened:**

- Prints **got 1** then **got 2** — order preserved (FIFO).
- **move** on the closure: **tx** ownership moves to the worker; main keeps **rx**.
- When all **tx** senders are **dropped**, the **for val in rx** loop **ends** — channel closed.
- **Message passing** often beats shared mutable state for work queues and command streams.

### 15.5.4 Level 4 — Medium: Arc<Mutex<T>> shared counter

When threads must **mutate the same data**, wrap it in **Mutex** (exclusive access) and **Arc** (shared ownership):

```
// Playground
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
```

```
let counter = Arc::new(Mutex::new(0));
let mut handles = vec![];

for _ in 0..4 {
    let c = Arc::clone(&counter);
    handles.push(thread::spawn(move || {
        let mut n = c.lock().unwrap();
        *n += 1;
    }));
}
for h in handles {
    h.join().unwrap();
}
println!("count = {}",
    *counter.lock().unwrap());
}
```

| Piece                | Role                                                                    |
|----------------------|-------------------------------------------------------------------------|
| Arc                  | Reference-counted handle — many threads hold the <b>same</b> allocation |
| Mutex                | Only one thread mutates inner T at a time                               |
| lock().unwrap()      | Block until lock acquired; <b>panic</b> if mutex is <b>poisoned</b>     |
| Arc::clone(&counter) | Cheap handle copy — points at same Mutex                                |

Arc::clone vs cloning the inner value — easy to confuse:

| Call                               | What it copies                                    | Cost                                                |
|------------------------------------|---------------------------------------------------|-----------------------------------------------------|
| Arc::clone(&counter)               | Pointer + atomic ref-count only — same heap Mutex | <b>Cheap</b> — O(1), no data duplicate              |
| (*counter.lock().unwrap()).clone() | Inner T when T: Clone (e.g. whole String, Vec)    | <b>Expensive</b> — full deep copy of protected data |

In the loop, **Arc::clone(&counter)** gives each thread a **handle** to the **one** shared counter. Calling **.clone() on the inner value** copies the data under the lock. That is the wrong tool for “many threads, one counter.” See Chapter 10 — Arc vs deep clone.

What happened:

- Prints **count** = 4 — each of four threads incremented once; no lost updates.
- Without **Mutex**, sharing `&mut` across threads would not compile — Rust blocks the data race at compile time.
- **Lock poisoning**: if a thread **panics while holding the lock**, others get **PoisonError** on `lock()` — the lock may be inconsistent (Chapter 8).

### 15.5.5 Level 5 — Hard: Send trap — Rc cannot enter a thread

Types must implement **Send** to be **moved** into another thread. **Rc** is not **Send**:

```
// Playground --- does not compile
use std::rc::Rc;
use std::thread;

fn main() {
    let data = Rc::new(0);
    thread::spawn(move || {
        println!("{data}");
    });
    // ERROR: `Rc<i32>` cannot be sent between
    threads safely
}
```

What happened:

- Compiler **rejects** the spawn — Rc ref-count is not thread-safe.
- **Fix**: use **Arc::new(0)** (atomic ref-count) — same pattern as Level 4.

```
// Playground
use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(0);
    let d = Arc::clone(&data);
    thread::spawn(move ||
        println!("{d}")).join().unwrap();
    println!("main still has {data}");
}
```



```
}
```

**What happened:** compiles and prints **0** twice — Arc is **Send + Sync** when the inner type allows it. **Arc::clone(&data)** is cheap (ref-count bump); main and the worker share **one** i32 on the heap, not two copies.

## 15.5.6 Level 6 — Hard: join without unwrap + sensor poll worker

**6a. Handle panics at the boundary** (Chapter 8 — don't unwrap production join in unattended services):

```
// Playground
use std::thread;

fn main() {
    let handle = thread::spawn(|| {
        // simulate failure: panic!("device
        fault");
        100
    });
    match handle.join() {
        Ok(reading) => println!("ok {reading}"),
        Err(_) => eprintln!("worker panicked
        --- log and continue supervisor
        loop"),
    }
}
```

**What happened:**

- **Ok(100)** -> prints **ok 100**.
- If you uncomment the panic -> **Err(\_)** arm runs; **main survives** — contrast with panic in the worker killing the whole process if unhandled.

**6b. Poll worker sketch — poll thread, main logs readings:**

```
// Playground
use std::sync::mpsc;
use std::thread;
use std::time::Duration;
```

```
fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        for id in 1..=3 {
            thread::sleep(Duration::from_millis(
                5));
            tx.send(format!("sensor-{{id}}:
                {:.1}", 20.0 + id as
                f64)).unwrap();
        }
        // tx dropped here --- channel closes
    });

    for reading in rx {
        println!("log {}", reading);
    }
    println!("poll loop ended");
}
```

What happened:

- Prints **log sensor-1: 21.0, sensor-2: 22.0, sensor-3: 23.0**, then **poll loop ended**.
- Worker **produces**; main **consumes** — classic **message-passing work queue** without shared mutable state.
- Dropping **tx** when the worker finishes closes the channel; main's **for reading in rx** exits cleanly.

## 15.6 RwLock — many readers, rare writers

Sensor cache: many threads read, one thread reloads config:

```
// Playground
use std::collections::HashMap;
use std::sync::{Arc, RwLock};

fn main() {
    let cache: Arc<RwLock<HashMap<String,
```

```

f64>>> =
    Arc::new(RwLock::new(HashMap::from([("temp".into(), 22.5)])));

let reader = Arc::clone(&cache);
let t = std::thread::spawn(move || {
    let map = reader.read().expect("lock poisoned");
    println!("read temp={}", map["temp"]);
});
t.join().unwrap();

{
    let mut map =
        cache.write().expect("lock poisoned");
    map.insert("temp".into(), 23.1);
}
let map = cache.read().unwrap();
println!("after reload temp={}", map["temp"]);
}

```

.read() allows concurrent readers. .write() exclusive for updates. If a thread panics while holding a lock, the lock is **poisoned** — .expect or match on the Result.

## 15.7 OnceLock — initialize once

Lazy global config without hand-rolled double-checked locking:

```

// Playground
use std::sync::OnceLock;

struct Settings {
    port: u16,
}

static CONFIG: OnceLock<Settings> =

```

```

    OnceLock::new();

fn settings() -> &'static Settings {
    CONFIG.get_or_init(|| Settings { port: 502
    })
}

fn main() {
    println!("port={}", settings().port);
    println!("same={}", settings().port);
}

```

First call runs the closure; later calls return the same reference. For stack-local one-time init, `std::sync::LazyLock` (Rust 1.80+) works similarly without `static`.

## 15.8 `thread::scope` — borrow stack data into threads

Parallel checksum over chunks — each thread borrows `&chunks[i]` safely:

```

// Playground
use std::thread;

fn checksum(chunk: &[u8]) -> u32 {
    chunk.iter().map(|&b| b as u32).sum()
}

fn main() {
    let chunks: Vec<[u8; 4]> = vec![[1, 2, 3,
    4], [5, 6, 7, 8], [9, 10, 11, 12]];

    thread::scope(|s| {
        let handles: Vec<_> = chunks
            .iter()
            .map(|chunk| s.spawn(||
                checksum(chunk)))
            .collect();
    });
}

```

```

    let total: u32 =
        handles.into_iter().map(|h|
            h.join().unwrap()).sum();
        println!("total={}", total);
    });
}

```

scope guarantees threads join before returning — so borrows from chunks cannot dangle. Plain `spawn` cannot borrow local `Vec` elements without 'static data.

### 15.8.1 Sync primitive edge cases

**RwLock vs Mutex:** use `RwLock` when reads dominate; `Mutex` is simpler when writes are frequent or critical sections are tiny.

**Poisoned lock:** if a writer panics, `read()` returns `Err`. Log and rebuild cache, or use `into_inner()` on the poison error to recover the inner value.

## 15.9 Techniques at a glance

Popular primitives — one line each. Details for starred rows live in Afterparty or linked chapters.

| Technique                              | One-line use                                         | Where                                                       |
|----------------------------------------|------------------------------------------------------|-------------------------------------------------------------|
| <code>thread::spawn / join</code>      | Start OS thread; wait for result                     | Levels 1–2, 6                                               |
| <code>move</code> closures             | Transfer ownership into thread                       | Levels 2–4                                                  |
| <code>mpsc</code>                      | Message passing, work queues                         | Levels 3, 6b                                                |
| <code>Arc&lt;Mutex&lt;T&gt;&gt;</code> | Shared mutable state, short locks                    | Level 4                                                     |
| <b>Send</b>                            | Safe to <b>move</b> <code>T</code> to another thread | Level 5                                                     |
| <b>Sync</b>                            | Safe to share <b>&amp;T</b> across threads           | Level 4 ( <code>Arc</code> shares <code>&amp;Mutex</code> ) |
| <code>RwLock</code>                    | Many readers, rare writers                           | Section above                                               |
| <code>rayon / thread pools</code>      | CPU-bound parallelism                                | Afterparty / Go deeper                                      |
| <code>atomics</code>                   | Lock-free counters, flags                            | Chapter 15                                                  |

| Technique                   | One-line use              | Where      |
|-----------------------------|---------------------------|------------|
| <code>async / .await</code> | Many concurrent I/O waits | Chapter 16 |

### Send / Sync in practice:

- Most plain types (`i32`, `String`, `Vec`) are **Send** and **Sync**.
- **Rc**, **RefCell** — not **Sync** (single-thread interior mutability).
- **Raw pointers** — neither unless you enforce safety yourself (`unsafe`).
- **Arc<Mutex<T>>** — share **Sync** handle; mutation only inside **lock()**.

## 15.10 Idiom spotlight

**Prefer channels for work queues and command streams; use `Mutex` for short critical sections.** Long-held locks hurt poll latency (a Modbus cycle can miss its slot).

**I/O-bound services:** one thread blocked on serial read, another on network publish — overlap waits without fighting the GIL.

**Handle `join()` like `Result`** at the supervisor boundary — worker panic is data, not necessarily process death.

## 15.11 Go deeper

- [The Rust Book — Fearless Concurrency](#)
- [Mutex basics](#) — 986
- [Thread pool](#) — 923
- [Arc threads](#) — 109

## 15.12 See also

- Chapter 8: Errors and panic — `join()` after worker panic, why `unwrap` in loops is risky
- Chapter 10: Arc and smart pointers — `Rc` vs `Arc`
- Chapter 12: Closures — `move` and `Fn` traits for `spawn`
- Chapter 15: Atomics — lock-free counters, `OnceLock` vs hand-rolled `init`
- Chapter 16: Async — when threads are not the right tool

### 15.12.1 Afterparty

Use these for worker pools and topics not covered above.

#### 15.12.1.1 Spawn, move, join

1. **Race quiz** — “Which snippets are data races in C++ but rejected by Rust compiler?”
2. **Move fix** — “Show spawn without move that fails; fix with move or clone.”
3. **Join panic** — “Worker panics; rewrite main to match `join()` and keep supervisor alive.”
4. **Detached threads** — “Why is `mem::forget(handle)` after spawn dangerous? Better pattern?”

#### 15.12.1.2 Channels

5. **Channel design** — “Worker pool with `mpsc`: I describe throughput; sketch thread count + channel shape.”
6. **Drop tx footgun** — “Main exits before worker sends — diagram who holds tx/rx.”
7. **Multiple producers** — “Clone tx to two workers; main receives merged stream — sketch code.”
8. **Bounded vs unbounded** — “When does unbounded `mpsc` blow memory in automation? Bounded alternative?”

#### 15.12.1.3 Mutex and shared state

9. **Mutex vs RwLock** — “Read-heavy sensor cache — pick primitive and why.”
10. **Hold lock briefly** — “Refactor bad code that calls network I/O while holding Mutex lock.”
11. **Deadlock sketch** — “Two mutexes, lock order A then B vs B then A — show hang scenario.”
12. **Poison recovery** — “Thread panics holding lock; show `PoisonError` and `into_inner()` recovery.”
13. **Send fix** — “I try to move `Rc` into thread; show fix with `Arc`.”
14. **RefCell trap** — “Why `Arc<RefCell<T>>` is not Sync; fix pattern for shared mutation.”

**15.12.1.4 Production and automation**

15. **PLC gateway layout** — “Sketch poll thread + command channel + main supervisor; no code over 40 lines.”
16. **Lock latency** — “Modbus cycle 20 ms — max time holding mutex for register cache update?”
17. **Level ladder recap** — “Explain Levels 1–6 in one paragraph each for a Java teammate.”

**15.12.1.5 RwLock, OnceLock, and scope**

18. **RwLock cache** — “Read-heavy sensor map — sketch `Arc<RwLock<HashMap>>`; when does write starve readers?”
19. **Mutex vs RwLock** — “Same cache with 50% writes — pick Mutex or RwLock and justify in two sentences.”
20. **OnceLock init** — “`get_or_init` fails second init with different value — show `OnceLock` behaviour.”
21. **Scope borrow** — “Parallel sum over `Vec<u8; 512>` with `thread::scope` — why plain spawn fails on `&chunk`.”
22. **Poison recovery** — “Writer thread panics holding `RwLock` — show poisoned `read()` error and recovery options.”
23. **Capstone sync** — “Design: lazy config (`OnceLock`), shared cache (`RwLock`), scoped batch workers — list types only.”



# Chapter 16

## Chapter 15: Atomics and Lock-Free Basics

### 16.1 Hook

Lock-free primitives appear in many runtimes (**Java** `java.util.concurrent.atomic.`  
**Python** atomics behind the GIL in CPython, ...). Rust exposes `std::sync::atomic` for lock-free counters and flags when mutex overhead is too high. **Memory ordering** is part of the contract.

### 16.2 Scope — a brief tour

Practical atomics for counters and flags — not lock-free data structures or formal memory-model proofs.

| This chapter covers                                                  | Deferred to See also / Afterparty         |
|----------------------------------------------------------------------|-------------------------------------------|
| AtomicBool, AtomicUsize, load / store / fetch_add / compare_exchange | Lock-free queues, stacks, hazard pointers |
| Practical <b>Ordering</b> subset                                     | Full formal memory-model proofs           |
| <b>Data races</b> + happens-before intuition                         | Formal verification                       |
| Shared flags/counters in <b>threads and async</b>                    | crossbeam, hand-rolled lock-free DS       |
| When <b>not</b> to use atomics                                       | SIMD atomics, portable-atomic edge cases  |

## 16.3 Where atomics fit — threads, async, and Chapter 14

Atomics solve **concurrent read/write of one machine word** without a **Mutex lock**. They appear in **OS-thread** code (Chapter 14) and **async** services (Chapter 16). Same types, same rules.

| Context                          | Shared pattern                                  | Atomics role                                                   |
|----------------------------------|-------------------------------------------------|----------------------------------------------------------------|
| <b>OS threads</b> (Ch14 Level 4) | Arc<AtomicUsize> across thread::spawn           | Hot counter without mutex contention                           |
| <b>Async tasks</b> (Ch16)        | Arc<AtomicBool> read by many tokio::spawn tasks | Cooperative <b>shutdown</b> without locking the runtime        |
| <b>Both</b>                      | Arc wraps the atomic — Arc::clone is cheap      | Metrics: Relaxed; flags with dependent data: Release / Acquire |

Ch14: thread::spawn → Arc<AtomicUsize>  
fetch\_add

Ch16: tokio::spawn → Arc<AtomicBool>  
load/store

**Key message:** Async does **not** remove concurrency. Tasks on a thread pool still share memory. Atomics are **Send + Sync**. Wrap them in **Arc** when many tasks or threads need the same counter or flag.

Atomics do **not** replace channels (message streams) or **Mutex** (multi-field invariants). One atomic word is not a lock-free entire struct.

## 16.4 Data races — why atomics exist

A **data race** (informal): two threads access the **same memory**, at least one **writes**, with **no synchronization**. In C/C++ that is **undefined behavior**.

| Kind                             | Example                                | Safe Rust?                                            |
|----------------------------------|----------------------------------------|-------------------------------------------------------|
| <b>Prevented at compile time</b> | two &mut to same String in two threads | <b>compile error</b>                                  |
| <b>Non-atomic shared counter</b> | static mut COUNTER += 1 in two threads | <b>unsafe UB</b> — do not do this                     |
| <b>Atomic operations</b>         | fetch_add(1, Ordering::Relaxed)        | <b>defined</b> — each op is indivisible for that word |

| Kind                             | Example                                                                  | Safe Rust?                                                  |
|----------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>Visibility bug</b> (ordering) | flag set with Relaxed but reader never sees prior writes to other fields | <b>defined atomic op, wrong logic</b> — use Release/Acquire |

### Logic race vs memory race:

- **fetch\_add** on AtomicUsize does **not lose increments** — hardware makes the RMW atomic.
- Plain **+= 1** on shared mut (in C or unsafe Rust) **can** lose updates — two threads read 5, both write 6.
- **Relaxed** is fine for **standalone metrics**; it is **wrong** when a flag must publish **other memory** (config buffer, shutdown state) — see Level 4–5.

### Conceptual anti-pattern (UB — not safe Rust):

```
// Playground --- do not write this;
    illustrates why atomics exist
// static mut COUNTER: u32 = 0;
// thread::spawn(|| unsafe { COUNTER += 1 });
// thread::spawn(|| unsafe { COUNTER += 1 });
// two unsynchronized writes = data race
    (undefined behavior)
```

Same job as Chapter 14 Level 4 Arc<Mutex<usize>> — atomics trade **expressiveness** for **speed** on simple ops.

## 16.5 Examples: elementary -> hard

Work through in order. After each snippet: **run it**, then read **what happened**.

### 16.5.1 Level 1 — Elementary: AtomicBool shutdown flag

Worker loops until main sets **shutdown** with **Release**; worker reads with **Acquire**:

```
// Playground
use std::sync::atomic::{AtomicBool, Ordering};
use std::sync::Arc;
```

```

use std::thread;
use std::time::Duration;

fn main() {
    let shutdown =
        Arc::new(AtomicBool::new(false));
    let flag = Arc::clone(&shutdown);

    let worker = thread::spawn(move || {
        while !flag.load(Ordering::Acquire) {
            thread::sleep(Duration::from_millis(
                5));
            // simulate poll work
        }
        println!("worker stopped");
    });

    thread::sleep(Duration::from_millis(20));
    shutdown.store(true, Ordering::Release);
    worker.join().unwrap();
}

```

### What happened:

- Main sleeps ~20 ms, sets shutdown **true**, worker exits loop, prints **worker stopped**, **join** succeeds.
- **Release store** (main) + **Acquire load** (worker) = **happens-before**: worker **sees** all memory writes main made **before** the store(**true**).
- Same pattern works behind **Arc<AtomicBool>** in **tokio::spawn** (Chapter 16) — tasks poll **load(Acquire)** between **.await** points.

## 16.5.2 Level 2 — Elementary: **fetch\_add** counter

Lock-free increment — port of Ch14 Mutex counter:

```

// Playground
use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::Arc;
use std::thread;

```

```
fn main() {
    let n = Arc::new(AtomicUsize::new(0));
    let mut handles = vec![];

    for _ in 0..4 {
        let c = Arc::clone(&n);
        handles.push(thread::spawn(move || {
            c.fetch_add(1, Ordering::Relaxed);
        }));
    }
    for h in handles {
        h.join().unwrap();
    }
    println!("{}", n.load(Ordering::Relaxed));
}
```

| Piece                 | Role                                                                 |
|-----------------------|----------------------------------------------------------------------|
| AtomicUsize           | One shared counter word — no Mutex                                   |
| fetch_add(1, Relaxed) | Atomic read-modify-write; <b>Relaxed</b> OK for metrics-only counter |
| Arc::clone(&n)        | Cheap handle — same atomic on heap (Chapter 14)                      |

What happened:

- Prints **4** — four threads each added 1; **no lost updates** (contrast non-atomic static mut).
- **Relaxed** does not synchronize **other** memory — fine here because **only** the atomic is touched.

16.5.3 Level 3 — Medium: compare\_exchange — cap a counter

**Compare-and-swap (CAS):** update only if current value matches expected — retry on contention:

```
// Playground
use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::Arc;
```

```

use std::thread;

const MAX: usize = 100;

fn try_inc(counter: &AtomicUsize) {
    loop {
        let current =
            counter.load(Ordering::Relaxed);
        if current >= MAX {
            return;
        }
        if counter
            .compare_exchange_weak(current,
                                   current + 1, Ordering::AcqRel,
                                   Ordering::Relaxed)
            .is_ok()
        {
            return;
        }
        // spurious failure --- retry
    }
}

fn main() {
    let n = Arc::new(AtomicUsize::new(98));
    let mut handles = vec![];

    for _ in 0..4 {
        let c = Arc::clone(&n);
        handles.push(thread::spawn(move ||
            try_inc(&c)));
    }
    for h in handles {
        h.join().unwrap();
    }
    println!("capped at {}",
            n.load(Ordering::Relaxed));
}

```

**What happened:**

- Starts at **98**; four threads race to increment; at most **2** succeed -> prints **capped at 100** (never exceeds MAX).
- **compare\_exchange\_weak** may fail spuriously — **always retry** in a loop.
- **AcqRel** on success: typical for RMW that publishes/consumes in one step.

**16.5.4 Level 4 — Medium: Release/Acquire publish pattern**

Worker updates **config**, then bumps **version**; reader observes version with **Acquire** before reading config:

```
// Playground
use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::{Arc, Mutex};
use std::thread;
use std::time::Duration;

fn main() {
    let config = Arc::new(Mutex::new(502u16));
    let version = Arc::new(AtomicUsize::new(0));

    let cfg_w = Arc::clone(&config);
    let ver_w = Arc::clone(&version);
    let writer = thread::spawn(move || {
        {
            let mut p = cfg_w.lock().unwrap();
            *p = 8080;
        } // release lock before publishing
        ver_w.fetch_add(1, Ordering::Release);
    });

    let cfg_r = Arc::clone(&config);
    let ver_r = Arc::clone(&version);
    let reader = thread::spawn(move || {
        loop {
            let seen =
                ver_r.load(Ordering::Acquire);
```

```
        if seen > 0 {
            let p = *cfg_r.lock().unwrap();
            println!("reader saw version {}
                    port {}", seen, p);
            break;
        }
        thread::sleep(Duration::from_millis(
            1));
    }
});

writer.join().unwrap();
reader.join().unwrap();
}
```

What happened:

- Writer sets port 8080, then **fetch\_add(1, Release)** publishes.
- Reader **load(Acquire)** on version -> guaranteed to see port 8080, not stale 502 from before the write.
- Happens-before chain:

writer: \*config = 8080 → version Release+1  
          → reader Acquire load → read config

Wrong — Relaxed on version only (visibility bug story):

Using **Relaxed** for both version load and store can let the reader see a new version **without** the config write on some architectures. **Release/Acquire** fixes the handoff. Metrics-only atomics do not need this; **publish patterns** do.

16.5.5 Level 5 — Hard: when Relaxed lies — ordering foot-gun

| Scenario                        | Ordering                     | Verdict                                        |
|---------------------------------|------------------------------|------------------------------------------------|
| Frames-processed counter        | Relaxed load/store           | <b>OK</b> — standalone stat                    |
| Shutdown flag + prior log flush | Relaxed both sides           | <b>Risky</b> — reader may not see flushed logs |
| Shutdown flag + prior log flush | Release store / Acquire load | <b>Correct</b> publish pattern                 |



| Scenario                     | Ordering          | Verdict                                        |
|------------------------------|-------------------|------------------------------------------------|
| "I want simple mental model" | SeqCst everywhere | <b>OK</b> at low frequency;<br>slightly slower |

### Sophisticated edge case — atomic flag + non-atomic payload:

```
// Playground --- conceptual bug pattern (do
    not ship)
use std::sync::atomic::{AtomicBool, Ordering};

struct BadHandoff {
    ready: AtomicBool,
    port: u16,
    // NOT atomic --- paired with ready
}

// Writer: port = 8080; ready.store(true,
    Ordering::Relaxed);
// Reader: if ready.load(Relaxed) { use port }
// may read stale port without Release/Acquire
```

Fix: **Release** after writing port, **Acquire** before reading port — or protect both under **Mutex** (Chapter 14).

**What happened (conceptually):** the atomic op is well-defined. **Your program logic** is wrong if ordering does not establish happens-before between the flag and other fields.

## 16.5.6 Level 6 — Hard: gateway sketch

**Metrics** (Relaxed) + **shutdown** (Release/Acquire) — same struct works for threads today, async tasks tomorrow:

```
// Playground
use std::sync::atomic::{AtomicBool,
    AtomicUsize, Ordering};
use std::sync::Arc;
use std::thread;
use std::time::Duration;
```

```

struct Gateway {
    polls: AtomicUsize,
    shutdown: AtomicBool,
}

fn main() {
    let gw = Arc::new(Gateway {
        polls: AtomicUsize::new(0),
        shutdown: AtomicBool::new(false),
    });

    let worker_gw = Arc::clone(&gw);
    let worker = thread::spawn(move || {
        while
            !worker_gw.shutdown.load(Ordering::Acquire) {
                thread::sleep(Duration::from_millis(
                    5));
                worker_gw.polls.fetch_add(1,
                    Ordering::Relaxed);
            }
    });

    thread::sleep(Duration::from_millis(25));
    println!("metrics polls={}",
        gw.polls.load(Ordering::Relaxed));
    gw.shutdown.store(true, Ordering::Release);
    worker.join().unwrap();
    println!("supervisor done");
}

```

### What happened:

- Worker increments **polls** several times with **Relaxed** (metrics-only).
- Supervisor prints poll count, sets **shutdown** with **Release**, worker observes **Acquire** and exits.
- In **async** (Chapter 16): replace `thread::spawn` with `tokio::spawn`, same `Arc<Gateway>` — atomics unchanged.

## 16.6 Memory ordering (practical subset)

| Ordering       | Typical use                                | Example                                  |
|----------------|--------------------------------------------|------------------------------------------|
| <b>Relaxed</b> | Standalone counters, stats                 | frames processed, bytes read, poll count |
| <b>Release</b> | Publish — “data is ready”                  | store shutdown true after flushing state |
| <b>Acquire</b> | Observe — “I see your prior writes”        | worker checks shutdown flag              |
| <b>AcqRel</b>  | Read-modify-write (CAS, fetch_add handoff) | Level 3 cap counter                      |
| <b>SeqCst</b>  | Simplest global order; slightly slower     | low-frequency flags when unsure          |

**When Relaxed is wrong:** any atomic that **guards other memory** (config version, shutdown + shared buffer) needs **Release/Acquire** (or **Mutex**).

**ABA (one line):** `compare_exchange` on **pointers** can succeed spuriously if the same bit pattern reappears after free/realloc. Relevant for lock-free queues. Use Afterparty for depth — not production queues here.

## 16.7 Atomics vs Mutex

| Aspect        | Atomics                          | Mutex (Ch14)                        |
|---------------|----------------------------------|-------------------------------------|
| Best for      | counters, flags, CAS on one word | multi-field invariants, Vec updates |
| Composability | single-word ops                  | arbitrary critical sections         |
| Debugging     | subtle ordering bugs             | coarser, clearer                    |
| Overhead      | no lock for hot counter          | lock + possible contention          |

### Decision sketch:

|                                      |           |
|--------------------------------------|-----------|
| one word, simple op (count, flag)?   | -> atomic |
| several fields must stay consistent? | -> Mutex  |
| stream of messages / commands?       | ->        |
| channel (Ch14)                       |           |
| custom lock-free queue?              | -> crate  |
| or Afterparty --- not hand-roll      |           |

**Port exercise:** swap Ch14 Level 4 Arc<Mutex<usize>> for Level 2 AtomicUsize when profiling shows lock contention on a hot path.

## 16.8 Edge cases and compiler traps

| Trap                      | Symptom                                      | Idiom                                    |
|---------------------------|----------------------------------------------|------------------------------------------|
| Non-atomic shared mut     | lost counts, UB in C/unsafe                  | Atomic* or Mutex                         |
| Relaxed for publish flag  | stale reads of non-atomic fields             | Release / Acquire                        |
| CAS without retry loop    | spurious<br>compare_exchange_weak<br>failure | loop until is_ok() or give up            |
| Atomics for large struct  | wrong tool — one word only                   | Mutex or channel                         |
| False sharing (advanced)  | cache line ping-pong<br>between cores        | pad/separate hot atomics —<br>Afterparty |
| ABA on pointer CAS        | wrong lock-free pop                          | epoch / RCU — Afterparty                 |
| Confusing panic with race | thread died vs data race                     | Ch8 join vs atomics                      |

**Wrong — compare\_exchange once, no retry:**

```
// Playground --- may silently fail to
    increment under contention
use std::sync::atomic::{AtomicUsize, Ordering};

fn bad_inc(n: &AtomicUsize) {
    let c = n.load(Ordering::Relaxed);
    let _ = n.compare_exchange(c, c + 1,
        Ordering::Relaxed, Ordering::Relaxed);
    // spurious failure -> lost increment
}
```

**Wrong — expect Mutex-less Vec push from many threads:**

```
// Playground --- does not compile
use std::sync::Arc;
use std::thread;

fn main() {
    let v = Arc::new(vec![1, 2, 3]);
```

```

let v2 = Arc::clone(&v);
thread::spawn(move || {
    // v2.push(4);
    // ERROR: cannot borrow as mutable
});
}

```

Use `Mutex<Vec<_>>`, a `channel`, or a dedicated concurrent crate — not a lone atomic.

## 16.9 Idiom spotlight

Start with `Mutex`; switch to atomics when profiling shows lock contention on a hot counter or flag. Use `Relaxed` for metrics-only words; use `Release/Acquire` when another thread must see your **prior non-atomic writes**.

Same `Arc<Atomic*>` behind **OS threads** (Chapter 14) and **async tasks** (Chapter 16). Do not hand-roll lock-free queues without study.

## 16.10 Go deeper

- [The Rust Book — Fearless Concurrency](#) — mutex-first baseline; atomics go deeper below
- [Atomic types — lock-free primitives](#) — 452
- [Rust nomicon — atomics](#)

## 16.11 See also

- Chapter 14: Multithreading — `Arc`, `Mutex`, threads
- Chapter 16: Async — tasks sharing `Arc<AtomicBool>`
- Chapter 8: Errors and panic — `join()` after panic vs race bugs

### 16.11.1 Afterparty

Use these for lock-free structures and formal memory-model topics not covered above.

**16.11.1.1 Concepts and placement**

1. **Threads vs async atomics** — “Same `Arc<AtomicBool>` in `thread::spawn` vs `tokio::spawn` — what differs, what stays the same?”
2. **Data race vs logic race** — “Define both; classify `lost static mut increment` vs `fetch_add`.”
3. **Ch14 port** — “Rewrite Mutex counter (Ch14 L4) as `AtomicUsize`; when is each better?”

**16.11.1.2 Memory ordering**

4. **Ordering quiz** — “Shutdown flag + published config pointer — pick orderings; justify.”
5. **When Relaxed lies** — “Metrics OK, config handoff broken — show Release / Acquire fix.”
6. **Fence intuition** — “Draw happens-before for Release store + Acquire load (Level 4).”
7. **Relaxed polls vs version** — “Why is Relaxed OK for Level 2 polls but wrong for Level 4 version? One sentence each.”

**16.11.1.3 CAS and compare\_exchange**

8. **Counter port** — “Cap counter with CAS loop; explain spurious `compare_exchange_weak` failure.”
9. **ABA problem** — “Explain ABA in 80 words for pointer `compare_exchange` — no full queue.”
10. **Retry loop** — “Show single-shot CAS anti-pattern; fix with loop.”
11. **Double-checked locking** — “Show broken lazy init with atomics; fix with `OnceLock` or explain why not hand-roll.”

**16.11.1.4 Races and debugging**

12. **Race quiz** — “Mark 6 snippets: safe atomic, UB static mut, ordering bug, needs Mutex.”
13. **Visibility story** — “Writer updates port then Relaxed flag — what can reader see?”

**16.11.1.5 Atomics vs Mutex vs channels**

14. **When not** — “Three cases atomics are the wrong tool; prefer channels or Mutex.”

15. **Mutex vs RwLock** — “Read-heavy sensor cache — atomic counter vs RwLock vs Mutex.”
16. **Vec push** — “Why many threads cannot push to shared Vec; three fixes.”
17. **Profile-first** — “Gateway uses Mutex for counter; profiler shows lock contention — minimal atomic refactor.”
18. **Channel vs atomic flag** — “When is crossbeam/bounded channel + shutdown better than lone AtomicBool?”

#### 16.11.1.6 Production and Java port

19. **Gateway metrics** — “Design Gateway { polls, shutdown } for async; orderings per field.”
20. **False sharing** — “Two hot atomics on same cache line — problem and mitigation in 60 words.”
21. **Lock-free queue** — “Why this chapter says don’t hand-roll; name crates/patterns instead.”

# Chapter 17

## Chapter 16: Async Rust and Tokio

### 17.1 Hook

Async/await shows up in many stacks (**Java** `CompletableFuture`, **Python** `asyncio`, JavaScript promises, ...). Rust feels familiar in shape, different in mechanics.

`async fn` returns a **Future** — lazy work until you `.await` or `spawn` it.

**Tokio** is the usual runtime for network and concurrent I/O.

### 17.2 Scope — a brief tour

Practical Tokio intro — not runtime internals or web-framework guides.

| This chapter covers                                                                                                        | Deferred to See also / Afterparty                                  |
|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <code>async / .await</code> , Future mental model                                                                          | Pin/Unpin formalism, <code>async</code> trait impls in depth (Ch7) |
| Tokio: <code>#[tokio::main]</code> , <code>spawn</code> , <code>join!</code> , <code>select!</code> , <code>timeout</code> | <code>async-std</code> , embedded runtimes, runtime tuning         |
| Blocking footgun + <code>spawn_blocking</code>                                                                             | <code>block_in_place</code> , worker thread counts                 |
| Async I/O overview ( <code>TcpListener</code> , <code>fs</code> )                                                          | Production TCP/Modbus stacks — Chapter 19                          |



This chapter covers

Deferred to See also / Afterparty

Shutdown via `Arc<AtomicBool>` (Ch15 L6)`axum`, `tonic`, stream combinators

## 17.2.1 What this chapter skips — and where to learn it

The table above is honest scope, not a teaser list. Each deferred row is safe to ignore until you hit the trigger in the right column:

| Deferred topic                                  | Learn it when...                                                                                          | Start here                                                                                                                                |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pin / Unpin</b>                              | You implement a manual Future, build self-referential async state, or read compiler errors mentioning Pin | <a href="#">Rustonomicon — Pin</a> ; return here after Chapter 7 traits                                                                   |
| <b>async fn in traits</b>                       | A gateway trait needs async methods (connect, poll_device)                                                | Chapter 7 — <code>async fn</code> in traits, Send bounds, dyn limits                                                                      |
| <b>Other runtimes / tuning</b>                  | Tokio defaults starve your workload or you target <code>no_std</code> / embedded                          | <a href="#">async-std</a> for API comparison; <a href="#">Embassy</a> for embedded; <a href="#">Tokio tuning</a> for worker thread counts |
| <b>block_in_place</b>                           | CPU-heavy or blocking work must run <i>on</i> an async worker thread, not the blocking pool               | <a href="#">Tokio — block_in_place</a> after you have measured executor latency (Level 5)                                                 |
| <b>Production TCP / Modbus stacks</b>           | One echo connection works; you need timeouts, retry, framing, and logging in production                   | Chapter 19 sync I/O first, then port the poll loop async                                                                                  |
| <b>axum, tonic, streams</b>                     | Raw <code>TcpListener</code> + spawn is understood; you want HTTP/gRPC routes or stream pipelines         | <a href="#">axum</a> , <a href="#">tonic</a> , <a href="#">tokio_stream</a> combinators                                                   |
| <b>tokio::sync::mpsc + cancellation hygiene</b> | Tasks need async message passing, or <code>select!</code> drops work mid-flight                           | Tokio sync docs; Level 4 caveat + <b>Cancellation hygiene</b> in Afterparty                                                               |

You do **not** need every row to ship a Level 6 supervisor. Add depth when a concrete project forces the question.

## 17.3 What async is

**Async** is cooperative concurrency: one OS thread hosts many **tasks**, each pausing at `.await` so the executor runs others. Unlike Chapter 14 threads (OS-preempted), async tasks **yield voluntarily** — which makes blocking calls inside them dangerous.

The [Rust Book — Async, Await, Futures, and Streams](#) covers the std mental model: **futures**, **tasks**, and **executors**. This chapter applies that model with **Tokio** — a third-party runtime the book uses in examples but does not standardize.

| Idea                             | Plain language                                                                                                                             |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>async fn</b>                  | Returns a <b>Future</b> — a state machine describing work; <b>does not run</b> until polled                                                |
| <b>.await</b>                    | Pause <b>this</b> task until the step is ready; the <b>executor</b> runs other tasks                                                       |
| <b>Executor (Tokio)</b>          | Thread pool + scheduler that <b>polls</b> futures to completion                                                                            |
| vs <b>OS thread</b> (Chapter 14) | Threads are preempted by the OS; async <b>cooperates</b> — a blocking call in one task can stall many                                      |
| <b>Shared state</b>              | Same concurrency rules: Arc, atomics (Chapter 15), <b>tokio::sync::Mutex</b> — not <b>std::sync::Mutex</b> held across <code>.await</code> |

When a task hits `.await`, control returns to the executor until the waited-on step is ready:

```
caller .await → executor runs other tasks →
               future ready → caller resumes
```

### 17.3.1 Level 0 — Mental model (sync stand-in, not async)

Before adding Tokio, anchor the **names** (`fetch`, `parse`, `log`) with ordinary sequential Rust. Real async needs a **runtime**; this Playground snippet is **not async** — it runs one step after another on one thread:

```
// Playground --- conceptual; NOT async (no
    executor)
fn main() {
    let tasks = vec!["fetch", "parse", "log"];
```

```

    for t in tasks {
        println!("step: {}", t);
    }
}

```

**What happened:** prints **step: fetch, parse, log** in order — one thread, no concurrency. An `async fn` would **not** interleave until `.await` yields to Tokio.

All Tokio examples below are **Cargo projects**. Add this dependency once:

```

[dependencies]
tokio = { version = "1", features = ["full"] }

```

## 17.4 Examples: elementary -> hard

The levels below build from “start a runtime” to a small async supervisor. Work through in order; after each snippet, read **what happened** before moving on.

### 17.4.1 Level 1 — Elementary: minimal Tokio spawn

Start here: boot Tokio with `#[tokio::main]`, spawn a child task, and `await` its join handle so `main` does not exit early.

```

// Cargo project
use tokio::time::{sleep, Duration};

#[tokio::main]
async fn main() {
    let handle = tokio::spawn(async {
        sleep(Duration::from_millis(50)).await;
        42
    });
    println!("result = {:?}", handle.await);
}

```

**What happened:**

- `#[tokio::main]` starts the Tokio runtime and runs `async fn main`.
- Child task sleeps 50 ms, returns **42**; `handle.await` waits for **Ok(42)**.

- Without `.await` on the handle, `main` could exit before the task finishes — dropped join handle **aborts** the task.

## 17.4.2 Level 2 — Elementary: sequential vs concurrent `.await`

Writing two `.await`s in one function does **not** run them in parallel. Compare sequential awaits with `tokio::join!` to see real async concurrency:

```
// Cargo project
use tokio::time::{sleep, Duration, Instant};

async fn sequential() {
    sleep(Duration::from_millis(100)).await;
    sleep(Duration::from_millis(100)).await;
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    sequential().await;
    println!("sequential ~{} ms",
        start.elapsed().as_millis());

    let start = Instant::now();
    tokio::join!(
        sleep(Duration::from_millis(100)),
        sleep(Duration::from_millis(100)),
    );
    println!("join! ~{} ms",
        start.elapsed().as_millis());
}
```

What happened:

- **sequential**  $\approx 200$  ms — second sleep starts after the first completes.
- **tokio::join!**  $\approx 100$  ms — both sleeps run **concurrently** on the executor.
- For background work that outlives the caller, use `tokio::spawn` (Level 1).

### 17.4.3 Level 3 — Medium: `join!` and `timeout`

Services often wait on **multiple** device polls at once and must **give up** when hardware is slow. `join!` waits for all branches; `timeout` caps how long one future may run:

```
// Cargo project
use tokio::time::{sleep, timeout, Duration};

async fn fast_poll() -> u16 {
    sleep(Duration::from_millis(10)).await;
    502
}

async fn slow_poll() -> u16 {
    sleep(Duration::from_millis(500)).await;
    8080
}

#[tokio::main]
async fn main() {
    let (a, b) = tokio::join!(fast_poll(),
                             fast_poll());
    println!("join ports {} {}", a, b);

    match timeout(Duration::from_millis(50),
                  slow_poll()).await {
        Ok(port) => println!("slow ok {}",
                             port),
        Err(_) => eprintln!("slow poll timed
                             out"),
    }
}
```

**What happened:**

- `join!` runs both `fast_poll` tasks concurrently -> prints **502 502** quickly.
- `timeout(50ms, slow_poll())` returns `Err(_)` — slow poll needs 500 ms. This is the **deadline** pattern for Modbus/serial timeouts.

### 17.4.4 Level 4 — Medium: `select!` — first branch wins

When you want the **fastest** response and can abandon slower paths, `select!` races branches and **cancels** the losers:

```
// Cargo project
use tokio::time::{sleep, Duration};

async fn fast_cache_hit() -> &'static str {
    sleep(Duration::from_millis(5)).await;
    "cached"
}

async fn slow_device_read() -> &'static str {
    sleep(Duration::from_millis(200)).await;
    "device"
}

#[tokio::main]
async fn main() {
    let source = tokio::select! {
        v = fast_cache_hit() => v,
        v = slow_device_read() => v,
    };
    println!("source = {}", source);
}
```

What happened:

- Prints **source = cached** — fast branch completes first; **slow branch is cancelled** (dropped future).
- **Caveat:** cancelled work may stop mid-flight — ensure partial I/O is safe; see Afterparty for cancellation hygiene.

### 17.4.5 Level 5 — Hard: `blocking` footgun + `spawn_blocking`

Tokio runs many async tasks on a **small** set of OS worker threads. When a task hits `.await`, it **steps aside** so the worker can run other tasks. **Blocking** code never steps aside — it holds the worker until it finishes.

| What you call                                                  | What actually happens                                                                    |
|----------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <code>thread::sleep(100ms)</code> inside <code>async fn</code> | Worker thread is <b>stuck</b> for 100 ms. Every other task on that worker <b>waits</b> . |
| <code>tokio::time::sleep(100ms).await</code>                   | Task releases the worker. Other tasks run during the wait.                               |
| <code>std::fs::read</code> / heavy CPU on hot path             | Same as <code>thread::sleep</code> — blocks the worker.                                  |
| <code>tokio::task::spawn_blocking(...)</code>                  | Moves blocking work to a <b>separate</b> thread pool. Async workers stay free.           |

**Restaurant analogy:** one waiter (worker) serves many tables (tasks). `.await` = “I’ll check back when the kitchen is ready.” `thread::sleep` = waiter stands frozen at one table while others go ignored.

Under load, a few `thread::sleep` or `std::fs::read` calls in `async` handlers can slow **every** connection — not just the one that blocked.

The snippet below runs two experiments with `tokio::join!`: first with `std::thread::sleep` (bad), then with `tokio::time::sleep` (good), then `spawn_blocking` for work you cannot make `async`.

```
// Cargo project
use std::thread;
use std::time::Duration as StdDuration;
use tokio::time::{sleep, Duration, Instant};

async fn bad_blocking() {
    thread::sleep(StdDuration::from_millis(100))
        ; // stalls executor worker
}

async fn good_async_sleep() {
    sleep(Duration::from_millis(100)).await;
    // yields --- other tasks run
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    tokio::join!(bad_blocking(),
```

```

        sleep(Duration::from_millis(10));
        println!("bad path elapsed ~{} ms (10 ms
            task delayed)",
            start.elapsed().as_millis());

    let start = Instant::now();
    tokio::join!(good_async_sleep(),
        sleep(Duration::from_millis(10)));
    println!("good path elapsed ~{} ms",
        start.elapsed().as_millis());

    let n = tokio::task::spawn_blocking(|| {
        thread::sleep(StdDuration::from_millis(5
            0));
        99
    })
    .await
    .unwrap();
    println!("spawn_blocking -> {}", n);
}

```

Quick reference for what belongs inside an async task:

| Call                                     | In async task? | Effect                                                                       |
|------------------------------------------|----------------|------------------------------------------------------------------------------|
| <code>std::thread::sleep</code>          | <b>Bad</b>     | blocks executor worker — other tasks wait                                    |
| <code>tokio::time::sleep().await</code>  | <b>Good</b>    | yields until deadline                                                        |
| <code>std::fs::read</code> on hot path   | <b>Risky</b>   | blocks — prefer <code>tokio::fs</code> or <b><code>spawn_blocking</code></b> |
| <code>tokio::task::spawn_blocking</code> | <b>Good</b>    | runs closure on blocking thread pool                                         |

### What happened:

- **bad\_blocking + 10 ms task** — prints elapsed  $\approx 100+$  ms. The 10 ms sleep **started late** because `thread::sleep` hogged the worker. Two “concurrent” tasks ran **one after another**.
- **good\_async\_sleep + 10 ms task** — prints elapsed  $\approx 100$  ms. Both tasks **shared** the worker during waits; wall time is  $\sim \max(100, 10)$ , not  $100 +$



10.
- **spawn\_blocking** — prints **spawn\_blocking** -> **99**. The 50 ms `thread::sleep` ran on a **blocking** thread; async workers were not stuck.

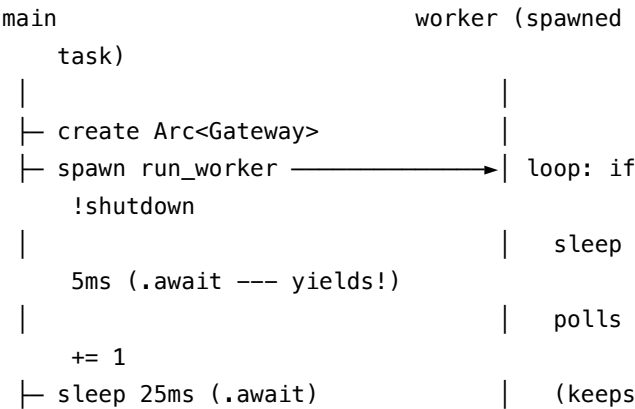
**Rule of thumb:** in Chapter 14, `thread::sleep` on its own OS thread only blocks **that** thread. Inside Tokio async code, it blocks a **shared** worker — so it hurts **all** tasks on that worker.

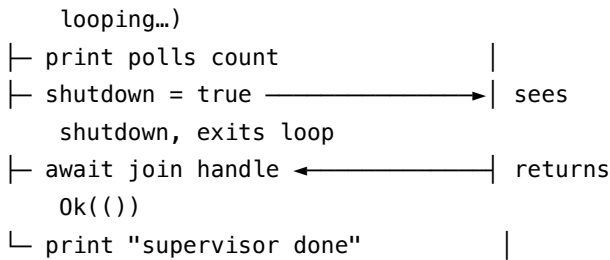
### 17.4.6 Level 6 — Hard: async gateway supervisor

This mirrors Chapter 15 Level 6, but the worker is a **Tokio task** instead of an OS thread:

| Piece                     | Role                                                                     |
|---------------------------|--------------------------------------------------------------------------|
| <b>Gateway</b>            | Shared state: poll counter + shutdown flag                               |
| <b>Arc&lt;Gateway&gt;</b> | Lets <b>main</b> and the worker <b>both</b> hold the same struct safely  |
| <b>run_worker</b>         | Loop: sleep 5 ms -> increment polls -> repeat until shutdown             |
| <b>tokio::spawn</b>       | Starts the worker <b>in the background</b> without blocking <b>main</b>  |
| <b>shutdown flag</b>      | <b>main</b> sets <b>true</b> ; worker sees it and exits the loop cleanly |

**main is the supervisor; run\_worker is the poll loop** on a device gateway. Supervisor runs for 25 ms, prints poll count, then stops the worker and waits for a clean exit.





**Why atomics again?** Same as Chapter 15: `polls` is a **metric** (Relaxed is fine). `shutdown` is a **signal** — Release when main writes, Acquire when the worker reads.

**Why `sleep(...).await` in the worker?** Each 5 ms pause **yields** to Tokio. `thread::sleep` here would block a worker on every iteration (Level 5 foot-gun).

```
// Cargo project
use std::sync::atomic::{AtomicBool,
    AtomicUsize, Ordering};
use std::sync::Arc;
use tokio::time::{sleep, Duration};

struct Gateway {
    polls: AtomicUsize,
    shutdown: AtomicBool,
}

async fn run_worker(gw: Arc<Gateway>) ->
    Result<(), &'static str> {
    while !gw.shutdown.load(Ordering::Acquire) {
        sleep(Duration::from_millis(5)).await;
        gw.polls.fetch_add(1,
            Ordering::Relaxed);
    }
    Ok(())
}

#[tokio::main]
async fn main() -> Result<(), &'static str> {
    let gw = Arc::new(Gateway {
```

```

        polls: AtomicUsize::new(0),
        shutdown: AtomicBool::new(false),
    });

    let worker_gw = Arc::clone(&gw);
    let handle = tokio::spawn(async move {
        run_worker(worker_gw).await });

    sleep(Duration::from_millis(25)).await;
    println!("metrics polls={}",
        gw.polls.load(Ordering::Relaxed));

    gw.shutdown.store(true, Ordering::Release);
    handle.await.map_err(|_| "worker join
        failed")??;
    println!("supervisor done");
    Ok(())
}

```

### What happened (summary):

1. Worker polls concurrently with main while supervisor sleeps 25 ms.
2. **polls.load(Relaxed)** prints something like **metrics polls=4** (~25 ms ÷ 5 ms per loop).
3. **shutdown.store(true, Release)** stops the worker after the current iteration.
4. **handle.await** waits for the task to finish. The **??** applies **?** to both the join result and **run\_worker**'s **Result**.
5. **supervisor done** — no dangling background task.

**Compared to Chapter 15:** replace `thread::spawn + thread::sleep` with `tokio::spawn + tokio::time::sleep().await`. **Atomics** and **shutdown** pattern stay the same — only the runtime changes.

**main() -> Result** (Chapter 8): errors bubble to one place instead of scattering `unwrap()` in the supervisor.

### 17.4.7 Level 7 — Medium: bounded concurrency with Semaphore

Unbounded `tokio::spawn` on thousands of devices can overwhelm databases or serial buses. A **Semaphore** caps how many in-flight operations run at once:

```
// Cargo project
use std::sync::Arc;
use tokio::sync::Semaphore;
use tokio::time::{sleep, Duration};

async fn probe_device(id: u32) -> u32 {
    sleep(Duration::from_millis(20)).await;
    id
}

#[tokio::main]
async fn main() {
    let sem = Arc::new(Semaphore::new(2));
    // at most 2 probes at once
    let mut handles = Vec::new();

    for id in 0..5u32 {
        let permit = Arc::clone(&sem);
        handles.push(tokio::spawn(async move {
            let _permit =
                permit.acquire().await.unwrap();
            // held until task finishes
            probe_device(id).await
        }));
    }

    for h in handles {
        println!("id {}", h.await.unwrap());
    }
}
```

What happened:

- Five tasks are spawned, but `acquire().await` blocks when two permits

are already taken.

- The `_permit` guard releases the slot when the task ends — no manual release.
- Same pattern protects MongoDB cursors, HTTP fan-out, and Modbus batch reads.

### 17.4.8 Level 8 — Medium: batch spawn and await??

Fan out independent async work with `tokio::spawn`, then collect results. Each `handle.await` yields `Result<T, JoinError>` — use `??` when the inner task also returns `Result` (Chapter 8):

```
// Cargo project
use tokio::time::{sleep, Duration};

async fn fetch_reading(seed: u32) ->
    Result<f64, &'static str> {
    sleep(Duration::from_millis(10)).await;
    if seed == 3 {
        Err("device offline")
    } else {
        Ok(seed as f64 * 1.5)
    }
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn
    std::error::Error>> {
    let seeds = [1u32, 2, 3];
    let mut handles = Vec::new();

    for seed in seeds {
        handles.push(tokio::spawn(fetch_reading(
            seed)));
    }

    for handle in handles {
        match handle.await? {
            Ok(v) => println!("reading {}", v),
```

```

        Err(e) => eprintln!("skip: {}", e),
    }
}
Ok(())
}

```

| Layer | Type                                   | ? / ??                               |
|-------|----------------------------------------|--------------------------------------|
| Outer | JoinError from panicked/cancelled task | first ? on <code>handle.await</code> |
| Inner | Result<T, E> from your async fn        | second ? when you want to propagate  |

Use **match** on the inner Result when one failure should not abort the whole batch (as above for seed 3).

### 17.4.9 Async trait boundaries for testing

Async methods in traits are not natively object-safe without **async\_trait** (crate). Production and test code share one interface — MongoDB in prod, mock in tests (Chapter 8 — trait mocks):

```

// Cargo project
// async-trait = "0.1"
use async_trait::async_trait;
use std::sync::{Arc, Mutex};

#[async_trait]
trait PipelineEffects: Send + Sync {
    async fn write_batch(&self, tags: &[&str])
        -> Result<usize, String>;
}

struct LiveEffects;

#[async_trait]
impl PipelineEffects for LiveEffects {
    async fn write_batch(&self, tags: &[&str])
        -> Result<usize, String> {
        Ok(tags.len())
    }
}

```

```

    }
}

struct MockEffects {
    pub calls: Mutex<Vec<Vec<String>>>,
}

#[async_trait]
impl PipelineEffects for MockEffects {
    async fn write_batch(&self, tags: &[&str])
        -> Result<usize, String> {
        self.calls
            .lock()
            .unwrap()
            .push(tags.iter().map(|s|
                s.to_string()).collect());
        Ok(tags.len())
    }
}

async fn run_pipeline(effects: Arc<dyn
    PipelineEffects>) -> Result<(), String> {
    effects.write_batch(&["temp",
        "press"]).await?;
    Ok(())
}

#[tokio::main]
async fn main() -> Result<(), String> {
    let live: Arc<dyn PipelineEffects> =
        Arc::new(LiveEffects);
    run_pipeline(live).await?;

    let mock: Arc<dyn PipelineEffects> =
        Arc::new(MockEffects {
            calls: Mutex::new(Vec::new()),
        });
    run_pipeline(Arc::clone(&mock)).await?;
}

```

```

    assert_eq!(mock.calls.lock().unwrap().len(),
               1);
    Ok(())
}

```

Bounds **Send** + **Sync** on the trait allow **Arc<dyn PipelineEffects>** across tasks. Keep the trait **small** — same decomposition advice as Chapter 7.

### 17.4.10 Pipeline sketch — reader task, channel, workers

Long-running services decouple **read throughput** from **write latency** with bounded channels. One task pulls documents/frames; workers consume batches:

```

// Cargo project --- outline
use tokio::sync::mpsc;

async fn reader_task(tx: mpsc::Sender<Vec<u8>>)
    -> Result<(), String> {
    for i in 0u8..3 {
        tx.send(vec![i]).await.map_err(|e|
            e.to_string())?;
    }
    Ok(())
}

async fn worker_task(mut rx:
    mpsc::Receiver<Vec<u8>>) -> Result<(),
    String> {
    while let Some(batch) = rx.recv().await {
        println!("process {} bytes",
            batch.len());
    }
    Ok(())
}

#[tokio::main]
async fn main() -> Result<(), String> {
    let (tx, rx) = mpsc::channel(4);

```



```

// backpressure when full
let worker = tokio::spawn(worker_task(rx));
reader_task(tx).await?;
// dropping tx closes the channel
worker.await.unwrap()?;
Ok(())
}

```

**Bounded capacity** (4 here) applies backpressure: a fast reader blocks on `send().await` when workers fall behind — preferable to unbounded memory growth.

### 17.4.11 Polling loop — long-running HTTP or job status

APIs that return **202 + poll URL** fit a **loop**, **`sleep().await`**, and exit on success or timeout:

```

// Cargo project
use tokio::time::{sleep, timeout, Duration,
    Instant};

async fn job_ready(tick: u32) ->
    Option<&'static str> {
    if tick >= 3 {
        Some("done")
    } else {
        None
    }
}

async fn poll_until_ready(max_wait: Duration)
    -> Result<&'static str, &'static str> {
    let start = Instant::now();
    let mut tick = 0;
    loop {
        if let Some(status) =
            job_ready(tick).await {
            return Ok(status);
        }
    }
}

```

```

        if start.elapsed() >= max_wait {
            return Err("timed out waiting for
                       job");
        }
        tick += 1;
        sleep(Duration::from_millis(50)).await;
    }
}

#[tokio::main]
async fn main() {
    match timeout(Duration::from_secs(1),
        poll_until_ready(Duration::from_secs(5))
    ).await {
        Ok(Ok(s)) => println!("status = {}", s),
        Ok(Err(e)) => eprintln!("{}", e),
        Err(_) => eprintln!("outer timeout"),
    }
}

```

Wrap the whole poll in `timeout(...)` (Level 3) so a stuck remote cannot loop forever.

## 17.5 Techniques at a glance

Use this table as a cheat sheet while reading Tokio code elsewhere. Each row maps to a level above.

| Technique                         | One-line use                                        | Level           |
|-----------------------------------|-----------------------------------------------------|-----------------|
| <code>async fn / .await</code>    | lazy futures; yield points                          | 0–2             |
| <code>#[tokio::main]</code>       | start Tokio runtime                                 | 1               |
| <code>tokio::spawn</code>         | concurrent background task                          | 1, 6, 8         |
| <code>tokio::join!</code>         | wait for <b>all</b> branches                        | 2–3             |
| <code>tokio::select!</code>       | <b>first</b> ready branch                           | 4               |
| <code>tokio::time::timeout</code> | deadline / device timeout                           | 3, poll loop    |
| <code>spawn_blocking</code>       | sync I/O or <code>thread::sleep</code> off executor | 5               |
| Semaphore                         | cap in-flight async work                            | 7               |
| <code>mpsc</code> channel         | decouple reader and workers                         | pipeline sketch |

| Technique                          | One-line use               | Level            |
|------------------------------------|----------------------------|------------------|
| <code>async_trait</code>           | async methods on dyn Trait | testing boundary |
| <code>Arc&lt;AtomicBool&gt;</code> | shutdown flag across tasks | 6, Ch15          |

**Mutex choice:** use `tokio::sync::Mutex` if the lock may be held across `.await`. Holding `std::sync::MutexGuard` across `.await` makes the future **!Send** — the compiler rejects it (see edge cases below).

## 17.6 Async I/O (brief)

Networking and file I/O are where async pays off: many connections wait on kernel I/O while a small thread pool keeps working. Tokio mirrors std I/O with **async** APIs — `.await` instead of blocking the thread:

| Sync (Ch19)                      | Async (Tokio)                                        |
|----------------------------------|------------------------------------------------------|
| <code>std::fs::read</code>       | <code>tokio::fs::read</code>                         |
| <code>TcpListener::accept</code> | <code>tokio::net::TcpListener::accept().await</code> |

The sketch below accepts one connection, reads bytes, and echoes them back — a minimal pattern before a multi-connection server:

```
// Cargo project --- outline
use tokio::io::{AsyncReadExt, AsyncWriteExt};
use tokio::net::TcpListener;

#[tokio::main]
async fn main() -> Result<(), Box<dyn
    std::error::Error>> {
    let listener =
        TcpListener::bind("127.0.0.1:0").await?;
    let (mut socket, _) =
        listener.accept().await?;
    let mut buf = [0u8; 64];
    let n = socket.read(&mut buf).await?;
    socket.write_all(&buf[..n]).await?;
    Ok(())
}
```

Many connections -> **one process, many tasks** — each accept can `tokio::spawn` a handler. Full protocols and error handling: Chapter 19 + Afterparty.

## 17.7 Async vs OS threads

You already have OS threads from Chapter 14. Async is not a replacement — it is a different tool for **many concurrent waits** on a small thread pool:

| Aspect          | OS threads (Ch14)                         | Async tasks (Tokio)                                            |
|-----------------|-------------------------------------------|----------------------------------------------------------------|
| Best for        | CPU-bound parallel work                   | Many <b>I/O waits</b> (TCP, timers)                            |
| Stack / memory  | higher per thread                         | many tasks on thread pool                                      |
| Blocking call   | blocks one thread only                    | <b>blocks executor worker</b> — hurts all tasks on that worker |
| Shared shutdown | <code>Arc&lt;AtomicBool&gt;</code> (Ch15) | same atomics behind <code>tokio::spawn</code>                  |

When choosing for a device gateway or network service:

```
many idle TCP/serial waits?    -> async
CPU-bound parallel compute?    -> threads (or
    rayon --- Ch14 Afterparty)
one simple PLC poll loop?      -> single
    thread may suffice
1000 connections, one process? -> async
```

## 17.8 Edge cases and compiler traps

These mistakes show up constantly in real Tokio code. The table lists the symptom and the fix; the snippets below are **wrong on purpose** so you recognize them in the wild.

| Trap                       | Symptom                                      | Idiom                                            |
|----------------------------|----------------------------------------------|--------------------------------------------------|
| Forgot <code>.await</code> | logic never runs; unused future warning      | always <code>.await</code> or <code>spawn</code> |
| No runtime                 | <code>async fn main</code> won't run futures | <code>#[tokio::main]</code>                      |

| Trap                                                  | Symptom                       | Idiom                                                     |
|-------------------------------------------------------|-------------------------------|-----------------------------------------------------------|
| <code>std::thread::sleep</code> in <code>async</code> | latency spikes for all tasks  | <code>tokio::time::sleep / spawn_blocking</code>          |
| <code>MutexGuard</code> across <code>.await</code>    | Future not Send compile error | <code>tokio::sync::Mutex</code> ; drop guard before await |
| Blocking <code>std::fs</code> on hot path             | executor starvation           | <code>tokio::fs / spawn_blocking</code>                   |
| <code>select!</code> cancels losing branch            | partial work / leaked state   | scope cancellation; abort handles — Afterparty            |

### Wrong — `std::sync::MutexGuard` across `.await`:

```
// Cargo project --- does not compile
use std::sync::Mutex;
use tokio::time::{sleep, Duration};

async fn bad(m: Mutex<i32>) {
    let _guard = m.lock().unwrap();
    sleep(Duration::from_millis(10)).await;
    // ERROR: future cannot be sent between
    // threads safely (MutexGuard not Send)
}

// fn main() {
//     tokio::runtime::Runtime::new().unwrap().block_on(
//         bad(Mutex::new(0));
//     );
// }
```

### Wrong — drop future without `.await`:

```
// Cargo project
async fn important() {
    println!("never runs if not awaited");
}

#[tokio::main]
async fn main() {
    important();
    // warning: unused Future --- does NOT run
    // important().await; // correct
}
```

## 17.9 Idiom spotlight

One paragraph to carry into production code:

**Async for many I/O waits; threads for CPU-heavy or a single simple poll loop.** Never block the executor — **`tokio::time::sleep`**, not **`thread::sleep`**. Share shutdown with **`Arc<AtomicBool>`** (Chapter 15). Bubble errors with **`?`** and **`main()`** -> **`Result`** at the boundary (Chapter 8).

**Cap fan-out with `Semaphore`. Decouple stages with bounded `mpsc`. `async_trait`** for swappable I/O in tests.

## 17.10 Go deeper

When this chapter's ladder is not enough, these links cover fundamentals, channels, I/O, and the official Tokio walkthrough:

- [The Rust Book — Async, Await, Futures, and Streams](#)
- [async fn and .await fundamentals](#) — 321
- [Async channels mpsc](#) — 328
- [Async I/O](#) — 342, 921
- [Tokio tutorial](#)

## 17.11 See also

Related chapters — read these when you need threads, atomics, errors, sync I/O, or async traits in depth:

- Chapter 14: Multithreading — when threads beat async
- Chapter 15: Atomics — `Arc<AtomicBool>` shutdown in async
- Chapter 8: Errors — `Result` boundary in `main`
- Chapter 19: I/O — sync I/O and processes
- Chapter 7: Traits — async traits (advanced)

### 17.11.1 Afterparty

Use these for runtime internals and web-framework topics not covered above.

### 17.11.1.1 Concepts and mental model

Prompts to solidify how futures, polling, and scope fit together:

1. **Future diagram** — “Draw state machine for `async fn` with two `.await` points.”
2. **Poll vs await** — “Who polls the Future — caller, executor, or Tokio runtime?”
3. **Executor vs thread** — “One paragraph: cooperative async vs preemptive OS threads.”
4. **Forgotten await** — “Show unused Future bug; fix with `.await` or `spawn`.”

### 17.11.1.2 Tokio basics

Runtime setup, spawn lifecycle, and error boundaries:

5. **Tokio scaffold** — “Minimal `#[tokio::main]` + `spawn` + `join handle` — explain each line.”
6. **Spawn vs await** — “What if `main` drops `join handle` without `.await`?”
7. **Join handle ??** — “Explain the ?? on `handle.await` in Level 6 — outer `join Result` vs inner `run_worker Result`.”

### 17.11.1.3 Level 5–6 drills

Reinforce blocking vs yielding and the async gateway supervisor:

8. **Bad join timing** — “Walk through Level 5 bad path: why does a 10 ms task wait ~100 ms when paired with `thread::sleep`?”
9. **Ch15 port** — “Compare Level 6 to Ch15 L6 — what changes with `tokio::spawn`, what stays the same?”
10. **Supervisor timeline** — “Trace Level 6 step by step: when does the worker stop, and why must it use `tokio::time::sleep` not `thread::sleep`?”

### 17.11.1.4 Concurrency primitives

When to wait for all vs first, deadlines, and cancellation:

11. **join vs select** — “Two slow tasks: when `join!` vs `select!?` One automation example each.”
12. **select! scenario** — “Cancel slow request when fast path returns — full `select!` sketch.”

13. **Timeout drill** — “Wrap Modbus read async fn with 100 ms timeout; handle `Elapsed`.”
14. **Cancellation hygiene** — “What runs when `select!` drops the losing branch? Modbus read vs cache hit example.”

#### 17.11.1.5 Blocking and I/O

Keeping the executor healthy while doing real device and file work:

15. **Blocking fix** — “Audit async snippet with `thread::sleep`, `std::fs::read`; fix each.”
16. **spawn\_blocking** — “When `tokio::fs` vs `spawn_blocking` for config file read?”
17. **Tcp echo** — “Expand Async I/O outline to multi-connection echo with spawn per accept.”

#### 17.11.1.6 Async vs threads

Architecture choices for gateways and connection counts:

18. **async vs thread** — “1000 Modbus polls — argue async vs thread pool for latency.”
19. **200 TCP connections** — “One process — async vs thread-per-connection memory story.”
20. **When thread enough** — “Single serial port poll — justify thread loop over Tokio.”

#### 17.11.1.7 Atomics and shared state (Ch15)

Bridging lock-free flags from Chapter 15 into async tasks:

21. **Async sleep in worker** — “Why must Level 6’s poll loop use `tokio::time::sleep().await` on every iteration, not `thread::sleep`?”
22. **tokio Mutex** — “Rewrite bad `std::sync::MutexGuard` across `await` with `tokio::sync::Mutex`.”



## **Part III**

# **Metaprogramming and Unsafe**

## Chapter 18

# Chapter 17: Metaprogramming

### 18.1 Hook

Metaprogramming varies by ecosystem — **Java** annotations and reflection, **Python** metaclasses, Rust macros at compile time. Rust metaprogramming is mostly **compile-time**. Macros expand source **before** type checking — zero runtime cost for `macro_rules!` and `derive`.

| Approach                                             | When it runs            | Typical cost            |
|------------------------------------------------------|-------------------------|-------------------------|
| Java reflection                                      | Runtime                 | Slow; errors at runtime |
| Lombok / annotation processors                       | Compile time            | IDE-dependent           |
| Python metaclass                                     | Import / class creation | Runtime setup           |
| Rust <code>macro_rules!</code> / <code>derive</code> | Compile time            | No runtime overhead     |
| Rust <code>const fn</code> / <code>const</code>      | Compile time            | Baked into binary       |

### 18.2 Scope — a brief tour

Intro to `macro_rules!`, `derive`, and `const` — not proc-macro authoring with `syn/quote`.

| This chapter covers                                                                                                                                                                | Deferred to See also / Afterparty                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>macro_rules!</code> , fragments, repetition, hygiene                                                                                                                         | Writing proc macros with <code>syn/quote</code>                                  |
| <code>#[derive]</code> syntax, annotations vs decorators, <code>std</code> + ecosystem + <code>custom</code> derives, <code>Clone/Copy</code> , <code>serde JSON</code> round-trip | Proc-macro authoring with <code>syn/quote</code> , <code>build.rs</code> codegen |
| <code>const</code> / <code>const fn</code> / <code>env!</code> / <code>include_str!</code>                                                                                         | <code>generic_const_exprs</code> , advanced const traits                         |
| Forbidden matcher clauses, <code>edge cases</code> , <code>why macros are hard to trace</code>                                                                                     | Miri, macro fuzzing, proc-macro testing                                          |

## 18.3 Compile-time pipeline

**Mental model:** the compiler sees your source as **tokens**, expands every macro invocation, then type-checks the **expanded** program.

```
source tokens -> macro expansion -> type check
      + borrow check -> LLVM
```

Macros match **syntax trees** (tokens), not types. A macro can compile while the **expanded** code fails. Errors often say in expansion of macro ....

## 18.4 Under the hood — what the compiler actually does

Brief peek inside — enough to make restrictions and debugging feel logical:

*[Diagram omitted in print edition — see the web repository.]*

**Declarative (`macro_rules!`):**

- The **matcher** compares token trees against patterns with fragment specifiers (`expr`, `ty`, ...).
- The **transcriber** pastes captured fragments into a template — substitution, not evaluation of Rust code.
- No access to types, trait bounds, or name resolution at match time — only token shape.

**Procedural (`#[derive]`, `sql!`, ...):**

- The compiler calls a proc-macro function with a **TokenStream** (tokens + span metadata).
- The macro runs as **normal Rust at compile time** (often using `syn` + `quote`) and returns another `TokenStream`.
- A **proc-macro = true crate** may export **only** proc macros. Helpers live in a sibling crate — a hard platform rule.

### Hygiene (why names “disappear”):

- Macro-generated identifiers get a fresh **syntax context**. They do not collide with caller locals, but also **won’t resolve** to caller bindings unless passed as `$x:expr`.
- `$crate` exists so exported macros refer to **their** crate’s paths, not the caller’s.

## 18.5 macro\_rules!

```
// Playground
macro_rules! say_hello {
    () => { println!("Hello!"); };
    ($name:expr) => { println!("Hello, {}!",
                             $name); };
}

fn main() {
    say_hello!();
    say_hello!("Automation");
}
```

**What happened:** the macro matches **token patterns** — empty `()` or one expression — and expands to `println!` calls before type checking.

### 18.5.1 Fragment specifiers

| Specifier          | Matches    | Use when                           |
|--------------------|------------|------------------------------------|
| <code>expr</code>  | expression | values, calls, literals            |
| <code>ident</code> | identifier | names, field keys                  |
| <code>ty</code>    | type       | generic params, type slots         |
| <code>pat</code>   | pattern    | match arms in macro-generated code |

| Specifier | Matches                   | Use when                             |
|-----------|---------------------------|--------------------------------------|
| stmt      | statement (no trailing ;) | one-line statements                  |
| block     | { ... } block             | multi-statement bodies               |
| item      | fn, struct, mod, ...      | generating whole items               |
| literal   | 42, "hi", true            | numeric / string constants           |
| tt        | token tree                | escape hatch — match almost anything |

18.5.2 Repetition

```
// Playground
macro_rules! vec_str {
    ($($x:expr),*) => {{
        let mut v = Vec::new();
        $( v.push(String::from($x)); )*
        v
    }};
}

fn main() {
    let v = vec_str!["a", "b"];
    println!("{:?}", v);
}
```

What happened:

- `$(...)*` repeats zero or more times — `vec_str! []` would work with `*`, not with `+`.
- `$( ... )*` in the output expands once per matched input.
- Trailing comma: `$(x),*` allows "a", "b", — common Rust style.

| Modifier | Meaning      |
|----------|--------------|
| *        | zero or more |
| +        | one or more  |
| ?        | zero or one  |

18.5.3 Level 2 — automation: register map

Map symbolic register names to addresses — typical Modbus-style tables:

```
// Playground
macro_rules! register_map {
    ($($name:ident = $addr:literal),* $(,)? ) =>
    {{
        fn addr_of(name: &str) -> Option<u16> {
            match name {
                $( stringify!($name) =>
                    Some($addr), )*
                _ => None,
            }
        }
        addr_of
    }};
}

fn main() {
    let lookup = register_map!(temp = 0x01,
                               pressure = 0x02);
    println!("{:?}", lookup("temp"));
}
```

**What happened:** `ident` captures names; `literal` captures `u16` values; repetition builds a match over `stringify!` keys — all at compile time, zero runtime parsing of a table file.

### 18.5.4 Hygiene and `$crate`

Exported macros should use `$crate::` for paths inside **your** crate so re-exports and dependency trees resolve correctly. Macro-generated `let` names are **hygienic** — they won't accidentally shadow or capture caller locals.

Debug expanded code:

```
cargo install cargo-expand
cargo expand --bin my_app    # in a Cargo project
```

Conceptual `cargo expand on #[derive(Debug)]` for `Point` emits an `impl Debug for Point { fn fmt(...) { ... } }` — the `derive` proc macro wrote that `impl`; you never see it in source unless you expand.

## 18.6 Syntax forbidden or restricted in macros — and why

The macro matcher is a **separate, weaker parser** that must stay **unambiguous today and in future Rust editions** (follow-set rules).

**Important nuance:** follow-set rules limit what can follow `$e:expr` in a **matcher** — not what a macro can **emit**. You can still generate `for` loops in expanded code.

What is restricted: (1) keywords **after** `$e:expr` / `$s:stmt` in a matcher, and (2) expanding **multiple statements** where Rust expects a **single expression**.

### 18.6.1 Follow-set: keywords after `expr` / `stmt`

For `$e:expr` and `$s:stmt`, the only tokens allowed **immediately after** the fragment in a matcher are `=>`, `,`, and `;`. Nothing else — not `=`, not `for`, not `let`, not `if`, not `>>`.

| Fragment                    | May only be followed by                                                                                                                                      | Forbidden example                                                               | Why                                                                      |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <code>expr, stmt</code>     | <code>=&gt;, ,, ;</code>                                                                                                                                     | <code>\$a:expr for</code><br><code>\$i:ident in</code><br><code>\$r:expr</code> | <code>for</code> could start a trailing expression — parser won't commit |
| <code>expr, stmt</code>     | <code>=&gt;, ,, ;</code>                                                                                                                                     | <code>\$a:expr let</code><br><code>\$x:ident = \$v:expr</code>                  | same follow-set rule                                                     |
| <code>expr, stmt</code>     | <code>=&gt;, ,, ;</code>                                                                                                                                     | <code>\$a:expr &gt;&gt; \$b:expr</code>                                         | <code>&gt;&gt;</code> could be part of a future expression               |
| <code>pat, pat_param</code> | <code>=&gt;, ,, =, \  , if, in, ...</code>                                                                                                                   | <code>\$p:pat + \$q:pat</code>                                                  | <code>+</code> not in follow-set — but <b><code>in</code> is allowed</b> |
| <code>ty, path</code>       | <code>=&gt;, ,, =, \  , ;, :, &gt;, &gt;&gt;, ', {, as, where, ...</code><br><code> \$t:ty + \$u:ty  +'</code><br>might start a different grammar production |                                                                                 |                                                                          |

**Wrong — `$e:expr` followed by `for` (matcher fails at macro definition):**

```
// Playground --- does not compile
macro_rules! bad_foreach_matcher {
```

```

($e:expr for $i:ident in $r:expr) => { $e };
}
// error: `e:expr` is followed by `for`, which
//       is not allowed for `expr` fragments
// note: allowed there are: `=>`, ` `, or `;`

```

**Wrong — `$e:expr` followed by `=`:**

```

// Playground --- does not compile
macro_rules! set_reg {
    ($addr:expr = $val:expr) => { /* ... */ };
}
// error: `$addr:expr` is followed by `=`,
//       which is not allowed for `expr` fragments
// FIX: set_reg!(0x01, 42) or set_reg! { 0x01
//       => 42 }

```

**Right — `$p:pat` followed by `in` (foreach-style matcher):**

The keyword `in` is in the follow-set for `pat` / `pat_param`, so this pattern is **legal**. Literal `for` comes *before* the pattern, not after an `expr`:

```

// Playground
macro_rules! foreach {
    ($p:pat in $r:expr => $body:expr) => {
        for $p in $r { $body }
    };
}

fn main() {
    foreach!(i in 0..3 => println!("tick {}",
        i));
}

```

**What happened:** the matcher uses `$p:pat in $r:expr` — allowed because `in` follows `pat`. The transcriber emits a normal `for` loop. This is how you write “mini foreach” macros without breaking follow-set rules.



## 18.6.2 Expression position vs statement expansion (let, for, multiple lines)

A macro invoked where Rust expects an **expression** (`let x = mac!(); mac!() + 1`) must expand to **one** expression. A bare `{ let a = 1; for i in 0..n { ... } }` block is a **block expression** and can work.

A transcribe template with **multiple statements** separated at the top level often breaks.

**Wrong — let + for in expression position (single-brace body):**

```
// Playground --- does not compile when used as
// expression
macro_rules! poll_twice {
    () => {
        let n = 2;
        for i in 0..n {
            println!("poll {}", i);
        }
    };
}

fn main() {
    let _x = poll_twice!();
    // error: expected expression, found `let`
    // statement
    // note: macro expansion ignores keyword
    // `for` and any tokens following
}
```

**Fix — double braces `{{ ... ; last_expr }}`:**

```
// Playground
macro_rules! poll_twice {
    () => {{
        let n = 2;
        for i in 0..n {
            println!("poll {}", i);
        }
    }}
    ()
    // block's value --- unit, or return a
}
```

```
        // count/Result
    }
};

fn main() {
    let _x = poll_twice!();
}
```

**What happened:** outer { ... } is the macro’s transcribe delimiter. Inner { ... ; () } is a **block expression** — one value Rust can assign to `_x`. Without the inner block, `let` and `for` are loose statements, invalid in expression position.

| Situation                                         | Works?                            | Pattern                                                                     |
|---------------------------------------------------|-----------------------------------|-----------------------------------------------------------------------------|
| for loop only, used as expression                 | often yes (loop is an expression) | => { for ... { } }                                                          |
| let then for, used as expression                  | no (single braces)                | => {{ let ...; for ...; value }}                                            |
| for after <code>\$e:expr</code> in <b>matcher</b> | never                             | use <code>\$p:pat</code> in <code>\$r:expr</code> or change DSL punctuation |
| for in macro <b>output</b> at statement position  | yes                               | <code>poll_twice!();</code> as its own statement                            |

Same logic applies to **let**, **while**, and **match** when they appear **after** an expr fragment in a matcher. It also applies to **let-heavy expansions** in expression context.

**Other important limits:**

| Rule                                               | What breaks                                                         | Why                                             |
|----------------------------------------------------|---------------------------------------------------------------------|-------------------------------------------------|
| Forwarded fragments are <b>opaque</b>              | Inner macro can’t match literals inside outer <code>\$e:expr</code> | Fragment is an AST blob                         |
| No type introspection in <code>macro_rules!</code> | “Derive Clone only for Clone fields”                                | Macro sees tokens, not types — needs proc macro |
| Recursion depth cap (~128)                         | <code>recursion limit exceeded</code>                               | Protects compile time                           |
| <code>ident</code> excludes <code>\$crate</code>   | Reserved for path injection                                         | Hygiene                                         |

**Escape hatch:** `tt` (token tree) matches delimited chunks — **you** validate structure (often in a proc macro with `syn`).

## 18.7 Standard library macros you already use

| Macro                                                                      | Role                                                  |
|----------------------------------------------------------------------------|-------------------------------------------------------|
| <code>vec!</code> , <code>format!</code> , <code>println!</code>           | collection / formatting                               |
| <code>matches!</code>                                                      | pattern guard sugar ([Chapter 6 — Advanced patterns]) |
| <code>env!</code> , <code>option_env!</code>                               | build-time strings (Preface)                          |
| <code>include_str!</code> , <code>concat!</code> , <code>stringify!</code> | embed / stringify at compile time                     |
| <code>cfg!</code> , <code>assert!</code> , <code>debug_assert!</code>      | conditional compile / tests                           |
| <code>todo!</code> , <code>unimplemented!</code>                           | stub markers                                          |

```
// Playground
fn main() {
    let name = env!("CARGO_PKG_NAME");
    // compile-time; needs Cargo project
    let msg = format!("running {}", name);
    assert!(matches!(msg.as_str(), s if
        s.starts_with("running")));
    println!("{}", msg);
}
```

## 18.8 Derive attributes

`#[derive(...)]` is **core Rust syntax** — not optional decoration. You place it **above** a struct or enum to auto-implement one or more **traits** before type checking runs. You will see it from the first error enum (Chapter 3) through config types (Chapter 13), `thiserror` (Chapter 8), and `serde` below.

```
// Playground
#[derive(Debug, Clone, PartialEq)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let a = Point { x: 1, y: 2 };
}
```

```
let b = a.clone();
println!("{:?} == {:?} ? {}", a, b, a == b);
}
```

Comma-separated trait names in one attribute: `#[derive(Debug, Clone)]`. Field-level helpers (e.g. `#[serde(rename = "...")]`) sit on individual fields and are interpreted by the same ecosystem crate — see Serde JSON below.

### 18.8.1 Not annotations or decorators

If you come from **Java**, **Python**, **C#**, or **TypeScript**, do **not** read `#[derive]` as an annotation or decorator. The mental model is different.

| Habit from other languages                                      | What you might assume                              | What Rust <code>#[derive]</code> actually does                                                    |
|-----------------------------------------------------------------|----------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Java <code>@Override</code> ,<br><code>@SuppressWarnings</code> | Hints to compiler / IDE                            | <b>Generates Rust source</b> (impl blocks) at compile time                                        |
| Java Spring <code>@Autowired</code> ,<br>JAX-RS annotations     | Runtime wiring via reflection                      | <b>No runtime reflection</b> — expanded code is normal Rust                                       |
| Python <code>@property</code> ,<br><code>@staticmethod</code>   | Wraps or replaces the function at <b>call time</b> | <b>Does not wrap</b> your type — emits trait impls once at <b>compile time</b>                    |
| Python <code>@dataclass</code>                                  | Runtime class builder                              | Closest cousin — but Rust expand is <b>before</b> type check, zero runtime cost                   |
| C# / TypeScript <b>decorators</b>                               | Metadata or method interception at runtime         | Rust derive is <b>not</b> interception — it is <b>codegen</b>                                     |
| Lombok <code>@Data</code><br>(compile-time Java)                | Closest analogy                                    | Still different: Rust errors are in <b>your</b> crate's type check, not a separate processor pass |

#### Rules of thumb for migrants:

- `#[derive(Name)]` -> “run the proc macro registered for `Name`” — usually emits impl for a trait also called `Name` (compile time).
- `#[some_attr]` on a **function** (e.g. `#[test]`, `#[tokio::main]`) -> attribute proc macro — also compile time, but not derive; see [Proc macros — three kinds](#).
- **No attribute runs when you call a function** unless you explicitly invoked a macro that generated that call.

## 18.8.2 Mechanics

1. `#[derive(Name)]` looks up a **registered derive proc macro** for that identifier — built into the compiler (`Debug`, `Clone`, ...) or shipped in a dependency (`Serialize`, `Error`, ...).
2. The macro **expands** and typically emits `impl SomeTrait for YourType { ... }` (and sometimes extra items). **Name is usually the same as `SomeTrait`**, but the compiler invokes the **macro registration**, not “any trait you defined.”
3. The compiler type-checks the expanded code like hand-written Rust.

**Not automatic:** writing `trait Foo { ... }` in your application module does **not** enable `#[derive(Foo)]`. A proc-macro = true crate must register `#[proc_macro_derive(Foo)]` ([custom derives below](#)). You can always `impl Foo for Bar { ... }` by hand without a derive.

**No derive for every trait:** `Display` is a real std trait with **no** `#[derive(Display)]`. Unknown names fail at compile time: *cannot find derive macro Foo in this scope*.

Inspect expansion when confused: `cargo expand` ([Go deeper](#)).

**Field rule:** `derive` succeeds only when **every field** (or every variant’s fields) already implements the trait. Otherwise: “field X doesn’t implement Y”.

**Enum vs struct:** enums get per-variant match arms in generated `Debug`, `Clone`, `PartialEq`, ...; structs get field-wise impls.

**Chained requirements:**

| Derive            | Also requires                                      |
|-------------------|----------------------------------------------------|
| <code>Copy</code> | <code>Clone</code> (all fields <code>Copy</code> ) |
| <code>Eq</code>   | <code>PartialEq</code>                             |
| <code>Ord</code>  | <code>PartialEq + Eq</code>                        |
| <code>Hash</code> | usually paired with <code>Eq</code> for map keys   |

**What happened in the `Point` example:** `Clone` generated `fn clone(&self) -> Self { ... }` field-wise. `PartialEq` generated `==` field-wise.

**Manual vs derive:** `derive` when default behaviour fits. Hand-write for redacted `Debug`, custom `Clone` (share `Arc` instead of deep copy), or enum

Default with `#[default]` on one variant (Rust 1.62+). Chapter 13 lists which std traits can be derived vs must be written by hand (Display has no derive).

**Wrong — Eq on floats:**

```
// Playground --- does not compile
// #[derive(Eq, PartialEq)]
// struct Reading { value: f64 }
// FIX: integer fixed-point, `PartialEq` only,
//    or ordered-float crate --- see Ch7
```

### 18.8.3 Std derives, ecosystem derives, and custom derives

Three sources — same **syntax**, different implementers:

| Source                           | Examples                                               | Provided by                                             |
|----------------------------------|--------------------------------------------------------|---------------------------------------------------------|
| <b>Standard library</b>          | Debug, Clone, Copy, PartialEq, Eq, Hash, Default, Ord  | Compiler-built-in proc macros                           |
| <b>Ecosystem crates</b>          | Serialize, Deserialize, Error, Parser, instrument, ... | serde, thiserror, clap, tracing, ... (Derive ecosystem) |
| <b>Custom (your org / crate)</b> | <code>#[derive(RegisterMap)]</code> , ...              | <b>Your</b> proc-macro = true<br>crate (syn + quote)    |

instrument is **ecosystem proc-macro tooling** from tracing, but it is an **attribute** on fn (`#[tracing::instrument]`), not a name inside `#[derive(...)]`. Same crate family, different proc-macro kind — [Proc macros — three kinds](#).

You **cannot register** a new derive macro name in a normal application module. Custom derives live in a **dependency** crate with `proc-macro = true`. **Using** a registered name looks identical to std — the lookup at step 1 above is the same:

```
// Cargo.toml: my-derive = { path =
//    "../my-derive" }
// my-derive crate registers
//    #[proc_macro_derive(RegisterMap)] and
//    defines trait RegisterMap
```

```
// #[derive(RegisterMap)]
// struct DeviceId(u16);
// expands to: impl RegisterMap for DeviceId {
//     ... }
```

Defining trait `RegisterMap` only in your app without the proc-macro crate gives you manual `impl` — not `#[derive(RegisterMap)]`.

## 18.9 Clone and Copy — derive in practice

Cloning ties directly to ownership (Chapter 1) and shows up in config reload, channel messages, and async task state.

| Aspect         | Copy                                                                       | Clone                                                          |
|----------------|----------------------------------------------------------------------------|----------------------------------------------------------------|
| Mechanism      | implicit bitwise copy on move/assign                                       | explicit <code>.clone()</code>                                 |
| Trait relation | <code>Copy</code> : <code>Clone</code>                                     | standalone                                                     |
| Typical types  | <code>i32</code> , <code>bool</code> , small arrays                        | <code>String</code> , <code>Vec</code> , most structs/enums    |
| Derive         | <code>#[derive(Copy, Clone)]</code> if <b>all fields</b> <code>Copy</code> | <code>#[derive(Clone)]</code> if all fields <code>Clone</code> |
| Cost           | free (stack)                                                               | may heap-allocate (deep copy)                                  |

### When to derive `Clone` in automation:

- Config structs from TOML passed into worker threads (Chapter 14 move closures).
- Command enums — clone before send if both sides need a copy.
- Snapshot sensor readings for logging without holding locks.

### When not to derive blindly:

- Large buffers — prefer `Arc<[u8]>` or `&T`.
- Types with `Mutex`/`RefCell` inside — usually not `Clone` (Chapter 10).
- Cheap shared clone — manual `Clone` calling `Arc::clone`, not derived deep copy.

```
// Playground
#[derive(Clone)]
```

```
struct Label(String);

#[derive(Copy, Clone)]
struct RegisterAddr(u16);

fn main() {
    let a = Label("sensor-A".into());
    let b = a.clone(); // heap string duplicated
    let r1 = RegisterAddr(0x01);
    let r2 = r1; // Copy --- no .clone() needed
    println!("{}", b.0, r2.0);
}
```

Wrong — Copy on heap field:

```
// Playground --- does not compile
// #[derive(Copy, Clone)]
// struct Bad { name: String }
// String is not Copy
```

| Trap                       | Symptom                    | Fix                                      |
|----------------------------|----------------------------|------------------------------------------|
| Clone in hot poll loop     | CPU + allocator churn      | borrow, Arc, or move                     |
| Deep clone of nested graph | accidental megabyte copies | Arc, intern, or move                     |
| Java-style dyn Clone       | not in std                 | enum of variants or T: Clone (Chapter 7) |
| Copy on non-Copy field     | compile error              | drop Copy; keep Clone                    |

18.10 Derive ecosystem — existing solutions

Std derives (compact reference):

| Derive         | Use for                                                   |
|----------------|-----------------------------------------------------------|
| Debug          | logs, tests, REPL-style printing                          |
| Clone / Copy   | duplicates — see above                                    |
| PartialEq / Eq | equality (not f64 total order — Ch7)                      |
| Hash           | HashMap keys                                              |
| Default        | ..Default::default() fills; enum needs #[default] variant |



| Derive           | Use for                         |
|------------------|---------------------------------|
| Ord / PartialOrd | sorting when total order exists |

### Automation-relevant proc-macro crates (consumer view):

| Crate / macro                        | Typical use                             |
|--------------------------------------|-----------------------------------------|
| <b>serde</b> Serialize/Deserialize   | config TOML/JSON, log payloads, IPC     |
| <b>thiserror</b> Error               | library error enums (Chapter 8)         |
| <b>clap</b> Parser, Subcommand, Args | CLI gateways                            |
| <b>tracing</b> instrument            | spans around poll loops                 |
| <b>tokio</b> main, select!, join!    | async runtime (Chapter 16)              |
| <b>strum</b> / <b>enum-as-inner</b>  | enum <-> string for protocols           |
| <b>async-trait</b>                   | object-safe async traits (escape hatch) |

```
// Cargo project --- composite sketch
// serde = { version = "1", features =
//     ["derive"] }
// thiserror = "2"
// clap = { version = "4", features =
//     ["derive"] }

use clap::Parser;
use serde::Deserialize;
use thiserror::Error;

#[derive(Debug, Clone, Deserialize)]
struct GatewayConfig {
    port: u16,
}

#[derive(Error, Debug)]
enum GatewayError {
    #[error("config read failed")]
    ConfigRead,
}

#[derive(Parser)]
```

```
struct Cli {
    #[arg(long, default_value = "502")]
    port: u16,
}
```

**Version coupling:** `serde derive` feature must match `serde` version; same for `clap` majors.

**Serde + Clone pairing:** `#[derive(Debug, Clone, Deserialize)]` — deserialize once, then `clone()` hands copies to threads without re-reading the file.

### 18.10.0.1 Serde JSON — serialize and deserialize

`serde` defines the traits; **`serde_json`** is the format crate for JSON text and bytes. Derive **`Serialize`** and **`Deserialize`**, then call **`to_string / from_str`** at boundaries — config files, log lines, HTTP bodies (Chapter 19).

```
// Cargo project
// [dependencies]
// serde = { version = "1", features =
//     ["derive"] }
// serde_json = "1"

use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, Deserialize)]
struct SensorReading {
    device_id: String,
    value: f64,
}

fn main() -> Result<(), serde_json::Error> {
    let reading = SensorReading {
        device_id: "plc1".into(),
        value: 23.5,
    };

    let json = serde_json::to_string(&reading)?;
```

```
println!("{}", json);
// {"device_id":"plc1","value":23.5}

let back: SensorReading =
    serde_json::from_str(&json)?;
println!("{}", back);
Ok(())
}
```

**Serialize** turns Rust values into JSON text. **Deserialize** parses JSON back into typed structs. Failures return `serde_json::Error` — propagate with `?` or map into your app error enum (Chapter 8).

| API                                 | Input                                               | Typical use                                             |
|-------------------------------------|-----------------------------------------------------|---------------------------------------------------------|
| <code>serde_json::to_string</code>  | <code>&amp;T</code> where <code>T: Serialize</code> | log line, HTTP response body                            |
| <code>serde_json::from_str</code>   | <code>&amp;str</code>                               | config file read into <code>String</code> , API payload |
| <code>serde_json::from_slice</code> | <code>&amp;[u8]</code>                              | socket/frame buffer before UTF-8 validation             |

For files: `std::fs::read_to_string(path)?` then `from_str` (Chapter 19).  
For outgoing logs: `to_string` and write one line.

**Field names in JSON often differ from Rust idioms** — use `serde` attributes instead of manual string parsing:

```
// Cargo project
// serde = { version = "1", features =
//     ["derive"] }
// serde_json = "1"

use serde::{Deserialize, Serialize};

#[derive(Debug, Serialize, Deserialize)]
struct PollConfig {
    #[serde(rename = "pollIntervalMs")]
    poll_ms: u64,
}
```

```
fn main() -> Result<(), serde_json::Error> {
    let json = r#"{"pollIntervalMs":250}"#;
    let cfg: PollConfig =
        serde_json::from_str(json)?;
    println!("poll every {} ms", cfg.poll_ms);

    let out = serde_json::to_string(&cfg)?;
    println!("{}", out);
    // {"pollIntervalMs":250}
    Ok(())
}
```

Rename attrs keep Rust field names idiomatic while matching external JSON. After a refactor, add an integration test with a fixture file — silent drift is a common production trap (see Afterparty **Serde rename**).

## 18.11 Proc macros — three kinds

[Derive attributes](#) above is the **derive** kind in depth. Two other proc-macro shapes share the same compile-time expansion pipeline:

| Kind                 | Syntax                                | Example                                                                                  | Differs from derive how?                                       |
|----------------------|---------------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| <b>derive</b>        | <code>#[derive(Trait)]</code> on type | Debug, Deserialize, Error                                                                | Emits <code>impl Trait</code> for Type                         |
| <b>attribute</b>     | <code>#[attr(...)]</code> on item     | <code>#[tokio::main]</code> , <code>#[test]</code> , <code>#[tracing::instrument]</code> | Rewrites the <b>annotated item</b> (function, module, ...)     |
| <b>function-like</b> | <code>name!(...)</code>               | <code>include_bytes!</code> , <code>sqlx::query!</code>                                  | Expands at the <b>call site</b> like <code>macro_rules!</code> |

Do not mix these up with **annotations/decorators** from other languages — all three kinds expand to Rust source **before** runtime ([Not annotations or decorators](#)).

Proc macros live in separate `proc-macro = true` crates — `rust-analyzer` sometimes stops at the macro boundary. **Authoring proc macros is out of scope here**. Know how to **consume** and **debug** them with `cargo expand`.

## 18.12 `const` and compile-time evaluation

Complementary to macros — typed computation without generating new syntax.

| Form                           | Role                                            |
|--------------------------------|-------------------------------------------------|
| <code>const X: T = ...</code>  | compile-time constant embedded in binary        |
| <code>static X: T = ...</code> | fixed memory location; 'static lifetime         |
| <code>const fn</code>          | function callable in <code>const</code> context |

**`const fn` limits (conservative):** no I/O, no threads, no `Mutex`; heap allocation limited — prefer simple numeric / bitwise logic.

```
// Playground
const MAX_REGISTERS: usize = 128;

const fn crc_nibble(x: u8) -> u8 {
    x ^ 0x0F
}

fn main() {
    const CRC: u8 = crc_nibble(0xA5);
    println!("max {} crc {:02x}",
        MAX_REGISTERS, CRC);
}
```

**`env!` vs `runtime`:** `env!("CARGO_PKG_NAME")` is substituted at **compile time** (Preface). `std::env::var("HOME")` reads the process environment at **runtime**. Different tools.

**When to use which:**

|                                |                                                                     |
|--------------------------------|---------------------------------------------------------------------|
| need typed compile-time value? | -> <code>const</code> /<br><code>const fn</code>                    |
| repeated syntax across types?  | -> <code>derive</code> or<br><code>macro_rules!</code>              |
| single generic algorithm?      | -> <code>fn</code> +<br>generics + traits (Ch7)                     |
| external schema/codegen file?  | -> <code>build.rs</code><br>or <code>include!</code> --- Afterparty |

## 18.13 When custom metaprogramming pays off

| Benefit                         | Example                                             |
|---------------------------------|-----------------------------------------------------|
| DRY impl explosion              | <code>#[derive(Error)]</code> vs 40 lines manual    |
| Domain DSL at zero runtime cost | <code>register_map! -&gt; const + match arms</code> |
| Compile-time validation         | <code>env! missing -&gt; build fails</code>         |
| Type-safe codegen               | field rename breaks compile, not silent JSON drift  |
| API ergonomics                  | <code>command! subcommand tables</code>             |
| Cross-cutting hooks             | <code>tracing::instrument</code>                    |

## 18.14 Restrictions — what macros cannot do

| Restriction                    | Consequence                                                                    |
|--------------------------------|--------------------------------------------------------------------------------|
| Token matching, not types      | macro ok; expanded code fails confusingly                                      |
| Recursion limit (~128)         | recursion limit exceeded                                                       |
| Hygiene                        | can't see caller locals unless <code>\$x:expr</code>                           |
| Proc-macro crate isolation     | helpers in sibling crate                                                       |
| Orphan rule on generated impl  | <code>impl ForeignTrait for LocalType</code> still illegal                     |
| <code>const</code> eval bounds | no file I/O, no threads                                                        |
| Follow-set violation           | <code>`\$x:expr`</code> is followed by <code>`X`</code> , which is not allowed |
| Derive bound failure           | field doesn't implement trait                                                  |
| IDE / rust-analyzer            | jump-to-def stops at macro                                                     |

### Wrong — type check after expansion:

```
// Playground --- macro parses; expanded code
// fails type check
macro_rules! bad_impl {
    ($t:ty) => { impl Clone for $t { fn
        clone(&self) -> Self { unimplemented!()
        } } };
}
// bad_impl!(i32);
// ERROR: conflicting Clone impl or incomplete
// --- i32 already has Clone
```

## 18.15 Why macro code is hard to trace

**Yes — genuinely harder than hand-written Rust**, for structural reasons:

| Reason                | What you experience                              | Mitigation                                          |
|-----------------------|--------------------------------------------------|-----------------------------------------------------|
| Expansion indirection | in expansion of<br><code>macro_rules! foo</code> | <code>cargo expand</code> ; read call site<br>first |
| No stable source file | generated code not in repo                       | <code>expand to stdout</code> ; test public<br>API  |
| Stacked expansions    | derive inside macro inside<br>attribute          | expand one layer at a time                          |
| Hygiene               | <code>grep</code> misses generated<br>names      | search macro definition;<br><code>expand</code>     |
| IDE limits            | no “go to impl” through<br>derive                | <code>cargo expand</code> side buffer               |
| Proc macro opacity    | <code>serde</code> logic in another crate        | read crate docs/tests                               |
| Debuggers             | breakpoints at call site or<br>odd spans         | test expanded or public API                         |

**Java contrast:** annotation processors also generate code, but IDE culture around generated sources is mature. Rust macro output is **invisible unless you expand**. Same compile-time idea, rougher tooling surface.

**Team guidance:** macros at **stable boundaries** (`serde`, `thiserror`, documented internal DSLs) are fine. Business logic buried in macro layers nobody expands is a liability.

## 18.16 Edge cases and compiler traps

### 18.16.1 Follow-set: `for`, `let`, and expression position

Same rules as Syntax forbidden — wrong vs right examples below.

#### 18.16.1.1 Edge 1 — `$e:expr` then `for` in the matcher (illegal)

The macro **definition** fails — you never get to call it.

```
// Playground --- does not compile
macro_rules! repeat_body {
    ($body:expr for $i:ident in $range:expr) =>
    {
```

```

    for $i in $range { $body }
  };
}
// error: `$body:expr` is followed by `for`,
        which is not allowed for `expr` fragments

```

#### 18.16.1.2 Edge 2 — `$e:expr` then `let` in the matcher (illegal)

Same follow-set as `for` — only `=>`, `,`, `;` may follow `expr`.

```

// Playground --- does not compile
macro_rules! init_then {
  ($setup:expr let $x:ident = $v:expr) => {
    $setup; let $x = $v; };
}
// error: `$setup:expr` is followed by `let`,
        which is not allowed for `expr` fragments

```

#### 18.16.1.3 Edge 3 — `$e:expr` then `while` in the matcher (illegal)

```

// Playground --- does not compile
macro_rules! while_wrap {
  ($cond:expr while $c:expr) => { while $c {
    $cond } };
}
// error: `$cond:expr` is followed by `while`,
        which is not allowed for `expr` fragments

```

#### 18.16.1.4 Edge 4 — legal `foreach` matcher: `$p:pat in $r:expr`

Literal `for` sits **before** the pattern; `in` after `pat` is allowed.

```

// Playground
macro_rules! foreach {
  ($p:pat in $r:expr => $body:expr) => {
    for $p in $r { $body }
  };
}

fn main() {

```



```
foreach!(reg in 0u16..3 => println!("read
    {}", reg));
}
```

**What happened:** compiles and prints three lines — the matcher never put `for` after an `expr` fragment.

#### 18.16.1.5 Edge 5 — `let` + `for` expansion in expression position (illegal, single braces)

The macro **definition** is fine; the **call site** fails when you need a value back.

```
// Playground --- macro ok; main fails
macro_rules! poll_twice {
    () => {
        let n = 2;
        for i in 0..n {
            println!("poll {}", i);
        }
    };
}

fn main() {
    let _done = poll_twice!();
    // error: expected expression, found `let`
    //         statement
    // note: macro expansion ignores keyword
    //       `for` and any tokens following
}
```

#### 18.16.1.6 Edge 6 — same macro as a statement (legal)

Same `poll_twice!` body — but no assignment — works.

```
// Playground
macro_rules! poll_twice {
    () => {
        let n = 2;
        for i in 0..n {
            println!("poll {}", i);
        }
    };
}
```

```

    }
  };
}

fn main() {
  poll_twice!(); // ok --- statement position
}

```

#### 18.16.1.7 Edge 7 — let + for in expression position: double-brace fix

```

// Playground
macro_rules! poll_twice {
  () => {{
    let n = 2;
    for i in 0..n {
      println!("poll {}", i);
    }
    n
    // block tail expression --- value
    // returned to caller
  }};
}

fn main() {
  let count = poll_twice!();
  println!("polled {} times", count);
}

```

**What happened:** prints poll 0, poll 1, then polled 2 times — inner { ... ; n } is one block expression.

#### 18.16.1.8 Edge 8 — lone for as expression (often legal)

A **single** for loop (no leading let) can expand where an expression is expected — the loop itself is the expression.

```

// Playground
macro_rules! count_to {
  ($n:expr) => {
    for i in 0..$n {

```

```
        println!("{}", i);
    }
};
}

fn main() {
    let _ = count_to!(3);
    // ok --- for-loop expression (value is ())
}
```

18.16.1.9 Edge 9 — let inside \$e:expr argument (legal)

The restriction is on matcher **grammar**, not on what tokens the caller passes. This compiles:

```
// Playground
macro_rules! id {
    ($e:expr) => { $e };
}

fn main() {
    id!({
        let x = 10;
        x + 1
    });
    // ok --- one block expression passed as
    $e:expr
}
```

**What happened:** caller wrapped `let + value` in `{ ... }` — one `expr` fragment. Contrast Edge 5, where the **macro body** emitted bare `let` without a wrapping block expression.

| Edge | Layer   | Fails?    | Takeaway                                                                                            |
|------|---------|-----------|-----------------------------------------------------------------------------------------------------|
| 1-3  | matcher | macro def | no keyword after <code>\$e:expr</code> except <code>=&gt;</code> , <code>,,</code> , <code>;</code> |
| 4    | matcher | no        | use <code>\$p:pat</code> in <code>\$r:expr</code> , literal <code>for</code> in transcriber         |

| Edge | Layer     | Fails?    | Takeaway                                                 |
|------|-----------|-----------|----------------------------------------------------------|
| 5    | expansion | call site | let + for need {{<br>... ; tail }} for<br>let x = mac!() |
| 6    | expansion | no        | statement-position<br>call needs no return<br>value      |
| 7    | expansion | no        | double braces + tail<br>expr                             |
| 8    | expansion | no        | single for is often<br>enough                            |
| 9    | caller    | no        | block { let ...;<br>expr } is one<br>expression          |

18.16.2 macro\_rules! (other traps)

| Trap                                         | Symptom                                                                                 | Fix                                                                         |
|----------------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>\$e:expr for ... in matcher</code>     | <code>`\$e:expr`</code> is followed by<br><code>`for`</code> at macro <b>definition</b> | use <code>\$p:pat</code> in <code>\$r:expr</code> —<br>see Syntax forbidden |
| let + for in expr position                   | expected expression,<br>found let; ignores keyword<br>for                               | {{ let ...; for ...;<br>value }}                                            |
| <code>\$()+</code> on empty input            | no rules expected this<br>token on <code>my_macro!()</code>                             | use <code>\$()*</code> or add <code>() =&gt; ...</code><br>arm              |
| Expression-position macro                    | expected expression,<br>found statement                                                 | wrap body in {{ ... }}                                                      |
| Arm overlap                                  | wrong arm fires                                                                         | most-specific arm first                                                     |
| <code>#[macro_export]</code>                 | macro at crate root path                                                                | document path; use <code>\$crate</code>                                     |
| <code>stringify!</code> vs <code>eval</code> | <code>stringify!(a + b) -&gt; "a +<br/>b"</code>                                        | diagnostics only                                                            |

```
// Playground --- WRONG with `+` on empty call
macro_rules! my_vec {
    ($($x:expr),+) => { vec![$($x),*] };
}
// my_vec!() fails --- FIX: use `*` or add ()
=> { vec![] }
```

18.16.3 Derive / attributes

| Trap                                           | Symptom                 | Fix                                    |
|------------------------------------------------|-------------------------|----------------------------------------|
| Default on enum                                | derive error            | <code>#[default]</code> on one variant |
| Manual + derived same trait                    | E0119 conflicting impls | remove one                             |
| Generic T in struct                            | T: Clone not satisfied  | bound on struct definition             |
| Recursive enum                                 | infinite size           | Box, Arc, manual Clone                 |
| <code>#[serde(default)]</code> without Default | odd runtime defaults    | add Default derive                     |

```
// Playground --- does not compile
// #[derive(Clone)]
// struct Id(u64);
// impl Clone for Id { fn clone(&self) -> Self
//     { Self(self.0) } }
// ERROR: conflicting implementations of trait
// `Clone`
```

18.16.4 Clone / build-time / automation

| Trap                                   | Symptom                             | Fix                                                           |
|----------------------------------------|-------------------------------------|---------------------------------------------------------------|
| Arc<String> derived Clone              | cheap pointer bump, not deep string | know semantics                                                |
| Clone before channel send              | double alloc when move suffices     | <code>send(value)</code> not <code>send(value.clone())</code> |
| <code>env!("VAR")</code> missing in CI | hard compile error                  | <code>option_env!</code> or document <code>env</code>         |
| <code>#[cfg]</code> + macro            | code “missing” in release           | <code>cfg</code> -gated code not typechecked when off         |
| Serde rename + refactor                | silent config mismatch              | integration test with fixture TOML                            |
| Debug on secrets                       | keys in logs                        | redacted manual Debug                                         |

```
// Playground --- conceptual; needs
// compile-time env or fails in playground
// let version = env!("RELEASE_VERSION");
// prefer:
//     option_env!("RELEASE_VERSION").unwrap_or("dev")
// in a const context pattern
```

## 18.17 Cons, warnings, and team hygiene

| Warning                | Symptom                   | Mitigation                    |
|------------------------|---------------------------|-------------------------------|
| Macro when fn suffices | opaque errors             | fn first                      |
| Proc-macro sprawl      | slow builds, version hell | minimal derives; pin versions |
| DSL too clever         | onboarding cost           | document expansion            |
| Attribute magic        | logic hidden from grep    | explicit calls in hot paths   |
| Trust proc macros      | build-time arbitrary code | audit deps                    |
| Over-derive            | compile bloat             | derive only what you use      |
| Clone everywhere       | hidden perf cost          | audit hot paths               |
| Skipping cargo expand  | guesswork debugging       | expand on confusion           |
| Deep macro stacks      | untraceable errors        | max 1–2 layers in app code    |

## 18.18 Idiom spotlight

**Macros for syntax repetition; generics/traits for logic reuse.**

If a macro could be a function, prefer the function.

**Derive Clone on small config/command types; reach for**

**.clone() sparingly in poll loops** — prefer move or borrow.

When follow-set rules fight your DSL, change punctuation or use a proc macro. **Audit expanded code** before shipping custom macros.

## 18.19 Go deeper

- [The Rust Book — Macros](#)
- [macro\\_rules! basics](#) — 411
- [Derive macro concepts](#) — 422
- [The Little Book of Rust Macros](#)
- [Rust Reference — Follow-set ambiguity](#)
- [Rust Reference — Macros](#)
- [cargo-expand](#)
- [serde derive](#), [serde\\_json](#), [thiserror](#), [clap derive](#)

## 18.20 See also

- Preface — `env!` vs runtime `env`
- Chapter 3: Functions — first `#[derive(Debug)]` on error enums
- Chapter 6: Enums and pattern matching — `matches!`, advanced patterns
- Chapter 7: Traits — `derive` usage, `float/Eq`
- Chapter 13: Standard traits — which traits to `derive` vs `implement` (`Display`, `Debug`, cheat sheet)
- Chapter 8: Errors — `#[derive(Error)]` with `thiserror`
- Chapter 14: Multithreading — `Clone` + channels
- Chapter 16: Async — Tokio attribute macros
- Chapter 18: Unsafe
- Chapter 19: I/O — config files

### 18.20.1 Afterparty

Use these for proc-macro authoring and advanced DSL topics not covered above.

#### 18.20.1.1 Concepts and mental model

1. **Expansion order** — “List compiler phases from tokens to LLVM; where do macros run?”
2. **Tokens vs types** — “Why can a macro compile but expanded code fail? One example.”
3. **Follow-set why** — “Explain in 80 words why `$a:expr = $b:expr` is forbidden in matchers.”
4. **Scope honesty** — “List 5 metaprogramming topics Ch17 skips and where to learn each.”
5. **Trace checklist** — “Give 6 reasons macro code is hard to trace and one mitigation each.”

#### 18.20.1.2 `macro_rules!` drills

6. **Macro vs fn** — “Rewrite macro as generic fn if possible; when impossible, say why.”
7. **Fragment picker** — “I describe a DSL shape; you pick `expr/ident/tt` for each slot.”

8. **Expr equals fix** — “Fix `set_reg!($addr = $val)` matcher; show two valid surface syntaxes.”
9. **For after expr** — “Why is `$e:expr` for `$i:ident` in `$r:expr` illegal? Show legal `$p:pat` in `$r:expr` `foreach` macro.”
10. **Double-brace fix** — “Fix `poll_twice!` macro that mixes `let` and `for` for use in `let x = poll_twice!().`”
11. **Trailing comma** — “Explain `$(x),*` vs `$(x),+` on empty input; show failing and fixed macro.”
12. **Hygiene** — “Explain macro hygiene in 60 words with `$crate` mention.”
13. **Register DSL** — “Extend `register_map!` with a third register; explain `stringify! arm.`”

### 18.20.1.3 How derive works

14. **Debug expand** — “Walk me through `cargo expand` on `derive Debug` output (conceptual).”
15. **Clone expand** — “Show conceptual expanded `impl Clone` for struct with two `i32` fields.”
16. **Enum vs struct** — “How does derived `PartialEq` differ for `enum` vs `struct`? Sketch match arms.”
17. **Field bound failure** — “Struct with `Mutex<i32>` field + `#[derive(Clone)]` — quote error and fix.”
18. **Redacted Debug** — “When hand-write `Debug` instead of `derive` on command `enum` with secrets.”

### 18.20.1.4 Clone and Copy

19. **Copy vs Clone quiz** — “Classify 8 types: `Copy`, `Clone` only, or neither; justify.”
20. **Hot-loop clone audit** — “Audit poll loop with `.clone()` each tick; suggest `move/Arc/borrow.`”
21. **Arc vs derive Clone** — “Explain cheap `Arc` clone vs deep `String` clone with one snippet.”

### 18.20.1.5 Derive ecosystem

22. **derive need** — “List derives I want for `config` struct loaded from `TOML` — justify each.”



- 23. **Serde rename** — “Field `poll_ms` in JSON as `pollIntervalMs` — show attr; trap on refactor.”
- 24. **thiserror vs manual** — “Same error enum: count lines derive vs hand-written (Ch8 style).”
- 25. **Float Eq trap** — “Show `#[derive(Eq)]` on `f64` field failure; two fixes from Ch7.”

#### 18.20.1.6 Debugging and tooling

- 26. **cargo expand walkthrough** — “Step-by-step: install, run, read output for one derive.”
- 27. **In expansion of** — “Decode a 3-note compiler error chain from nested macro + derive.”
- 28. **tt vs expr escape** — “When switch matcher from `expr` to `tt`; tradeoffs in 80 words.”
- 29. **Three-layer trace** — “Derive inside attribute inside `macro_rules` — how to debug layer by layer.”

#### 18.20.1.7 const vs macro

- 30. **Port to const fn** — “Replace tiny numeric macro with `const fn`; when macro still needed?”
- 31. **env vs var** — “Compare `env!`, `option_env!`, `std::env::var` — table with one use case each.”
- 32. **include\_str config** — “Embed default TOML with `include_str!`; deserialize at startup sketch.”

#### 18.20.1.8 Pros / cons decisions

- 33. **Macro vs fn audit** — “Mark 6 snippets: should be macro, derive, or plain fn — justify.”
- 34. **When not proc macro** — “Three scenarios where `proc macro` is overkill; alternative each.”
- 35. **Minimal derive set** — “Gateway config + error + CLI: smallest derive list that still ships.”

#### 18.20.1.9 Edge case audit

- 36. **Trap quiz** — “Mark 8 snippets: empty +, double brace, Default enum, Arc Clone, `env!`, duplicate `impl`, `cfg` macro, `serde rename`.”

- 37. **Duplicate register DSL** — “Design compile-time error for duplicate keys in `register_map!`”
- 38. **Serde refactor test** — “Integration test plan after renaming TOML field with `serde attrs.`”

#### 18.20.1.10 Derive and attribute proc macros

- 39. **Derive vs decorator** — “I come from Python/Java. Explain why `#[derive(Debug)]` is not a decorator or annotation that runs at call time — what actually happens at compile time?”
- 40. **Three kinds quiz** — “Classify 8 snippets: derive proc macro, attribute proc macro, function-like macro, compiler attribute (`#[inline]`, `#[cfg]`), field meta parsed inside a derive (`#[serde(rename)]`).”
- 41. **tokio::main expand** — “Sketch the conceptual expansion of `#[tokio::main] async fn main() { ... }`. Why is this an attribute proc macro, not `#[derive]`?”
- 42. **Custom attribute inputs** — “For `#[my_attr(some = \"config\")] fn poll() { ... }`, what two token streams does the proc macro receive? Give two things the macro might emit.”
- 43. **Field attr vs item attr** — “Contrast `#[serde(rename = \"pollIntervalMs\")]` on a struct field vs `#[tracing::instrument]` on `fn poll` — same proc-macro kind or not?”
- 44. **Custom attribute when** — “Three scenarios (poll-loop tracing, type-level JSON mapping, wrapping main with a runtime). Pick: custom attribute proc macro, derive, or plain helper fn — justify each.”
- 45. **Runtime myth** — “Does `#[tracing::instrument]` or `#[test]` run every time I call the function? Explain what runs at compile time vs run time.”

#### 18.20.1.11 Automation capstone

- 46. **DSL sketch** — “Design tiny command! macro for CLI subcommands — tokens only.”
- 47. **Modbus table macro** — “Spec register table macro generating lookup + const max address.”
- 48. **Derive soup review** — “I paste 40-line struct with 12 derives; trim to minimal set with reasons.”

# Chapter 19

## Chapter 18: Unsafe and When to Stop

### 19.1 Hook

Every ecosystem has escape hatches — **Java** `sun.misc.Unsafe`, **Python** C extensions, Rust **unsafe blocks**. Inside `unsafe`, the borrow checker is off. You must uphold invariants the compiler cannot verify.

### 19.2 Scope — a brief tour

Practical intro to reading `unsafe`, wrapping FFI, and knowing when to stop — not the full Nomicon.

| This chapter covers                                                                      | Deferred to See also / Afterparty                                  |
|------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Four powers of <code>unsafe</code> + <b>why it exists</b>                                | Full <a href="#">Rustonomicon</a>                                  |
| <b>Soundness</b> vs <b>safety</b>                                                        | Custom allocators, <code>Pin</code> / <code>Unpin</code> formalism |
| Safe wrappers over raw pointers                                                          | Lock-free data structures (Chapter 15)                             |
| <code>unsafe fn</code> , <code>unsafe trait</code> , <code>Send/Sync</code> proof sketch | Writing proc macros with <code>unsafe</code> traits (Chapter 17)   |
| FFI orientation + checklist                                                              | Full <code>bindgen</code> / <code>cxx</code> walkthrough           |
| When <b>not</b> to use <code>unsafe</code> + <code>Miri</code> intro                     | <code>Miri</code> deep dives, fuzzing proc macros                  |

### 19.3 Why unsafe exists — the aim

Rust’s default contract: **safe Rust that compiles cannot cause undefined behavior (UB)**. That promise is **soundness**. `unsafe` implements that promise under the hood. It is also how **you** opt in when the compiler cannot see the full story.

| Role               | Plain language                                                                                                                       |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Implement safe std | <code>Vec::push</code> , <code>str</code> , <code>Box</code> — internally use <code>unsafe</code> so <b>you</b> call sites stay safe |
| FFI / C ABI        | Call vendor PLC drivers, <code>libc</code> , OS APIs — memory layout and ownership cross a boundary                                  |
| Certify invariants | <code>unsafe impl Send</code> for a thread-safe file descriptor wrapper the compiler cannot prove                                    |
| Rare performance   | After <b>profiling</b> , sometimes a hot path needs intrinsics or hand-tuned memory — last resort                                    |

| Language | Escape hatch                                                        | Who certifies safety?                                               |
|----------|---------------------------------------------------------------------|---------------------------------------------------------------------|
| Java     | <code>sun.misc.Unsafe</code> , JNI                                  | JVM spec + you at JNI boundary                                      |
| Python   | C extension modules                                                 | Extension author; CPython users trust maintainers                   |
| Rust     | <code>unsafe blocks</code> / <code>fns</code> / <code>traits</code> | <b>You</b> document invariants; safe API must still be <b>sound</b> |

**Key idea:** `unsafe` does **not** turn off all rules. Safe functions that call `unsafe` must uphold Rust’s global soundness. Callers of your `safe open_port()` must never get UB from correct use.

### 19.4 What unsafe allows

Four operations only possible inside `unsafe` (or in an `unsafe fn` body):

| Power                    | Rust                                    | Typical Java / Python parallel      |
|--------------------------|-----------------------------------------|-------------------------------------|
| Dereference raw pointers | <code>*const T, *mut T</code>           | JNI pointers; ctypes address        |
| Call unsafe fn           | Vec growth, <code>from_raw_parts</code> | Native method calls                 |
| unsafe impl trait        | Send / Sync for custom handles          | @GuardedBy — manual proof           |
| Mutable static           | global state (avoid in app code)        | static fields; module-level globals |

Safe Rust **around** unsafe must still be **sound**. A safe public API must not let callers trigger UB through normal use.

### 19.4.1 Soundness vs safety

| Term             | Meaning                                                                                                   |
|------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Safety</b>    | No <code>unsafe</code> in <i>this</i> function body — compiler enforces borrow rules                      |
| <b>Soundness</b> | <i>Entire program</i> cannot UB if only safe APIs are used — includes your <code>unsafe</code> invariants |

A **safe** wrapper can still be **unsound** if its `unsafe` block is wrong:

```
// Playground --- unsound safe API (do not ship)
fn always_five() -> i32 {
    let x = 5;
    let p = &x as *const i32;
    unsafe { *p } // safe fn, but...
    // x dropped here --- p would dangle if we
    // returned *p
}

fn main() {
    println!("{}", always_five());
    // OK today only because we copy before drop
}
```

**What happened:** compiles because we **copy** `*p` before `x` is dropped. Returning `&i32` or storing `p` past `x`'s lifetime would be UB — the safe signature would lie. Sound wrappers **narrow** `unsafe` to a proof you document.

## 19.5 Examples: elementary -> hard

Work through in order. After each snippet: **run it**, then read **what happened**.

### 19.5.1 Level 1 — Elementary: deref a stack pointer

```
// Playground
fn main() {
    let n = 5;
    let p = &n as *const i32;
    unsafe {
        println!("{}", *p);
    }
}
```

**What happened:**

- `&n as *const i32` **coerces** a valid borrow to a raw pointer — no allocation change.
- **unsafe block** is required to **dereference** `p`; outside the block, `*p` is forbidden.
- `n` is still alive on the stack; `*p` reads 5. The borrow checker is **off inside** the block — you must ensure `n` outlives the deref.

### 19.5.2 Level 2 — Elementary: viewing bytes + dangling trap

Safe pattern: slice from **valid** `&[u8]` — no unsafe needed for everyday buffer work:

```
// Playground
fn as_hex_preview(buf: &[u8], max: usize) ->
    String {
    let take = buf.len().min(max);
    buf[..take]
        .iter()
        .map(|b| format!("{b:02x}"))
        .collect::<Vec<_>>()
        .join(" ")
}
```

```

}

fn main() {
    let frame = [0x01, 0x03, 0x00, 0x10, 0xFF];
    println!("{}", as_hex_preview(&frame, 3));
}

```

**What happened:** prints **01 03 00** — Modbus-style frames are handled with **safe slices** most of the time.

When unsafe appears: `slice::from_raw_parts(ptr, len)` — **your** invariants must guarantee `ptr` is valid for `len` bytes:

```

// Playground --- conceptual; invariants listed
    in comments
use std::slice;

/// INVARIANTS caller must ensure:
/// 1. ptr is non-null and aligned for u8
/// 2. ptr.ptr+len is initialized and readable
/// 3. no mutable alias to same memory while
    &slice lives
/// 4. len does not exceed allocation
/// 5. memory outlives returned slice (or copy
    out)
unsafe fn view_bytes(ptr: *const u8, len:
    usize) -> &'static [u8] {
    slice::from_raw_parts(ptr, len)
}

fn main() {
    let data = [10u8, 20, 30];
    let p = data.as_ptr();
    let s = unsafe { view_bytes(p, 3) };
    println!("{}", s);
}

```

**What happened:** works here because `data` is stack-allocated and lives for `'static` in this toy. -> `&'static` is **misleading** in real code. Production wrappers return `&[u8]` tied to an **owner** lifetime, or **copy** into `Vec`.

**Wrong (UB — do not run):** use `p` after data is dropped -> dangling deref. Afterparty drills invariant lists.

### 19.5.3 Level 3 — Intermediate: `unsafe fn` in a small module

Encapsulate unsafe in the **smallest** module; expose only safe constructors:

```
// Playground
mod ring {
    pub struct RingBuf {
        data: Vec<u8>,
        len: usize,
    }

    impl RingBuf {
        pub fn new(cap: usize) -> Self {
            Self { data: vec![0; cap], len: 0 }
        }

        /// INVARIANT: len <= data.len()
        pub unsafe fn set_len_unchecked(&mut self, len: usize) {
            self.len = len;
        }

        pub fn push_byte(&mut self, b: u8) -> bool {
            if self.len >= self.data.len() {
                return false;
            }
            self.data[self.len] = b;
            // Safe because we just wrote index
            // `len` and len < cap
            unsafe {
                self.set_len_unchecked(self.len + 1);
            }
            true
        }
    }
}
```



```

        pub fn as_slice(&self) -> &[u8] {
            &self.data[..self.len]
        }
    }
}

fn main() {
    let mut rb = ring::RingBuf::new(4);
    rb.push_byte(0xAA);
    rb.push_byte(0xBB);
    println!("{}", rb.as_slice());
}

```

**What happened:** prints **[170, 187]** (0xAA, 0xBB). `set_len_unchecked` is unsafe because a wrong `len` makes `as_slice()` read **uninitialized** or **out-of-bounds** memory. Same idea as `Vec`'s internal length updates.

### 19.5.4 Level 4 — Hard: `unsafe impl Send` for an opaque handle

Chapter 14: types moved into `thread::spawn` must be **Send**. Raw pointers are not **Send**; OS handles sometimes need a **manual proof**:

```

// Playground
use std::sync::Arc;
use std::thread;

/// Wraps a fictional OS file descriptor (i32).
/// PROOF for Send: fd is owned, close-on-drop
///    only on owning thread,
///    and we never share &mut access --- only Arc
///    move into one thread at a time.
struct SerialHandle {
    fd: i32,
}

unsafe impl Send for SerialHandle {}

impl SerialHandle {

```

```

fn open_fake() -> Self {
    Self { fd: 3 }
}

fn read_line(&self) -> String {
    format!("fd={} ok", self.fd)
}
}

fn main() {
    let h = Arc::new(SerialHandle::open_fake());
    let h2 = Arc::clone(&h);
    let t = thread::spawn(move ||
        h2.read_line());
    println!("main: {}", h.read_line());
    println!("thread: {}", t.join().unwrap());
}

```

**What happened:** both threads print **fd=3 ok**. Without `unsafe impl Send, thread::spawn(move || h2...)` would **not compile**. Production code often uses crates like `serialport` (safe API, unsafe inside). You rarely write this yourself — but you may **read** it in driver wrappers.

`Sync` is a separate proof (sharing `&T` across threads) — often `Arc<Mutex<...>>` instead of hand-rolled `unsafe impl Sync`.

### 19.5.5 Level 5 — Hard: FFI sketch (Cargo only)

Calling C from Rust — needs `libc`, link flags, and ABI discipline:

```

// Cargo only --- conceptual; needs libc in
// Cargo.toml
// use std::ffi::{CStr, CString};
// use std::os::raw::c_char;
//
// extern "C" {
//     fn strlen(s: *const c_char) -> usize;
// }
//
// fn main() -> Result<(), Box<dyn
//     std::error::Error>> {

```

```
// let msg = CString::new("PING")?;
// let len = unsafe { strlen(msg.as_ptr())
// };
// println!("C strlen = {len}");
// Ok(())
// }
```

**What happened (conceptually):** `CString` owns a nul-terminated buffer; `as_ptr()` borrows it for the call. **unsafe** because C may read arbitrary memory if invariants fail. Never call `after drop(msg);` mind **who frees** (Rust owns `CString`; C must not free it unless documented).

| Tool           | When                         |
|----------------|------------------------------|
| <b>bindgen</b> | Generate extern "C" from .h  |
| <b>cxx</b>     | Safe-ish C++ interop bridge  |
| <b>libc</b>    | Common C types and functions |

## 19.6 Practical cases in services and embedded

| Situation                 | Typical approach                                                                              |
|---------------------------|-----------------------------------------------------------------------------------------------|
| Serial / GPIO / async I/O | Use <b>serialport</b> , <b>tokio</b> , <b>rpapal</b> — safe API, <b>unsafe</b> inside crate   |
| Parse JSON/TOML config    | <b>serde</b> — no hand-written pointer tricks                                                 |
| Vendor C PLC SDK          | Thin safe Rust module; extern "C" + ownership docs; or ask for Rust bindings                  |
| Shared-memory ring buffer | <b>unsafe</b> + atomics (Chapter 15); <b>Miri</b> + tests                                     |
| "Speed up JSON"           | Profile -> better algorithm -> <b>simd-json</b> / <b>serde</b> features -> <b>unsafe</b> last |

### Decision flow:

Need C library or OS API?

- └ no -> stay in safe Rust
- └ yes -> safe wrapper crate exists?
  - └ yes -> use crate (preferred)
  - └ no -> **bindgen/cxx** + checklist below

Hot path slow?

- └ profile first

- └ algorithm / batching / fewer allocations
- └ then SIMD crate or expert-reviewed unsafe

**CRC / Modbus example:** prefer the Rust `crc` crate or pure-Rust protocol code over pasting C `unsafe`. Use C only when a hardware vendor **requires** it and the library is audited.

## 19.7 unsafe fn and unsafe trait

- **unsafe fn:** caller must uphold preconditions (e.g. `set_len_unchecked`). Mark the **contract** in doc comments.
- **unsafe trait:** `Send`, `Sync` — implementing asserts thread-safety the compiler cannot derive.
- **Std library:** `Vec::push`, `String::from_utf8_unchecked` (in std internals) — pattern: **unsafe inside, safe outside**.

Ecosystem Chapter 17 derives may emit `unsafe impl` in generated code — application authors rarely write those by hand.

## 19.8 FFI — checklist and pitfalls

| Step      | Check                                                                                        |
|-----------|----------------------------------------------------------------------------------------------|
| ABI       | extern "C" unless docs say otherwise                                                         |
| Strings   | CString / CStr; nul-terminated; encoding (UTF-8 vs locale)                                   |
| Ownership | Who allocates? Who frees? Same allocator?                                                    |
| Pointers  | Valid for entire call; not dangling after Rust drop                                          |
| Panics    | Unwinding across C is UB — <code>panic=abort</code> or <code>catch_unwind</code> at boundary |
| Threads   | Is the C API thread-safe? Match with <code>Send</code> / <code>Sync</code> proofs            |
| Errors    | <code>errno</code> vs return codes — map to <code>Result</code> in safe wrapper              |

**Java JNI parallel:** local/global refs, exception checking, and “who owns the buffer” mirror Rust’s pointer + lifetime discipline. Rust catches more at compile time in safe code. **FFI is still manual proof.**

## 19.9 Miri — when to run it

**Miri** is an interpreter that detects undefined behavior in unsafe code (use-after-free, aliasing violations, etc.).

```
rustup +nightly component add miri
cargo +nightly miri test
```

| When                           | Why                               |
|--------------------------------|-----------------------------------|
| After adding / changing unsafe | Catch UB tests miss               |
| Before merging FFI wrapper PR  | Cheap extra audit                 |
| Teaching / learning            | See <i>why</i> a pattern is wrong |

Miri does not replace code review or fuzzing — it complements them. See Afterparty for drill prompts.

## 19.10 When not to use unsafe

| Bad reason                        | Better move                                                  |
|-----------------------------------|--------------------------------------------------------------|
| “Borrow checker annoys me”        | Restructure ownership (Chapter 5),<br>Arc/Mutex (Chapter 14) |
| “Faster without measuring”        | cargo bench, flamegraph, fewer allocations                   |
| “Avoid learning lifetimes”        | Fix API shape; unsound shortcuts break<br>production         |
| “Replace Mutex with raw pointers” | Data races -> UB; use atomics or channels<br>(Chapter 15)    |

Most application code **never** needs unsafe in *your* crate — depend on maintained libraries instead.

## 19.11 Edge cases and compiler traps

| Trap                 | Symptom          | Idiom                                            |
|----------------------|------------------|--------------------------------------------------|
| Dangling raw pointer | UB, Miri failure | Tie pointer to owner lifetime;<br>copy if needed |

| Trap                                               | Symptom              | Idiom                                             |
|----------------------------------------------------|----------------------|---------------------------------------------------|
| <code>from_raw_parts</code> wrong <code>len</code> | read past buffer     | Assert <code>len &lt;= cap</code> ; test boundary |
| Two <code>*mut</code> aliases + write              | UB (stacked borrows) | Mutex, single owner, or proven uniqueness         |
| <code>static mut</code> + threads                  | data race UB         | Atomic* or Mutex (Chapter 15)                     |
| Unwinding into C                                   | UB                   | <code>panic=abort</code> or no panic across FFI   |
| “Safe” API returns dangling ref                    | unsound library      | code review + Miri                                |

## 19.12 Idiom spotlight

**Encapsulate unsafe in the smallest module; document invariants in comments; test aggressively.** Prefer safe crates maintained by experts for FFI.

**Profile before unsafe for speed.** A safe algorithm beats a wrong unsafe patch.

**Safe wrapper, documented proof:** every unsafe block should name what safe callers rely on.

## 19.13 Go deeper

- [The Rust Book — Unsafe Rust](#)
- [Rustonomicon](#) — ownership, FFI, unwinding
- [Miri](#) — UB detection
- [Procedural macro intro](#) — 423 (boundary with unsafe traits)

## 19.14 See also

- Chapter 1: Paradigm shift — raw pointers vs references
- Chapter 8: Errors and testing — no panic across unattended service boundaries
- Chapter 14: Multithreading — Send / Sync
- Chapter 15: Atomics — `static mut` vs atomics
- Chapter 17: Metaprogramming — derive-generated `unsafe impl`

- Chapter 19: I/O — Read/Write, processes, binary frames

## 19.14.1 Afterparty

### 19.14.1.1 Why unsafe and soundness

1. **Invariant list** — “For raw pointer to buffer + length, list 5 invariants a safe wrapper must enforce.”
2. **Soundness** — “Explain ‘safe Rust can’t cause UB’ vs unsafe — one paragraph; include unsound safe wrapper example.”
3. **Scope honesty** — “List 6 topics Ch18 skips and where to learn each (nomicon, Miri, Pin, ...).”
4. **Aim table** — “Fill: why Vec needs unsafe internally while push stays safe for callers.”
5. **Promise diagram** — “Draw safe API -> unsafe block -> invariants -> caller cannot UB; label soundness.”

### 19.14.1.2 Raw pointers and safe wrappers

6. **\*const vs &T quiz** — “Give 5 snippets: legal ref, needs unsafe block, compile error; I classify each.”
7. **from\_raw\_parts design** — “Design fn view\_frame(ptr, len) -> Result<&[u8], Error> without &'static; list invariants.”
8. **Dangling audit** — “Show stack pointer used after drop; I explain UB; you show Miri-style symptom.”
9. **Modbus buffer** — “Register table as &[u8] vs from\_raw\_parts — when is each idiomatic in a gateway?”
10. **Hex preview port** — “Port Level 2 as\_hex\_preview to return Result on empty buffer; no unwrap.”

### 19.14.1.3 unsafe fn, Send, and Sync

11. **set\_len contract** — “Document pre/post conditions for set\_len\_unchecked; what breaks as\_slice if violated?”
12. **Send proof** — “I claim Rc<\*mut u8> is Send; you disprove with compiler error quote.”
13. **SerialHandle Sync** — “When would SerialHandle need unsafe impl Sync vs Arc<Mutex<...>>? Two sentences each.”
14. **Ch14 port** — “Rewrite Level 4 spawn example using only safe types — when is it impossible?”

15. **Proc-macro boundary** — “Why do `serde/tokio crates` use `unsafe impl` you don’t write? Link Ch17.”

#### 19.14.1.4 FFI and automation

16. **FFI checklist** — “Checklist for calling a C Modbus library from a Rust binary; include panic and ownership rows.”
17. **CString trap** — “Show `into_raw` forgotten `from_raw` leak; fix with RAI pattern sketch.”
18. **Vendor SDK** — “Diagram ownership: Rust owns config, C owns connection, callback pointer — boxes and arrows only.”
19. **serialport hide** — “Where does `unsafe` live in a typical serial crate vs my application code?”
20. **CRC decision** — “C `crc16` vs Rust `crc` crate vs hand-rolled — decision tree for production gateway.”
21. **Java JNI** — “Compare JNI pitfalls (refs, exceptions, pinning) to Rust FFI ownership rules.”

#### 19.14.1.5 Miri, testing, and review

22. **Miri** — “What is Miri and when should I run it relative to `unsafe` changes? Include one command.”
23. **Trap quiz** — “Mark 6 snippets: `safe`, `UB`, ordering bug, needs Miri, needs `Mutex`, unsound `safe API`.”
24. **Review rubric** — “10-point code-review checklist for an `unsafe` PR in an automation repo.”
25. **Test plan** — “Unit + Miri + integration tests for new extern `'C'` wrapper — bullet list only.”

#### 19.14.1.6 When not to use `unsafe`

26. **Avoid** — “Review use case: speed up JSON — `unsafe` vs `simd-json` vs algorithm; pick with justification.”
27. **Borrow checker fight** — “I paste `fight-the-borrow-checker` code; you refactor to safe Rust without `unsafe`.”
28. **static mut** — “Compare `static mut` counter vs `AtomicUsize` from Ch15 — `UB` vs defined behavior.”



**19.14.1.7 Capstone**

29. **PlcDriver API** — “Design safe PlcDriver Rust API over fictional external 'C' — types, Result, no raw pointers in public API.”
30. **Level ladder recap** — “Explain Levels 1–5 in one paragraph each for a Java teammate who knows JNI.”

## **Part IV**

# **Standard Library I/O**

# Chapter 20

## Chapter 19: I/O, Processes, and Bits

### 20.1 Hook

I/O APIs differ by language (**Python** `open()/subprocess`, **Java** `Files/ProcessBuilder`, ...). Rust unifies files, sockets, serial ports, and pipes behind **Read** and **Write**. It uses **Command** for subprocesses with explicit error types. CLIs, services, and protocol code all live here.

### 20.2 Scope — a brief tour

Practical capstone for files, processes, and binary frames — not kernel or embedded-HAL depth.

| This chapter covers                              | Deferred to See also / Afterparty                        |
|--------------------------------------------------|----------------------------------------------------------|
| Read / Write, buffering, <code>io::Result</code> | Zero-copy I/O, <code>io_uring</code> , <code>mmap</code> |
| File and line processing (sync)                  | Full logging/observability stacks                        |
| Command, pipes, shell vs direct exec             | Job orchestration (Kubernetes, <code>systemd</code> )    |
| Bit packing, endianness, CRC sketch              | Full Modbus/TCP stacks, <code>nom</code> parsers         |
| Serial orientation ( <code>serialport</code> )   | GPIO/MCU bare-metal                                      |
| Sync vs async I/O choice                         | Tokio tuning (Chapter 16)                                |

## 20.3 Why Read and Write — the aim

Application code follows the same patterns: **pull bytes in, push bytes out, handle errors, don't panic on bad input.**

| Medium                | Rust surface                                   | Same trait?                               |
|-----------------------|------------------------------------------------|-------------------------------------------|
| File                  | File                                           | Read / Write                              |
| stdin / stdout        | <code>std::io::stdin() / stdout()</code>       | yes                                       |
| In-memory test double | <code>Cursor&lt;[u8]&gt;, Vec&lt;u8&gt;</code> | yes                                       |
| TCP socket (sync)     | <code>std::net::TcpStream</code>               | yes                                       |
| Serial port           | serialport open handle                         | yes                                       |
| Async TCP             | <code>tokio::net::TcpStream</code>             | <code>.await</code> variants (Chapter 16) |

**Aim:** write **one** fn `process<R: Read, W: Write>(...)` and reuse it for files, pipes, and test buffers. Java `InputStream/OutputStream` and Python file objects play the same role. Rust encodes errors in **`io::Result`** instead of exceptions.

## 20.4 Read and Write essentials

| Method                                | Contract                                                            |
|---------------------------------------|---------------------------------------------------------------------|
| <code>read(&amp;mut buf)</code>       | Up to <code>buf.len()</code> bytes; <b><code>0k(0)</code></b> = EOF |
| <code>read_exact(&amp;mut buf)</code> | Fills <code>buf</code> or returns <code>UnexpectedEof</code>        |
| <code>write(&amp;mut buf)</code>      | May write <b>partial</b> slice                                      |
| <code>write_all(&amp;mut buf)</code>  | Loops until all bytes sent or error                                 |

Prefer **`write_all`** and **`read_exact`** when the protocol has fixed-size fields. Use **`read`** in loops when length is unknown until EOF.

## 20.5 Examples: elementary -> hard

Work through in order. After each snippet: **run it**, then read **what happened**.

### 20.5.1 Level 1 — Elementary: generic echo

```
// Playground --- in-memory stand-in (no
    filesystem)
use std::io::{self, Cursor, Read, Write};

fn echo(mut r: impl Read, mut w: impl Write) ->
    io::Result<()> {
    let mut buf = [0u8; 64];
    loop {
        let n = r.read(&mut buf)?;
        if n == 0 { break; }
        w.write_all(&buf[..n])?;
    }
    Ok(())
}

fn main() -> io::Result<()> {
    let input = b"sensor:42\n";
    let mut reader = Cursor::new(&input[..]);
    let mut out = Vec::new();
    echo(&mut reader, &mut out)?;
    println!("{}",
        String::from_utf8_lossy(&out));
    Ok(())
}
```

**What happened:** prints **sensor:42** — `Cursor` implements `Read` over a byte slice; `Vec` implements `Write`. Same `echo` works for files or sockets without changing the loop.

### 20.5.2 Level 2 — Elementary: line-oriented config

```
// Playground
use std::io::{BufRead, BufReader, Cursor};

fn parse_kv_lines(data: &str) -> Vec<(String,
    String)> {
    let mut out = Vec::new();
```

```

let reader =
    BufReader::new(Cursor::new(data.as_bytes
        ()));
for line in reader.lines() {
    let Ok(line) = line else { continue };
    let Some((k, v)) = line.split_once('=')
        else { continue };
    out.push((k.trim().into(),
        v.trim().into()));
}
out

fn main() {
    let cfg = "poll_ms=100\nport=502\n#
        comment\n";
    for (k, v) in parse_kv_lines(cfg) {
        println!("{k} -> {v}");
    }
}

```

**What happened:** prints **poll\_ms -> 100** and **port -> 502** — BufReader reduces syscalls for real files; # comment line skipped (no =). In production, return `io::Result` and map errors (Chapter 8) instead of expect on lines.

### 20.5.3 Level 3 — Intermediate: frame struct + CRC

Wrap bytes in a type once the layout stabilizes:

```

// Playground
#[derive(Debug, PartialEq)]
struct Frame {
    id: u8,
    value: u16,
    valid: bool,
    alarm: bool,
}

impl Frame {

```

```

fn encode(&self) -> [u8; 5] {
    let vb = self.value.to_be_bytes();
    let mut status = 0u8;
    if self.valid { status |= 1; }
    if self.alarm { status |= 2; }
    let body = [self.id, vb[0], vb[1],
                status];
    let crc = body.iter().fold(0u8, |a, &b|
                                a ^ b);
    let mut out = [0u8; 5];
    out[..4].copy_from_slice(&body);
    out[4] = crc;
    out
}

fn decode(buf: &[u8; 5]) -> Option<Self> {
    let body = &buf[..4];
    if body.iter().fold(0u8, |a, &b| a ^ b)
        != buf[4] {
        return None;
    }
    let value =
        u16::from_be_bytes([body[1],
                             body[2]]);
    Some(Self {
        id: body[0],
        value,
        valid: body[3] & 1 != 0,
        alarm: body[3] & 2 != 0,
    })
}

fn main() {
    let f = Frame { id: 0x01, value: 1025,
                    valid: true, alarm: false };
    let bytes = f.encode();
    println!("{:02x?}", bytes);
}

```

```
println!("{:?}", Frame::decode(&bytes));
}
```

**What happened:** prints hex frame and `Some(Frame { ... })` — XOR CRC is toy-grade; real Modbus uses CRC-16. Pattern: **encode/decode on struct**, not loose indices.

## 20.5.4 Level 4 — Intermediate: subprocess output

| Python                      | Java                        | Rust                                     |
|-----------------------------|-----------------------------|------------------------------------------|
| <code>subprocess.run</code> | <code>ProcessBuilder</code> | <code>Command::new(...).output()</code>  |
| <code>shell=True</code>     | —                           | <code>sh -c "..."</code> (use sparingly) |

**Cargo only:**

```
use std::process::Command;

fn main() -> std::io::Result<> {
    let out = Command::new("echo").arg("hello
        automation").output()?;
    println!("status: {}", out.status);
    println!("stdout: {}",
        String::from_utf8_lossy(&out.stdout));
    Ok(())
}
```

**What happened:** `output()` waits for the child, captures stdout/stderr into `Vec<u8>`, returns exit status. `from_utf8_lossy` handles non-UTF8 tool output without panicking.

**Safer than shell:** pass arguments with `.arg()` — no injection surface. Use `sh -c` only when you need `|`, globs, or shell variables.

## 20.5.5 Level 5 — Hard: pipeline sketch (Cargo only)

Wire **program A** -> **program B** with pipes (conceptual):

```
// Cargo only --- Unix-oriented; needs
    std::process::{Command, Stdio}
// use std::process::{Command, Stdio};
```



```
//
// fn main() -> std::io::Result<()> {
//     let mut first = Command::new("cat")
//         .arg("input.txt")
//         .stdout(Stdio::piped())
//         .spawn()?;
//     let first_out =
//         first.stdout.take().expect("piped");
//     let status = Command::new("grep")
//         .arg("ERROR")
//         .stdin(first_out)
//         .status()?;
//     first.wait()?;
//     println!("pipeline status: {status}");
//     Ok(())
// }
```

**What happened (conceptually):** first child's **stdout** becomes second child's **stdin**. Always wait() children to avoid zombies. For complex graphs, consider duct crate — std is verbose but explicit.

## 20.5.6 Level 6 — Hard: sync file transform (Cargo only)

```
// Cargo only
use std::fs::File;
use std::io::{BufRead, BufReader, Write};

fn process_file(input: &str, output: &str) ->
    std::io::Result<()> {
    let reader =
        BufReader::new(File::open(input)?);
    let mut out = File::create(output)?;
    for line in reader.lines() {
        let line = line?;
        writeln!(out, "{}",
            line.trim().to_uppercase());
    }
    out.flush()?;
}
```

```
Ok(())
}
```

**What happened:** ? propagates `io::Error` from `open`, `read`, or `write` — `main` can use `fn main() -> io::Result<()>`. **flush** before `exit` ensures buffered bytes hit disk (important for logs and PLC command files).

## 20.6 File I/O patterns

| Pattern                                        | When                                     |
|------------------------------------------------|------------------------------------------|
| <code>File::open + BufReader</code>            | Line or chunk input                      |
| <code>read_to_string</code>                    | Small text configs only                  |
| <code>BufWriter + write_all</code>             | Many small writes                        |
| <code>main() -&gt; io::Result&lt;()&gt;</code> | CLI tools — map to exit code at boundary |

**Boundary discipline** (Chapter 8): use internal `fn load_config() -> Result<Config, Error>`. In `main`, print `eprintln!("{e}")` and `exit 1`. Never `unwrap` on user-supplied paths in unattended services.

## 20.7 Bitwise and binary protocols

Register and status words on the wire are **bit fields**, not separate `bool` variables:

| Technique                                    | Use                                      |
|----------------------------------------------|------------------------------------------|
| <code>value  = 1 &lt;&lt; n</code>           | Set bit <i>n</i>                         |
| <code>value &amp; (1 &lt;&lt; n) != 0</code> | Test bit <i>n</i>                        |
| <code>to_be_bytes / from_be_bytes</code>     | Modbus-style <b>big-endian</b> registers |
| <code>to_le_bytes / from_le_bytes</code>     | Intel layouts, many CAN/USB payloads     |
| <code>struct + encode/decode</code>          | Stable frame layout                      |

Use **unsigned** types (`u8`, `u16`, `u32`) for shifts — signed shifts are a footgun.

## 20.8 Sync vs async I/O

| Situation                                        | Prefer                                                                              |
|--------------------------------------------------|-------------------------------------------------------------------------------------|
| CLI tool, batch file transform                   | <b>sync</b> <code>std::fs</code> , <code>BufReader</code>                           |
| One serial poll loop, blocking read with timeout | <b>sync</b> + dedicated thread (Chapter 14)                                         |
| Many TCP connections, one process                | <b>async</b> Tokio (Chapter 16)                                                     |
| Blocking read inside <code>async fn</code>       | <b>Bad</b> — stalls executor; <code>tokio::fs</code> or <code>spawn_blocking</code> |

Same **traits** mentally. Async adds `.await` and a runtime. Chapter 16 covers production protocol code.

## 20.9 Practical I/O scenarios

| Task                                    | Approach                                                                                               |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------|
| Read CSV/TOML config                    | <code>BufReader</code> + parse, or <code>serde</code> + file (Chapter 17)                              |
| JSON config / log line / REST body      | <code>read_to_string</code> or <code>buffer -&gt; serde_json::from_str / to_string</code> (Chapter 17) |
| Run <code>systemctl</code> / vendor CLI | Command direct args; capture stdout; check status                                                      |
| Modbus-style register                   | Frame struct, BE u16, CRC per spec                                                                     |
| Serial sensor / PLC                     | <code>serialport</code> crate — same Read/Write                                                        |
| Vendor C driver only                    | Safe Rust wrapper (Chapter 18)                                                                         |

### Serial orientation (Cargo only):

```
# Cargo.toml
# [dependencies]
# serialport = "4"

// Cargo only --- list ports; optional open
//   when arg provided
// use std::io::{Read, Write};
// use std::time::Duration;
//
// fn main() -> Result<(), Box<dyn
//   std::error::Error>> {
```

```
//      for p in serialport::available_ports()? {
//          println!("{}", p.port_name);
//      }
//
//  serialport::new("/dev/ttyUSB0",
//  9600).timeout(...).open()?
//
//  port.write_all(b"PING\n"); port.read(&mut
//  buf)?;
//      Ok(())
//  }
```

Poll loops: set **read timeout**, log the port on error, and **retry with backoff** (Afterparty P345). Do not spin tight on failure.

## 20.10 CLI utility — paths, env, and time

One tool thread: resolve config path from env -> check file size -> read key=value lines.

### 20.10.1 Path and env

```
// Cargo only --- conceptual main fragment
// use std::path::{Path, PathBuf};
// use std::env;
//
// fn config_path() -> PathBuf {
//     env::var("APP_CONFIG")
//         .map(PathBuf::from)
//         .unwrap_or_else(|_| {
//
//             Path::new(&env::var("HOME").unwrap_or_else(|
//             _| ".".into()))
//                 .join(".config")
//                 .join("gateway.toml")
//         })
// }
```

Path / PathBuf join segments without manual slash handling. Accept

impl AsRef<Path> in helpers (Chapter 13).

## 20.10.2 File metadata before read

```
// Cargo only
// use std::fs;
//
// fn read_if_small(path: &Path) ->
//     std::io::Result<String> {
//     let meta = fs::metadata(path)?;
//     if meta.len() > 1_000_000 {
//         return Err(std::io::Error::new(
//             std::io::ErrorKind::InvalidData,
//             "config too large",
//         ));
//     }
//     fs::read_to_string(path)
// }
```

Check `metadata().len()` before allocating a huge buffer — cheap guard for untrusted paths.

## 20.10.3 Stdin prompt alongside args

```
// Playground --- stdin blocks until Enter
// (compile-only in CI)
use std::io::{self, Write};

fn main() -> io::Result<()> {
    let arg_port = std::env::args().nth(1);
    let port = if let Some(p) = arg_port {
        p
    } else {
        print!("port: ");
        io::stdout().flush()?;
        let mut buf = String::new();
        io::stdin().read_line(&mut buf)?;
        buf.trim().to_string()
    };
};
```

```
println!("using port {}", port);
Ok(())
}
```

Args for scripts; stdin for interactive runs — same binary, two entry paths.

#### 20.10.4 Timeout with Duration

```
// Playground
use std::time::Duration;

fn retry_delay(attempt: u32) -> Duration {
    let secs: u64 = std::env::var("RETRY_SECS")
        .ok()
        .and_then(|s| s.parse().ok())
        .unwrap_or(2);
    Duration::from_secs(secs.saturating_mul(attempt as u64))
}

fn main() {
    println!("attempt 3 sleep {:?}",
        retry_delay(3));
}
```

Duration pairs with `thread::sleep` and serial timeouts — `env::var` gives ops a knob without recompile.

#### 20.10.5 CLI I/O edge cases

**Wrong — assume path is UTF-8 string on all OSes:** use `Path` end to end; convert to `str` only when displaying.

**Line endings:** `read_line` keeps `\n`; call `.trim()` before parsing numbers.

### 20.11 PTY (note)

Interactive CLI tools sometimes need a **pseudo-terminal** (line discipline, echo). Not in `std`; crates like `pty-process` or `expectrl` fill the gap. Start

with `Command`; add `PTY` when the tool checks `isatty`.

## 20.12 Edge cases and compiler traps

| Trap                                        | Symptom                    | Idiom                                       |
|---------------------------------------------|----------------------------|---------------------------------------------|
| read once assuming full buffer              | short read, corrupt parse  | loop read or <code>read_exact</code>        |
| write without <code>write_all</code>        | partial send on socket     | always <code>write_all</code> for protocols |
| Wrong endianness                            | register value off by 256× | document BE vs LE per spec                  |
| <code>unwrap</code> on <code>lines()</code> | panic on bad UTF-8         | ? + error type                              |
| <code>shell -c</code> with user input       | command injection          | <code>.arg()</code> list                    |
| Forgot <code>flush</code>                   | lost tail bytes on crash   | <code>flush</code> before exit              |
| Blocking serial in async task               | stalled Tokio worker       | dedicated thread or async crate             |

## 20.13 Idiom spotlight

**Parse bytes with types, not magic indices forever.** Once a frame layout stabilizes, wrap it in `struct` + `encode/decode` methods.

**Generic over `Read/Write` for testability** — `Cursor` in unit tests, real `File` in integration tests.

**Treat I/O as `Result` at boundaries** — long-running binaries should survive bad paths and timeouts without panic (Chapter 8).

## 20.14 Go deeper

- [Csv parser / writer](#) — 958–959
- [Rust book](#) — I/O

## 20.15 See also

- Chapter 8: Errors — `io::Error`, ?, no panic in production loops
- Chapter 14: Multithreading — blocking I/O on worker threads

- Chapter 16: Async I/O — `tokio::fs`, `TcpListener`
- Chapter 17: Metaprogramming — `include_str!`, `serde` JSON serialize/deserialize
- Chapter 18: Unsafe — FFI and safe wrappers over I/O handles
- Chapter 20: Production standards — review checklist before merge

### 20.15.0.1 Read / Write and buffering

1. **Trait refactor** — “Refactor file copy loop to generic `copy<R: Read, W: Write>`; discuss error propagation.”
2. **read vs read\_exact** — “Give 3 protocol shapes; I pick `read` loop vs `read_exact` each time; you verify.”
3. **Cursor test** — “Write unit test for `parse_kv_lines` using `Cursor` — no `filesystem`.”
4. **BufReader why** — “Explain in 60 words why `BufReader` matters for 10k-line log files.”

### 20.15.0.2 Files and CLI

5. **CSV tool** — “Spec for CLI: read two-column CSV, emit `name=value`; I implement with `BufRead`.”
6. **Boundary errors** — “Sketch main mapping `io::Error` to exit code 1 with context path — no `unwrap`.”
7. **read\_to\_string trap** — “When is `read_to_string` wrong for automation configs? Give size threshold rule.”

### 20.15.0.3 Binary frames and endianness

8. **Packet layout** — “Add CRC byte to 4-byte packet; update encode/decode with XOR — show tests.”
9. **Endian trap** — “Quiz: 3 scenarios pick LE vs BE for Modbus-style register.”
10. **Bit field port** — “Java status int with flags — port to Rust encode with `|=` and `&` masks.”
11. **CRC upgrade** — “Replace XOR toy CRC with CRC-16-Modbus — out-line steps, no full crate required.”

### 20.15.0.4 Processes and pipelines

12. **Command safety** — “Review `shell=True` style command; rewrite without shell when possible.”



13. **Pipeline** — “Design program A | program B using only Rust std (two processes, pipe).”
14. **Exit status** — “Child exited 2 — how should gateway log and retry? Table: fatal vs transient.”
15. **Env and cwd** — “Show Command with `.env("PORT", "502")` and `.current_dir` — when needed?”

#### 20.15.0.5 Sync vs async and serial

16. **Sync vs async pick** — “Three gateway designs: I pick sync thread vs Tokio per scenario; you justify.”
17. **Serial debug** — “I get timeout on read; give systematic checklist (baud, cable, permissions).”
18. **serialport traits** — “Explain how serialport maps to Read/Write — diagram only.”
19. **Blocking in async** — “Show wrong `std::fs::read` inside `async fn`; fix with `tokio::fs` or `spawn_blocking`.”

#### 20.15.0.6 Capstone and operations

20. **Capstone scaffold** — “Generate module tree and function signatures for `sensor_gateway`; no bodies.”
21. **Retry policy** — “Design exponential backoff for Modbus-style poll errors; Rust pseudocode.”
22. **Log schema** — “Propose JSON log lines for sensor events with timestamps and error codes.”
23. **GPIO next step** — “After serial works on Pi, outline migration to `gpio-dev` for one LED.”
24. **Code review** — “I paste capstone main loop; review for panic risks and missing flush.”
25. **Gateway capstone** — “End-to-end: config file -> serial poll -> JSON log line — module list and error types only.”
26. **Ch16 bridge** — “Same echo server: sketch sync thread version vs Tokio version — tradeoffs table.”

#### 20.15.0.7 Paths, env, and CLI

27. **Path join** — “Build config path from `HOME + .config/app.toml` — show `Path::join` vs string concat trap.”

28. **Env default** — “TIMEOUT env var with default 30 — `var().unwrap_or_else` pattern.”
29. **Metadata guard** — “Reject config files over 1MB before `read_to_string` — `sketch_metadata().len()` check.”
30. **Stdin fallback** — “Port from `argv` or interactive prompt — one main with both paths.”
31. **Line parser** — “BufRead lines, skip #, parse `key=value` — handle trailing newline.”
32. **Capstone CLI** — “End-to-end: env path -> metadata check -> line parse -> print port — function list only.”

## **Part V**

# **Production Standards**

# Chapter 21

## Chapter 20: Production Rust Standards

### 21.1 Hook

You can write Rust that compiles and still miss what experienced reviewers look for: typed boundaries, errors as data, minimal cloning, and panic-free production paths. This chapter is a **review checklist** — the habits teams enforce in code review and the patterns worth asking an AI to verify after every Rust change.

### 21.2 Scope — a brief tour

| This chapter covers                         | Assumes you have read   |
|---------------------------------------------|-------------------------|
| Review checklist with examples + motivation | Ch 6–8 types and errors |
| Ownership, allocation, and pointer idioms   | Ch 1, Ch 10             |
| Workspace and test conventions              | Ch 9, Ch 8              |

Run these checks **only when Rust code changed**. No Rust diff -> skip this chapter.

## 21.3 How to use the checklist

Each section follows the same shape:

1. **Anti-pattern** (or weak pattern) — what reviewers reject.
2. **Idiomatic fix** — runnable example.
3. **Why** — short motivation: what breaks in production if you ignore it.

At the end, a **one-page table** collects every rule for quick scans.

---

## 21.4 Types — newtypes and enums

Production code often smuggles meaning through **raw numbers**: two `u8` parameters that look interchangeable, and status codes checked with `== 0`. Newtypes and enums make that meaning visible to the compiler.

### 21.4.1 Weak — primitives hide two different mistakes

```
// Playground --- weak
fn write_register(unit_id: u8, register: u8,
    value: u16) {
    let _ = (unit_id, register, value);
}

fn is_healthy(status_code: u8) -> bool {
    status_code == 0
    // 0 = ok, 1 = fault --- magic number at
    // the call site
}

fn main() {
    write_register(3, 1, 100);
    // unit 3, register 1
    write_register(1, 3, 100);
    // compiles --- same types, wrong order

    let status_code = 1;
    if !is_healthy(status_code) {
```

```
println!("device fault");
}
}
```

Two separate problems:

| Problem                      | What goes wrong                                                   |
|------------------------------|-------------------------------------------------------------------|
| unit_id: u8 and register: u8 | Swapping arguments <b>compiles</b> — both are u8.                 |
| status_code: u8 with == 0    | Callers pass undocumented magic numbers; easy to typo a valid u8. |

Different primitive types (u16 vs String) already prevent some swaps — the real footgun is **two parameters of the same type**, plus **open-coded status values**.

## 21.4.2 Strong — newtypes for identity, enums for state

```
// Playground
struct UnitId {
    id: u8,
}

struct Register {
    address: u8,
}

enum DeviceStatus {
    Healthy,
    Fault { code: u8 },
}

fn write_register(unit: UnitId, reg: Register,
    value: u16) {
    println!(
        "unit {} reg {} := {}",
        unit.id, reg.address, value
    );
}
```

```

}

fn describe(status: DeviceStatus) -> &'static
    str {
    match status {
        DeviceStatus::Healthy => "healthy",
        DeviceStatus::Fault { .. } => "fault",
    }
}

fn main() {
    write_register(
        UnitId { id: 3 },
        Register { address: 1 },
        100,
    );
    // write_register(
    //     Register { address: 1 },
    //     UnitId { id: 3 },
    //     100,
    // );
    // ERROR: expected UnitId, found Register

    let status = DeviceStatus::Fault { code:
        0x07 };
    println!("{}", describe(status), match
        status {
            DeviceStatus::Fault { code } =>
                format!("code 0x{:02X}", code),
            DeviceStatus::Healthy => "no
                code".into(),
        });
}

```

| Fix               | Effect                                                                |
|-------------------|-----------------------------------------------------------------------|
| UnitId / Register | Swapping arguments is a <b>type error</b> , not a silent wrong write. |

| Fix               | Effect                                                                                        |
|-------------------|-----------------------------------------------------------------------------------------------|
| DeviceStatus enum | States are <b>named variants</b> — add Degraded and the compiler lists every match to update. |

**Why:** newtypes attach **identity** to values that share a representation (u8, u16, String). Enums attach **behaviour** to state instead of scattered integer checks. Together they turn whole classes of field and wiring bugs into compile-time failures (Chapter 6, Chapter 7).

## 21.5 Lifetimes — correct and minimal

**Weak:** 'static everywhere because it silences the borrow checker.

```
// Playground --- smell: unnecessary 'static
fn label(_s: &'static str) -> &'static str {
    "fixed"
}
```

**Strong:** the **shortest** lifetime that describes the data — often elided on simple helpers.

```
// Playground
fn first_token(s: &str) -> Option<&str> {
    s.split_whitespace().next()
}

fn main() {
    let line = String::from("temp 22.5");
    println!("{:?}", first_token(&line));
}
```

**Why:** over-long lifetimes hide bugs (returning references to temporaries) and force callers to leak or allocate. Minimal signatures stay reusable and honest (Chapter 5). Add explicit lifetimes only when the compiler asks or when two inputs must outlive the same scope.



## 21.6 Option and Result — not null-like sentinels

**Weak:** magic values (−1, empty string) or nullable references for “missing”.

```
// Playground --- weak
fn find_port(config: &str) -> i32 {
    if config.is_empty() {
        -1
    } else {
        502
    }
}
```

**Strong:** Option for absence, Result for failure with a reason.

```
// Playground
fn find_port(config: &str) -> Option<u16> {
    if config.is_empty() {
        None
    } else {
        Some(502)
    }
}

fn parse_port(config: &str) -> Result<u16,
    &'static str> {
    let p = find_port(config).ok_or("missing
        port"?);
    if p < 1024 {
        Err("privileged port")
    } else {
        Ok(p)
    }
}

fn main() {
    println!("{:?} {:?}", find_port("modbus"),
        parse_port("8080"));
}
```

**Why:** sentinels collide with valid data and skip the type system. `Option/Result` force callers to handle absence and failure — the core of reliable automation (Chapter 6, Chapter 8).

---

## 21.7 Avoid unwrap and expect in production

**Weak:** I/O and parsing that panic on bad input.

```
// Playground --- production anti-pattern
fn load_port(s: &str) -> u16 {
    s.parse().unwrap()
}
```

**Strong:** propagate or handle at a boundary.

```
// Playground
fn load_port(s: &str) -> Result<u16, String> {
    s.parse().map_err(|e| e.to_string())
}

fn main() {
    match load_port("502") {
        Ok(p) => println!("port {}", p),
        Err(e) => eprintln!("skip tick: {}", e),
    }
}
```

**Why:** `unwrap/expect` **panic** — they stop the thread (often the whole gateway) on expected failures like bad config or a dropped device. Reserve them for **tests**, **prototypes**, or truly impossible states after invariants you control (Chapter 8).

---

## 21.8 Panic audit — indexing, `panic!`, and production paths

**Weak:** unchecked indexing and explicit panics on user data.

```
// Playground --- can panic in production
fn first_byte(frame: &[u8]) -> u8 {
    frame[0] // panics if empty
}

fn validate_tag(tag: &str) {
    if tag.is_empty() {
        panic!("empty tag");
    }
}
```

**Strong:** safe access and Result for invalid input.

```
// Playground
fn first_byte(frame: &[u8]) -> Option<u8> {
    frame.first().copied()
}

fn validate_tag(tag: &str) -> Result<(),
    &'static str> {
    if tag.is_empty() {
        Err("empty tag")
    } else {
        Ok(())
    }
}

fn main() {
    println!("{:?} {:?}", first_byte(&[]),
        validate_tag("temp"));
}
```

**Why:** every `vec[i]`, `unwrap`, `expect`, and `panic!` on a production path is an **unhandled incident** waiting for bad wire data. Tests may panic freely; field code should return `Err` and let the supervisor retry or alarm.

---

## 21.9 Propagate with ? consistently

**Weak:** nested match noise in fallible pipelines.

```
// Playground --- verbose
fn load(s: &str) -> Result<u16, String> {
    let t = match s.parse::<u16>() {
        Ok(v) => v,
        Err(e) => return Err(e.to_string()),
    };
    Ok(t)
}
```

**Strong:** ? on the railway (Chapter 8).

```
// Playground
fn load(s: &str) -> Result<u16, String> {
    let t = s.parse::<u16>().map_err(|e|
        e.to_string())?;
    Ok(t)
}

fn main() {
    println!("{}", load("502"));
}
```

**Why:** consistent ? keeps helpers thin and makes the **happy path** readable. Reviewers spot missing error handling when ? is the default and match is reserved for recovery.

---

## 21.10 Custom errors — thiserror in libraries, anyhow in small binaries

**Weak:** library API returning Result<T, String> or using anyhow in public interfaces.

```
// Playground --- weak public API
pub fn connect(_host: &str) -> Result<(),
    String> {
```

```
Err("timeout".into())
}
```

**Strong:** focused enum + **thiserror** in library crates; **anyhow** only in binary/main helpers.

```
// Cargo project --- library pattern (thiserror
    = "2")
use thiserror::Error;

#[derive(Debug, Error)]
pub enum ConnectError {
    #[error("timeout after {ms} ms")]
    Timeout { ms: u64 },
    #[error("invalid host")]
    InvalidHost,
}

pub fn connect(_host: &str) -> Result<(),
    ConnectError> {
    Err(ConnectError::Timeout { ms: 500 })
}
```

```
// Cargo project --- binary only (anyhow = "1")
use anyhow::Context;

fn run() -> anyhow::Result<()> {
    let _port =
        std::env::var("PORT").context("PORT
        must be set");
    Ok(())
}
```

**Why:** typed errors are **matchable** for recovery; strings are not. **thiserror** removes boilerplate while keeping a stable library contract. **anyhow** is fine for **one-off scripts and main**, not for code other crates depend on (Chapter 8).

---

## 21.11 Errors store data, not messages

**Weak:** formatting errors into strings — context lost, no structured logging.

```
// Playground --- weak
enum AppError {
    Io(String),
    // "failed to read /etc/gateway.toml:
    // permission denied"
}
```

**Strong:** variants carry **fields**; display text comes from `thiserror` or `Display`.

```
// Playground
#[derive(Debug)]
enum AppError {
    ConfigRead {
        path: String,
        source: std::io::Error,
    },
}

fn main() {
    let e = AppError::ConfigRead {
        path: "gateway.toml".into(),
        source:
            std::io::Error::new(std::io::ErrorKind::NotFound, "missing"),
    };
    println!("{:?}", e);
    // logs match on kind + path, not string
    // grep
}
```

**Why:** structured variants support **metrics, retries, and tests** (match on `ErrorKind::NotFound`). Stringly errors force fragile substring checks and duplicate messages across call sites.

---

## 21.12 Focused error enums per layer

**Weak:** one mega `AppError` for validation, Mongo, config, and serial — returned from a string trim helper.

```
// Playground --- weak: validation returns
    unrelated variants
enum AppError {
    InvalidPort,
    MongoTimeout,
    ConfigParse,
}

fn trim_port(s: &str) -> Result<u16, AppError> {
    s.trim().parse().map_err(|_|
        AppError::InvalidPort)
    // why would MongoTimeout appear here?
}
```

**Strong:** each **module or concern** owns a small enum; map at boundaries.

```
// Playground
#[derive(Debug)]
enum ValidateError {
    InvalidPort,
    OutOfRange(u16),
}

#[derive(Debug)]
enum StorageError {
    Timeout { ms: u64 },
}

fn trim_port(s: &str) -> Result<u16,
    ValidateError> {
    let p: u16 = s.trim().parse().map_err(|_|
        ValidateError::InvalidPort)?;
    if p < 1024 {
        Err(ValidateError::OutOfRange(p))
    } else {
```

```

        Ok(p)
    }
}

fn save(_port: u16) -> Result<(), StorageError>
{
    Err(StorageError::Timeout { ms: 500 })
}

fn main() {
    println!("{:?} {:?}", trim_port("8080"),
        save(8080));
}

```

**Why:** callers of `trim_port` should not handle database timeouts. Focused enums document **who can recover from what** and keep match arms honest. Compose or wrap at the service boundary (Chapter 8 — error aggregation).

## 21.13 Cloning — appropriate, not a band-aid

**Weak:** `.clone()` to silence borrow checker errors without understanding ownership.

```

// Playground --- weak: clone hides that you
// could borrow
fn print_twice(s: &String) {
    println!("{}", s.clone(), s.clone());
}

```

**Strong:** clone when you **need** an owned copy; borrow otherwise.

```

// Playground
fn print_twice(s: &str) {
    println!("{}", s, s);
}

fn store_label(s: &str) -> String {

```



```

    s.to_string()
    // one intentional allocation for storage
}

fn main() {
    let label = String::from("sensor-1");
    print_twice(&label);
    let owned = store_label(&label);
    println!("{}", owned);
}

```

**Why:** unnecessary clones cost CPU and memory in hot paths (parsers, poll loops). Cloning to “make it compile” often means the API should take `&str`, return owned data once, or use Cow (Chapter 13).

## 21.14 Avoid unnecessary `.clone()` and `.to_owned()`

**Weak:** cloning collections on every iteration.

```

// Playground --- weak
fn sum_tags(tags: &Vec<String>) -> usize {
    tags.clone().iter().map(|t| t.len()).sum()
}

```

**Strong:** iterate by reference.

```

// Playground
fn sum_tags(tags: &[String]) -> usize {
    tags.iter().map(|t| t.len()).sum()
}

fn main() {
    let tags = vec!["a".into(), "bb".into()];
    println!("{}", sum_tags(&tags));
}

```

**Why:** each stray clone duplicates heap data. Accept `&[T]` / `&str` in helpers; clone only when crossing threads, storing long-term, or transforming into an owned value.

---

## 21.15 Rc::clone/Arc::clone — not .clone() on the smart pointer

**Weak:** `arc.clone()` — ambiguous (clone the pointer or the inner value?).

```
// Playground --- works but unclear in review
use std::sync::Arc;

fn share(data: Arc<String>) -> Arc<String> {
    data.clone()
}
```

**Strong:** explicit `Arc::clone(&data)` (same for `Rc::clone`).

```
// Playground
use std::sync::Arc;

fn share(data: Arc<String>) -> Arc<String> {
    Arc::clone(&data)
}

fn main() {
    let a = Arc::new("config".to_string());
    let b = share(Arc::clone(&a));
    println!("{}", a, b);
}
```

**Why:** `Arc::clone` increments the **reference count** only — cheap and obvious in code review. `.clone()` on `Arc` does the same thing but reads like a deep copy of the inner `String`. Team style guides often require the explicit form (Chapter 10).

---

## 21.16 Borrow instead of move when you only read

**Weak:** taking `String` by value when the caller still needs it.

```
// Playground --- weak
fn log_name(name: String) {
    println!("{}", name);
}

fn main() {
    let name = String::from("plc-1");
    log_name(name);
    // println!("{}", name); // ERROR: moved
}
```

**Strong:** `&str` / `&T` for read-only use (Chapter 3).

```
// Playground
fn log_name(name: &str) {
    println!("{}", name);
}

fn main() {
    let name = String::from("plc-1");
    log_name(&name);
    println!("still have {}", name);
}
```

**Why:** moving values forces clones at call sites or loses data after one call. Borrowing keeps ownership with the caller and documents read-only intent.

---

## 21.17 Heap allocations — only when needed

**Weak:** `Box`, `Vec`, and `String` for tiny stack-sized data.

```
// Playground --- weak
fn port() -> Box<u16> {
```

```
Box::new(502)
}
```

**Strong:** stack types and iterators; heap when size is dynamic or ownership must move.

```
// Playground
fn port() -> u16 {
    502
}

fn readings(count: usize) -> Vec<f64> {
    (0..count).map(|i| i as f64).collect()
}

fn main() {
    println!("{}", port(), readings(3));
}
```

**Why:** heap allocations add latency and fragmentation. Prefer stack values, `&str`, and `impl Iterator` return types until you need dynamic size or trait objects (Chapter 3, Chapter 7).

---

## 21.18 `Vec::with_capacity` when size is bounded

**Weak:** repeated push on an empty `Vec` — many reallocations.

```
// Playground --- weak for known upper bound
fn build_tags(n: usize) -> Vec<String> {
    let mut v = Vec::new();
    for i in 0..n {
        v.push(format!("tag-{}", i));
    }
    v
}
```

**Strong:** `with_capacity` when max length is known.

```
// Playground
fn build_tags(n: usize) -> Vec<String> {
    let mut v = Vec::with_capacity(n);
    for i in 0..n {
        v.push(format!("tag-{}", i));
    }
    v
}

fn main() {
    println!("len={}", build_tags(100).len());
}
```

**Why:** pre-allocation avoids  $O(n)$  realloc copies in parsers and batch builders. If capacity is unknown, start empty; if you know “at most  $N$  registers”, reserve  $N$  (Chapter 11).

---

## 21.19 Workspace dependencies — { workspace = true }

**Weak:** duplicate version pins in every crate of a workspace.

```
# member Cargo.toml --- weak
[dependencies]
tokio = { version = "1.40", features = ["full"] }
serde = "1.0.210"
```

**Strong:** centralize in the **root** Cargo.toml, inherit in members.

```
# root Cargo.toml
[workspace.dependencies]
tokio = { version = "1", features = ["full"] }
serde = "1"
thiserror = "2"
```

```
# member Cargo.toml
[dependencies]
```

```
tokio = { workspace = true }
serde = { workspace = true }
thiserror = { workspace = true }
```

**Why:** one bump updates every crate; CI and local builds stay on the same versions. Exception: a member-only dependency that is large and unrelated to the rest — pin it locally with a comment (Chapter 9).

---

## 21.20 Tests — equality, not field-by-field drift

**Weak:** asserting each field manually — tests break on every harmless derive change.

```
// Playground --- brittle
#[derive(Debug, PartialEq)]
struct Reading { tag: String, value: f64 }

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn reading() {
        let r = Reading { tag: "t".into(),
                          value: 1.0 };
        assert_eq!(r.tag, "t");
        assert_eq!(r.value, 1.0);
    }
}

fn main() {}
```

**Strong:** `assert_eq!` on the whole value when `PartialEq` is implemented; use `pretty_assertions` in Cargo projects for diffs.

```
// Playground
#[derive(Debug, PartialEq)]
struct Reading { tag: String, value: f64 }
```

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn reading() {
        let got = Reading { tag: "t".into(),
                           value: 1.0 };
        let want = Reading { tag: "t".into(),
                             value: 1.0 };
        assert_eq!(got, want);
    }
}

fn main() {}
```

```
# Cargo --- dev-dependencies
[dev-dependencies]
pretty_assertions = "1"
```

```
// Cargo test module
use pretty_assertions::assert_eq;
```

**Why:** struct equality catches **any** field drift in one failure. `pretty_assertions` prints side-by-side diffs for nested data — essential for parser golden tests (Chapter 8).

---

## 21.21 Tests — avoid time and global state pitfalls

**Weak:** tests that sleep real time or mutate static globals — flaky in CI.

```
// Playground --- flaky pattern (conceptual)
// static mut COUNTER: u32 = 0;
// #[test]
// fn depends_on_wall_clock() {
//
//     std::thread::sleep(Duration::from_secs(1));
//     assert!(true);
```

```
// }
```

**Strong:** inject clocks and stores; use temp dirs in integration tests.

```
// Playground
trait Clock {
    fn now_ms(&self) -> u64;
}

struct FixedClock(u64);

impl Clock for FixedClock {
    fn now_ms(&self) -> u64 {
        self.0
    }
}

fn is_stale(c: &impl Clock, last: u64, max_age:
    u64) -> bool {
    c.now_ms().saturating_sub(last) > max_age
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn stale_after_threshold() {
        let c = FixedClock(1000);
        assert!(is_stale(&c, 0, 500));
        assert!(!is_stale(&c, 900, 500));
    }
}

fn main() {}
```

**Why:** wall-clock sleeps slow CI and fail under load. Global mutable state makes tests **order-dependent**. Fake clocks and per-test fixtures keep suites deterministic (Chapter 8).



## 21.22 Quick reference — full checklist

| #  | Rule                                      | One-line test                                    |
|----|-------------------------------------------|--------------------------------------------------|
| 1  | Newtypes / enums for domain meaning       | Same-type args swappable? Status as raw u8/bool? |
| 2  | Minimal lifetimes                         | Any 'static that could be elided?                |
| 3  | Option / Result, not sentinels            | Any magic -1 / empty meaning "error"?            |
| 4  | No unwrap / expect in production paths    | Only tests and impossible invariants?            |
| 5  | Panic audit                               | Any [], panic!, or unwrap on external input?     |
| 6  | Consistent ?                              | Fallible helpers return Result?                  |
| 7  | thiserror in libs; anyhow in bins/scripts | Public API free of anyhow?                       |
| 8  | Errors carry <b>data</b>                  | Variants have fields, not preformatted strings?  |
| 9  | Focused error enums per layer             | Does validation return DB errors?                |
| 10 | Clone deliberately                        | Clone fixes borrow errors -> fix API first?      |
| 11 | No stray .clone() / .to_owned()           | Could this be &str / &[T]?                       |
| 12 | Rc::clone / Arc::clone                    | Smart-pointer clones explicit?                   |
| 13 | Borrow for read-only                      | Parameters own String without storing?           |
| 14 | Heap when needed                          | Box<u16>-style noise removed?                    |
| 15 | with_capacity when bounded                | Known max len -> reserve?                        |
| 16 | { workspace = true }                      | Member deps aligned with root?                   |
| 17 | Tests use assert_eq! on values            | Field-by-field asserts replaced?                 |
| 18 | pretty_assertions for rich diffs          | Dev-dependency in workspace?                     |
| 19 | No flaky time / global state              | Clocks and stores injected?                      |

## 21.23 Idiom spotlight

**Production Rust is typed, panic-averse, and layered.** Model meaning with newtypes and enums; propagate with `?` and **focused** `thiserror` enums; borrow by default; allocate with intent; test with **equality** and deterministic fixtures. Review Rust diffs against this table before merge.

## 21.24 Go deeper

- [Rust API Guidelines](#)
- [The Rust Book — Error Handling](#)
- [The Rust Book — Writing Automated Tests](#)
- [Clippy lints](#) — many rules overlap this checklist

## 21.25 See also

- Chapter 8: Errors and testing — `Result`, `thiserror`, mocks
- Chapter 7: Structs and traits — newtypes, trait boundaries
- Chapter 9: Modules and crates — workspaces
- Chapter 10: Smart pointers — `Arc` / `Rc`
- Chapter 11: Collections — capacity and maps
- Chapter 13: Standard traits — `From`, `AsRef`, `PartialEq`

### 21.25.1 Afterparty

#### 21.25.1.1 Checklist drills

1. **Diff review** — “Paste a 30-line Rust PR; I mark each checklist row pass/fail with one sentence.”
2. **Mega-error refactor** — “Split one `AppError` into `ValidateError` + `StorageError`; show boundary mapping.”
3. **Panic hunt** — “Find five panic sources in a snippet; replace with `Option/Result`.”
4. **Clone audit** — “Remove three unnecessary clones by fixing signatures to `&str` / `&[T]`.”

5. **Arc style** — “Rewrite `arc.clone()` call sites to `Arc::clone(&arc)`; explain review benefit.”

#### 21.25.1.2 Workspace and tests

6. **Workspace.toml** — “Sketch root + two members; all shared deps use `{ workspace = true }`.”
7. **Golden test** — “Parser output: one `assert_eq!(got, want) + pretty_assertions`; no per-field asserts.”
8. **Flaky test fix** — “Replace `thread::sleep` in test with injected `Clock` trait.”

## **Appendix A**

# **Appendices**

# Appendix B

## Playground Guide

### B.1 When to use the playground

| Use playground                    | Use Cargo locally                                                |
|-----------------------------------|------------------------------------------------------------------|
| Learning syntax, ownership, types | Files, <code>Command</code> , serial ports                       |
| Quick experiments (< 50 lines)    | External crates ( <code>tokio</code> , <code>serialport</code> ) |
| Sharing a link in chat or docs    | Integration tests, multi-file crates                             |

**Playground:** <https://play.rust-lang.org/>

### B.2 Rules (matches `STYLE_GUIDE.md`)

1. **One file** with `fn main()` — no `mod` tree in book snippets marked **Playground**.
2. **std only** — playground cannot add `Cargo.toml` dependencies.
3. **No filesystem** — use `Cursor<[u8]>`, `Vec<u8>`, or `&str` instead of `File::open`.
4. **No subprocess** — simulate with strings or skip to Cargo lab.
5. **Output via `println!`** — playground has no CLI args; hardcode sample input.

## B.3 Sharing code

1. Paste the snippet into the editor.
2. Click **Share** to get a permalink.
3. In your notes, store: Playground: `<url>` next to the chapter example.

## B.4 Edition and channel

Book examples target **Edition 2021**, **stable** channel — same as playground default.

## B.5 Playground stand-ins

| Real API                     | Playground substitute                                                            |
|------------------------------|----------------------------------------------------------------------------------|
| <code>File::open</code>      | <code>Cursor::new(bytes)</code> or <code>include_str!</code> in local Cargo only |
| <code>Command::output</code> | Print mock stdout string                                                         |
| <code>tokio::spawn</code>    | Describe in comment; run Cargo lab                                               |
| Serial port                  | Encode/decode bytes in memory                                                    |

## B.6 Local run for Cargo labs

```
cd your_project
cargo run
cargo test
cargo run --example name # if using examples/
```

## B.7 Troubleshooting

| Problem                       | Fix                                                                               |
|-------------------------------|-----------------------------------------------------------------------------------|
| cannot find crate             | You need a Cargo lab — add dependency to <code>Cargo.toml</code>                  |
| Playground timeout            | Infinite loop or huge print; reduce data                                          |
| Different output on Mac/Linux | Line endings or <code>echo</code> vs <code>cmd</code> — expected for process labs |

## **B.8 See also**

- [CONTENTS.md](#)
- [AI Prompt Index](#)

# Appendix C

## Java / Python / Rust — Mental Model Map

Optional one-page orientation — especially handy if you know **Java** or **Python**. Not required to read the notes; details live in linked chapters.

### C.1 Execution and memory

| Topic                | Java             | Python           | Rust                    |
|----------------------|------------------|------------------|-------------------------|
| Run model            | JVM bytecode     | Interpreter / VM | Native binary           |
| Memory               | GC, heap objects | GC + refcounts   | Ownership, stack + heap |
| Null                 | null             | None             | Option<T>               |
| Exceptions           | throw / catch    | raise            | Result + panic!         |
| Immutability default | fields optional  | variables rebind | let immutable           |

-> Ch 1 Paradigm

### C.2 Types and polymorphism



| Topic                | Java                             | Python                           | Rust                                                     |
|----------------------|----------------------------------|----------------------------------|----------------------------------------------------------|
| Classes              | <code>class</code> + inheritance | <code>class</code> , duck typing | <code>struct</code> + <code>impl</code> , no inheritance |
| Interfaces           | <code>interface</code>           | informal protocol                | <code>trait</code>                                       |
| Generics             | type erasure + bounds            | hints optional                   | monomorphized, zero-cost                                 |
| Runtime polymorphism | virtual methods                  | duck typing                      | <code>dyn Trait</code> or generics                       |

-> Ch 7 Traits

## C.3 Collections and loops

| Topic          | Java                     | Python                | Rust                                   |
|----------------|--------------------------|-----------------------|----------------------------------------|
| Dynamic array  | <code>ArrayList</code>   | <code>list</code>     | <code>Vec&lt;T&gt;</code>              |
| Map            | <code>HashMap</code>     | <code>dict</code>     | <code>HashMap&lt;K, V&gt;</code>       |
| Loop style     | indexed <code>for</code> | <code>for x in</code> | <code>for x in iter / iterators</code> |
| Comprehensions | streams (Java 8+)        | list comp             | iterator adapters                      |

-> Ch 4 Iterators · Ch 11 Collections (`Vec`, `HashMap`)

## C.4 Concurrency

| Topic        | Java                                            | Python                       | Rust                                                         |
|--------------|-------------------------------------------------|------------------------------|--------------------------------------------------------------|
| Threads      | <code>Thread</code> , executors                 | <code>threading</code> (GIL) | <code>std::thread</code>                                     |
| Shared state | <code>synchronized</code> , locks               | locks, GIL limits            | <code>Mutex</code> , <code>Arc</code> , <code>atomics</code> |
| Async        | virtual threads, <code>CompletableFuture</code> | <code>asyncio</code>         | <code>async/await</code> + Tokio                             |
| Data races   | possible at runtime                             | possible (C ext)             | prevented in safe code                                       |

-> Ch 14–16 (threads, atomics, async)

## C.5 I/O and systems

| Topic       | Java           | Python        | Rust                     |
|-------------|----------------|---------------|--------------------------|
| Files       | Files, streams | open()        | File + Read/Write traits |
| Subprocess  | ProcessBuilder | subprocess    | Command                  |
| Binary data | ByteBuffer     | bytes, struct | u8, slices, to_be_bytes  |

-> Ch 19

## C.6 Package management

| Java           | Python       | Rust              |
|----------------|--------------|-------------------|
| Maven / Gradle | pip / poetry | Cargo + crates.io |

-> Ch 3 Functions · Ch 9 Modules · Ch 2 Types

## C.7 Habits to unlearn

1. **Shared mutable aliasing** — default to one owner; borrow briefly.
2. **Null checks everywhere** — use `Option` and `match`.
3. **Catch-all exceptions** — use `Result` and typed errors.
4. **Clone by habit** — borrow first; clone when semantics require a copy.
5. **Ignore return values** — `Result` must be used or explicitly dropped with intent.

## C.8 AI Afterparty

Use `AI_PROMPT_INDEX.md` with this map: if you know Java or Python, ask the model to translate a snippet from either into idiomatic Rust using the right column.

# Appendix D

## AI Prompt Index

All **Afterparty** prompts from the book, numbered for reuse. Paste into any AI assistant after reading the linked chapter.

### D.1 How to use Afterparty

Full workflow (starter context, one chat per chapter, verification): Preface — Afterparty.

Quick loop (read and run examples first — see How to work through a chapter):

1. Open a **new chat** for this chapter; paste the starter context from the preface.
2. Paste **one** Afterparty prompt; read the answer and follow up until you understand it — verify code in the playground or with `cargo check` before the next prompt.
3. Push back if the answer is vague or wrong.

Find a prompt by ID (P046 -> table below), open the linked chapter for context, paste the prompt into the **same chapter chat**, and keep correcting until the model matches the compiler.

---

### D.2 Preface

| ID   | Prompt                                                                                                                                                                          |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P001 | <b>Learning plan</b> — I already program in at least one language. Based on this preface, build me a 2-week study plan using only Rust Core’s chapter list; 45 minutes per day. |
| P002 | <b>Gap check</b> — Ask me five quick questions to see if I should skip straight to Chapter 5 (lifetimes) or read Chapters 1–4 first.                                            |
| P003 | <b>Prompt practice</b> — Give me one sample Afterparty-style question about ownership; I will answer; you grade like a Rust teacher.                                            |
| P004 | <b>Motivation map</b> — List three real projects (automation, CLI, service) where Rust’s ownership model wins over GC; no hype.                                                 |
| P005 | <b>Glossary seed</b> — Define in one sentence each: ownership, borrow, trait, async, atomics — I will refine after reading Part I.                                              |

## D.3 Chapter 1 — Paradigm shift

| ID   | Prompt                                                                                                                                                                          |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P006 | <b>GC quiz</b> — Quiz me: for 8 snippets, say whether Java GC, Python refcount, or Rust drop applies; then explain Rust’s case.                                                 |
| P007 | <b>Move drill</b> — Give 5 tiny Rust snippets; I predict compile error or ok; you reveal answers with one-line fixes.                                                           |
| P008 | <b>Latency story</b> — Explain stop-the-world GC pauses vs Rust drop for a 1 kHz control loop — 200 words, no jargon pile-up.                                                   |
| P009 | <b>Java habit</b> — I wrote <code>clone()</code> everywhere in Java; how should I think about <code>clone()</code> in Rust? When is <code>.clone()</code> idiomatic vs a smell? |
| P010 | <b>Paradigm essay</b> — In 150 words: what does “fearless concurrency” mean if I only know Python’s GIL?                                                                        |
| P011 | <b>Refactor fantasy</b> — Take this Python class with shared mutable list; sketch the Rust struct + ownership split without full code.                                          |

| ID   | Prompt                                                                                                                                                                                                                    |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P012 | <b>Stack vs heap quiz</b> — For 8 Rust variable declarations, I say stack-only, heap involved, or both; you correct and draw the pointer picture for <code>String</code> and <code>Vec</code> .                           |
| P013 | <b>Scope and drop</b> — Given a nested-block snippet with <code>String</code> , trace exactly when heap memory is freed vs when stack slots disappear.                                                                    |
| P014 | <b>Stack frame drill</b> — Explain stack frames and pointer-bump allocation: I give you a 3-function call chain with locals; you trace frame push/pop and when each stack slot dies.                                      |
| P015 | <b>Heap owner trace</b> — I paste a snippet with <code>String</code> and <code>Vec</code> ; step through who owns the heap buffer after each line, when <code>drop</code> runs, and whether any heap copy happened.       |
| P016 | <b>Latency compare</b> — Compare stack pop vs heap allocate+free for a 1 kHz loop that creates a small <code>String</code> every tick vs reusing a stack <code>i32</code> counter — worst-case latency in plain language. |
| P017 | <b>Three-language map</b> — Same program: one <code>int</code> , one growable string, one function return. For Java, Python, and Rust, tabulate: stack vs heap for each, and who frees heap memory.                       |
| P018 | <b>Move at two levels</b> — When <code>let s2 = s1</code> for <code>String</code> , explain what moves on the stack vs what stays on the heap; why the heap buffer is not copied unless I call <code>.clone()</code> .    |
| P019 | <b>Borrow checker tutor</b> — I paste compiler errors; you explain the borrow conflict and show the smallest fix.                                                                                                         |
| P020 | <b>Five snippets</b> — Move vs borrow quiz: 5 code fragments, I label ok/error and why.                                                                                                                                   |
| P021 | <b>Python port</b> — This Python function mutates a list passed in; rewrite with <code>&amp;mut Vec</code> and explain aliasing rules.                                                                                    |
| P022 | <b>Java port</b> — This Java method stores the passed <code>List</code> in a field; show the Rust ownership split (return owned vs <code>Arc</code> ).                                                                    |
| P023 | <b>Slice drill</b> — Given <code>&amp;[i32]</code> , write <code>first</code> and <code>rest</code> without panicking on empty — use <code>Option</code> .                                                                |

| ID   | Prompt                                                                                             |
|------|----------------------------------------------------------------------------------------------------|
| P024 | <b>clone audit</b> — Review my 30-line Rust snippet; mark unnecessary <code>.clone()</code> calls. |

D.4 Chapter 2 — Types and expressions

| ID   | Prompt                                                                                                                                                                                                                                                          |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P025 | <b>Integer pick</b> — Quiz me: for 8 scenarios (Modbus port, byte buffer, collection index, money cents, timestamp millis, hash output, loop counter, enum discriminant), I pick <code>u8/u16/u32/u64/i32/usize</code> ; you correct and explain overflow risk. |
| P026 | <b>char vs byte</b> — Explain why ‘A’ is not the same type as <code>65u8</code> . Show one valid char literal and one invalid escape; mention UTF-8 only when discussing <code>str/String</code> .                                                              |
| P027 | <b>Tuple vs array</b> — When do I use <code>(u16, u16)</code> vs <code>[u16; 2]</code> for a coordinate pair? Give one idiomatic example of each.                                                                                                               |
| P028 | <b>&amp;str vs String API</b> — Five function signatures for logging labels. I choose <code>&amp;str</code> or <code>String</code> for each; you explain ownership and allocation cost.                                                                         |
| P029 | <b>Range quiz</b> — For half-open vs inclusive ranges, I predict output of four for loops using <code>..</code> and <code>..=</code> ; you correct with printed values.                                                                                         |
| P030 | <b>match warm-up</b> — Extend a match on HTTP status codes with <code>3xx, 4xx, 5xx</code> groupings using range patterns; I write arms; you check exhaustiveness.                                                                                              |

D.5 Chapter 3 — Functions and methods

| ID   | Prompt                                                                                                                                                                                                                                                          |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P031 | <b>Parameter audit</b> — Five function signatures for logging: <code>&amp;str</code> , <code>String</code> , <code>&amp;String</code> , <code>Cow&lt;str&gt;</code> , <code>impl AsRef&lt;str&gt;</code> . I pick one per use case; you explain ownership cost. |
| P032 | <b>Move vs borrow</b> — Snippet calls <code>process(s)</code> then uses <code>s</code> again. I explain the error and show two fixes ( <code>&amp;s</code> , <code>clone</code> ).                                                                              |
| P033 | <b>impl block</b> — Struct <code>Timer</code> with <code>start</code> , <code>elapsed</code> , <code>reset</code> . I write <code>impl</code> ; you check <code>&amp;self</code> vs <code>&amp;mut self</code> .                                                |
| P034 | <b>Associated fn</b> — When is <code>Type::new()</code> idiomatic vs <code>Default::default()</code> ? One example each.                                                                                                                                        |
| P035 | <b>Consume self</b> — Method <code>fn into_inner(self) -&gt; Vec&lt;u8&gt;</code> — why must it take <code>self</code> by value?                                                                                                                                |
| P036 | <b>Semicolon trap</b> — Three tiny functions: one returns <code>i32</code> correctly, two fail due to <code>;</code> . I fix them.                                                                                                                              |
| P037 | <b>Early return</b> — Rewrite nested <code>if</code> in a parser as early return <code>None / ?</code> style.                                                                                                                                                   |
| P038 | <b>Unit vs value</b> — Which functions should return <code>()</code> vs <code>bool</code> vs <code>Option&lt;T&gt;</code> ? Three CLI helper names, I choose.                                                                                                   |
| P039 | <b>Generic bounds</b> — Fix <code>fn max(a: T, b: T)</code> without bounds; add <code>T: Ord</code> or <code>PartialOrd</code> .                                                                                                                                |
| P040 | <b>Result signature</b> — Design <code>fn read_config(path: &amp;str) -&gt; Result&lt;Config, ...&gt;</code> ; list error variants, no body.                                                                                                                    |

P041 | **impl Iterator return** — `top_n` returns `impl Iterator` — why two iterator types in `if` arms fail. |

P042 | **mem take drain** — Buffer drains via `mem::take` — show before/after inner field. |

P043 | **where clause** — Rewrite cluttered generic bounds with `where` block. |

P044 | **Self return** — Method `into_inner(self) -> Vec<u8>` — why `Self` not concrete type? |

P045 | **const fn** — `const fn is_valid_port(p: u16) -> bool` — compile-time limits. |

P046 | **pick impl Trait** — Factory returning different iterator types — fix with `Box<dyn Iterator>`. |

P047 | **Drain method** — Design `Buffer::drain_into(&mut Vec<u8>)` signature and ownership. |

## D.6 Chapter 4 — Iterators

| ID   | Prompt                                                                                                                                                                                                           |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P048 | <b>iter vs into_iter</b> — Give 4 snippets using <code>Vec</code> ; I predict whether <code>v</code> is usable after the loop; you explain move vs borrow.                                                       |
| P049 | <b>collect turbofish</b> — Three <code>collect()</code> calls that fail without hints — I add type annotation or turbofish; you verify.                                                                          |
| P050 | <b>Adapter chain</b> — Task: parse lines, trim, keep non-empty, parse as <code>u16</code> . I sketch <code>.lines().map(...).filter(...).collect();</code> you refine.                                           |
| P051 | <b>enumerate vs index</b> — Same sum-over-evens task twice: index for vs <code>.enumerate()</code> . Compare readability and bounds-check risk.                                                                  |
| P052 | <b>Lazy vs eager</b> — Explain when <code>filter().map()</code> allocates vs when <code>.collect()</code> forces work. One example with <code>println</code> in <code>map</code> showing evaluation order.       |
| P053 | <b>for desugar quiz</b> — Four for loops ( <code>0..n</code> , <code>&amp;vec</code> , <code>vec</code> , <code>&amp;mut vec</code> ). I say <code>iter vs into_iter</code> vs <code>range</code> ; you correct. |
| P054 | <b>iter_mut drill</b> — Double every element in <code>Vec&lt;f64&gt;</code> in place; I write the loop; you review <code>*n</code> and borrows.                                                                  |
| P055 | <b>Range vs collect</b> — When is <code>for i in 0..n</code> better than collecting ( <code>0..n</code> )? Two automation examples.                                                                              |
| P056 | <b>zip pairs</b> — Zip two <code>Vec&lt;u16&gt;</code> into pairs; I write it; you handle length mismatch.                                                                                                       |
| P057 | <b>take and skip</b> — Paginate a log with <code>.skip().take()</code> on <code>.lines();</code> show one pitfall.                                                                                               |
| P058 | <b>chain iterators</b> — Concatenate two <code>&amp;[u8]</code> slices with <code>.chain()</code> without copying into one array first.                                                                          |



| ID   | Prompt                                                                                                                                                                 |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P059 | <b>find and Option</b> — Find first log line containing <code>ERROR</code> with <code>.find()</code> ; contrast <code>.filter().next()</code> .                        |
| P060 | <b>fold vs sum</b> — Max and count in one <code>.fold()</code> pass vs separate calls — when is fold worth it?                                                         |
| P061 | <b>any and all</b> — Validate ports and comment lines with <code>.all()</code> / <code>.any()</code> ; fix a double-reference mistake.                                 |
| P062 | <b>Moved Vec mistake</b> — Code uses <code>for x in v</code> then <code>v</code> again; I explain and fix; you rank fixes.                                             |
| P063 | <b>Borrow in chain</b> — <code>Vec&lt;&amp;str&gt;</code> from <code>String</code> lines then drop strings; I explain failure and fix.                                 |
| P064 | <b>Modbus-style scan</b> — Filter even <code>u16</code> registers, scale to <code>f64</code> , sum — write and check types/overflow.                                   |
| P065 | <b>Zero-cost check</b> — Does <code>iter().filter().map().sum()</code> allocate intermediate Vecs in <code>--release</code> ?                                          |
| P066 | <b>Trap sheet drill</b> — Six snippets mixing <code>for x in v</code> , <code>&amp;v</code> , <code>into_iter</code> , and collect errors; I predict ok/fail; you fix. |
| P067 | <b>&amp;&amp;i32 decoder</b> — Label closure param types in one <code>.iter().filter().map()</code> chain; show three compiling closure styles.                        |
| P068 | <b>Empty iterator policy</b> — Sum, find, all on empty <code>Vec</code> ; I state results and domain bugs; you correct.                                                |
| P069 | <b>zip truncation</b> — Zip unequal <code>Vec</code> lengths; explain silent loss and sketch a safe pairing strategy.                                                  |
| P070 | <b>PortScan impl</b> — Implement <code>Iterator</code> for ports <code>502..=505</code> ; collect and sum — show type <code>Item</code> and <code>fn next</code> .     |
| P071 | <b>Skip blanks</b> — <code>NonEmptyLines</code> iterator trims and skips empty lines; collect keys before <code>=</code> .                                             |
| P072 | <b>Infinite take</b> — Counter from 0 — why <code>.take(n)</code> before <code>.collect()</code> ? Show hang vs bounded.                                               |
| P073 | <b>IntoIterator pair</b> — Same struct implements <code>Iterator</code> and <code>IntoIterator</code> for <code>for</code> loop.                                       |

| ID   | Prompt                                                                                                           |
|------|------------------------------------------------------------------------------------------------------------------|
| P074 | <b>Stateful parser</b> — Byte buffer iterator yielding 4-byte frames — sketch <code>next()</code> state machine. |
| P075 | <b>Capstone iterator</b> — CSV fields parse col 2 as u16, filter > 0 — custom struct + consumer chain.           |

D.7 Chapter 5 — Lifetimes

| ID   | Prompt                                                                                                                                                                 |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P076 | <b>Error archaeology</b> — I paste a “lifetime may not live long enough” error; walk me through owner vs reference diagram.                                            |
| P077 | <b>Return type choice</b> — For API fn <code>title(book: &amp;Book) -&gt; ???</code> compare <code>&amp;str</code> vs <code>String</code> trade-offs for a library.    |
| P078 | <b>Struct lifetime</b> — Design <code>ConfigParser</code> holding <code>&amp;str</code> slices into input buffer — when is it sound vs use owned <code>String</code> ? |
| P079 | <b>Elision quiz</b> — Add explicit lifetimes to 4 function signatures where elision fails.                                                                             |
| P080 | <b>Fix mine</b> — I return <code>&amp;String</code> built inside function; show three idiomatic fixes ranked by simplicity.                                            |

- P081 | **static trap** — Return `&str` from `format!` — error and owned fix. |
- P082 | **two lifetimes** — `first<'a, 'b>(x: &'a str, y: &'b str) -> &'a str` — drop `y` while result lives. |
- P083 | **Config struct** — Parse `host:port` into `Config<'a>` — when must caller keep line alive? |
- P084 | **Owned refactor** — Owned `Config { host: String }` vs borrowed — three tradeoffs. |
- P085 | **T: 'a bound** — Explain `Holder<'a, T: 'a>` — failure when `T` shorter-lived. |
- P086 | **Elision fail** — Four signatures: elision works vs fails — I label each. |
- P087 | **Iterator borrow** — `Vec<&str>` from `String` lines — drop order trap. |

## D.8 Chapter 6 — Enums and pattern matching

| ID   | Prompt                                                                                                                                                                                 |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P088 | <b>Null replacement</b> — Translate 5 Java methods returning null into <code>Option</code> Rust; explain callsite changes.                                                             |
| P089 | <b>Exhaustive match</b> — I have enum with 4 variants; generate match that compiles; then add variant and show compiler error.                                                         |
| P090 | <b>Result railway</b> — Chain parse $\rightarrow$ validate $\rightarrow$ compute with <code>?</code> ; I fill blanks, you verify.                                                      |
| P091 | <b>if let vs match</b> — When is <code>if let</code> clearer than <code>match</code> ? Give 3 contrasting snippets.                                                                    |
| P092 | <b>State machine</b> — Model TCP connection states as enum; methods connect, send, close with illegal transition errors.                                                               |
| P093 | <b>Python Union</b> — This Python function accepts <code>int   str</code> ; design Rust enum + match without dynamic typing.                                                           |
| P094 | <b>unwrap audit</b> — Paste 20 lines with 4 <code>unwrap()</code> calls; I mark panic risk; you rewrite with <code>match</code> or <code>?</code> .                                    |
| P095 | <b>Combinator chain</b> — Parse port <code>Option&lt;u16&gt;</code> , double if <code>Some</code> , default 502 — I write <code>map/unwrap_or</code> ; you add <code>and_then</code> . |
| P096 | <b>let-else port</b> — Rewrite nested <code>match</code> on <code>parse()</code> into <code>let Ok(x) = ... else { ... }</code> ; preserve behavior.                                   |
| P097 | <b>Err arm missing</b> — Show <code>match</code> on <code>Result</code> without <code>Err</code> arm; I quote error and fix; add boundary <code>eprintln!</code> pattern.              |
| P098 | <b>unwrap vs ?</b> — Same parser: <code>unwrap</code> vs <code>fn</code> $\rightarrow$ <code>Result</code> with <code>?</code> ; compare panic risk and signature honesty.             |
| P099 | <b>Wildcard footgun</b> — Explain why <code>_</code> on your own enum hides new variants; show explicit arms vs <code>_</code> refactor story.                                         |
| P100 | <b>Partial move</b> — enum with <code>String</code> field: <code>match</code> moves field; I fix with <code>ref</code> or <code>match &amp;</code> ; show error text.                  |
| P101 | <b>Match on ref</b> — Owned <code>Status</code> : <code>match s</code> vs <code>match &amp;s</code> ; predict move errors; diagram ownership.                                          |

| ID   | Prompt                                                                                                          |
|------|-----------------------------------------------------------------------------------------------------------------|
| P102 | <b>Guard drill</b> — Classify i32 with guards; I write arms; you check exhaustiveness.                          |
| P103 | <b>Opcode table</b> — Design enum for 3 frame types + match opcodes; add fourth type as compile-break exercise. |
| P104 | <b>ReadOutcome extend</b> — Add Disconnected to ReadOutcome; list match sites the compiler forces to update.    |
| P105 | <b>Config sentinel</b> — Rewrite port() -> i32 returning -1 to Option<u16> + match in main.                     |
| P106 | <b>Checklist drill</b> — Match 6 Chapter 6 compiler errors to snippets; I name fix (match arm, ref, ?).         |
| P107 | <b>Java enum map</b> — Java enum State with method -> Rust enum + match + impl; contrast nullability.           |

P108 | **Slice split** — Parse HTTP request line with slice patterns — handle single-token input. |

P109 | **matches refactor** — Replace 6-arm bool match with matches! on Mode enum. |

P110 | **if let chain** — Parse host:port — chain rejects host:port:extra. |

P111 | **@ binding quiz** — Port range arms with @ — label which values hit which arm. |

P112 | **Exhaustive slice** — [a, b] on len 1 or 3 — predict \_ arm vs bug; suggest .. rest. |

P113 | **matches vs match** — When keep full match instead of matches!? Example returning String. |

P114 | **\*\*Guard + @\*\*** — Match port n @ 1024..=65535 with guard n % 2 == 0. |

P115 | **Capstone parse** — Frame header slice pattern from byte slice — sketch match arms only. |

## D.9 Chapter 7 — Structs, traits, generics

| ID   | Prompt                                                                                                 |
|------|--------------------------------------------------------------------------------------------------------|
| P116 | <b>dyn vs impl</b> — Quiz: 4 scenarios — pick dyn Trait or impl Trait and justify.                     |
| P117 | <b>Default trait methods</b> — Add default <code>summary()</code> on trait; override in one type only. |
| P118 | <b>Generic bounds</b> — Fix compiler error: T needs <code>Display + Clone</code> ; minimal bound set.  |

P119 | **Item type quiz** — Change `PortScan`'s type `Item` from `u16` to `(u16, bool)` — list broken call sites. |

P120 | **Summarizable design** — Summarizable with type `Output = String` for three sensor structs; one report fn. |

P121 | **Associated vs generic** — trait `Get<T>` vs trait `Get { type Value; }` — when is each painful? |

P122 | **Supertrait bounds** — Exportable: `Display + Debug` with default export — impl for `Port(u16)`. |

P123 | **UFCS fix** — Type implements A and B, both define `name()` — ambiguous call and UFCS fix. |

## D.10 Chapter 8 — Errors and testing

| ID   | Prompt                                                                                                                                       |
|------|----------------------------------------------------------------------------------------------------------------------------------------------|
| P124 | <b>? chain</b> — Refactor nested match on <code>Results</code> to <code>? style</code> ; explain each change.                                |
| P125 | <b>Error enum design</b> — Design <code>AppError</code> for CLI that reads config + talks serial; variants + <code>From</code> impls sketch. |
| P126 | <b>panic audit</b> — Mark which of 10 <code>unwrap()</code> calls should stay vs become <code>Result</code> .                                |
| P127 | <b>Test generation</b> — Write table-driven tests for <code>parse_port</code> including edge ports.                                          |
| P128 | <b>anyhow vs thiserror</b> — When would I pick each for a binary vs library crate?                                                           |

## D.11 Chapter 9 — Modules, paths, and crates

| ID   | Prompt                                                                                                                                                               |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P129 | <b>File tree</b> — Design module tree for a CLI that reads config and runs commands. Directories + mod lines only, no bodies.                                        |
| P130 | <b>Path quiz</b> — From <code>crate::service::worker::run</code> , how do I reach <code>crate::config::load</code> ? Show use and fully qualified call.              |
| P131 | <b>lib vs bin</b> — What belongs in <code>main.rs</code> vs <code>lib.rs</code> for a tool with 500 lines of logic?                                                  |
| P132 | <b>pub audit</b> — List items that should be <code>pub</code> vs <code>private</code> in a library crate exposing <code>Client::connect</code> .                     |
| P133 | <b>pub(crate)</b> — When is <code>pub(crate)</code> better than <code>pub</code> for test helpers?                                                                   |
| P134 | <b>Re-export</b> — Sketch <code>pub use</code> so users see <code>my_crate::Error</code> but you wrap <code>thiserror</code> internally.                             |
| P135 | <b>Workspace split</b> — Two crates: core library + cli binary. Write <code>Cargo.toml</code> dependency path only.                                                  |
| P136 | <b>Integration test</b> — Where does <code>tests/smoke.rs</code> live and how does it use the library?                                                               |
| P137 | <b>Orphan fix</b> — I want <code>Display</code> on <code>Vec&lt;u8&gt;</code> — show newtype wrapper module layout.                                                  |
| P138 | <b>Split monolith</b> — Given one <code>main.rs</code> with <code>config</code> + <code>parser</code> + <code>runner</code> , name three modules and what each owns. |
| P139 | <b>cfg test</b> — Explain why <code>mod tests</code> uses <code>#[cfg(test)]</code> and use <code>super::*</code> .                                                  |
| P140 | <b>Capstone</b> — Generate <code>src/</code> tree for <code>sensor_core</code> library + <code>sensor_cli</code> binary in one workspace; I implement.               |

P141 | **Feature flag** — Add serial feature gating `mod serial_io` — `Cargo.toml` and one `#[cfg].` |

P142 | **cfg vs cfg!** — `#[cfg(debug_assertions)]` vs `if cfg!(debug_assertions)` — release binary diff. |

P143 | **Optional dep** — Optional `tokio` behind feature `async` — show `dep:tokio` line. |

P144 | **Platform gate** — `#[cfg(target_os = "linux")]` module for

device path — sketch. |

P145 | **Integration layout** — Tree for tests/load\_config.rs — what API is invisible to test? |

P146 | **pub use prelude** — Re-export so users call my\_crate::connect not internal::connect. |

P147 | **doc hidden** — When mark helper `#[doc(hidden)]` on public re-export surface? |

P148 | **Capstone crate** — gateway crate: serial feature, integration test, /// on parse fn — tree only. |

## D.12 Chapter 10 — Smart pointers and interior mutability

| ID   | Prompt                                                                                                                                                          |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P149 | <b>Box why</b> — When is <code>Box&lt;T&gt;</code> better than <code>Vec&lt;T&gt;</code> on the stack? Two cases.                                               |
| P150 | <b>Rc cycle</b> — Explain why <code>Rc</code> cycles leak; contrast with Python reference cycles and <code>Weak</code> fix.                                     |
| P151 | <b>Arc Mutex sketch</b> — Diagram thread-safe cache with <code>Arc&lt;Mutex&gt;</code> — no full code.                                                          |
| P152 | <b>Java heap map</b> — Map Java “everything is reference” to Rust ownership — when <code>Arc</code> , when plain <code>&amp;</code> , when neither.             |
| P153 | <b>RefCell trap</b> — Show double <code>borrow_mut</code> panic; fix with scoped borrows.                                                                       |
| P154 | <b>Arc vs Rc</b> — Thread spawn with <code>Rc</code> — show error; fix with <code>Arc</code> ; explain atomic count overhead.                                   |
| P155 | <b>Deref coercion</b> — Why does <code>fn takes_str(s: &amp;str) accept &amp;String, &amp;Box&lt;String&gt;, and &amp;Rc&lt;String&gt;?</code> Trace steps.     |
| P156 | <b>Pick pointer</b> — Five scenarios (AST node, thread cache, GUI graph, config string, plugin list) — I pick <code>Box/Rc/Arc/RefCell/Weak</code> ; you grade. |
| P157 | <b>Recursive list</b> — Draw memory for <code>Cons(1, Box::new(Cons(2, Nil)))</code> — stack vs heap boxes.                                                     |

| ID   | Prompt                                                                                                                                                                               |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P158 | <b>Move out of Box</b> — What happens after <code>let s = *box_string</code> ? When idiomatic vs keeping the Box?                                                                    |
| P159 | <b>Trait object box</b> — Three plugin types implement <code>Plugin</code> — sketch <code>Vec&lt;Box&lt;dyn Plugin&gt;&gt;</code> ; why not <code>Vec&lt;Plugin&gt;</code> ?         |
| P160 | <b>Handle vs deep clone</b> — Audit snippet with <code>Rc&lt;String&gt;</code> and both <code>Rc::clone</code> and <code>(*rc).clone()</code> — label cost of each.                  |
| P161 | <b>Weak upgrade</b> — Parent/child with <code>Rc</code> parent and <code>Weak</code> child back-ref — I sketch types; you explain cycle break.                                       |
| P162 | <b>strong_count debug</b> — <code>strong_count</code> stays at 2 after I thought I dropped all refs — list 5 places handles hide.                                                    |
| P163 | <b>Immutable then mut</b> — Hold <code>let r = cell.borrow()</code> and call <code>borrow_mut</code> — explain panic; fix with nested block.                                         |
| P164 | <b>Cell vs RefCell</b> — Counter <code>u32</code> vs <code>Vec</code> cache — I pick <code>Cell</code> or <code>RefCell</code> each; you correct.                                    |
| P165 | <b>Rc RefCell graph</b> — Two nodes share <code>Rc&lt;RefCell&lt;Node&gt;&gt;</code> — one updates, one reads — sketch borrow rules on one thread.                                   |
| P166 | <b>Compile vs runtime</b> — Same overlapping-mut pattern: compile error with <code>&amp;mut</code> and runtime panic with <code>RefCell</code> side by side.                         |
| P167 | <b>Drop order</b> — Three <code>Drop</code> structs in one function — I predict print order; you confirm reverse declaration rule.                                                   |
| P168 | <b>Rc last handle</b> — Two <code>Rc</code> clones dropped at different times — when does inner <code>Drop</code> run? Step through with <code>println</code> in <code>Drop</code> . |
| P169 | <b>Drop panic</b> — Explain double-panic abort if <code>Drop</code> panics during unwind — link to Ch8.                                                                              |
| P170 | <b>Refactor to smart ptr</b> — I paste struct with <code>Box</code> , <code>Rc</code> , or raw tree — you suggest minimal smart pointer fix and justify.                             |
| P171 | <b>Leak hunt</b> — Sketch <code>Rc</code> cycle in observer pattern; refactor one edge to <code>Weak</code> and explain count after each drop.                                       |



## D.13 Chapter 11 — Collections

| ID   | Prompt                                                                                                                                             |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| P172 | <b>Loop port</b> — Rewrite C-style indexed loop as iterator chain; preserve behavior.                                                              |
| P173 | <b>HashMap merge</b> — Two maps of scores — merge by max per key; iterator + entry style.                                                          |
| P174 | <b>entry drill</b> — Word frequency from <code>Vec&lt;&amp;str&gt;</code> using only <code>.entry</code> — no double lookup.                       |
| P175 | <b>collect types</b> — Three <code>collect()</code> calls that need type hints — fix with <code>turbofish</code> .                                 |
| P176 | <b>Windows</b> — Detect rising edges in <code>Vec&lt;f64&gt;</code> with <code>.windows(2)</code> ; extend to <code>.windows(3)</code> .           |
| P177 | <b>Performance myth</b> — Do Rust iterators optimize to loops? When might they not?                                                                |
| P178 | <b>Pick collection</b> — Five tasks (dedup, range scan, FIFO, index by id, min-key) — I pick collection each.                                      |
| P179 | <b>Set ops</b> — Tags on two records — union, intersection, difference with <code>HashSet</code> .                                                 |
| P180 | <b>Hash vs BTree</b> — Same 10k insert + range scan — when <code>HashMap</code> wins vs <code>BTreeMap</code> .                                    |
| P181 | <b>Queue anti-pattern</b> — Review <code>v.remove(0)</code> queue loop — cost and <code>VecDeque</code> fix.                                       |
| P182 | <b>get vs index</b> — Four access patterns — I pick <code>[i]</code> vs <code>.get(i)</code> vs <code>.get_mut</code> vs <code>if let</code> Some. |
| P183 | <b>sort dedup</b> — Dedup <code>[3,1,4,1,5]</code> wrong vs right — sort + dedup pipeline.                                                         |
| P184 | <b>retain vs filter</b> — Remove evens in-place vs new <code>Vec</code> — compare <code>retain</code> and <code>filter().collect()</code> .        |
| P185 | <b>Borrow push trap</b> — Explain <code>let r = &amp;v[0]; v.push(1)</code> error; fix with scope.                                                 |
| P186 | <b>or_insert_with</b> — Lazy cache: expensive <code>Vec</code> built once per key — sketch with <code>or_insert_with</code> .                      |
| P187 | <b>insert overwrite</b> — Track old value on port remap using <code>insert</code> return.                                                          |
| P188 | <b>Stable dedup</b> — Unique <code>String</code> lines preserving first-seen order — no <code>HashSet</code> -only collect.                        |

| ID   | Prompt                                                                                                                    |
|------|---------------------------------------------------------------------------------------------------------------------------|
| P189 | <b>chunks vs windows</b> — Parse byte stream into 4-byte frames — <code>chunks(4)</code> vs <code>windows(4)</code> when? |
| P190 | <b>Duplicate keys collect</b> — collect to <code>HashMap</code> from duplicate-key pairs — predict final map.             |
| P191 | <b>Capacity hint</b> — Read 1M lines into <code>Vec</code> — when <code>with_capacity</code> matters; sizing rule.        |
| P192 | <b>Capstone store</b> — Register id -> reading + range scan — pick map type, list three API methods.                      |

## D.14 Chapter 12 — Closures and the Fn traits

| ID   | Prompt                                                                                                                                     |
|------|--------------------------------------------------------------------------------------------------------------------------------------------|
| P193 | <b>Fn quiz</b> — Four closures: I label each <code>Fn</code> / <code>FnMut</code> / <code>FnOnce</code> ; you correct and explain capture. |
| P194 | <b>move drill</b> — Thread spawn snippet missing <code>move</code> — show compile error and fix.                                           |
| P195 | <b>Iterator chain</b> — <code>.filter</code> closure that uses <code>&amp;config</code> — why <code>Fn</code> not <code>FnMut</code> ?     |
| P196 | <b>fn vs closure</b> — When can you pass <code>fn()</code> vs <code>impl Fn()</code> to the same helper?                                   |
| P197 | <b>Return closure</b> — Write <code>make_multiplier(f: f64) -&gt; impl Fn(f64) -&gt; f64</code> and explain <code>move</code> .            |
| P198 | <b>Double reference</b> — Fix <code>.iter().filter( x  ...)</code> type error on <code>Vec&lt;String&gt;</code> .                          |
| P199 | <b>sort_by</b> — Sort <code>Vec&lt;(String, u32)&gt;</code> by count descending with <code>sort_by</code> closure.                         |
| P200 | <b>for_each vs for</b> — Same side-effect loop twice: <code>for</code> vs <code>.for_each</code> — style tradeoffs.                        |
| P201 | <b>Box dyn Fn</b> — Store heterogeneous callbacks in a <code>Vec</code> — sketch trait object version.                                     |
| P202 | <b>Capstone</b> — Pipeline: lines, filter, parse <code>u16</code> , sum — all with closures; I write; you review <code>Fn</code> bounds.   |

| ID   | Prompt                                                                                                           |
|------|------------------------------------------------------------------------------------------------------------------|
| P203 | <b>RefCell bump</b> — Closure mutates RefCell<u32> counter — which Fn trait and why?                             |
| P204 | <b>Callback registry</b> — Three log filters in Vec<Box<dyn Fn(&str) -> bool>> — I add wrong signature; you fix. |
| P205 | <b>Loop move trap</b> — Building Vec<Box<dyn Fn()>> in for over String — show move error and clone fix.          |
| P206 | <b>sort_by_key</b> — Same sort with sort_by_key — when is key extraction cleaner?                                |
| P207 | <b>retain valid</b> — Drop invalid SensorReading rows with .retain — FnMut bound.                                |
| P208 | <b>Fn bound strict</b> — Helper takes impl Fn() but closure mutates — fix signature.                             |
| P209 | <b>Thread move</b> — spawn closure borrows String — show error without move and fix.                             |
| P210 | <b>Send Box Fn</b> — When does Box<dyn Fn() + Send> matter for thread callbacks?                                 |
| P211 | <b>Callback capstone</b> — Registry + sort pipeline + thread handoff — review Fn bounds on my code.              |
| P212 | <b>apply_twice generic</b> — impl Fn vs Box<dyn Fn> for helper called twice — cost comparison.                   |
| P213 | <b>Async move note</b> — One async move block capturing String — same rules as sync closure.                     |
| P214 | <b>Dedup_by closure</b> — Dedup adjacent SensorReading by id with dedup_by — FnMut.                              |
| P215 | <b>Filter config</b> — .filter reading &settings — prove closure is Fn not FnMut.                                |
| P216 | <b>FnOnce consume</b> — Closure that moves String out on first call — storage implications.                      |
| P217 | <b>Heterogeneous vec</b> — Why Vec<Box<dyn Fn(i32)>> cannot mix different capture sizes — one paragraph.         |
| P218 | <b>Iterator sort chain</b> — .filter().map().collect() then sort_by — label each closure's Fn trait.             |

| ID   | Prompt                                                                                                            |
|------|-------------------------------------------------------------------------------------------------------------------|
| P219 | <b>Scope vs move</b> — Compare <code>thread::scope</code> borrow closure vs <code>spawn(move)</code> — when each. |
| P220 | <b>Capstone registry</b> — Full callback registry filtering logs by severity — I write; you audit traits.         |

## D.15 Chapter 13 — Standard traits and conversions

| ID   | Prompt                                                                                                                                                                    |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P221 | <b>Debug vs Display</b> — When derive <code>Debug</code> only vs implement <code>Display</code> for a CLI status line?                                                    |
| P222 | <b>Redacted Debug</b> — Struct with <code>api_key: String</code> — sketch manual <code>Debug</code> with <code>[REDACTED]</code> .                                        |
| P223 | <b>From chain</b> — <code>String -&gt; MyLabel</code> via <code>From</code> ; add <code>From&lt;&amp;str&gt;</code> without duplicating logic.                            |
| P224 | <b>TryFrom port</b> — Port validation with custom enum error <code>OutOfRange</code> .                                                                                    |
| P225 | <b>parse vs TryFrom</b> — When <code>s.parse::&lt;u16&gt;()</code> vs <code>u16::try_from(x)</code> vs custom <code>FromStr</code> ?                                      |
| P226 | <b>AsRef drill</b> — Rewrite three functions taking <code>&amp;String</code> to <code>impl AsRef&lt;str&gt;</code> .                                                      |
| P227 | <b>Cow API</b> — Normalize slug: accept <code>Cow&lt;str&gt;</code> , return borrowed if valid else owned.                                                                |
| P228 | <b>Derive set quiz</b> — Map key, log line, sortable row, error enum — I list derives each needs.                                                                         |
| P229 | <b>Display impl</b> — Implement <code>Display</code> for <code>Port(u16)</code> showing <code>Port(8080)</code> .                                                         |
| P230 | <b>Mini crate API</b> — Public <code>HostPort</code> with <code>Display</code> , <code>TryFrom&lt;&amp;str&gt;</code> for <code>host:port</code> — list impl blocks only. |
| P231 | <b>Pretty debug</b> — Same struct with <code>{:#?}</code> vs <code>{:?}",</code> — when does pretty-print help in tests?                                                  |

| ID   | Prompt                                                                                          |
|------|-------------------------------------------------------------------------------------------------|
| P232 | <b>Default enum</b> — Three-variant mode enum — derive Default with #[default] on Auto.         |
| P233 | <b>Eq on floats</b> — Struct with f64 field — show Eq derive failure; three fixes.              |
| P234 | <b>HashMap key</b> — Why does UserId(String) need Eq + Hash for HashMap keys?                   |
| P235 | <b>FromStr type</b> — Parse host:port into struct — sketch FromStr with split.                  |
| P236 | <b>Silent cast trap</b> — Show 70000i32 as u16 vs TryFrom — predict values.                     |
| P237 | <b>From in errors</b> — Wire ParseIntError into AppError with From — list impl only.            |
| P238 | <b>AsRef bytes</b> — Log wire payload — impl AsRef<[u8]> accepts Vec, slice, array.             |
| P239 | <b>Borrow lookup</b> — Explain HashMap<String, V>.get(&str) — role of Borrow<str>.              |
| P240 | <b>into_owned</b> — When caller needs String after Cow helper — where into_owned()?             |
| P241 | <b>Newtype Display</b> — Wrap Vec<u8> as HexBytes — implement Display without orphan violation. |
| P242 | <b>Derive audit</b> — Config with secrets + TOML — list safe vs unsafe derives.                 |
| P243 | <b>Capstone traits</b> — Design RateLimit public API: parsing, display, equality — traits only. |

## D.16 Chapter 14 — Multithreading

| ID   | Prompt                                                                                                         |
|------|----------------------------------------------------------------------------------------------------------------|
| P244 | <b>Race quiz</b> — Which snippets are data races in C++ but rejected by Rust compiler?                         |
| P245 | <b>Channel design</b> — Worker pool with mpsc: I describe throughput; you sketch thread count + channel shape. |
| P246 | <b>Mutex vs RwLock</b> — Read-heavy sensor cache — pick primitive and why.                                     |
| P247 | <b>Send fix</b> — I try to move Rc into thread; show fix with Arc.                                             |

| ID   | Prompt                                                                     |
|------|----------------------------------------------------------------------------|
| P248 | <b>Join panic</b> — What happens if spawned thread panics? Handle in main. |

P249 | **RwLock cache** — Sketch `Arc<RwLock<HashMap>>` read-heavy cache; when write starves readers? |

P250 | **Mutex vs RwLock** — Same cache with 50% writes — pick primitive and justify. |

P251 | **OnceLock init** — `get_or_init` behaviour when called twice — same value guarantee. |

P252 | **Scope borrow** — Parallel sum over chunks with `thread::scope` — why plain spawn fails. |

P253 | **Poison recovery** — Writer panics holding `RwLock` — poisoned `read()` and recovery. |

P254 | **Capstone sync** — Lazy config (`OnceLock`), shared cache (`RwLock`), scoped workers — types only. |

## D.17 Chapter 15 — Atomics

| ID   | Prompt                                                                                                                                    |
|------|-------------------------------------------------------------------------------------------------------------------------------------------|
| P255 | <b>Ordering quiz</b> — For shutdown flag + published config pointer, which orderings? Justify briefly.                                    |
| P256 | <b>Counter port</b> — Replace <code>Mutex</code> counter with <code>AtomicUsize</code> ; discuss lost updates with <code>Relaxed</code> . |
| P257 | <b>ABA problem</b> — Explain ABA in 80 words for <code>compare_exchange</code> — no full queue impl.                                      |
| P258 | <b>When not</b> — Three cases atomics are the wrong tool; prefer channels or <code>Mutex</code> .                                         |
| P259 | <b>Fence intuition</b> — Draw happens-before arrow diagram for <code>Release store</code> + <code>Acquire load</code> .                   |

## D.18 Chapter 16 — Async and Tokio

| ID   | Prompt                                                                                  |
|------|-----------------------------------------------------------------------------------------|
| P260 | <b>Future diagram</b> — Draw state machine for async fn with two await points.          |
| P261 | <b>Tokio scaffold</b> — Generate minimal Tcp echo server skeleton; I fill body.         |
| P262 | <b>select! scenario</b> — Cancel slow request when fast path returns — outline select!. |
| P263 | <b>async vs thread</b> — 1000 Modbus polls — argue async vs thread pool for latency.    |
| P264 | <b>blocking fix</b> — Identify blocking calls in async snippet; suggest spawn_blocking. |
| P265 | <b>Python asyncio</b> — Map asyncio gather to Tokio join — API comparison table.        |

## D.19 Chapter 17 — Metaprogramming

| ID   | Prompt                                                                                        |
|------|-----------------------------------------------------------------------------------------------|
| P266 | <b>Macro vs fn</b> — Rewrite macro as generic fn if possible; when impossible, say why.       |
| P267 | <b>derive need</b> — List derives I want for config struct loaded from TOML — justify each.   |
| P268 | <b>Hygiene</b> — Explain macro hygiene in 60 words with \$crate mention.                      |
| P269 | <b>Debug expand</b> — Walk me through cargo expand on derive Debug output (conceptual).       |
| P270 | <b>DSL sketch</b> — Design tiny command! macro for CLI subcommands — tokens only.             |
| P271 | <b>Expansion order</b> — List compiler phases from tokens to LLVM; where do macros run?       |
| P272 | <b>Tokens vs types</b> — Why can a macro compile but expanded code fail? One example.         |
| P273 | <b>Follow-set why</b> — Explain in 80 words why \$a:expr = \$b:expr is forbidden in matchers. |
| P274 | <b>Scope honesty</b> — List 5 metaprogramming topics Ch17 skips and where to learn each.      |
| P275 | <b>Trace checklist</b> — Give 6 reasons macro code is hard to trace and one mitigation each.  |

| ID   | Prompt                                                                                                                     |
|------|----------------------------------------------------------------------------------------------------------------------------|
| P276 | <b>Fragment picker</b> — I describe a DSL shape; you pick <code>expr/ident/tt</code> for each slot.                        |
| P277 | <b>Expr equals fix</b> — Fix <code>set_reg!(\$addr = \$val)</code> matcher; show two valid surface syntaxes.               |
| P278 | <b>Trailing comma</b> — Explain <code>\$(x),*</code> vs <code>\$(x),+</code> on empty input; show failing and fixed macro. |
| P279 | <b>Register DSL</b> — Extend <code>register_map!</code> with a third register; explain <code>stringify!</code> arm.        |
| P280 | <b>Clone expand</b> — Show conceptual expanded <code>impl Clone</code> for struct with two <code>i32</code> fields.        |
| P281 | <b>Enum vs struct</b> — How does derived <code>PartialEq</code> differ for enum vs struct? Sketch match arms.              |
| P282 | <b>Field bound failure</b> — Struct with <code>Mutex&lt;i32&gt; field + #[derive(Clone)]</code> — quote error and fix.     |
| P283 | <b>Redacted Debug</b> — When hand-write <code>Debug</code> instead of <code>derive</code> on command enum with secrets.    |
| P284 | <b>Copy vs Clone quiz</b> — Classify 8 types: <code>Copy</code> , <code>Clone</code> only, or neither; justify.            |
| P285 | <b>Hot-loop clone audit</b> — Audit poll loop with <code>.clone()</code> each tick; suggest <code>move/Arc/borrow</code> . |
| P286 | <b>Arc vs derive Clone</b> — Explain cheap <code>Arc clone</code> vs deep <code>String clone</code> with one snippet.      |
| P287 | <b>Serde rename</b> — Field <code>poll_ms</code> in JSON as <code>pollIntervalMs</code> — show attr; trap on refactor.     |
| P288 | <b>thiserror vs manual</b> — Same error enum: count lines <code>derive</code> vs hand-written (Ch8 style).                 |
| P289 | <b>Float Eq trap</b> — Show <code>#[derive(Eq)]</code> on <code>f64</code> field failure; two fixes from Ch7.              |
| P290 | <b>cargo expand walkthrough</b> — Step-by-step: install, run, read output for one <code>derive</code> .                    |
| P291 | <b>In expansion of</b> — Decode a 3-note compiler error chain from nested macro + <code>derive</code> .                    |
| P292 | <b>tt vs expr escape</b> — When switch matcher from <code>expr</code> to <code>tt</code> ; tradeoffs in 80 words.          |



| ID   | Prompt                                                                                                                                                                                                 |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P293 | <b>Three-layer trace</b> — Derive inside attribute inside <code>macro_rules</code> — debug layer by layer.                                                                                             |
| P294 | <b>Port to const fn</b> — Replace tiny numeric macro with <code>const fn</code> ; when macro still needed?                                                                                             |
| P295 | <b>env vs var</b> — Compare <code>env!</code> , <code>option_env!</code> , <code>std::env::var</code> — table with one use case each.                                                                  |
| P296 | <b>include_str config</b> — Embed default TOML with <code>include_str!</code> ; deserialize at startup sketch.                                                                                         |
| P297 | <b>Macro vs fn audit</b> — Mark 6 snippets: should be macro, derive, or plain fn — justify.                                                                                                            |
| P298 | <b>When not proc macro</b> — Three scenarios where <code>proc macro</code> is overkill; alternative each.                                                                                              |
| P299 | <b>Minimal derive set</b> — Gateway config + error + CLI: smallest derive list that still ships.                                                                                                       |
| P300 | <b>Trap quiz</b> — Mark 8 snippets: empty <code>+</code> , double brace, Default enum, Arc Clone, <code>env!</code> , duplicate <code>impl</code> , <code>cfg</code> macro, <code>serde</code> rename. |
| P301 | <b>Duplicate register DSL</b> — Design compile-time error for duplicate keys in <code>register_map!</code>                                                                                             |
| P302 | <b>Serde refactor test</b> — Integration test plan after renaming TOML field with <code>serde</code> attrs.                                                                                            |
| P303 | <b>Modbus table macro</b> — Spec register table macro generating lookup + <code>const</code> max address.                                                                                              |
| P304 | <b>Derive soup review</b> — I paste 40-line struct with 12 derives; trim to minimal set with reasons.                                                                                                  |
| P305 | <b>For after expr</b> — Why is <code>\$e:expr</code> for <code>\$i:ident</code> in <code>\$r:expr</code> illegal? Show legal <code>\$p:pat</code> in <code>\$r:expr</code> <code>foreach</code> macro. |
| P306 | <b>Double-brace fix</b> — Fix <code>poll_twice!</code> macro that mixes <code>let</code> and <code>for</code> for use in <code>let x = poll_twice!()</code> .                                          |

| ID   | Prompt                                                                                                                                                                                                                                                                                                                                  |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P379 | <b>Derive vs decorator</b> — I come from Python/Java. Explain why <code>#[derive(Debug)]</code> is not a decorator or annotation that runs at call time — what actually happens at compile time?                                                                                                                                        |
| P380 | <b>Three kinds quiz</b> — Classify 8 snippets: <code>derive proc macro</code> , <code>attribute proc macro</code> , <code>function-like macro</code> , <code>compiler attribute</code> ( <code>#[inline]</code> , <code>#[cfg]</code> ), <code>field meta</code> parsed inside a <code>derive</code> ( <code>#[serde(rename)]</code> ). |
| P381 | <b>tokio::main expand</b> — Sketch the conceptual expansion of <code>#[tokio::main] async fn main() { ... }</code> . Why is this an <code>attribute proc macro</code> , not <code>#[derive]</code> ?                                                                                                                                    |
| P382 | <b>Custom attribute inputs</b> — For <code>#[my_attr(some = "config")] fn poll() { ... }</code> , what two token streams does the <code>proc macro</code> receive? Give two things the macro might emit.                                                                                                                                |
| P383 | <b>Field attr vs item attr</b> — Contrast <code>#[serde(rename = "pollIntervalMs")]</code> on a <code>struct field</code> vs <code>#[tracing::instrument]</code> on <code>fn poll</code> — same <code>proc-macro</code> kind or not?                                                                                                    |
| P384 | <b>Custom attribute when</b> — Three scenarios ( <code>poll-loop tracing</code> , <code>type-level JSON mapping</code> , <code>wrapping main with a runtime</code> ). Pick: custom attribute <code>proc macro</code> , <code>derive</code> , or plain helper <code>fn</code> — justify each.                                            |
| P385 | <b>Runtime myth</b> — Does <code>#[tracing::instrument]</code> or <code>#[test]</code> run every time I call the function? Explain what runs at compile time vs run time.                                                                                                                                                               |

## D.20 Chapter 18 — Unsafe

| ID   | Prompt                                                                                                                              |
|------|-------------------------------------------------------------------------------------------------------------------------------------|
| P307 | <b>Invariant list</b> — For raw pointer to buffer + length, list 5 invariants <code>safe wrapper</code> must enforce.               |
| P308 | <b>Soundness</b> — Explain “safe Rust can’t cause UB” vs unsafe — one paragraph; include unsound <code>safe wrapper</code> example. |

| ID   | Prompt                                                                                                                                                               |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P309 | <b>FFI checklist</b> — Checklist for calling C library from Rust binary.                                                                                             |
| P310 | <b>Miri</b> — What is Miri and when run it relative to unsafe changes?                                                                                               |
| P311 | <b>Avoid</b> — Review use case: speed up JSON — unsafe vs simd crate vs algorithm.                                                                                   |
| P312 | <b>Java JNI</b> — Compare JNI pitfalls to Rust FFI ownership rules.                                                                                                  |
| P313 | <b>Scope honesty</b> — List 6 topics Ch18 skips and where to learn each (nomicon, Miri, Pin, ...).                                                                   |
| P314 | <b>Aim table</b> — Fill: why Vec needs unsafe internally while push stays safe for callers.                                                                          |
| P315 | <b>Promise diagram</b> — Draw safe API -> unsafe block -> invariants -> caller cannot UB; label soundness.                                                           |
| P316 | <b>*const vs &amp;T quiz</b> — Give 5 snippets: legal ref, needs unsafe block, compile error; I classify each.                                                       |
| P317 | <b>from_raw_parts design</b> — Design fn <code>view_frame(ptr, len) -&gt; Result&lt;&amp;[u8], Error&gt;</code> without <code>&amp;'static</code> ; list invariants. |
| P318 | <b>Dangling audit</b> — Show stack pointer used after drop; I explain UB; you show Miri-style symptom.                                                               |
| P319 | <b>Modbus buffer</b> — Register table as <code>&amp;[u8]</code> vs <code>from_raw_parts</code> — when is each idiomatic in a gateway?                                |
| P320 | <b>Hex preview port</b> — Port Level 2 <code>as_hex_preview</code> to return <code>Result</code> on empty buffer; no <code>unwrap</code> .                           |
| P321 | <b>set_len contract</b> — Document pre/post conditions for <code>set_len_unchecked</code> ; what breaks <code>as_slice</code> if violated?                           |
| P322 | <b>Send proof</b> — I claim <code>Rc&lt;*mut u8&gt;</code> is <code>Send</code> ; you disprove with compiler error quote.                                            |
| P323 | <b>SerialHandle Sync</b> — When would <code>SerialHandle</code> need <code>unsafe impl Sync</code> vs <code>Arc&lt;Mutex&lt;...&gt;&gt;</code> ? Two sentences each. |
| P324 | <b>Ch14 port</b> — Rewrite Level 4 <code>spawn</code> example using only safe types — when is it impossible?                                                         |

| ID   | Prompt                                                                                                                                                                  |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P325 | <b>Proc-macro boundary</b> — Why do <code>serde/tokio crates</code> use <code>unsafe impl</code> you don't write? Link Ch17.                                            |
| P326 | <b>CString trap</b> — Show <code>into_raw</code> forgotten <code>from_raw</code> leak; fix with RAIL pattern sketch.                                                    |
| P327 | <b>Vendor SDK</b> — Diagram ownership: Rust owns <code>config</code> , C owns <code>connection</code> , <code>callback</code> pointer — boxes and arrows only.          |
| P328 | <b>serialport hide</b> — Where does <code>unsafe</code> live in a typical serial crate vs my application code?                                                          |
| P329 | <b>CRC decision</b> — C <code>crc16</code> vs Rust <code>crc</code> crate vs hand-rolled — decision tree for production gateway.                                        |
| P330 | <b>Trap quiz</b> — Mark 6 snippets: <code>safe</code> , UB, ordering bug, needs <code>Miri</code> , needs <code>Mutex</code> , unsound <code>safe API</code> .          |
| P331 | <b>Review rubric</b> — 10-point code-review checklist for an <code>unsafe PR</code> in an automation repo.                                                              |
| P332 | <b>Test plan</b> — Unit + <code>Miri</code> + integration tests for new extern <code>'C'</code> wrapper — bullet list only.                                             |
| P333 | <b>Borrow checker fight</b> — I paste <code>fight-the-borrow-checker</code> code; you refactor to <code>safe Rust</code> without <code>unsafe</code> .                  |
| P334 | <b>static mut</b> — Compare <code>static mut</code> counter vs <code>AtomicUsize</code> from Ch15 — UB vs defined behavior.                                             |
| P335 | <b>PlcDriver API</b> — Design <code>safe PlcDriver Rust API</code> over fictional extern <code>'C'</code> — types, <code>Result</code> , no raw pointers in public API. |
| P336 | <b>Level ladder recap</b> — Explain Levels 1–5 in one paragraph each for a Java teammate who knows JNI.                                                                 |

## D.21 Chapter 19 — I/O and processes

| ID   | Prompt                                                                                                         |
|------|----------------------------------------------------------------------------------------------------------------|
| P337 | <b>Trait refactor</b> — Refactor file copy loop to generic copy<R: Read, W: Write>; discuss error propagation. |
| P338 | <b>CSV tool</b> — Spec for CLI: read two-column CSV, emit name=value; I implement with BufRead.                |
| P339 | <b>Packet layout</b> — Add CRC byte to 4-byte packet; update encode/decode with XOR — show tests.              |
| P340 | <b>Command safety</b> — Review shell=True style command; rewrite without shell when possible.                  |
| P341 | <b>Endian trap</b> — Quiz: 3 scenarios pick LE vs BE for Modbus-style register.                                |
| P342 | <b>Pipeline</b> — Design program A \  program B using only Rust std (two processes, pipe).                     |
| P343 | <b>Capstone scaffold</b> — Generate module tree and function signatures for sensor_gateway; no bodies.         |
| P344 | <b>Serial debug</b> — I get timeout on read; give systematic checklist (baud, cable, permissions).             |
| P345 | <b>Retry policy</b> — Design exponential backoff for Modbus-style poll errors; Rust pseudocode.                |
| P346 | <b>Log schema</b> — Propose JSON log lines for sensor events with timestamp and error codes.                   |
| P347 | <b>GPIO next step</b> — After serial works on Pi, outline migration to gpio-cdev for one LED.                  |
| P348 | <b>Code review</b> — I paste capstone main loop; review for panic risks and missing flush.                     |
| P349 | <b>read vs read_exact</b> — Give 3 protocol shapes; I pick read loop vs read_exact each time; you verify.      |
| P350 | <b>Cursor test</b> — Write unit test for parse_kv_lines using Cursor — no filesystem.                          |
| P351 | <b>BufReader why</b> — Explain in 60 words why BufReader matters for 10k-line log files.                       |
| P352 | <b>Boundary errors</b> — Sketch main mapping io::Error to exit code 1 with context path — no unwrap.           |

| ID   | Prompt                                                                                                                |
|------|-----------------------------------------------------------------------------------------------------------------------|
| P353 | <b>read_to_string trap</b> — When is read_to_string wrong for automation configs? Give size threshold rule.           |
| P354 | <b>Bit field port</b> — Java status int with flags — port to Rust encode with \ = and \& masks.                       |
| P355 | <b>CRC upgrade</b> — Replace XOR toy CRC with CRC-16-Modbus — outline steps, no full crate required.                  |
| P356 | <b>Exit status</b> — Child exited 2 — how should gateway log and retry? Table: fatal vs transient.                    |
| P357 | <b>Env and cwd</b> — Show Command with .env("PORT", "502") and .current_dir — when needed?                            |
| P358 | <b>Sync vs async pick</b> — Three gateway designs: I pick sync thread vs Tokio per scenario; you justify.             |
| P359 | <b>serialport traits</b> — Explain how serialport maps to Read/Write — diagram only.                                  |
| P360 | <b>Blocking in async</b> — Show wrong std::fs::read inside async fn; fix with tokio::fs or spawn_blocking.            |
| P361 | <b>Gateway capstone</b> — End-to-end: config file -> serial poll -> JSON log line — module list and error types only. |
| P362 | <b>Ch16 bridge</b> — Same echo server: sketch sync thread version vs Tokio version — tradeoffs table.                 |

P363 | **Path join** — Config path from HOME + .config/app.toml — Path::join vs string concat. |

P364 | **Env default** — TIMEOUT env var default 30 — unwrap\_or\_else pattern. |

P365 | **Metadata guard** — Reject config over 1MB before read\_to\_string. |

P366 | **Stdin fallback** — Port from argv or interactive prompt — one main both paths. |

P367 | **Line parser** — BufRead lines, skip #, parse key=value — trailing newline. |

P368 | **Capstone CLI** — env path -> metadata check -> line parse -> print port — function list only. |

## D.22 Chapter 20 — Production standards

| ID   | Prompt                                                                                                                                            |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| P369 | <b>Diff review</b> — Mark each of the 19 checklist rows pass/fail on a 30-line Rust PR with one sentence each.                                    |
| P370 | <b>Mega-error refactor</b> — Split one <code>AppError</code> into <code>ValidateError</code> + <code>StorageError</code> ; show boundary mapping. |
| P371 | <b>Panic hunt</b> — Find five panic sources in a snippet; replace with <code>Option/Result</code> .                                               |
| P372 | <b>Clone audit</b> — Remove three unnecessary clones by fixing signatures to <code>&amp;str / &amp;[T]</code> .                                   |
| P373 | <b>Arc style</b> — Rewrite <code>arc.clone()</code> to <code>Arc::clone(&amp;arc)</code> ; explain review benefit.                                |
| P374 | <b>Workspace.toml</b> — Sketch root + two members; shared deps use <code>{ workspace = true }</code> .                                            |
| P375 | <b>Golden test</b> — One <code>assert_eq!(got, want) + pretty_assertions</code> ; no per-field asserts.                                           |
| P376 | <b>Flaky test fix</b> — Replace <code>thread::sleep</code> in test with injected <code>Clock</code> trait.                                        |
| P377 | <b>Pre-merge gate</b> — Checklist-only review of gateway <code>main.rs</code> + <code>lib.rs</code> — findings only.                              |
| P378 | <b>AI review prompt</b> — One paragraph prompt for an assistant to verify the Ch20 rules on a Rust diff.                                          |

Total: 385 prompts (P001–P378, P379–P385).

## D.23 By theme

| Theme               | IDs                  |
|---------------------|----------------------|
| Ownership / borrow  | P006–P024, P076–P087 |
| Stack / heap        | P012–P017            |
| Functions / methods | P031–P047            |
| Iterators           | P048–P075            |
| Enums / match       | P088–P115            |

| Theme                | IDs                             |
|----------------------|---------------------------------|
| Types / traits       | P025–P030, P116–P123, P221–P243 |
| Modules / crates     | P129–P148                       |
| Smart pointers       | P149–P171                       |
| Collections          | P172–P192                       |
| Closures             | P193–P220                       |
| Errors / tests       | P124–P128                       |
| Concurrency          | P244–P265                       |
| Systems / I/O        | P337–P368                       |
| Production standards | P369–P378                       |
| Meta / tooling       | P001–P005, P266–P306, P379–P385 |

D.24 See also

- `PLAYGROUND_GUIDE.md`
- `JAVA_PYTHON_RUST_MAP.md`
- `CONTENTS.md`